

FC7300F8MDQ SDK Quick Start

Rev.0.2

Table of Contents

Table of Contents	2
Chapter 1 Introduction	10
1.1 Audience	10
1.2 Conventions	10
1.2.1 SDK Directory Structure	10
1.2.2 Abbreviations	11
1.3 Supported IDE	15
1.3.1 FC_IDE	15
1.3.2 IAR Embedded Workbench	18
1.3.3 Keil	20
1.4 Hardware Environment	27
Chapter 2 Examples and Demos	29
2.1 ADC	29
2.1.1 ADC_Single	30
2.1.2 ADC_Single_DMA	32
2.1.3 ADC_Discontinuous	34
2.1.4 ADC_Compare	36
2.1.5 ADC_Sequence_Group	38
2.1.6 ADC_Standby	40
2.1.7 ADC_Standby_With_Tstmp	42
2.2 AONTIMER	42
2.2.1 AONTIMER_TimeCounter	42
2.2.2 AONTIMER_PulseCounter	44
2.3 CLK OUT	45
2.3.1 Description	45
2.3.2 Prerequisites	46
2.3.3 MCU Ports	47
2.3.4 Steps to Run	47
2.4 CMP	48
2.4.1 Introduction	48
2.4.2 CMP_Interrupt	49
2.4.3 CMP_DMA	51
2.4.4 CMP_ChannelScan_Interrupt	52
2.5 CMU	54
2.5.1 Description	54
2.5.2 Prerequisites	54

2.5.3	MCU Ports.....	54
2.5.4	Steps to Run	54
2.6	CORDIC	55
2.6.1	Description	55
2.6.2	Prerequisites	55
2.6.3	MCU Ports.....	55
2.6.4	Steps to Run	55
2.7	CPM	56
2.7.1	Description	56
2.7.2	Prerequisites	57
2.7.3	MCU Ports.....	57
2.7.4	Steps to Run	57
2.8	CRC.....	59
2.8.1	CRC_Demo.....	60
2.8.2	CRC_Demo_Hard_Soft_Compare.....	63
2.8.3	CRC_Demo_Multi_Step	64
2.9	DMA.....	67
2.9.1	DMA_Memcpy.....	67
2.9.2	DMA_CircularBuffer	68
2.10	EFTU.....	70
2.10.1	Eftu_SignalDetect	70
2.10.2	Eftu_PwmMeasurement.....	71
2.10.3	Eftu_Timer.....	72
2.10.4	Eftu_Pwm	73
2.11	EIM/ERM.....	73
2.11.1	Description	73
2.11.2	Prerequisites	74
2.11.3	MCU Ports.....	74
2.11.4	Steps to Run	74
2.12	ENET	74
2.12.1	ENET_lwIP_FreeRTOS	75
2.12.2	ENET_lwIP_NoSys.....	78
2.12.3	RT_THREAD	81
2.12.4	RT_THREAD_AMP.....	85
2.13	FCIIC.....	88
2.13.1	Description	88
2.13.2	Prerequisites	88
2.13.3	MCU Ports.....	88
2.13.4	Steps to Run	88

2.14	FCPIT	89
2.14.1	FCPIT_TimeCounter	89
2.14.2	FCPIT_ChainModeCounter	91
2.14.3	FCPIT_Trgsel	92
2.15	FCSMU	94
2.15.1	Description	94
2.15.2	Prerequisites	94
2.15.3	MCU Ports	94
2.15.4	Steps to Run	94
2.16	FCSPI	95
2.16.1	FCSPI_Master_Polling	95
2.16.2	FCSPI_Master_Interrupt	99
2.16.3	FCSPI_Master_DMA	102
2.16.4	FCSPI_Slave_Interrupt	106
2.17	FCUART	110
2.17.1	Description	110
2.17.2	Prerequisites	111
2.17.3	MCU Ports	112
2.17.4	Steps to Run	112
2.18	Flash	113
2.18.1	Flash_Async_Project	113
2.18.2	Flash_INT_Project	115
2.18.3	Flash_Sync_Project	117
2.19	FLEXCAN	119
2.19.1	CAN_DMA_Transmit	119
2.19.2	CAN_EnhanceFIFO_DMA	121
2.19.3	CAN_LegacyFIFO_DMA	123
2.19.4	CAN_MB_EnhancedFIFO	124
2.19.5	CAN_MB_LegacyFIFO	126
2.20	FPU&DSP	127
2.20.1	Description	127
2.20.2	Prerequisites	130
2.20.3	MCU Ports	130
2.20.4	Steps to Run	130
2.21	FreeRTOS	131
2.21.1	Description	131
2.21.2	Prerequisites	131
2.21.3	MCU Ports	132
2.21.4	Steps to Run	132

2.22	FREQM	132
2.22.1	Description	132
2.22.2	Prerequisites	132
2.22.3	MCU Ports	133
2.22.4	Steps to Run	133
2.23	FS26	133
2.23.1	Description	133
2.23.2	Prerequisites	133
2.23.3	MCU Ports	133
2.23.4	Steps to Run	134
2.24	FTU	134
2.24.1	InputCapture	135
2.24.2	TimeCounter	135
2.24.3	OutputCompare	137
2.24.4	PWM	138
2.24.5	QuadratureDecoder	139
2.25	GPIO	140
2.25.1	GPIO_Toggle	140
2.25.2	GPIO_EdgeDetect	142
2.25.3	GPIO_DigitalFilter	143
2.26	HRPWM	144
2.26.1	Description	144
2.26.2	Prerequisites	144
2.26.3	MCU Ports	144
2.26.4	Steps to Run	145
2.27	HSADC	145
2.27.1	HSADC_Single	147
2.27.2	HSADC_Sequence_Group	149
2.27.3	HSADC_Single_Internal_Trigger	151
2.27.4	HSADC_Fast_Compare	151
2.28	HSM	153
2.28.1	Description	153
2.28.2	Prerequisites	154
2.28.3	MCU Ports	154
2.28.4	Steps to Run	154
2.29	INTM	155
2.29.1	Description	155
2.29.2	Prerequisites	156
2.29.3	MCU Ports	157

2.29.4 Steps to Run	157
2.30 ISM	157
2.30.1 Description	157
2.30.2 Prerequisites	157
2.30.3 MCU Ports.....	157
2.30.4 Steps to Run	157
2.31 LIN	158
2.31.1 Description	158
2.31.2 Prerequisites	158
2.31.3 MCU Ports.....	159
2.31.4 Steps to Run	159
2.32 LU	161
2.32.1 Description	161
2.32.2 Prerequisites	161
2.32.3 MCU Ports.....	161
2.32.4 Steps to Run	161
2.33 Mailbox	162
2.33.1 Description	162
2.33.2 Prerequisites	162
2.33.3 MCU Ports.....	162
2.33.4 Steps to Run	163
2.34 MAM	165
2.34.1 Description	165
2.34.2 Prerequisites	165
2.34.3 MCU Ports.....	166
2.34.4 Steps to Run	166
2.35 MPU.....	167
2.35.1 Description	167
2.35.2 Prerequisites	167
2.35.3 MCU Ports.....	167
2.35.4 Steps to Run	167
2.36 MSC	168
2.36.1 Description	170
2.36.2 Prerequisites	170
2.36.3 MCU Ports.....	170
2.36.4 Steps to Run	170
2.37 Multicore.....	171
2.37.1 Description	171
2.37.2 Prerequisites	171

2.37.3	MCU Ports.....	171
2.37.4	Steps to Run	172
2.38	OTA	172
2.38.1	OTA_RegisterMode	172
2.38.2	OTA_NvrMode	176
2.39	Overlay.....	180
2.39.1	Description	180
2.39.2	Prerequisites	181
2.39.3	MCU Ports.....	181
2.39.4	Steps to Run	181
2.40	PMC	183
2.40.1	Description	183
2.40.2	Prerequisites	184
2.40.3	MCU Ports.....	184
2.40.4	Steps to Run	184
2.41	PTIMER.....	184
2.41.1	Description	184
2.41.2	Prerequisites	185
2.41.3	MCU Ports.....	185
2.41.4	Steps to Run	186
2.42	QDT	186
2.42.1	QDT_IC_Demo.....	186
2.42.2	QDT_ICDM_Demo	187
2.42.3	QDT_ICENM_Demo	188
2.42.4	QDT_ICEXPENM_Demo.....	189
2.42.5	QDT_ICPM_Demo	191
2.42.6	QDT_QUAD_Demo	192
2.43	RGM.....	193
2.43.1	Description	193
2.43.2	Prerequisites	194
2.43.3	MCU Ports.....	194
2.43.4	Steps to Run	194
2.44	RTC.....	194
2.44.1	RTC_Alarm.....	195
2.44.2	RTC_SecondInterrupt.....	196
2.44.3	RTC_Overflow	198
2.44.4	RTC_SetReadTime	199
2.45	SC6258XQ	201
2.45.1	Description	201

2.45.2	Prerequisites	201
2.45.3	MCU Ports.....	201
2.45.4	Steps to Run	201
2.46	SCST.....	203
2.46.1	Description	203
2.46.2	Prerequisites	203
2.46.3	MCU Ports.....	203
2.46.4	Steps to Run	204
2.47	SDADC.....	204
2.47.1	SDADC_Single	204
2.47.2	SDADC_Continuous	206
2.47.3	SDADC_Calibration.....	207
2.48	SEC.....	209
2.48.1	Description	209
2.48.2	Prerequisites	209
2.48.3	MCU Ports.....	209
2.48.4	Steps to Run	210
2.49	SEMA.....	210
2.49.1	Description	210
2.49.2	Prerequisites	210
2.49.3	MCU Ports.....	211
2.49.4	Steps to Run	211
2.50	SENT	211
2.50.1	SENT_Polling.....	211
2.50.2	SENT_Interrupt	213
2.50.3	SENT_Dma.....	214
2.50.4	SENT_SPC.....	216
2.51	SMC.....	217
2.51.1	Description	217
2.51.2	Prerequisites	218
2.51.3	MCU Ports.....	218
2.51.4	Steps to Run	219
2.52	STCU	219
2.52.1	Description	219
2.52.2	Prerequisites	220
2.52.3	MCU Ports.....	221
2.52.4	Steps to Run	221
2.53	Systick.....	221
2.53.1	Description	221

2.53.2 Prerequisites	221
2.53.3 MCU Ports	222
2.53.4 Steps to Run	222
2.54 TMU	222
2.54.1 Description	222
2.54.2 Prerequisites	224
2.54.3 MCU Ports	225
2.54.4 Steps to Run	225
2.55 TPU	225
2.55.1 Tpu_Pwm_NonFlexCore	225
2.55.2 Tpu_Capture_Host	227
2.55.3 Tpu_Host	229
2.55.4 Tpu_Pwm_FlexCore	232
2.55.5 Tpu_Capture_FlexCore	234
2.56 TSTMP	236
2.56.1 Description	236
2.56.2 Prerequisites	237
2.56.3 MCU Ports	238
2.56.4 Steps to Run	238
2.57 WDOG	238
2.57.1 Description	238
2.57.2 Prerequisites	240
2.57.3 MCU Ports	241
2.57.4 Steps to Run	241
Revision History	243
Disclaimer	244

Chapter 1 Introduction

The Flagchip FC7xxx MCU Software Development Kit (SDK) is a collection of software development tools that allows software engineers to develop diverse applications for the FC7xxx MCU family easier. This toolkit integrates essential software components and typical examples into one easy-to-use software package.

This document describes how to use the FC7300F8MDQ SDK to implement different use cases.

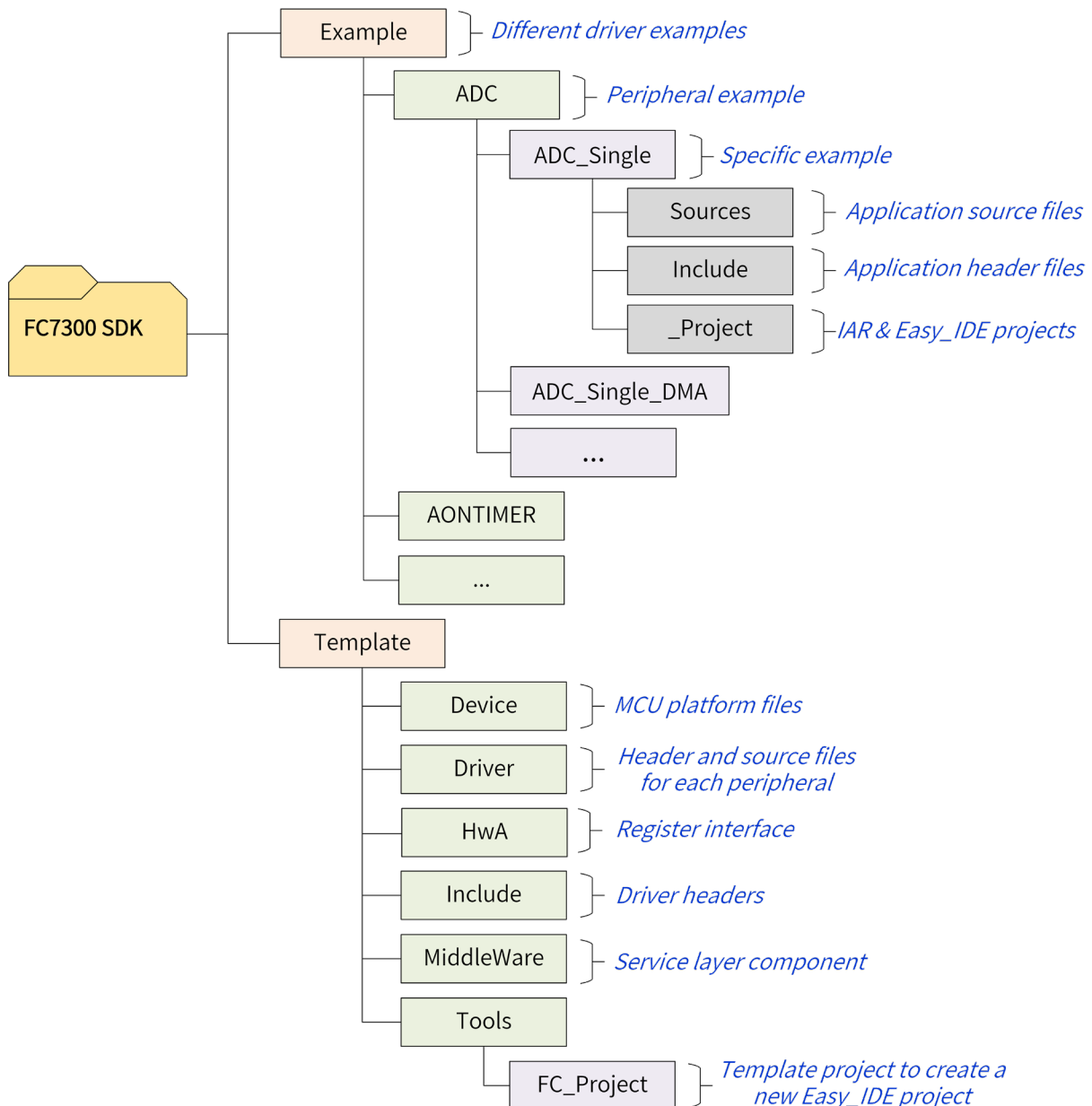
1.1 Audience

This user guide is intended for software developers who are using the FC7xxx MCU family to develop their own applications. The readers are expected to have basic knowledge of operating systems, logic circuits, microcontrollers, etc.

1.2 Conventions

1.2.1 SDK Directory Structure

The directory structure of the SDK is as follows:



1.2.2 Abbreviations

The following abbreviations are used in this document.

No.	Abbreviation	Description
1	ACC	ARMv7 Cache Controller
2	ADC	Analog-to-Digital Converter
3	AFCB	Advanced Flagchip Bus (APB Bridge)
4	AHB	Advanced High-Performance Bus
5	AHBD	AHB-Lite Debug
6	AHBP	AHB-Lite Peripheral
7	AHBS	AHB-Lite Slave
8	AMBA	Advanced Microcontroller Bus Architecture
9	AMP	Asymmetric Multi-Processing
10	AOI	AND/OR/INVERT
11	AONTIMER	Always-on Timer
12	AP	Access Port
13	APB	Advanced Peripheral Bus
14	AVB	Audio Video Bridging
15	AXI	Advanced eXtensible Interface
16	BG	Bandgap
17	BGA	Ball Grid Array
18	BIU	Bus Interface Unit
19	BJT	Bipolar Junction Transistor
20	BTB	Back-to-Back
21	CBC	Cipher Block Chaining
22	CCM	Counter with CBC-MAC
23	CCU	Code Cache Unit
24	CGC	Clock Gating Control
25	CIC	Cascaded Integrator Comb
26	CPM	CPU Private Module
27	CMAC	Cipher-based Message Authentication Code
28	CMU	Clock Monitor Unit
29	COCO	Conversion Complete
30	CPWM	Center-aligned Pulse-Width Modulation
31	CRC	Cyclic Redundancy Check
32	CSC	CPU System Control
33	CTCM	Code Tightly-Coupled Memory
34	CTI	Cross Trigger Interface
35	CTM	Cross Trigger Matrix
36	CTR	Counter
37	DAC	Digital-to-Analog Converter
38	DDR	Double Data Rate
39	DMA	Direct Memory Access

No.	Abbreviation	Description
40	DMAMUX	Direct Memory Access Multiplexer
41	DP	Debug Port
42	DPFPU	Double-precision Floating Point Unit
43	DSE	Driver Strength Enable
44	DSP	Digital Signal Processing
45	DTCM	Data Tightly-Coupled Memory
46	DWP	Domain Write Protection
47	DWT	Data Watchpoint and Trace
48	ECB	Electronic Codebook Book
49	ECC	Error Correction Code
50	ECM	Event Combiner Module
51	EEPROM	Electronically Erasable Programmable Read-Only Memory
52	eFTU	Enhanced Flexible Timer Unit
53	EIM	Error Injection Module
54	EPPB	External Private Peripheral Bus
55	EPWM	Edge-aligned Pulse-Width Modulation
56	ERM	Error Reporting Module
57	ETB	Embedded Trace Buffer
58	ETM	Embedded Trace Macro
59	EVITA	E-safety Vehicle Intrusion Protected Applications
60	EVTG	Event Generator
61	FCIIC	Flagchip (FC) Inter-Integrated Circuit
62	FCPIT	Flagchip (FC) Programmable Interrupt Timer
63	FCSMU	Fault Control and Safety Management Unit
64	FCSPI	Flagchip (FC) Serial Peripheral Interface
65	FCUART	Flagchip (FC) Universal Asynchronous Receiver/Transmitter
66	FF	Flip-Flop
67	FIFO	First In First Out
68	FIRC	Fast Internal Reference Clock
69	FLEXCAN	FLEX Controller Area Network
70	FMC	Flash Memory Controller
71	FOSC	Fast Oscillator
72	FPB	Flash Patch and Breakpoint
73	FPC	Filter & Prescaler Channel
74	FPM	Full Performance Mode
75	FPU	Floating Point Unit
76	FREQM	Frequency Measurement
77	FSE	Fast Select Enable
78	FSM	Finite State Machine
79	FTU	Flexible Timer Unit
80	GCM	Galois/Counter Mode
81	GPIO	General-Purpose Input/Output

No.	Abbreviation	Description
82	GTB	Global Time Base
83	HMI	Human-Machine Interface
84	HRPWM	High Resolution PWM
85	HSADC	High-Speed Analog-to-Digital Converter
86	HSM	Hardware Secure Module
87	HVD	High-Voltage Detect
88	IIC/I2C	Inter-Integrated Circuit
89	INTM	Interrupt Monitor
90	IRC	Internal Reference Clock
91	IRQ	Interrupt Request
92	ISM	Interface Safety Monitor
93	ISP	In-System Program
94	ITCM	Instruction Tightly-Coupled Memory
95	ITM	Instrumentation Trace Macrocell
96	LAM	Logic Analyzer Module
97	LBIST	Logic Built-In Self-Test
98	LDO	Low Dropout
99	LIN	Local Interconnect Network
100	LQFP	Low-profile Quad Flat Package
101	LU	Logic Unit
102	LVD	Low-Voltage Detect
103	LVDS	Low Voltage Differential Signaling
104	LVR	Low-Voltage Reset
105	MAM	Matrix Access Monitor
106	MAP	Memory Access Protection
107	MB	Mailbox
108	MBIST	Memory Built-In Self-Test
109	MDC	Management Data Clock
110	MDI	Mailbox Data Input
111	MDIO	Management Data Input/Output
112	MII	Media Independent Interface
113	MISO	Master In Slave Out
114	MOSI	Master Out Slave In
115	MPU	Memory Protection Unit
116	MSC	Micro Second Channel
117	MVA	Modified Virtual Address
118	NMI	Non-maskable Interrupt
119	NPN	Negative-Positive-Negative
120	NVIC	Nested Vectored Interrupt Controller
121	NVM	Non-volatile Memory
122	NVR	Non-volatile Region
123	OPAM	Operational Amplifier

No.	Abbreviation	Description
124	OSC	Oscillator
125	PAP	Peripheral Access Protection
126	PCC	Peripheral Configuration Control
127	PIT	Programmable Interrupt Timer
128	PLL	Phase-Locked Loop
129	PMC	Power Management Controller
130	PMIC	Power Management Integrated Circuit
131	PNET	Pretended Networking
132	POR	Power-on Reset
133	PPB	Private Peripheral Bus
134	PTAT	Proportional-to-Absolute Temperature
135	PTP	Precision Time Protocol
136	PWM	Pulse Width Modulation
137	QDT	Quadrature Decode Timer
138	RCC	Reset and Clock Controller
139	RGM	Reset Generation Module
140	RMII	Reduced Media Independent Interface
141	RPM	Reduce Power Mode
142	RSA	Rivest-Shamir-Adleman
143	RTC	Real-Time Clock
144	RWDS	Read/Write Data Strobe
145	RWW	Read-While-Write
146	SBC	System Basis Chip
147	SCG	System Clock Generator
148	SCL	Serial Clock Line
149	SCM	System Control Module
150	SCS	System Control Space
151	SDA	Serial Data Line
152	SDADC	Sigma Delta Analog-to-Digital Converter
153	SDR	Single Data Rate
154	SEC	Security Controller
155	SECCED	Single-bit Error Correction and Double-bit Error Detection
156	SEMA	Semaphore
157	SENT	Single Edge Nibble Transmission
158	SIMD	Single Instruction Multiple Data
159	SIRC	Slow Internal Reference Clock
160	SMBUS	System Management Bus
161	SMC	System Mode Controller
162	SMP	Symmetric Multi-Processing
163	SOG	Sea of Gate
164	SOSC	Slow Oscillator
165	SPC	Short PWM Code

No.	Abbreviation	Description
166	SPFPU	Single-precision Floating Point Unit
167	SPI	Serial Peripheral Interface
168	SRE	Slew Rate Enable
169	SSI	Speed and Sensor Interface
170	STCU	Safety Test and Control Unit
171	SWD	Serial Wire Debug
172	SWJ-DP	Serial Wire/JTAG Debug Port
173	SysTick	System Tick Timer
174	TAP	Test Access Port
175	TCM	Tightly-Coupled Memory
176	TMU	Temperature Monitor Unit
177	TPIU	Trace Port Interface Unit
178	TRGSEL	Trigger Select
179	TSN	Time-Sensitive Networking
180	TSTMP	Timer Stamp
181	UART	Universal Asynchronous Receiver and Transmitter
182	VCO	Voltage-Controlled Oscillator
183	WDOG	Watchdog
184	WIC	Wake-up Interrupt Controller
185	WKU	Wake-up Unit

1.3 Supported IDE

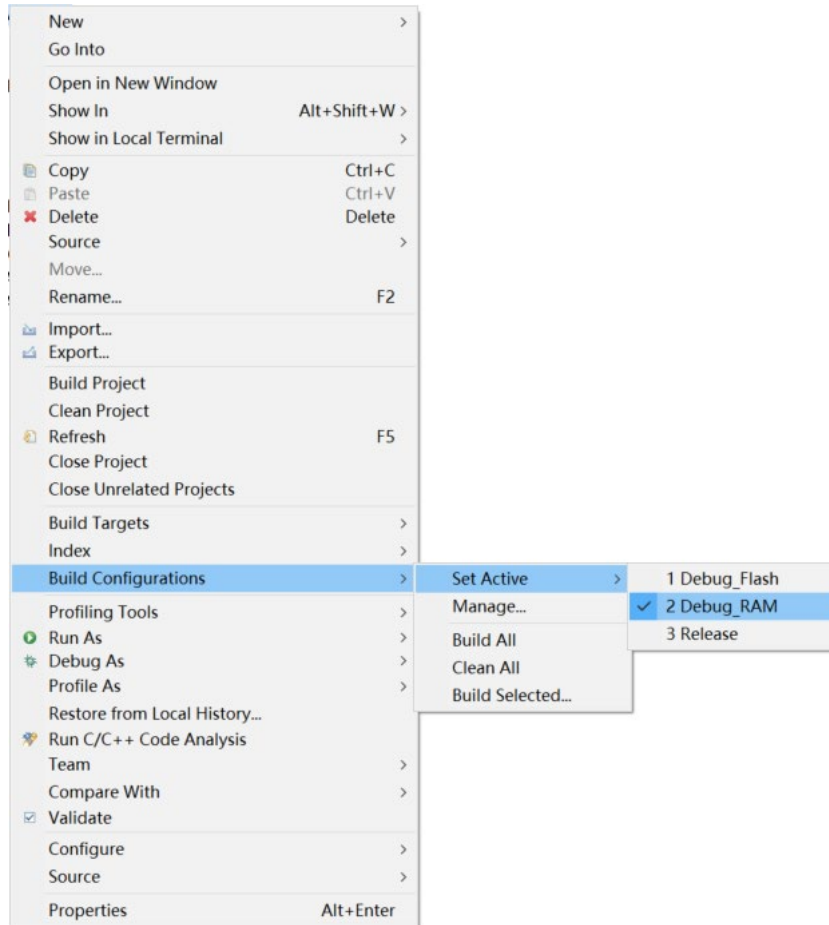
The SDK supports multiple IDEs including the FC_IDE, IAR Embedded Workbench, and Keil.

1.3.1 FC_IDE

To run an example by using the FC_IDE, take the following steps:

- 1) Import the project into the FC_IDE.
 - a. Open the IDE, and select **File > Import > General > Existing Project into Workspace**.
 - b. Browse and select the project (Directory: SDK\Example*Desired_Example_Name*_Project\FC_Project) you want to import and click **Finish**.
- 2) Right-click on the project name, and select **Build Configuration > Set Active**. "Debug_RAM" is selected as the active build configuration by default. You can select one of configurations according to your needs.

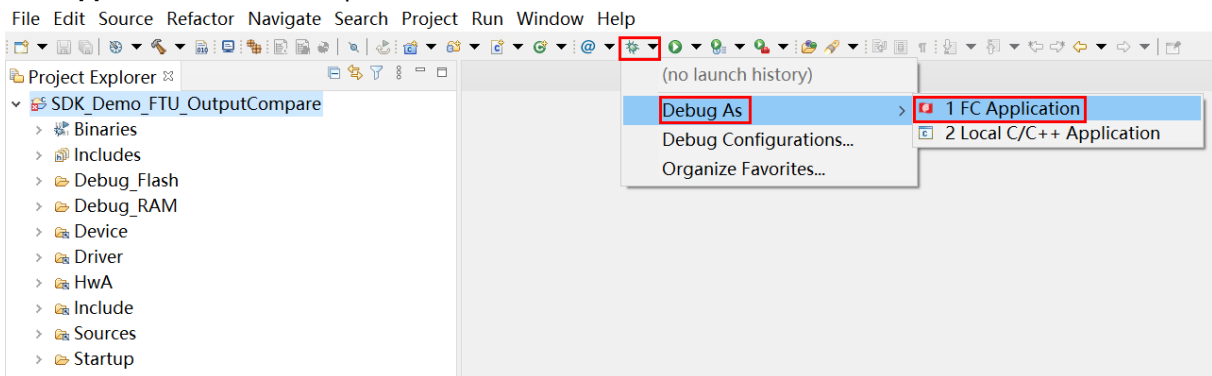
Note: There are two kinds of build configuration options: **Debug_Flash** and **Debug_RAM**. The major difference between the two options is the location where the program is loaded. If **Debug_Flash** is selected, the data will be loaded to the flash; if **Debug_RAM** is selected, the data will be loaded to the RAM.



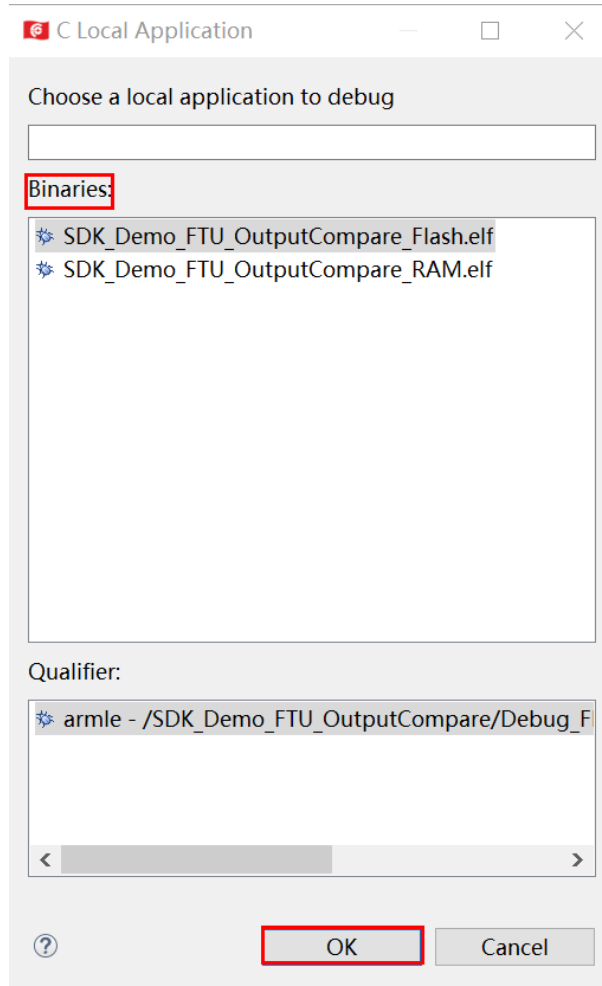
- 3) Select **Build Project** to build the project.
- 4) Create configuration and debug. There are two ways to create configuration and debug for a project.

Method 1:

- a) Click the upside-down triangle icon (▼) next to the Debug icon (🔍) in the toolbar, and select **Debug As > FC Application** from the drop-down list.



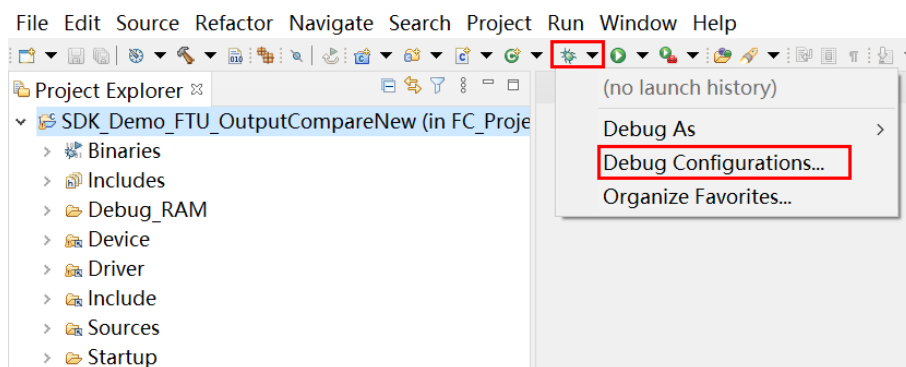
- b) In the pop-up **C Local Application dialog**, double-click one .elf file you need. Then the debug begins.



Note: If there is only one debug option for a project, there will be no pop-up dialog, and the debug will start immediately.

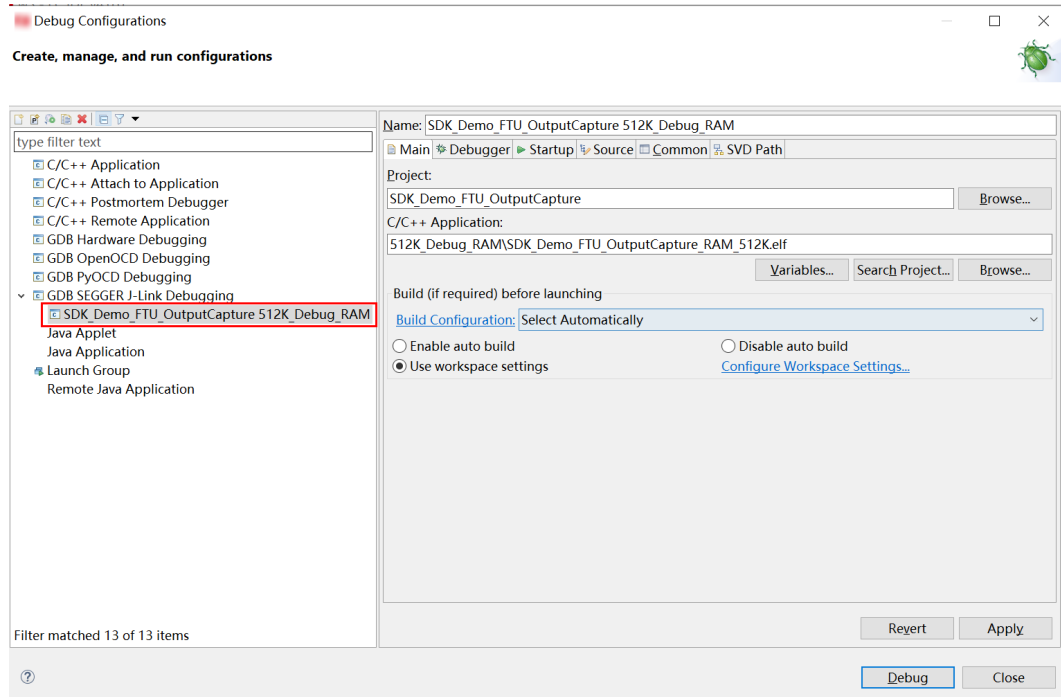
Method 2:

- Click the upside-down triangle icon (▼) next to the Debug icon (🔧) in the toolbar, and select **Debug Configurations** from the drop-down list.



- In the **Debug Configurations** dialog, double-click **GDB SEGGER J-Link Debugging** on the left pane to create a new configuration.

Note: If there is no ".elf" file displayed under the **C/C++ Application**, close the **Debug Configurations** dialog, right-click on the project name, and select **Refresh**. Then continue Step a.



c) Click **Debug** to start debugging.

5) At the same time, this process will download the binary file of the project program to the target MCU.

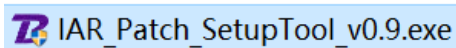
For details on how to get started with the FC_IDE, refer to the *FC_IDE User Guide*.

1.3.2 IAR Embedded Workbench

IAR's Embedded Workbench can also be used as an IDE for developing and debugging applications. For details on how to use the IAR, visit the IAR official website. The following is the example for IAR 8.2x/8.3x.

To add the support for the Flagchip series chips to the IAR, you need to install the IAR patch provided by Flagchip before using. The installation steps are as follows:

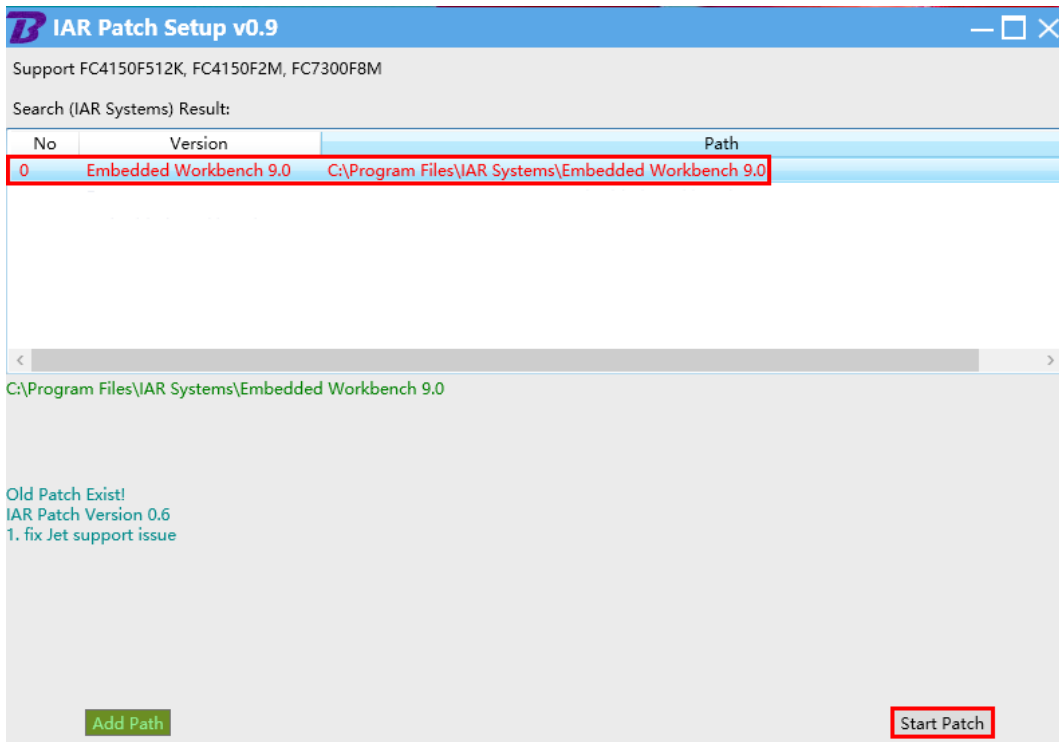
1) Double-click **IAR Patch Setup vX.X.X.exe** (Latest version).



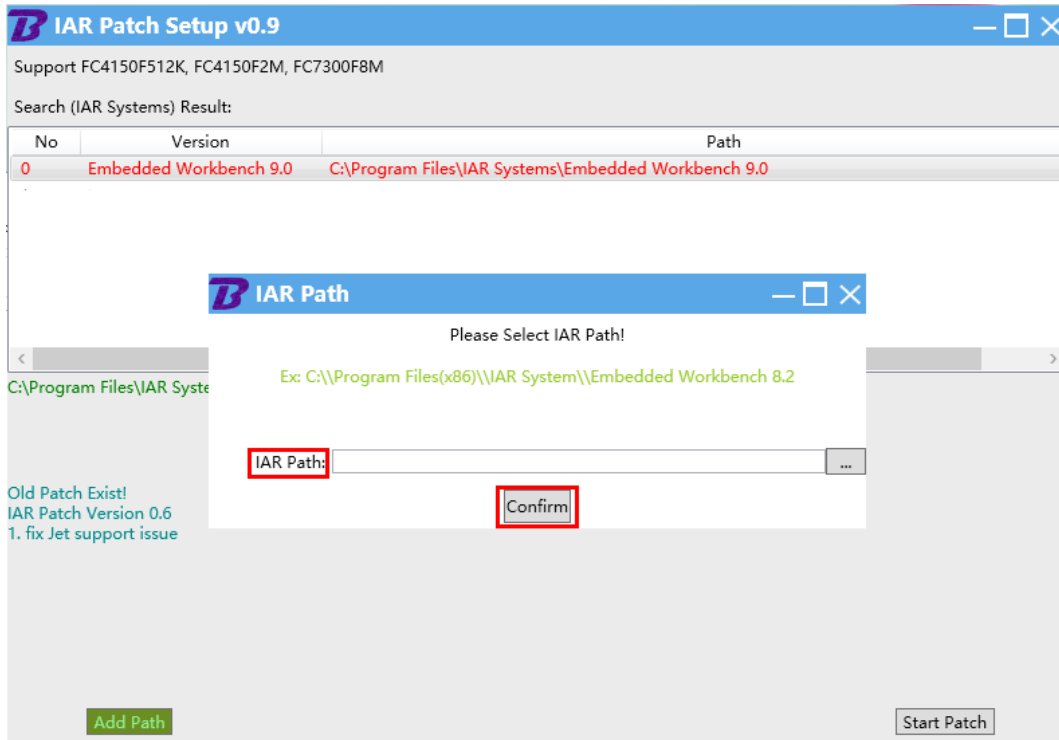
2) Select "我接受安装须知", and click **OK**.



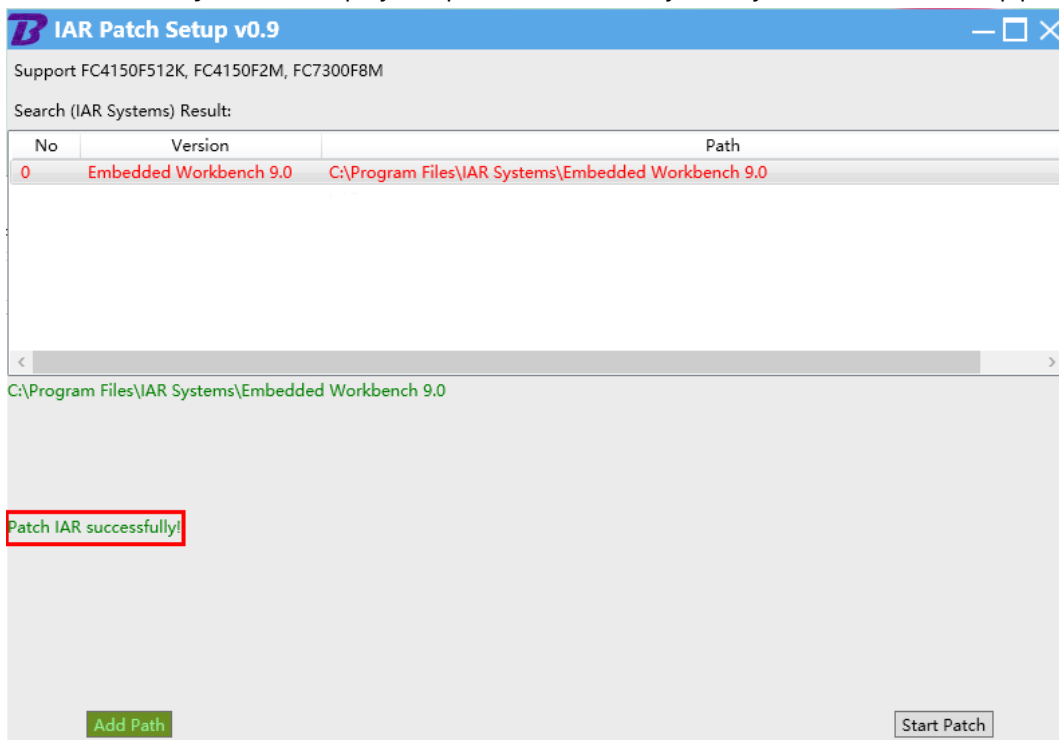
- 3) Select the installation path of Keil under **Search (IAR Systems) Result**, and click **Start Patch**.



Note: If there is no installation path shown, click **Add Path** to add the path manually, click **Confirm**, and then click **Start Patch**.



- 4) "Patch IAR successfully!" will be displayed if patched successfully. Then you can close the setup program.



1.3.3 Keil

Before running an example by using the Keil, take the following two steps:

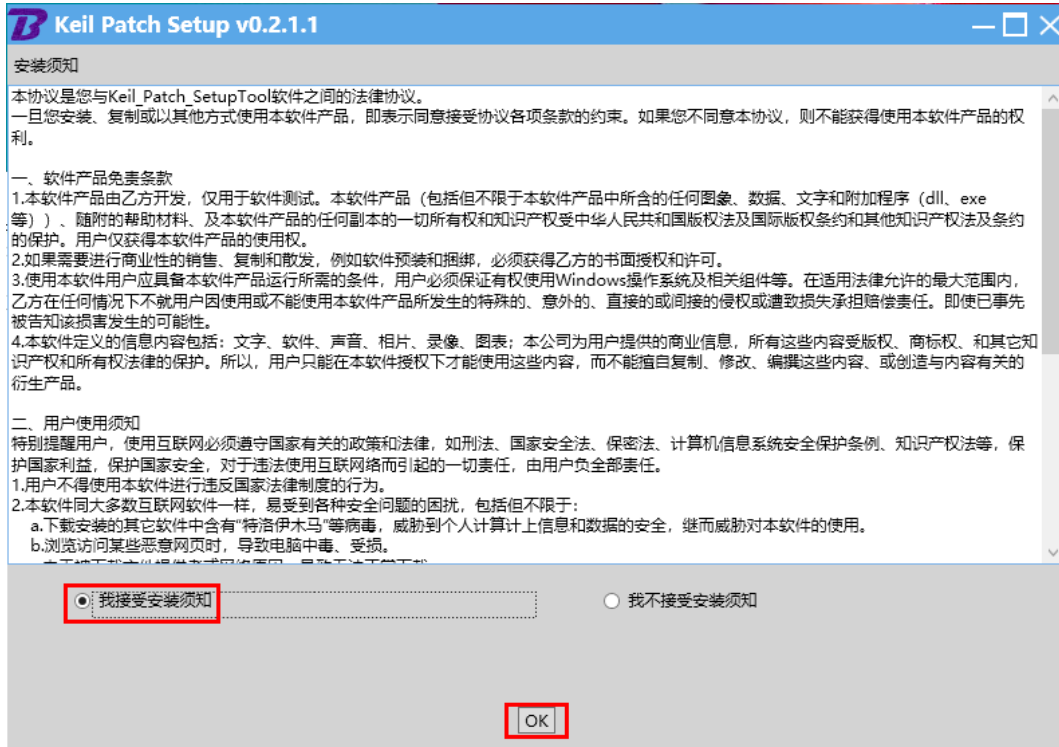
1. Install Patch for Keil.

To add the support for the Flagchip series chips to the Keil, you need to install the Keil patch provided by Flagchip before using. The installation steps are as follows:

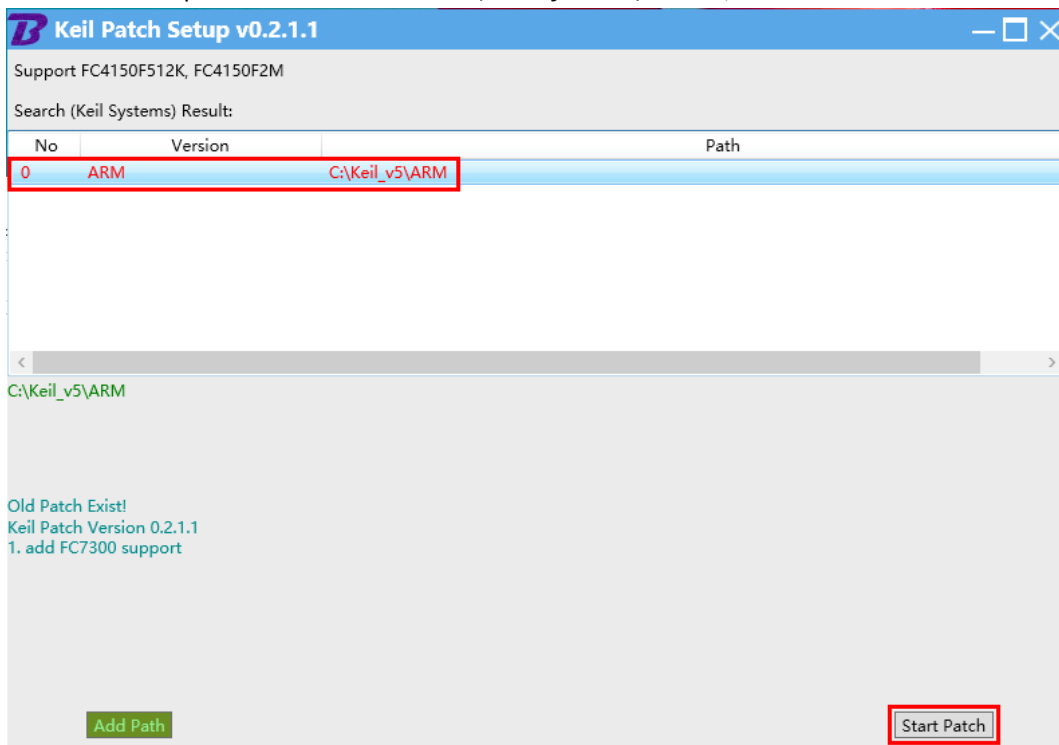
- 1) Double-click **Keil_Patch_SetupTool_vX.X.X.exe** (Latest version).

 [Keil_Patch_SetupTool_v0.2.1.1.exe](#)

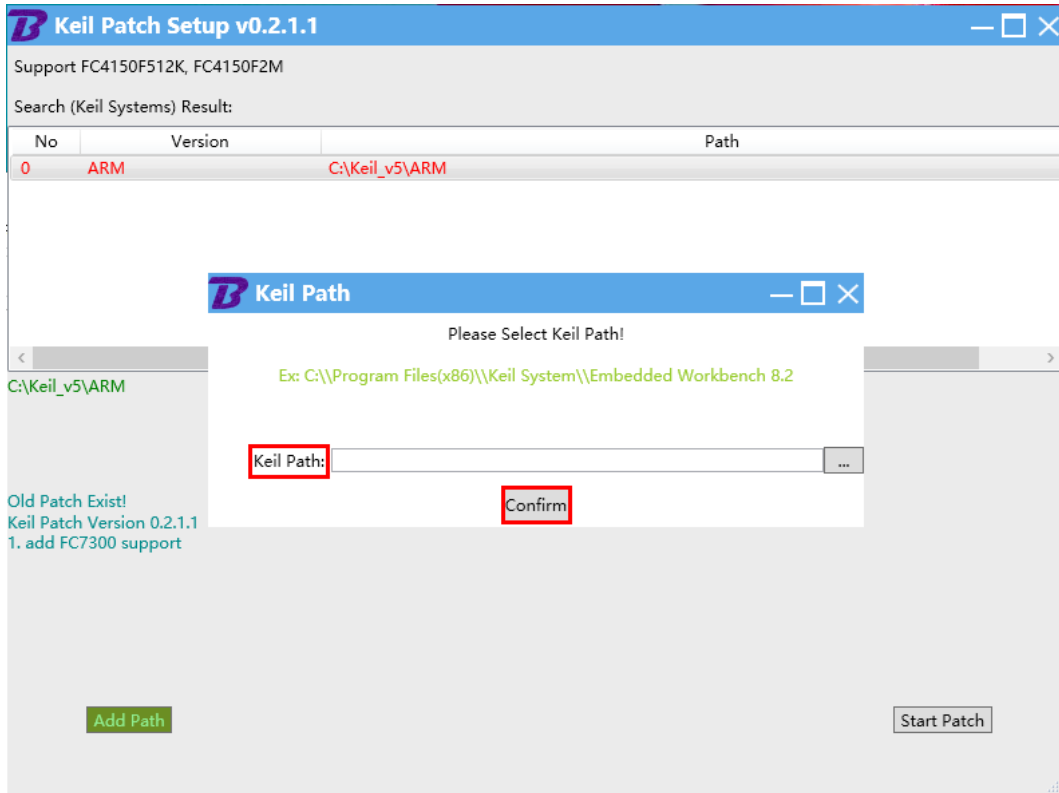
- 2) Select "我接受安装须知", and click OK.



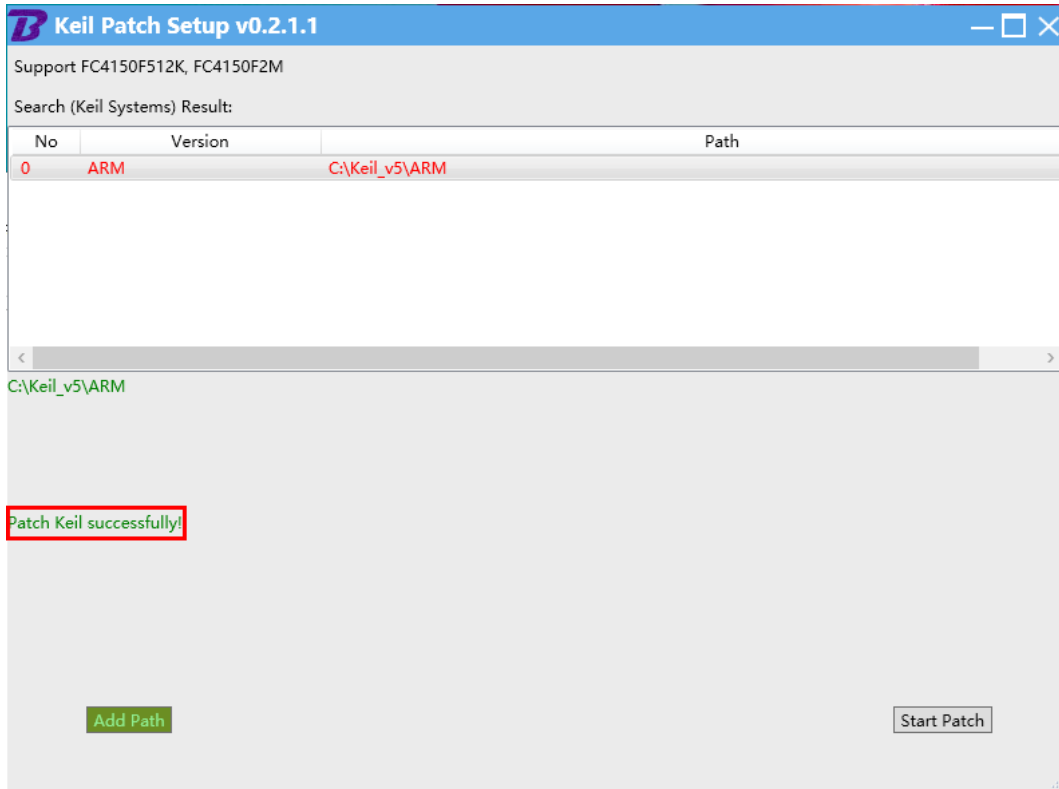
- 3) Select the installation path of Keil under **Search (Keil Systems) Result**, and click **Start Patch**.



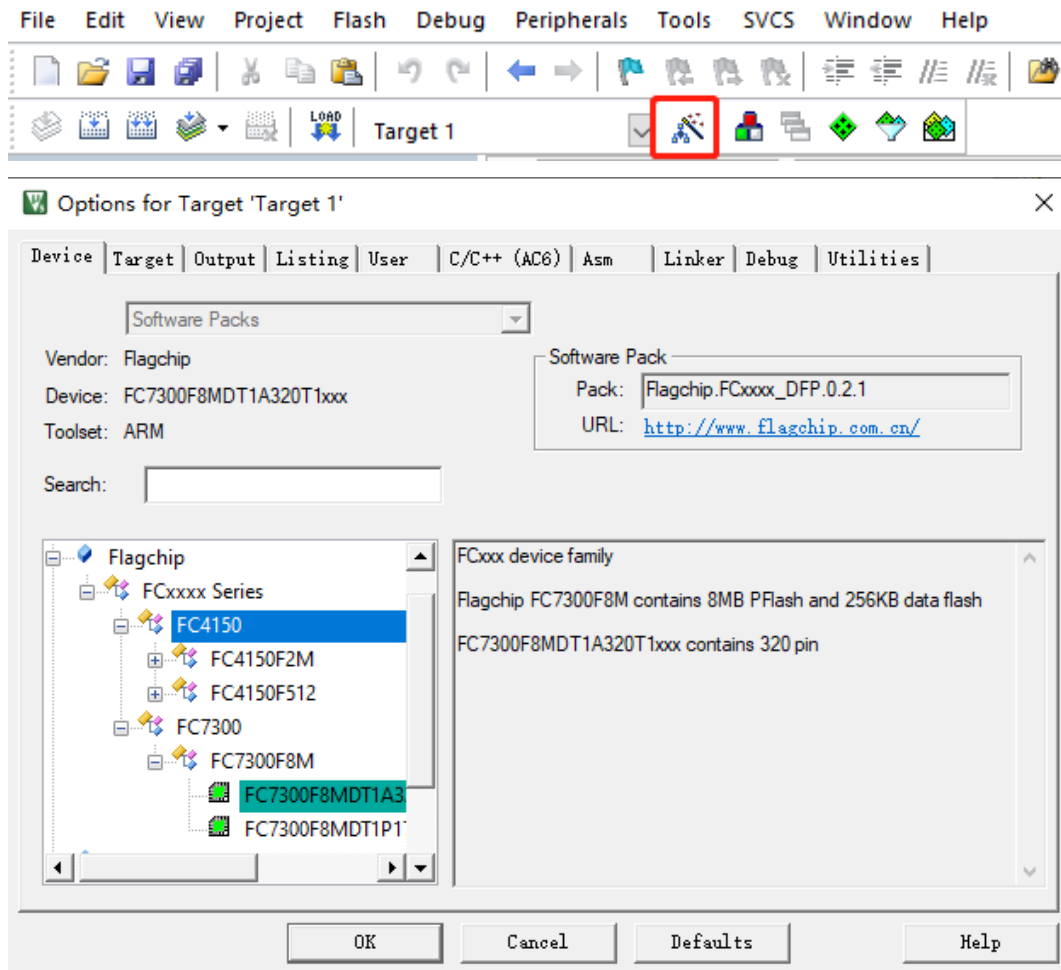
Note: If there is no installation path shown, click **Add Path** to add the path manually, click **Confirm**, and then click **Start Patch**.



- 4) "Patch Keil successfully!" will be displayed if patched successfully. Then you can close the setup program.



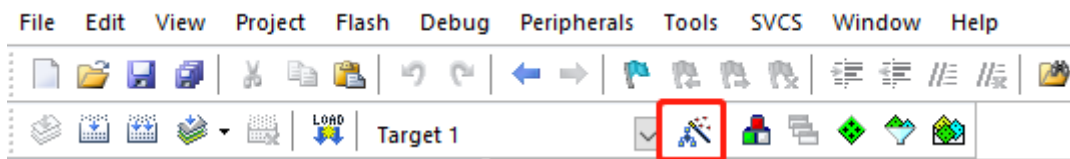
- 5) After opening a Keil project, click the icon  (**Options for Target**) in the toolbar, and click the **Device** tab, the Flagchip series pack information will be shown.



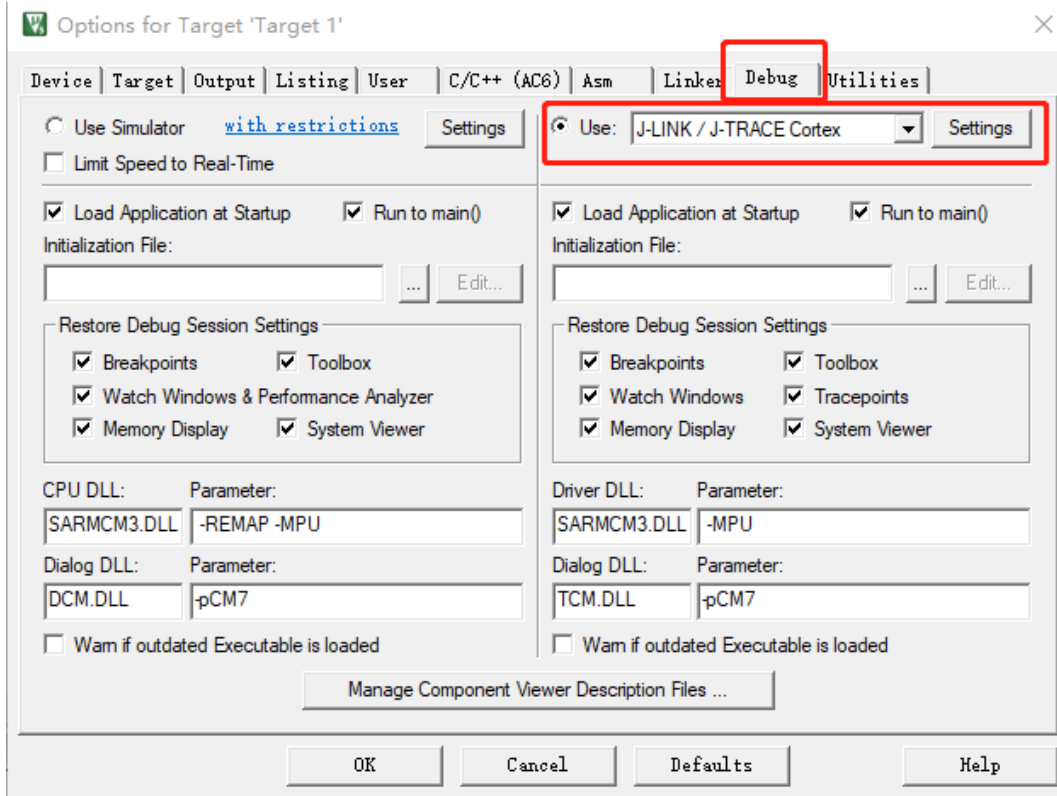
2. Configure Keil before downloading or debugging.

Before downloading or debugging a project, take the following steps:

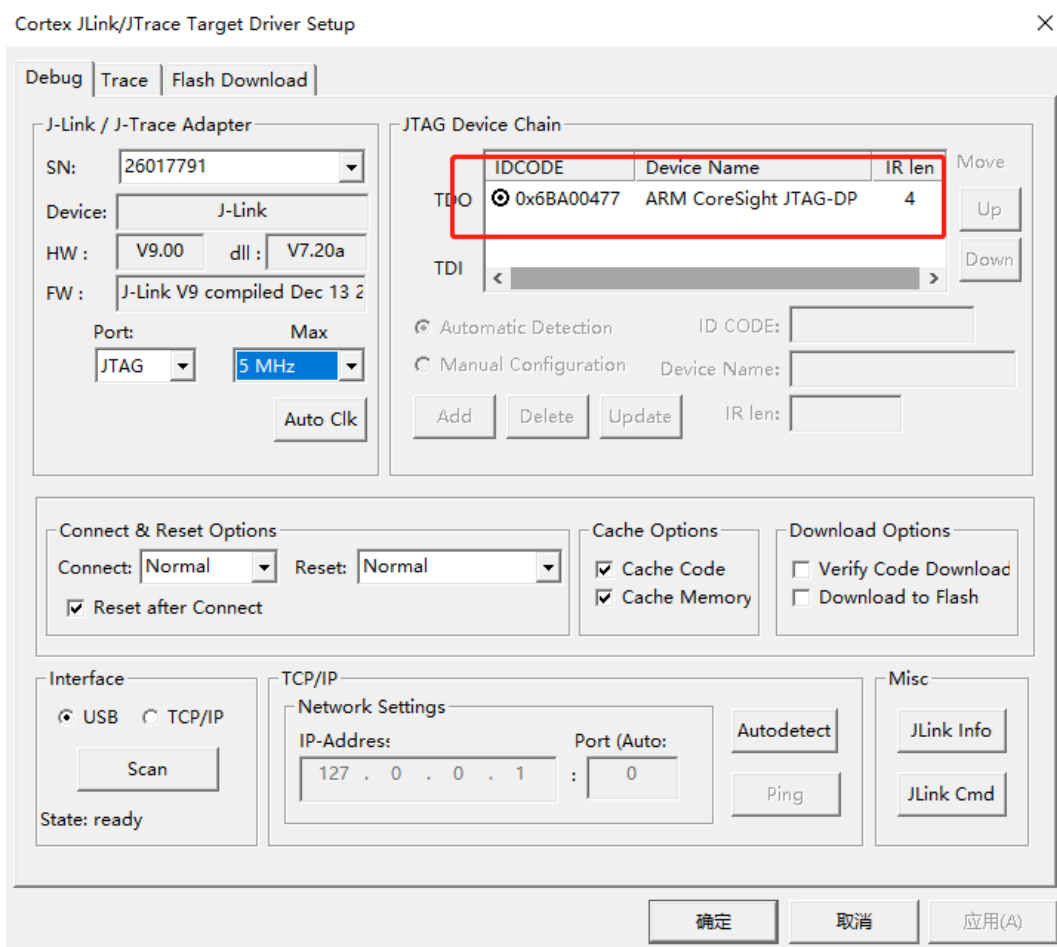
- 1) Open a Keil project, and click the icon  (**Options for Target**) in the toolbar.



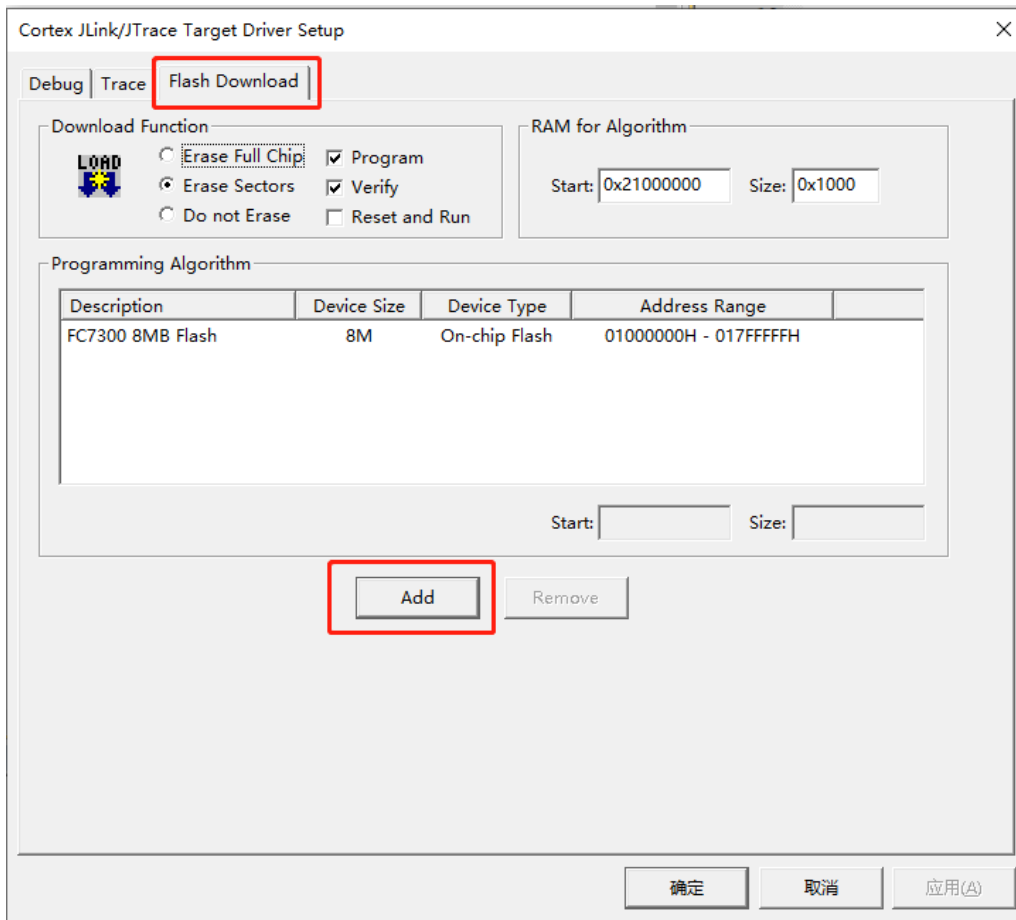
- 2) In the **Debug** tab, select **J-LINK / J-TRACE Cortex** from the **Use** drop-down list, and click **Settings**.



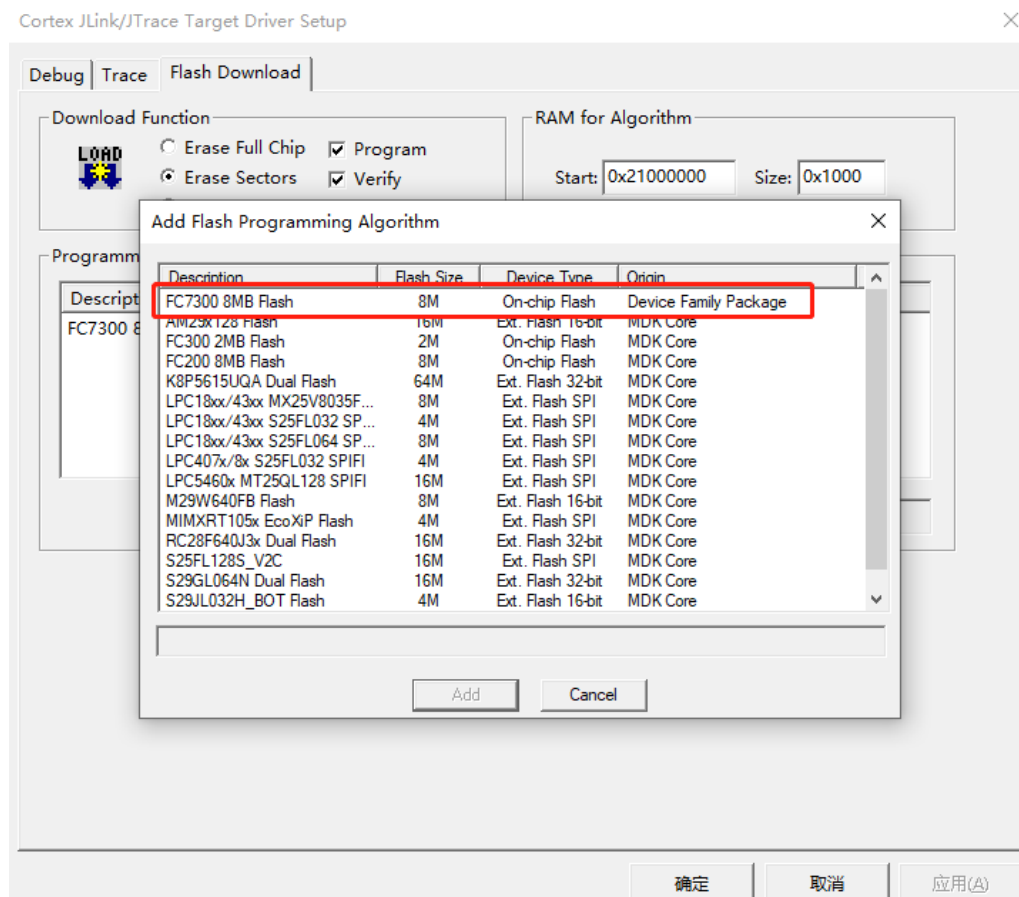
- 3) Under the **JTAG Device Tool Chain**, the FC7300 series-related information will be displayed. **Note:** Make sure "JTAG" is selected for **Ports**.



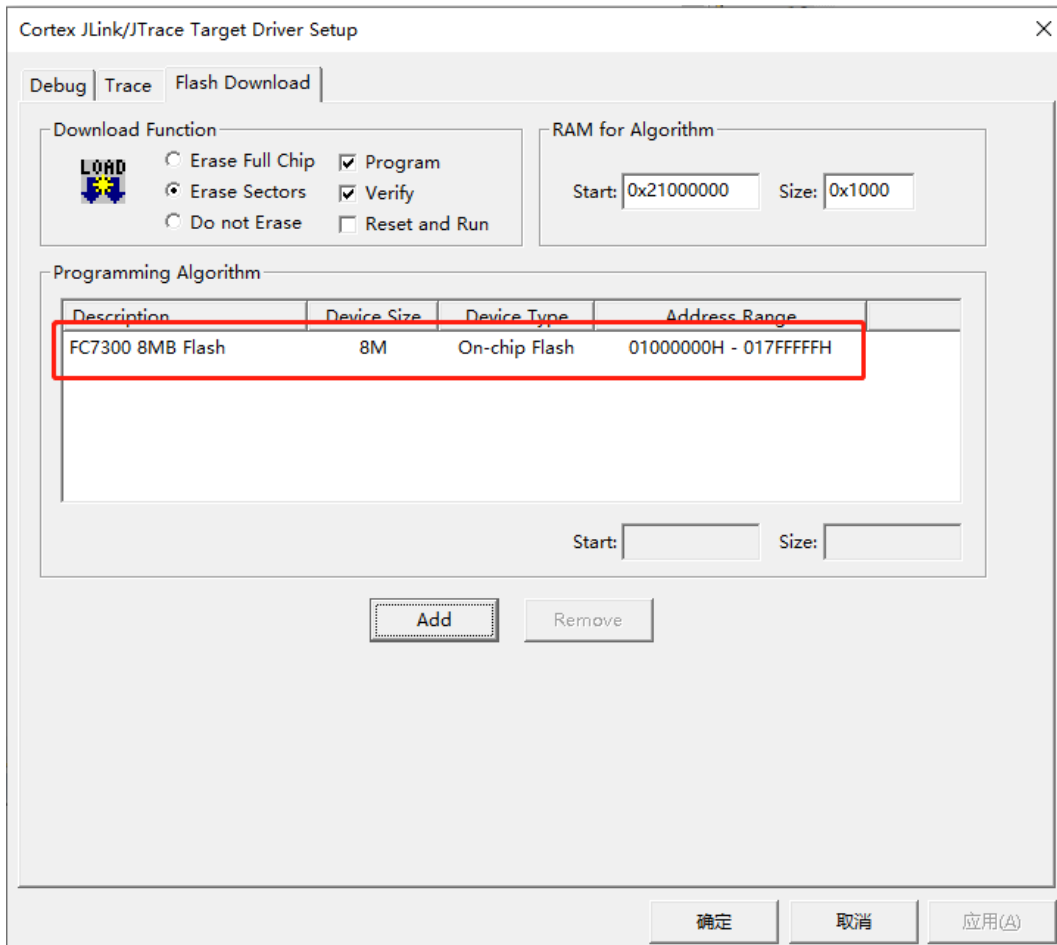
- 4) In the **Flash Download** tab, click **Add**.



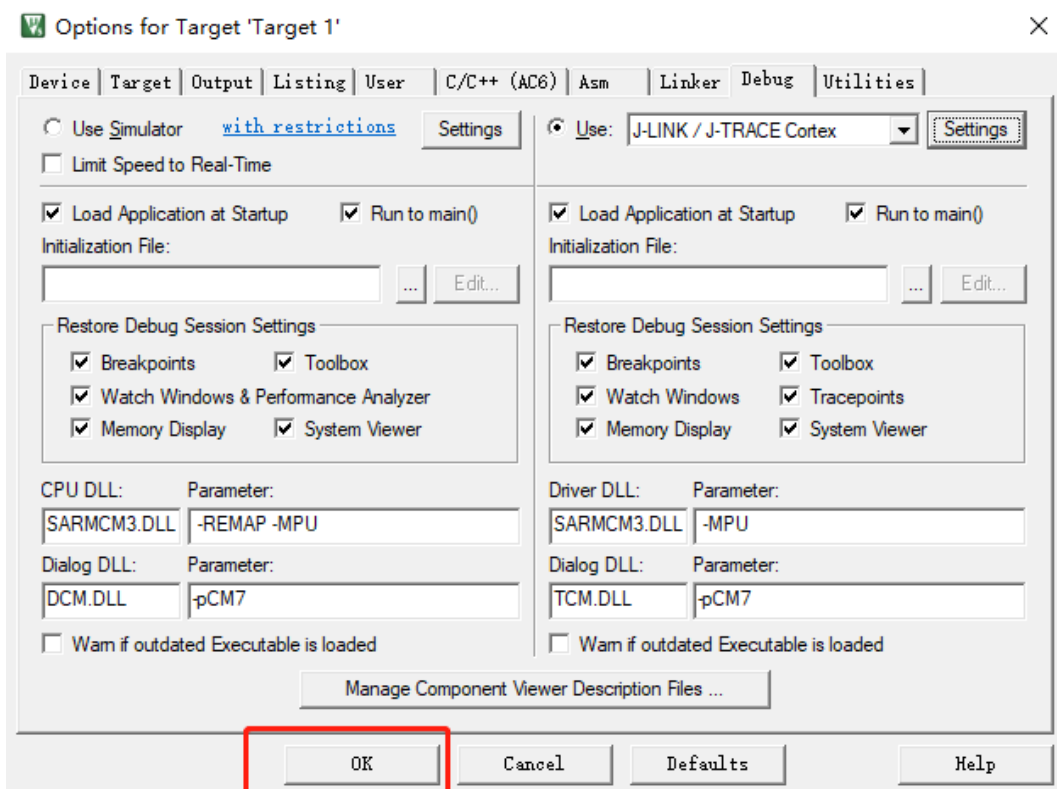
- 5) In the pop-up **Add Flash Programming Algorithm** dialog box, select "FC7300 8MB Flash" and click **Add**.



- 6) After completing the adding, FC7300-related information will be displayed in the **Programming Algorithm** box, and click 确定.



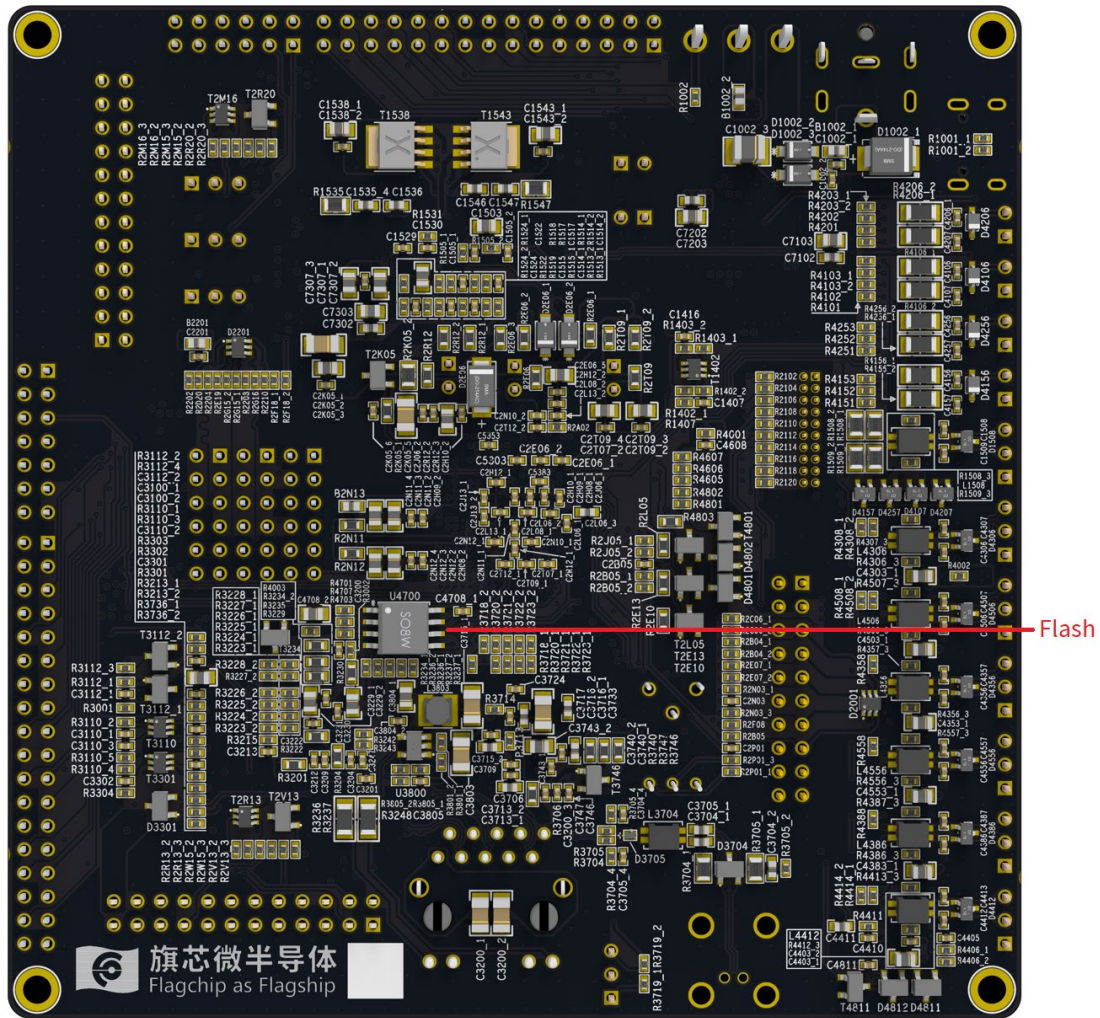
- 7) In the **Debug** tab, click **OK**, then you can start downloading or debugging.



Top view:



Bottom view:



Chapter 2 Examples and Demos

Besides the peripheral drivers described in *FC7300 SDK User Guide*, the SDK comes with a number of example applications covering the most common use cases.

This chapter describes several driver examples for different peripherals. With these examples, users can grasp basic use cases for specific drivers, and can modify the examples to create their own applications.

2.1 ADC

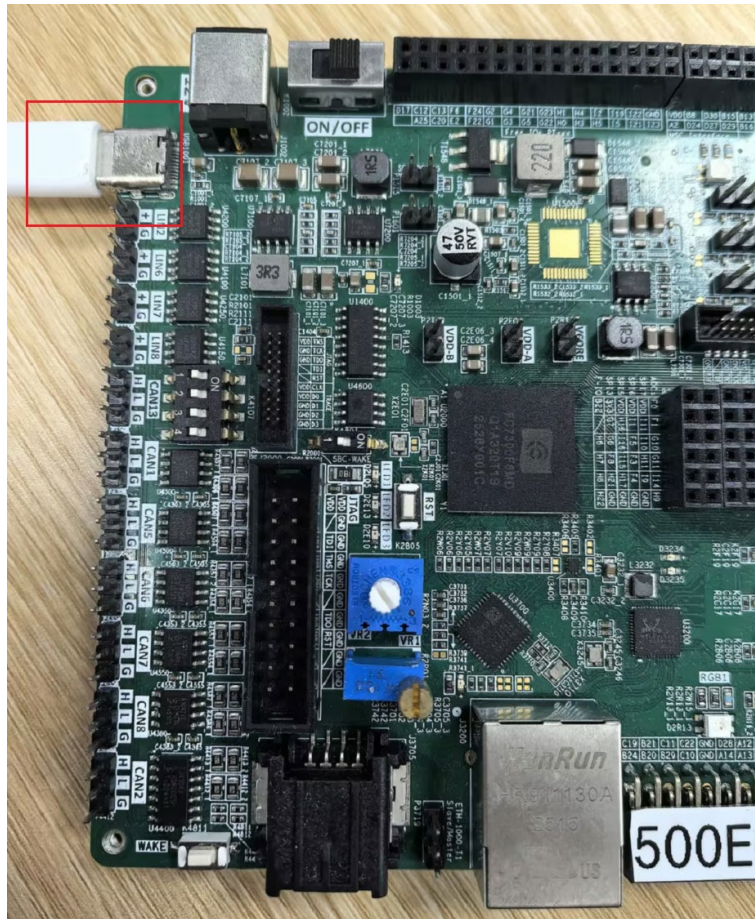
The ADC demos monitor three internal ADC signals and two external ADC channel, including:

Channel	Type	Description
ADC0_CHANNEL_VBG_BUFFER	Internal	VBG Buffer reference
ADC0_CHANNEL_V11	Internal	1.1V internal voltage
ADC0_CHANNEL_V25	Internal	2.5V internal voltage
ADC0_CHANNEL_11	External	Connected to PTA29 varistor

In the ADC demos, we use the internal voltage references and assume the voltage is 5.0V.



You may want to watch ADC result by UART, then you can connect a type-c USB cable to the USB-to-UART interface on demo board.



UART data format: 115200/8/N/1.

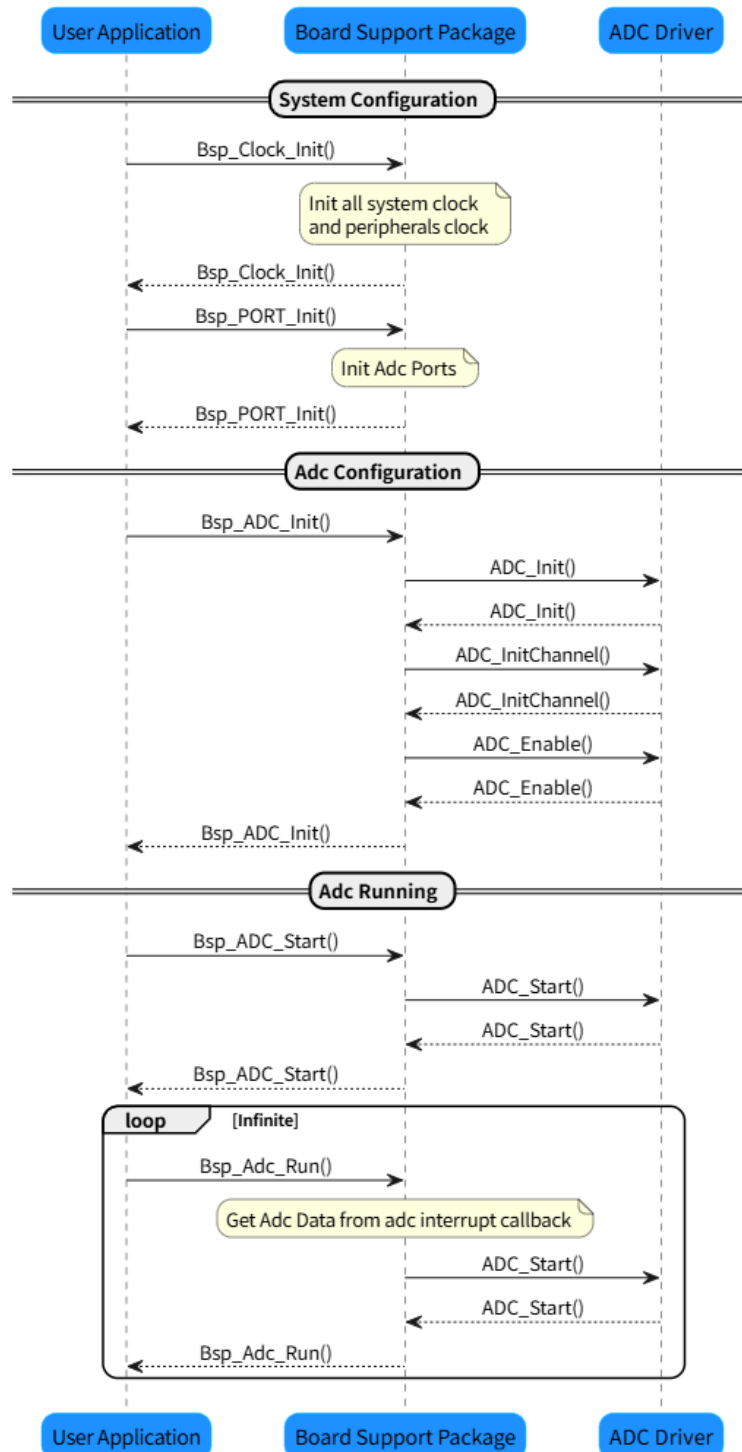
There are seven ADC demos:

2.1.1 ADC_Single

2.1.1.1 Description

This demo shows how to use the ADC single sequence with software trigger and interrupt.

The sequence diagram of the demo is as follows:



- 1) First, initialize the system and peripheral clock, and then the ADC ports.
- 2) In the ADC initialization function, specify the ADC resolution, trigger mode, and sequence mode. You can also configure the ADC sample time and average mode, etc.
- 3) After initializing the ADC instance, configure the ADC channels, in which you should specify which ADC channels you want to use.
- 4) If you want the ADC result to be transferred with an interrupt, then configure the ADC interrupt, in which you should specify the result buffer and the interrupt callback functions.
- 5) After the ADC instance is properly configured, you can enable the ADC instance.
- 6) After that, you can start the ADC instance, and the ADC conversion result will return in the interrupt callback function.

2.1.1.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 Personal computer
- 1 J-Link debugger
- 1 type-c USB cable (optional)

2.1.1.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
ADC0_SE11 (PTA29)	Varistor_0 – wired on board

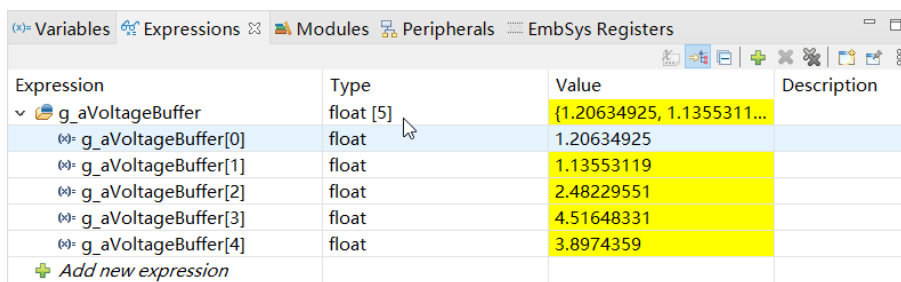
2.1.1.4 Steps to Run

2.1.1.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: Make sure "**Debug_Flash**" is selected as the active build configuration.

After building and debugging, you can see the g_aVoltageBuffer variable in the **Expressions** tab.



Expression	Type	Value	Description
g_aVoltageBuffer	float [5]	{1.20634925, 1.1355311...	
g_aVoltageBuffer[0]	float	1.20634925	
g_aVoltageBuffer[1]	float	1.13553119	
g_aVoltageBuffer[2]	float	2.48229551	
g_aVoltageBuffer[3]	float	4.51648331	
g_aVoltageBuffer[4]	float	3.8974359	
Add new expression			

2.1.1.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\ADC\ADC_Single_Project\IAR_Project, and double-click the **ADC_Single** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.1.1.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

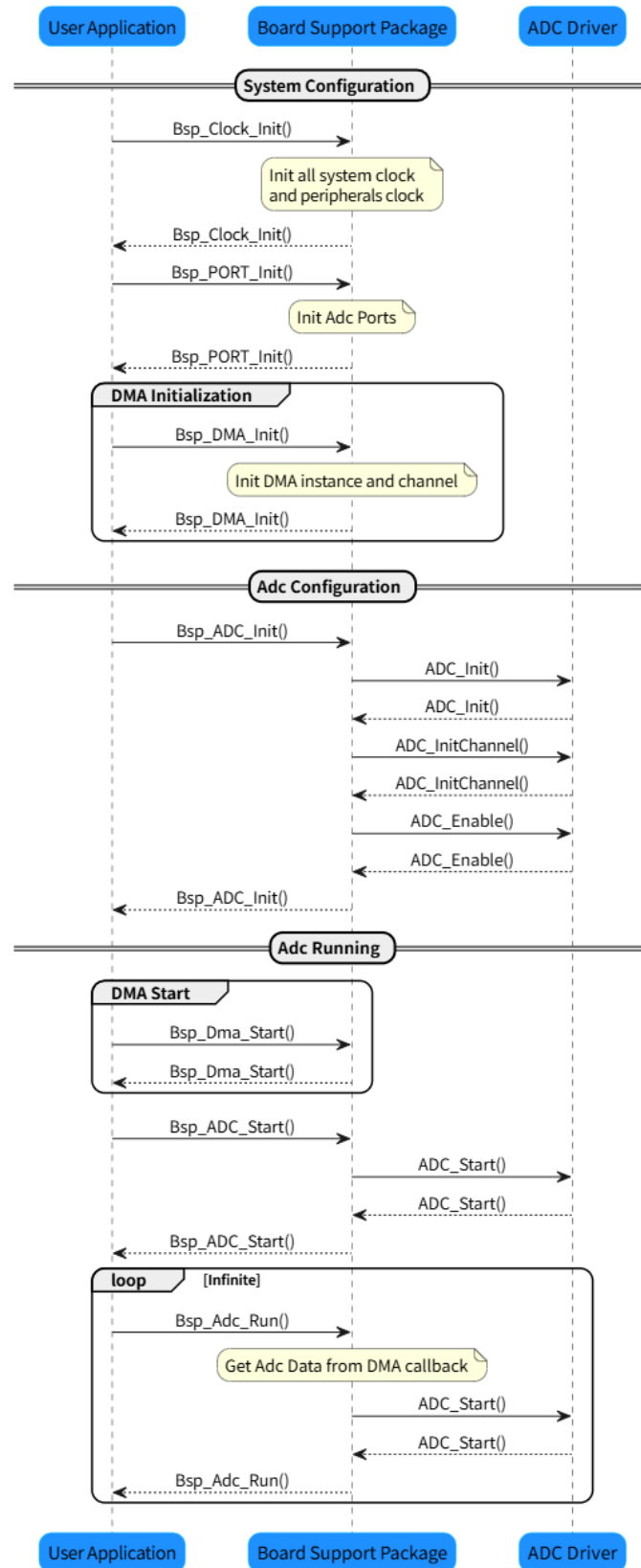
2.1.2 ADC_Single_DMA

The purpose of the demo is to show how to use ADC single sequence mode with software trigger and DMA transferring method.

The initialization and running sequence of this demo is quite likely the ADC_Single demo, except that we should initialize the DMA instance and DMA channel to transfer data.

The benefits of using DMA to transfer data is that it could offload the CPU from getting the data in the register.

The sequence diagram of the demo is as follows:



2.1.2.1 Prerequisites

Same as ADC_Single.

2.1.2.2 MCU Ports

Same as ADC_Single.

2.1.2.3 Steps to Run

Same as ADC_Single.

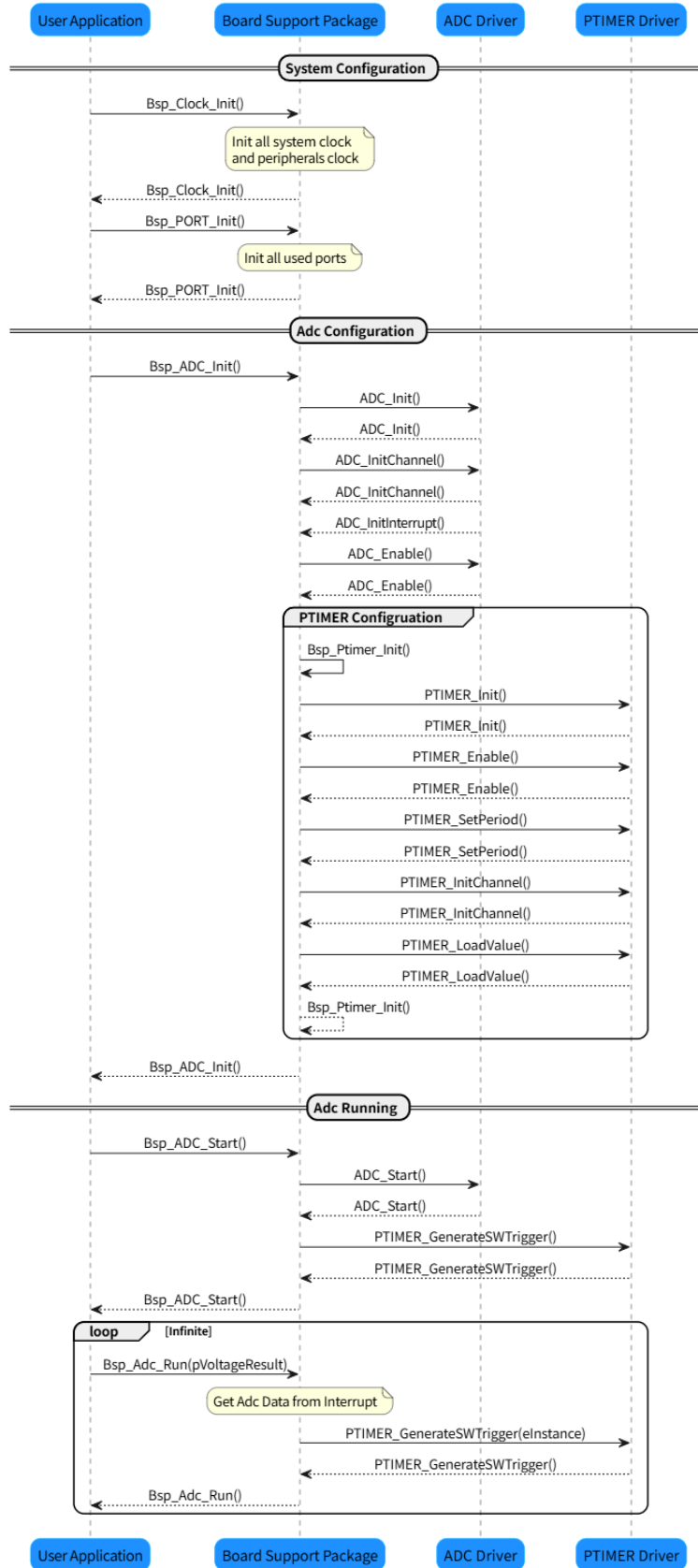
2.1.3 ADC_Discontinuous

2.1.3.1 Description

This demo shows how to use ADC discontinuous mode. In this mode, the user can use PTIMER as the pre-trigger source of the ADC module, and thus can control the time gap between different channels.

The initialization and running sequence of this demo is quite like the ADC_Single demo, except that the user should configure the PTIMER channels to trigger the ADC.

FC7300F8MDQ SDK Quick Start



2.1.3.2 Prerequisites

Same as ADC_Single.

2.1.3.3 MCU Ports

Same as ADC_Single.

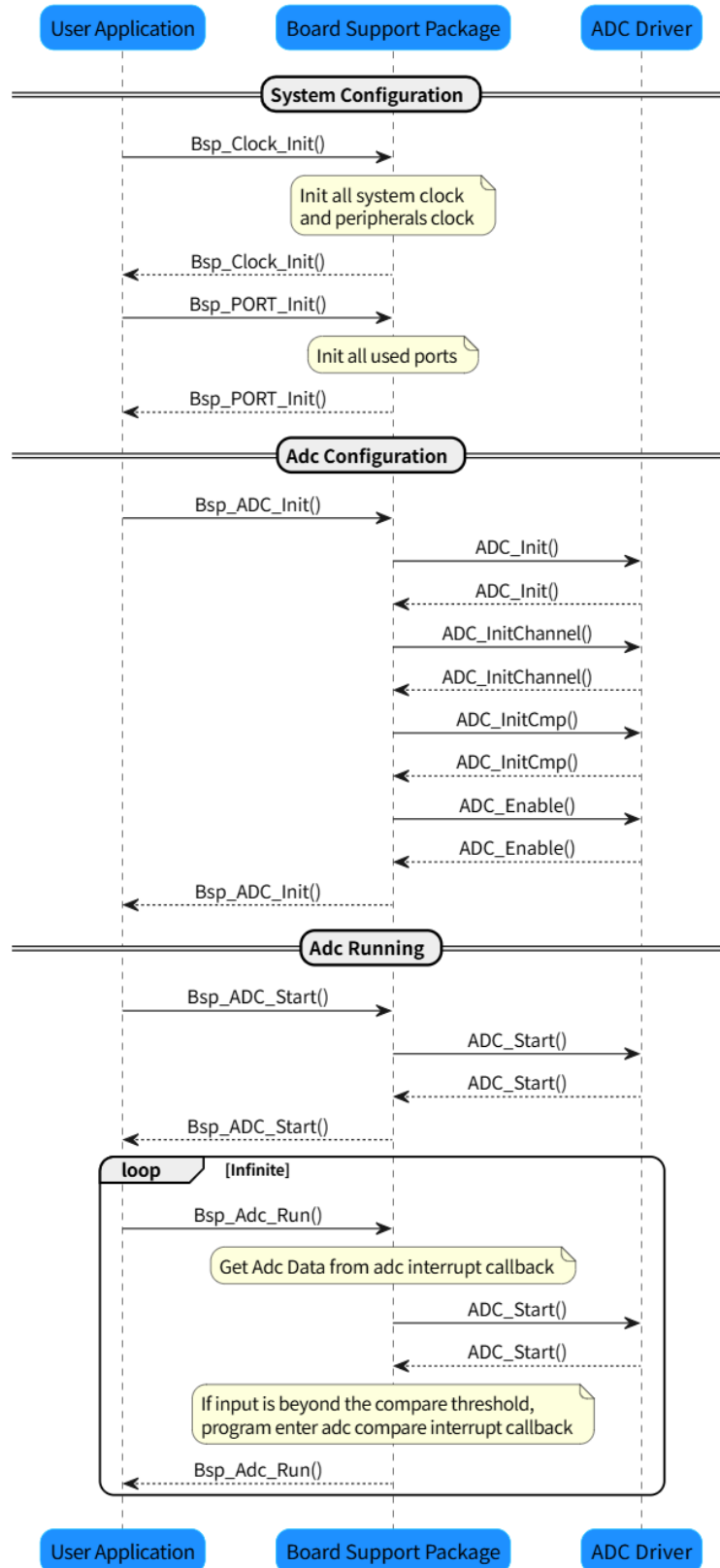
2.1.3.4 Steps to Run

Same as ADC_Single.

2.1.4 ADC_Compare

The purpose of this demo is to show you the usage of ADC compare feature mode. In this demo, you can configure the ADC module work in single mode, and thus program will enter compare callback if the input is beyond the compare threshold.

The initialization and running sequence of this demo is quite likely the ADC_Single demo.



2.1.4.1 Prerequisites

Same as ADC_Single

2.1.4.2 MCU Ports

Same as ADC_Single

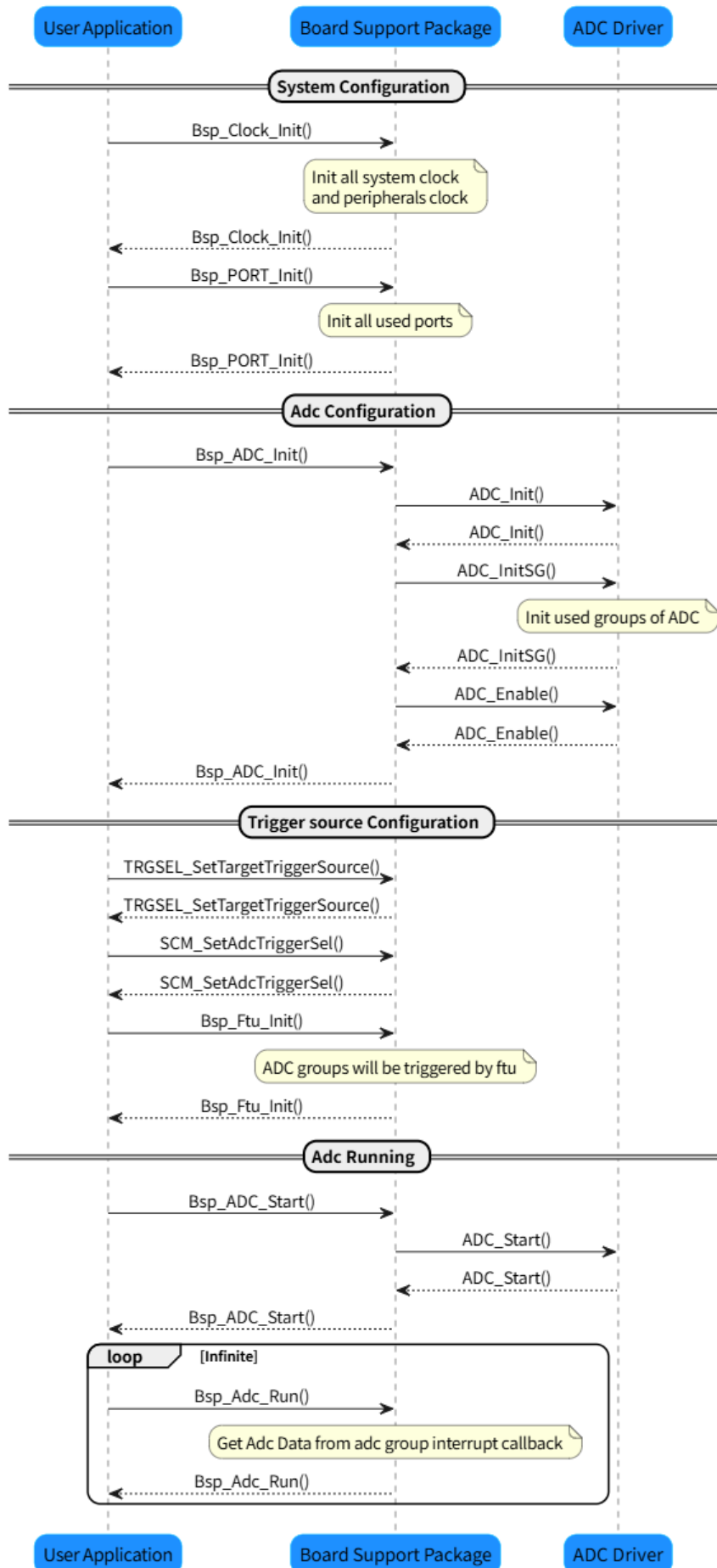
2.1.4.3 Steps to Run

The main steps are the same as ADC_Single. If you adjust the adjustable resistor on the board, the program will enter the interrupt callback function of the comparison function.

2.1.5 ADC_Sequence_Group

The purpose of this demo is to show you the usage of ADC sequence group mode. The FC7300F8MDQ chip's ADC supports a Sequence Group mode. In this mode, an ADC instance can be configured with four hardware groups, which can be individually triggered, as if they are converting in parallel.

In this demo, we initialize two groups of ADC, and they both be triggered by FTU reload trigger event.



2.1.5.1 Prerequisites

Same as ADC_Single.

2.1.5.2 MCU Ports

Same as ADC_Single.

2.1.5.3 Steps to Run

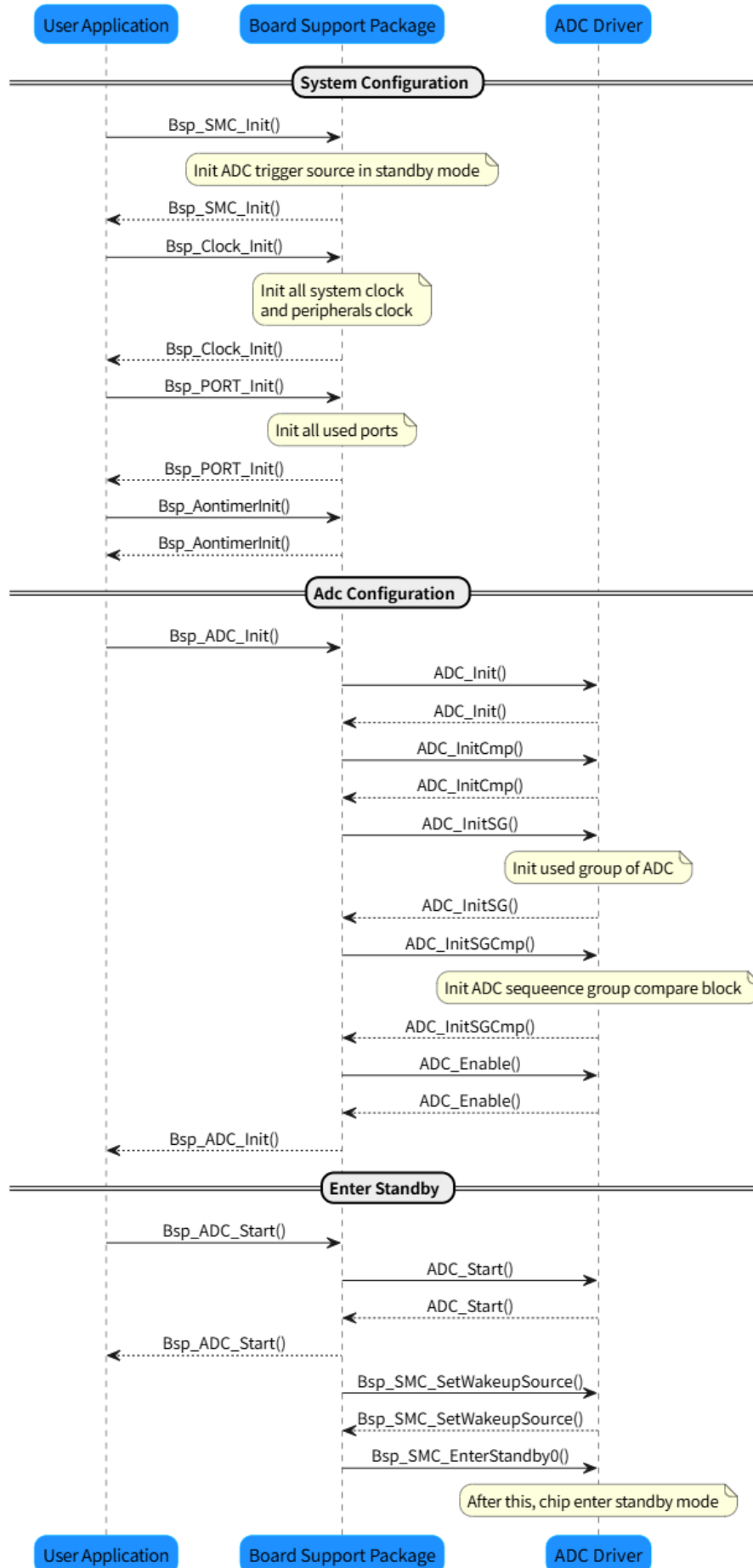
Same as ADC_Single.

2.1.6 ADC_Standby

The purpose of this demo is to show you the usage of ADC standby mode. The FC7300F8MDQ chip's ADC1 and ADC2 support standby mode.

The ADC Standby mode relies on the sequence group compare block. When the chip enters low-power mode, a periodic trigger signal can trigger the ADC to perform a conversion. The conversion result is received by the sequence group compare block. If the conversion result is beyond the threshold, the chip will be woken up from standby mode.

In this demo, we select AONTIMER as the source to wake up the ADC.



2.1.6.1 Prerequisites

Same as ADC_Single.

2.1.6.2 MCU Ports

Same as ADC_Single.

2.1.6.3 Steps to Run

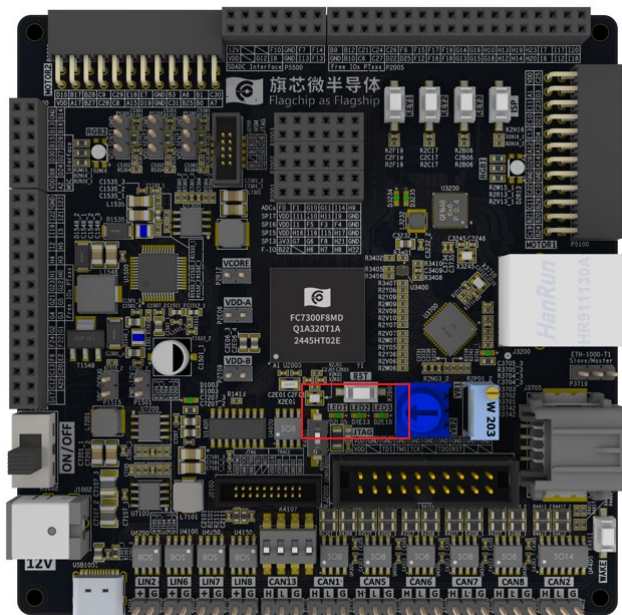
Same as ADC_Single.

2.1.7 ADC_Standby_With_Tstmp

This demo is quite like ADC_Standby, the only difference is that the ADC's wake-up source has changed from AONTIMER to TSTMP.

2.2 AONTIMER

There are two AONTIMER demos: AONTIMER_TimeCounter and AONTIMER_PulseCounter.



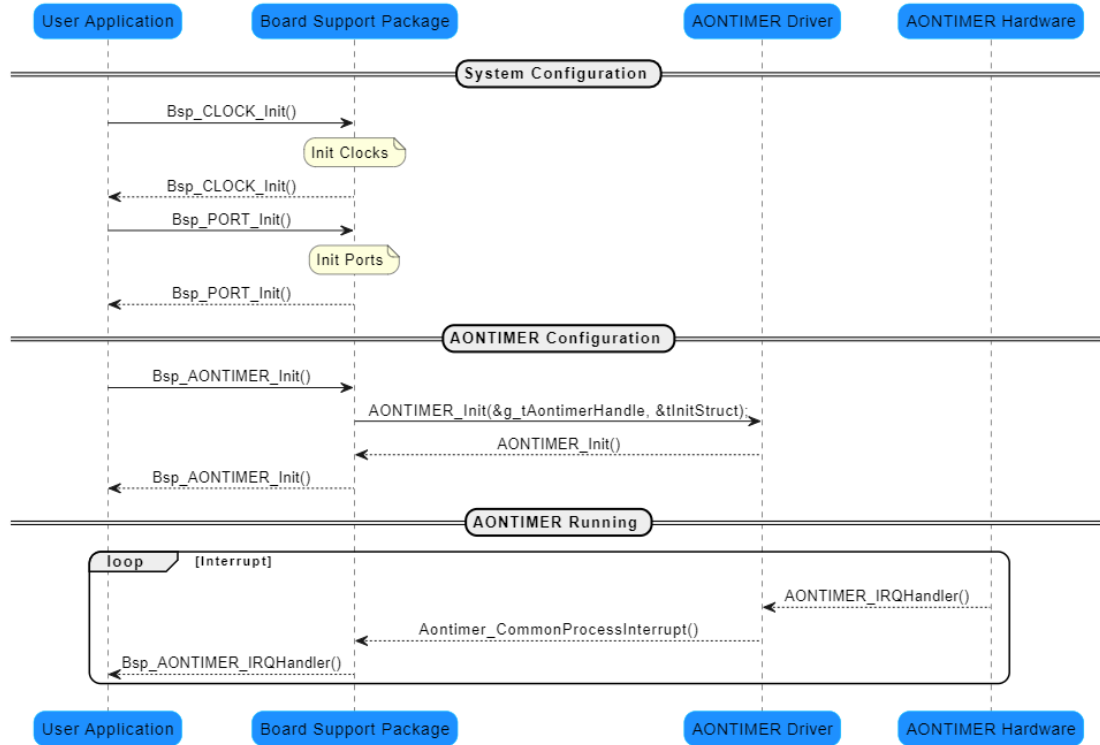
2.2.1 AONTIMER_TimeCounter

2.2.1.1 Description

This demo shows how to use the AONTIMER to generate 1-second period interrupts through the SDK.

This demo project will toggle LED every second.

The sequence diagram of the demo is as follows:



2.2.1.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V or micro-USB cable
- 1 Personal computer
- 1 J-Link debugger

2.2.1.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A26: MUX = ALT1, GPIO Output	LED1
Port D31: MUX = ALT1, GPIO Output	LED2

2.2.1.4 Steps to Run

2.2.1.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

2.2.1.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\AONTIMER\AONTIMER_TimeCounter_Project\IAR_Project, and double-click the **Aontimer_TimeCounter** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.2.1.4.3 For Keil

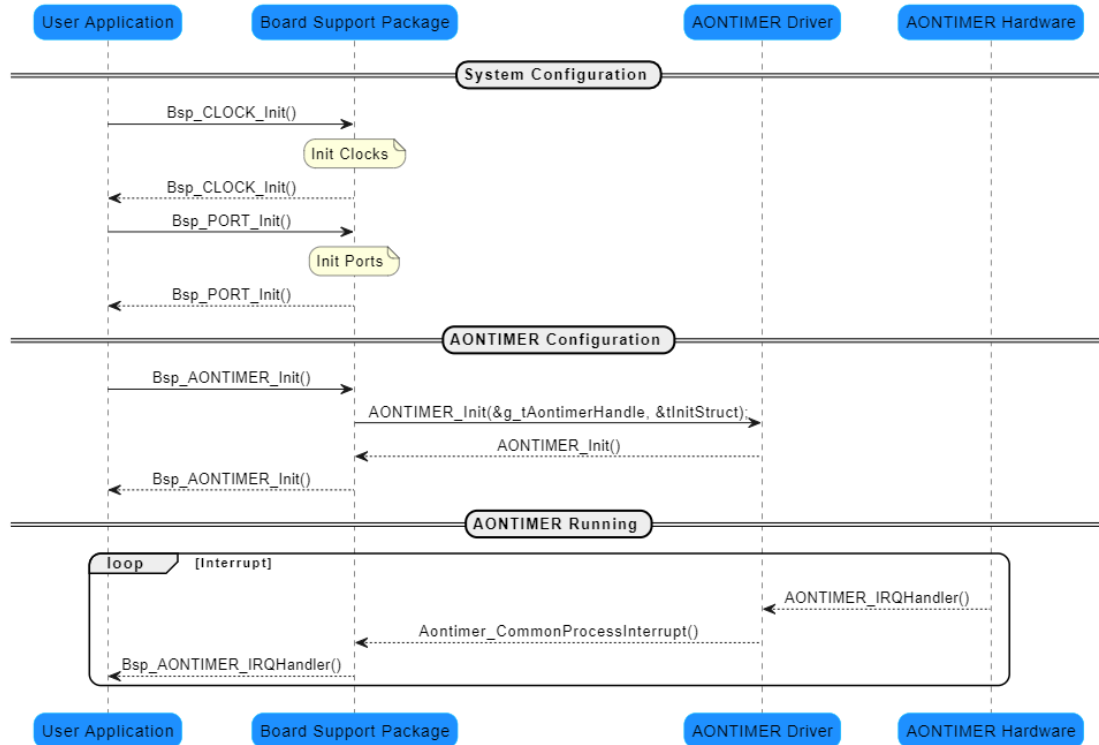
Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.2.2 AONTIMER_PulseCounter

2.2.2.1 Description

This demo shows how to use AONTIMER to generate an interrupt after capturing 10 input pulses, and LED will be toggled.

The sequence diagram of the demo is as follows:



2.2.2.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V or micro-USB cable
- 1 Personal computer
- 1 J-Link debugger

2.2.2.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A26: MUX = ALT1, GPIO Output	LED1
Port D31: MUX = ALT1, GPIO Output	LED2
Port A14: MUX = ALT3, AONTIMER0_CLK1	Use key or other connect to this pin to VCC_MCU

2.2.2.4 Steps to Run

2.2.2.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

2.2.2.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\AONTIMER\AONTIMER_PulseCounter_Project\IAR_Project, and double-click the **Aontimer_PluseCounter** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.2.2.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.3 CLK OUT

2.3.1 Description

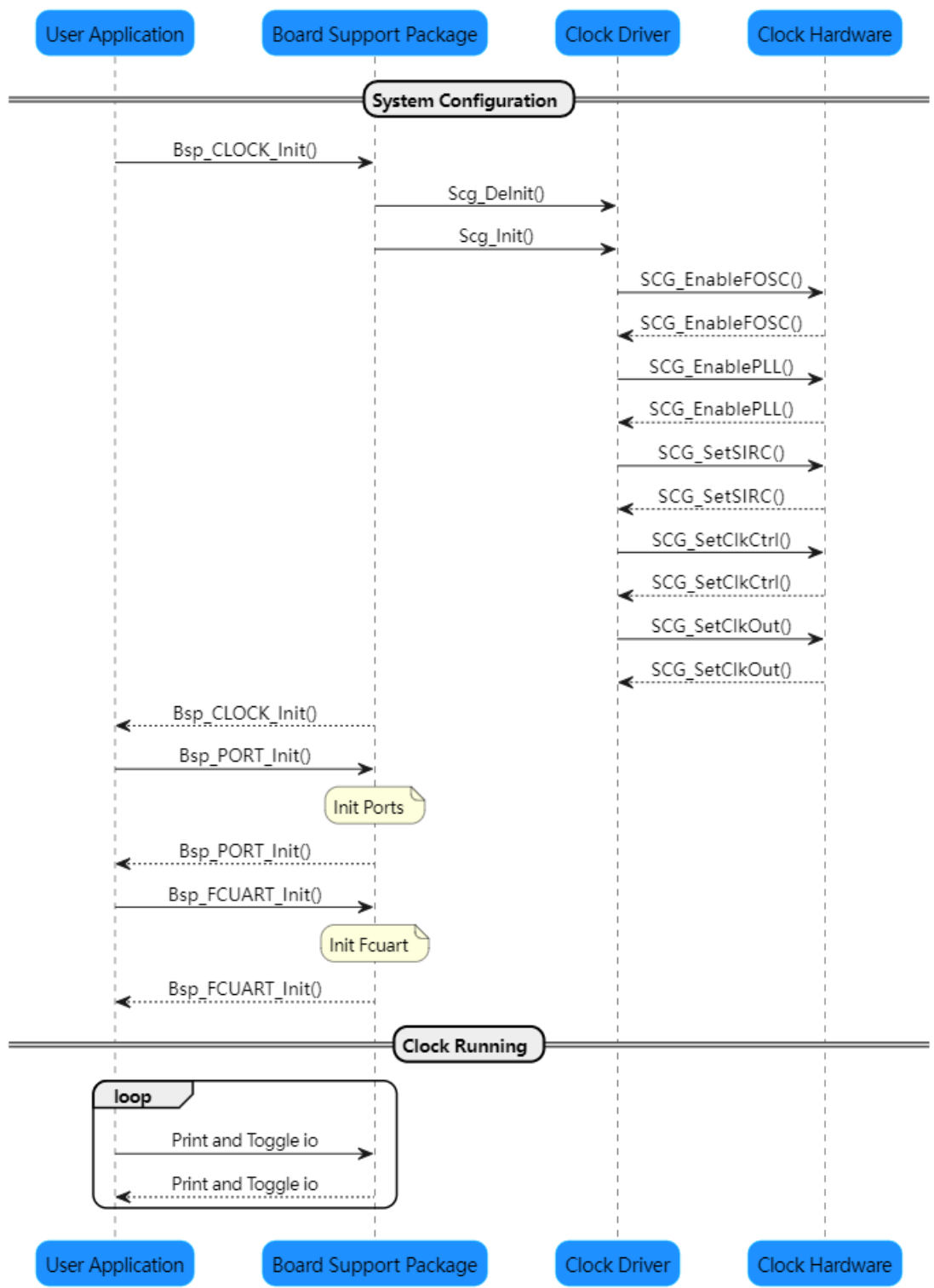
This demo shows how to set the system clock and clock out pin. The user can monitor the output clock to check if the expected clock is successfully configured.

Hardware requirements: Use UART by type-c port and connect 12V power.

This demo will toggle LED and send debug information regularly.

Note: The FOSC frequency configuration is defined by FOSC_FREQUENCY in device.h and SOSC frequency configuration is defined by SOSC_FREQUENCY in device.h.

The sequence diagram of the demo is as follows:



2.3.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger or other debug tools
- 1 Oscilloscope

2.3.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port E10: MUX = ALT2, SCG_CLOCKOUT	Clock out pin

2.3.4 Steps to Run

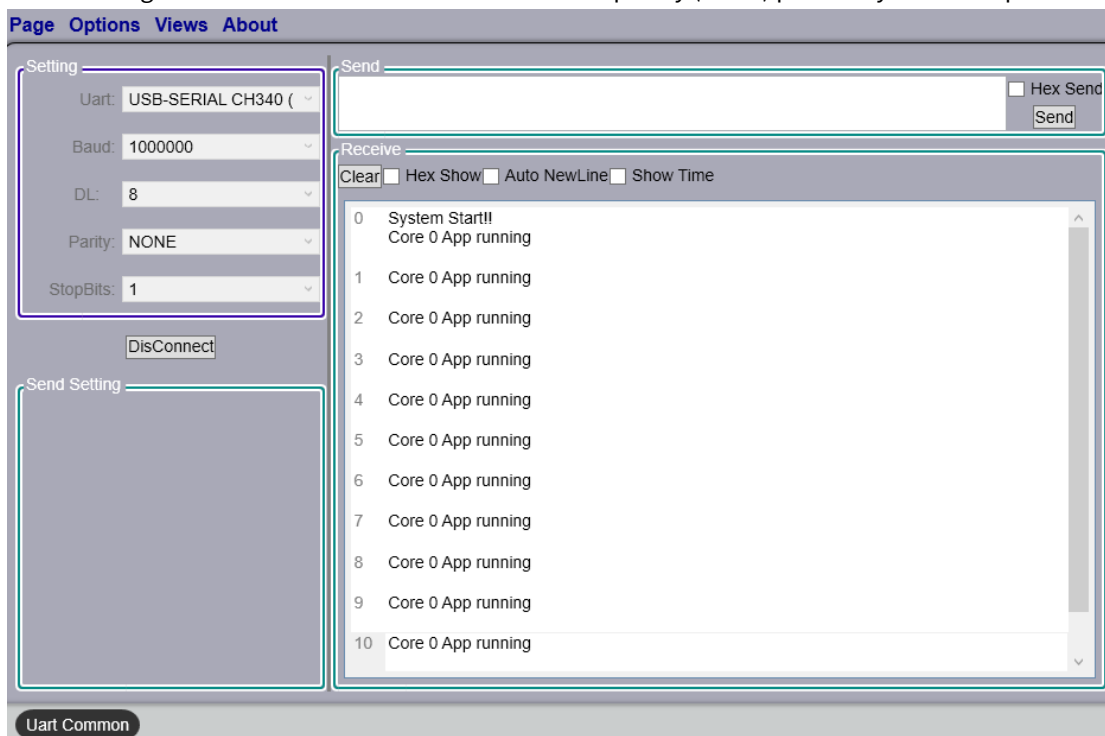
The steps to run the example are as follows:

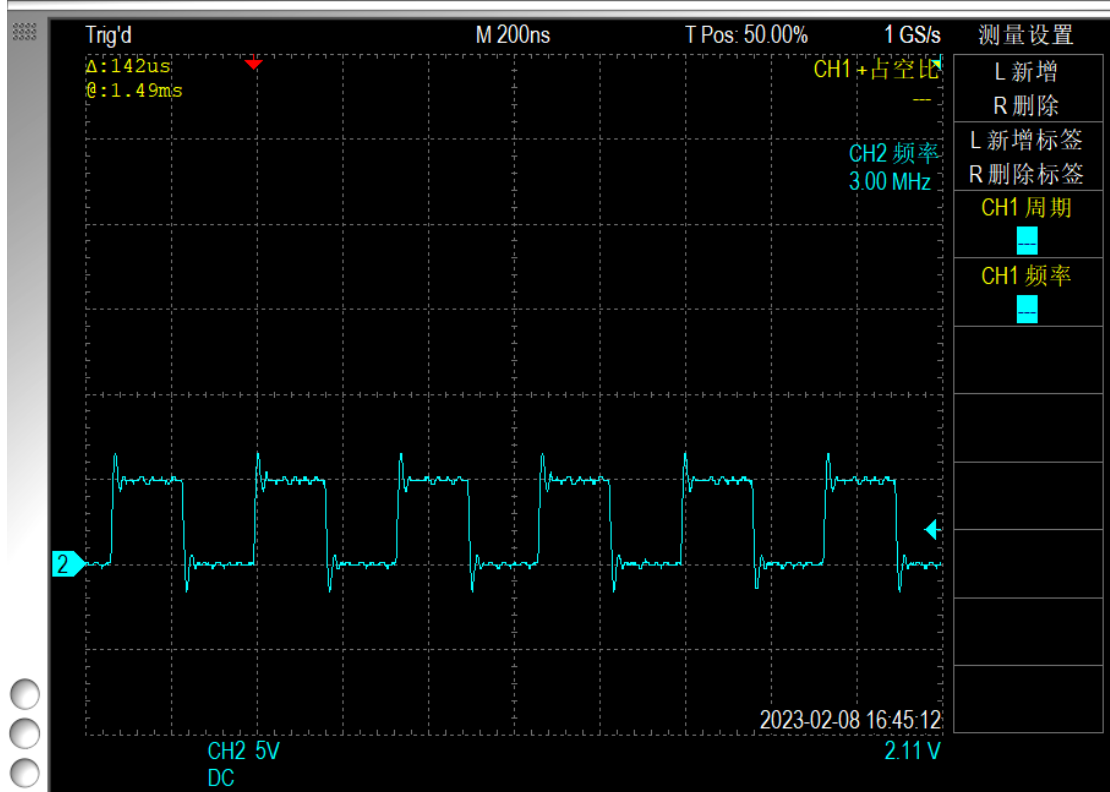
- 1) Plug in the USB Type-C cable, and connect the PC and demo board. Turn the power, and then the demo board will power on.
- 2) Download the program to the MCU by JTAG.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 3) Set baud rate to 1MHz, data bit to 8bit, stop bit to 1bit, parity bit to none for UART tool.
- 4) UART information will print regularly, and LED will toggle.
- 5) Use oscilloscope to monitor PortE10.
- 6) The UART debug information screen shot and PortE10 frequency (3MHz) picture by oscilloscope are shown below.





2.4 CMP

2.4.1 Introduction

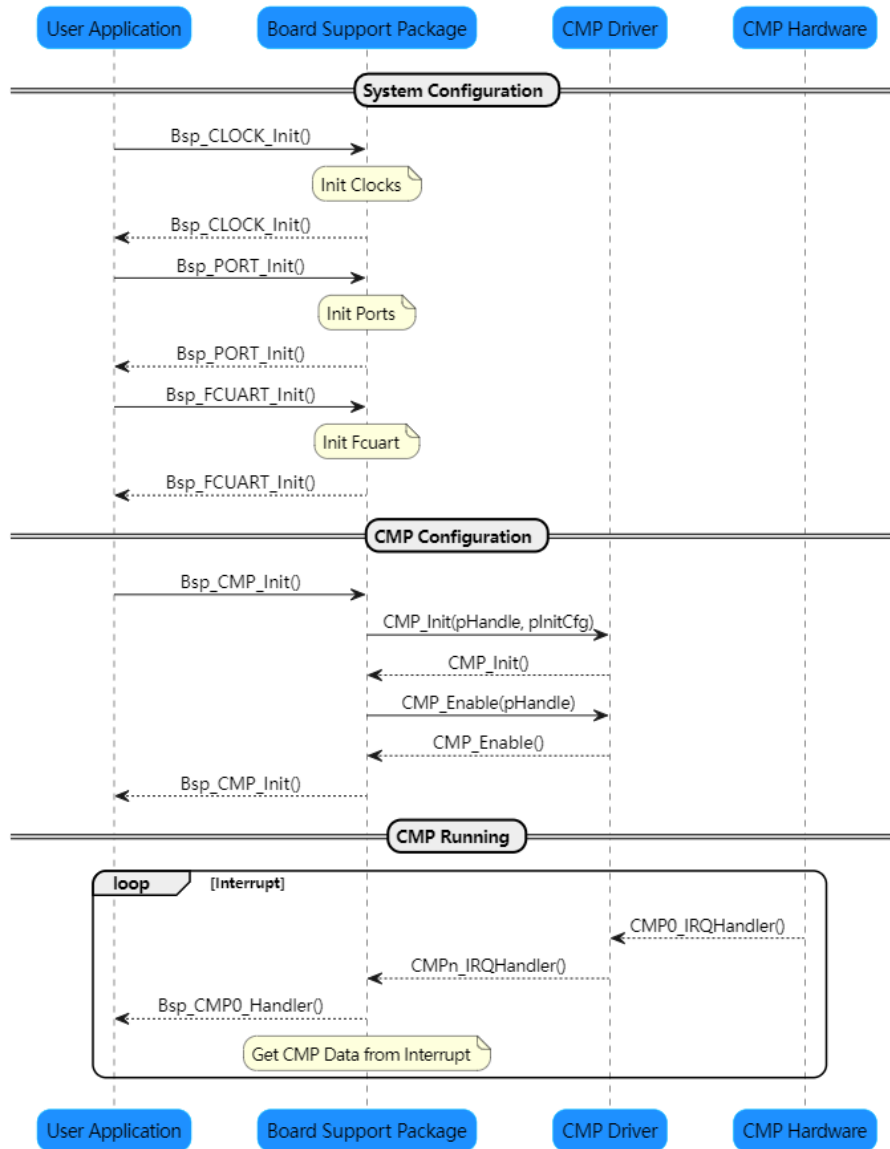
The input pins are shown in the following table:

CMP0	CMP0_IN0	PTA23
	CMP0_IN1	PTA24
	CMP0_IN2	PTG4
	CMP0_IN3	PTD17
	CMP0_IN4	PTA30
	CMP0_IN5	PTD16
	CMP0_IN6	PTD15
	CMP0_IN7	PTE9

In these demos, CMP0_IN2 (PTG4) and CMP0_IN3 (PTD17) are used as analog inputs of CMP module in FC7300F8MDQ-EVB, they will be compared to DAC output (configure in demo software) and force an interrupt or DMA transfer.

2.4.2.1 Description

The sequence diagram of the demo is as follows:



When analog voltage inputs of two channels are different, the CMP module will force an interrupt.

2.4.2.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.4.2.3 MCU Ports

N/A.

2.4.2.4 Steps to Run

2.4.2.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: Make sure "Debug_Flash" is selected as the active build configuration.

2.4.2.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\CMP\CMP_Interrupt_Project\IAR_Project, and double-click the **CMP_Interrupt** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.4.2.4.3 For Keil

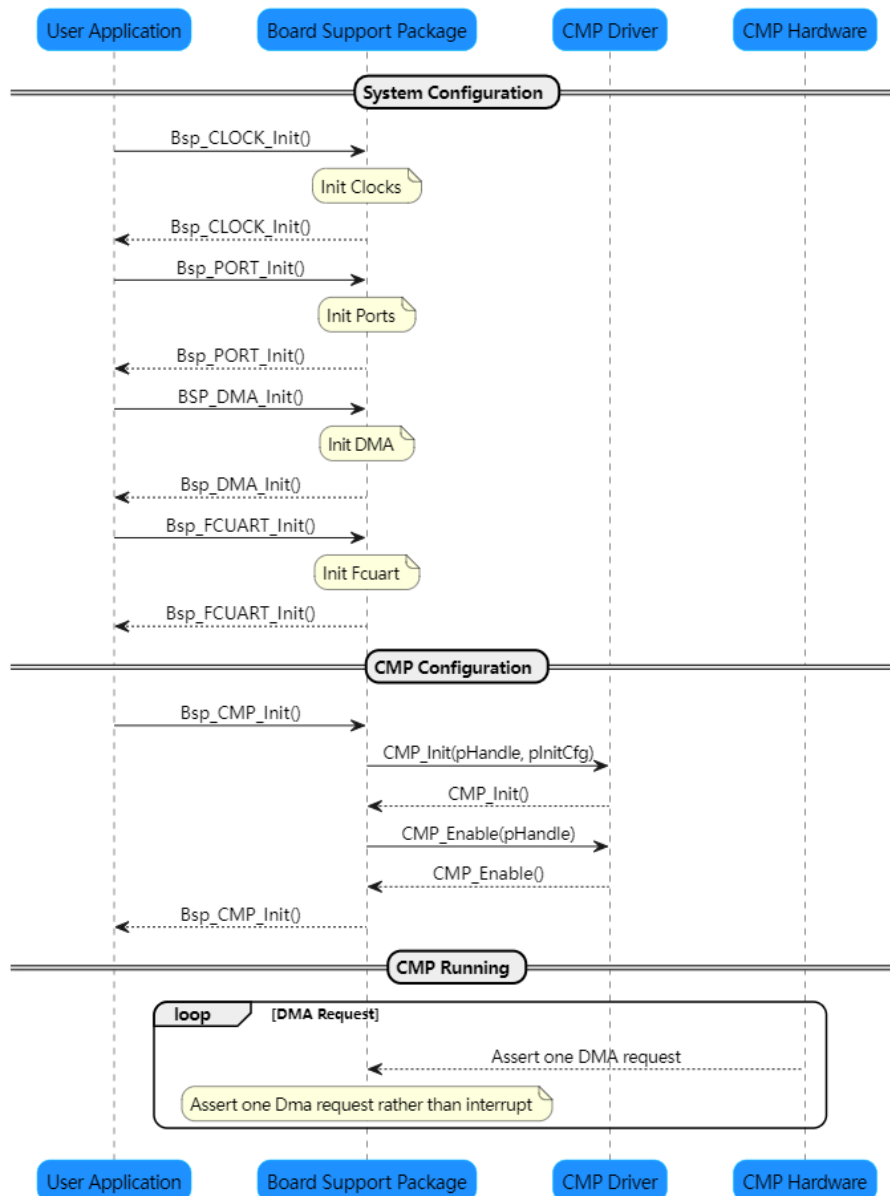
Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.4.3 CMP_DMA

2.4.3.1 Description

This demo is used in CMP module with DMA.

The sequence diagram of the demo is as follows:



When analog voltage inputs of two channels are different, the CMP module will force DMA transfer.

2.4.3.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.4.3.3 MCU Ports

N/A.

2.4.3.4 Steps to Run

2.4.3.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: Make sure "**Debug_Flash**" is selected as the active build configuration.

2.4.3.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\CMP\CMP_DMA_Project\IAR_Project, and double-click the **CMP_DMA** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.4.3.4.3 For Keil

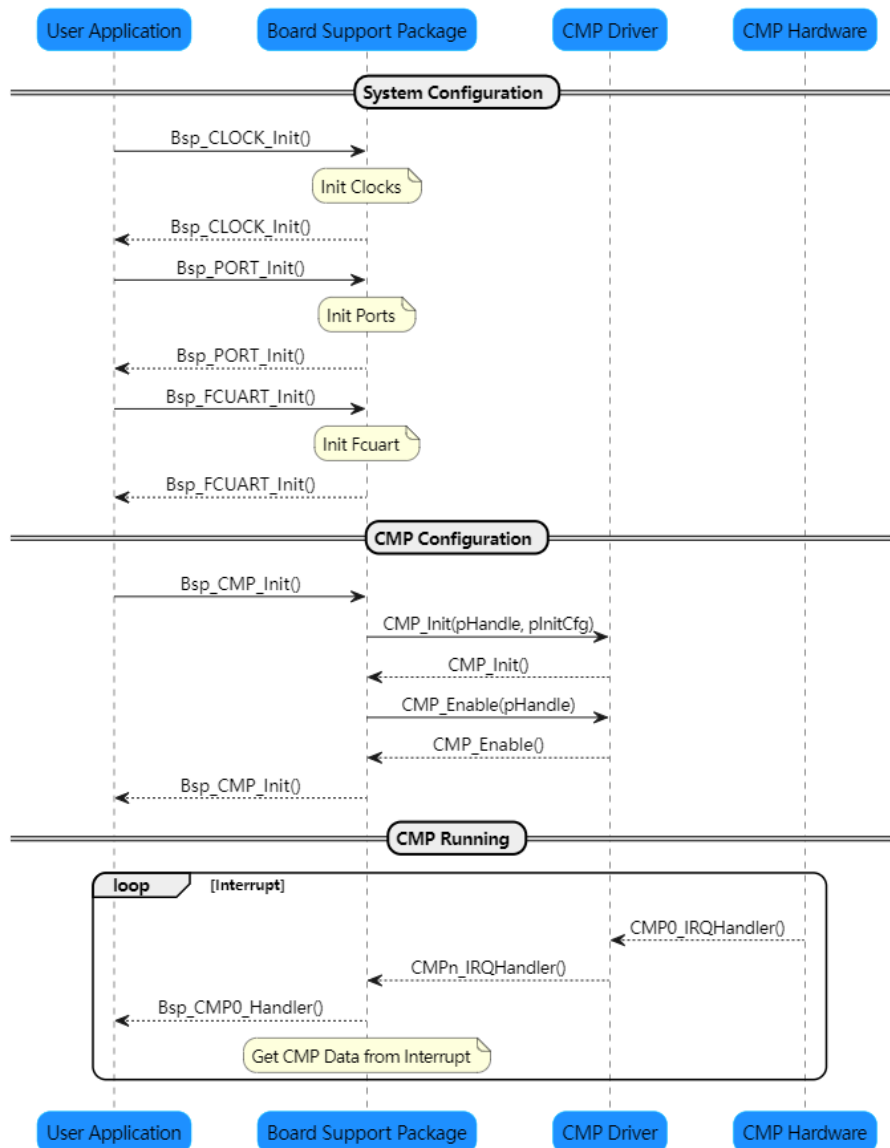
Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.4.4 CMP_ChannelScan_Interrupt

2.4.4.1 Description

The CMP module supports an additional Channel Scan mode. In Channel Scan mode, the CMP module compares all selected channels with a fixed port (any CMP input or DAC). The CMP module can assert an interrupt if any selected channel's voltage is different from the preset value.

This demo is used in CMP module with interrupt, the sequence diagram of the demo is as follows:



2.4.4.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.4.4.3 MCU Ports

N/A.

2.4.4.4 Steps to Run

2.4.4.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: Make sure "**Debug_Flash**" is selected as the active build configuration.

2.4.4.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\CMP\CMP_ChannelScan_Interrupt_Project\IAR_Project, and double-click the **CMP_ChannelScan_Interrupt** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.4.4.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.5 CMU

2.5.1 Description

This example is used for testing CMU function. In this example, we will use one 7300F8MDQ demo board. This example project will keep monitoring the clock using five CMU units.

2.5.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 USB-2-UART cable
- 1 Personal computer
- 1 J-Link debugger

2.5.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

2.5.4 Steps to Run

The steps to run the example are as follows:

- 1) Plug in the USB-2-UART cable and open UART debug terminal (UART1 115200/8/N/1) for the FC7300F8MDQ Demo Board.
- 2) Import the CMU_Demo project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 3) You will find the LED on board is blinking, and the log information can be read from the UART debug terminal.

FC7300F8MDQ SDK Quick Start

```

CMU0 SIRC->RTC min: 472 count: 489 max: 529
CMU1 SIRC->FOSC min: 1520 count: 1635 max: 1687
CMU2 SIRC->FIRC min: 4572 count: 4979 max: 5059
CMU3 FIRC->SIRC min: 543 count: 555 max: 606
CMU4 SIRC->SLOW min: 2080 count: 2192 max: 2305
CMU0 SIRC->RTC min: 472 count: 489 max: 529
CMU1 SIRC->FOSC min: 1520 count: 1635 max: 1687
CMU2 SIRC->FIRC min: 4572 count: 4979 max: 5059
CMU3 FIRC->SIRC min: 543 count: 555 max: 606
CMU4 SIRC->SLOW min: 2080 count: 2193 max: 2305
CMU0 SIRC->RTC min: 472 count: 488 max: 529
CMU1 SIRC->FOSC min: 1520 count: 1636 max: 1687
CMU2 SIRC->FIRC min: 4572 count: 4980 max: 5059
CMU3 FIRC->SIRC min: 543 count: 555 max: 606
CMU4 SIRC->SLOW min: 2080 count: 2193 max: 2305
CMU0 SIRC->RTC min: 472 count: 489 max: 529
CMU1 SIRC->FOSC min: 1520 count: 1635 max: 1687
CMU2 SIRC->FIRC min: 4572 count: 4981 max: 5059
CMU3 FIRC->SIRC min: 543 count: 555 max: 606
CMU4 SIRC->SLOW min: 2080 count: 2193 max: 2305

```

If the count value is between min and max, it means the clock is working normally.

2.6 CORDIC

2.6.1 Description

This demo is just for CORDIC drive test. CORDIC is a simplified algorithm module, which transforms many complex operations into an iterative operation of only shift and addition. It can accelerate some operations several hundred times.

2.6.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 USB-2-UART cable
- 1 Personal computer
- 1 J-Link debugger

2.6.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

For other ports, reference to the *FC7300F8MDQ Pinout* attached to *FC7300F8MDQ Reference Manual*.

2.6.4 Steps to Run

2.6.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: Make sure "**Debug_Flash**" is selected as the active build configuration.

After building and debugging the project, take the following steps:

- 1) Plug in the USB-2-UART cable and open the up-machine (UART1 115200/8/N/1).
- 2) Run the project, the up-machine will display as follows.

```
[13:45:17.602]收←◆System Start!!
cordic test start
sin45 angle = 0.7070, cos45 angle = 0.7071
Radical(4.2^2 + 6.4^2) = 7.6550
Radical(6.2^2 - 3.4^2) = 5.1845
arctan50.2 = 88.8584 angle
arctan96.5 = 89.4065 angle
arctan100.6 = 89.4068 angle
arctan1839/113 = 86.4666 angle
arctan1850/-124 = -86.1763 angle
sinh0.8 = 0.8880, cosh0.8 = 1.3374
ln2 = 0.6931
Sqrt2 = 1.4142
cordic test end
```

2.6.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\Cordic\Cordic_Demo_Project\IAR_Project, and double-click the **Cordic_Demo** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

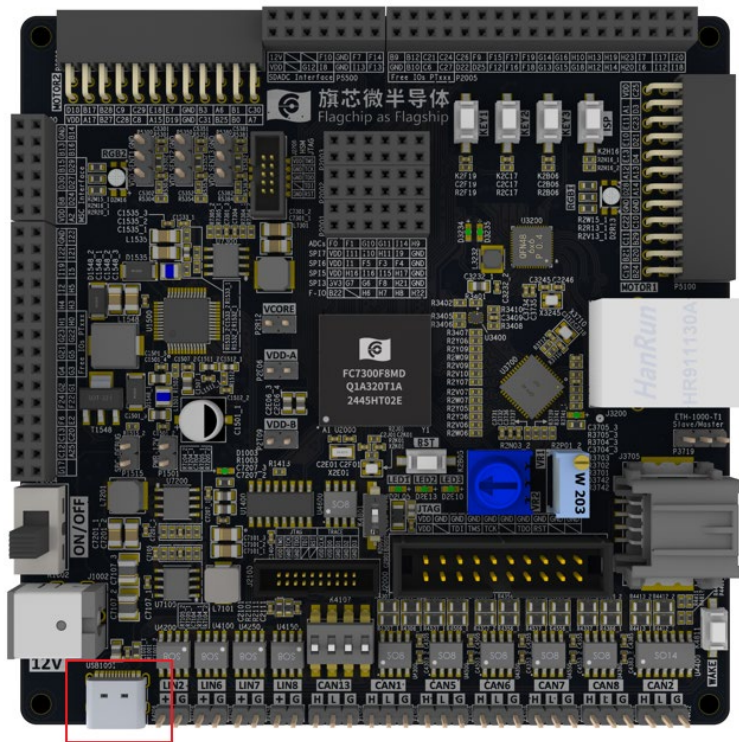
2.6.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.7 CPM

2.7.1 Description

This demo shows how to use the CPM to monitor the FPU-generated interrupt error and how to clear the FPU Interrupt Status.



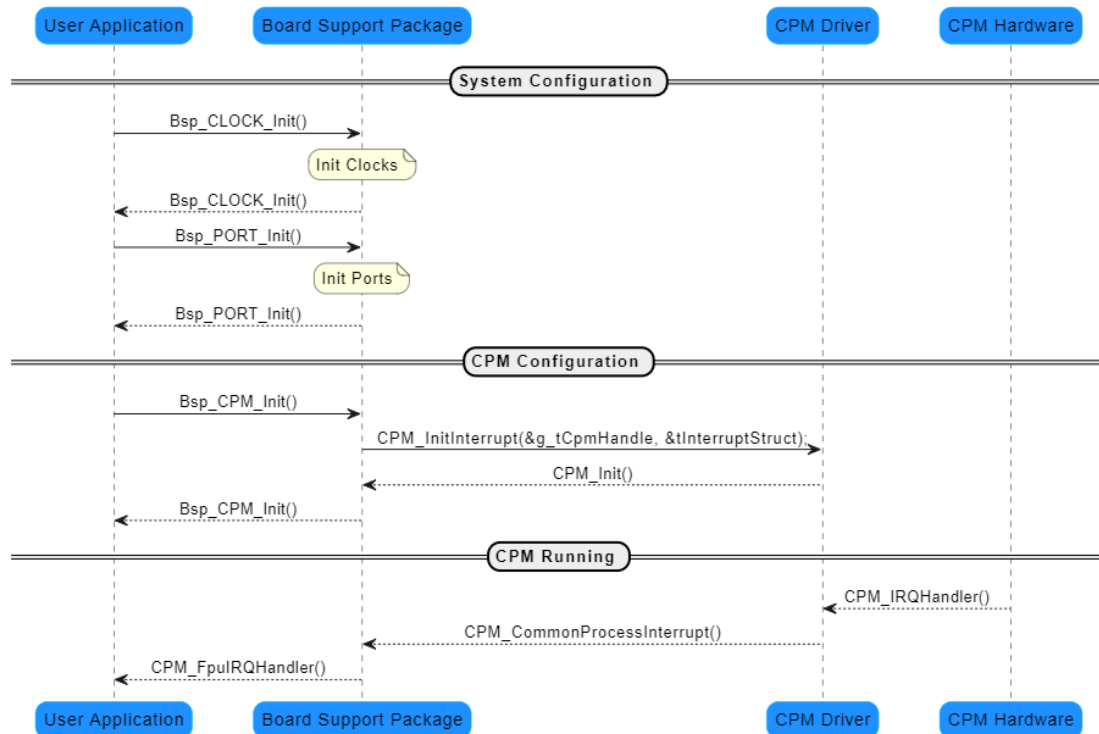
Note:

CPM_UserIRQHandler() is a callback function, and is called when a CPM interrupt occurs.

The demo tells the user how to check whether the interrupt is caused by the FPU or others when a CPM interrupt occurs. If the interrupt is caused by the FPU, the user needs to set a flag and disables the CPM interrupt.

The user cannot read FPSCR in ISR. In ISR, the value is read from FPDSCR. The user only can set the flag in the CPM interrupt function and treat it after out of ISR.

The sequence diagram of the demo is as follows:



2.7.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

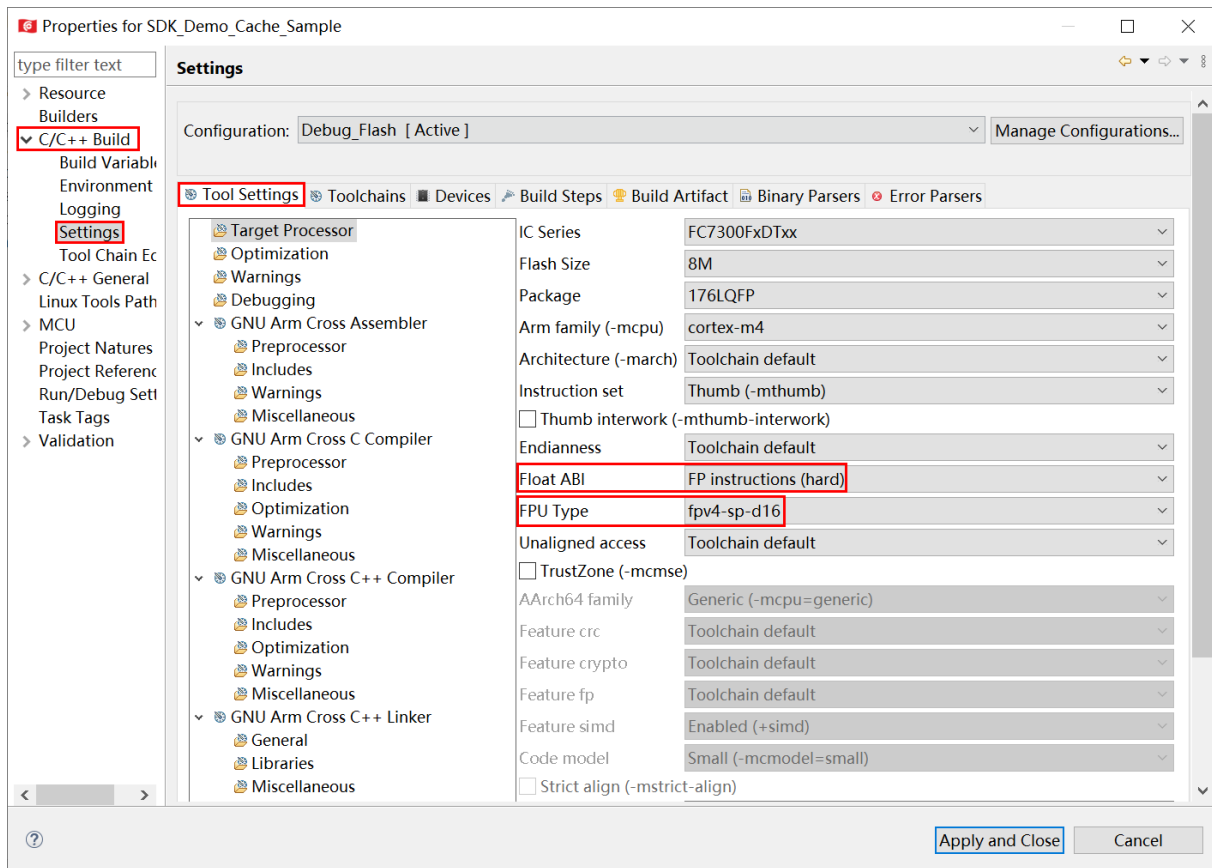
2.7.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

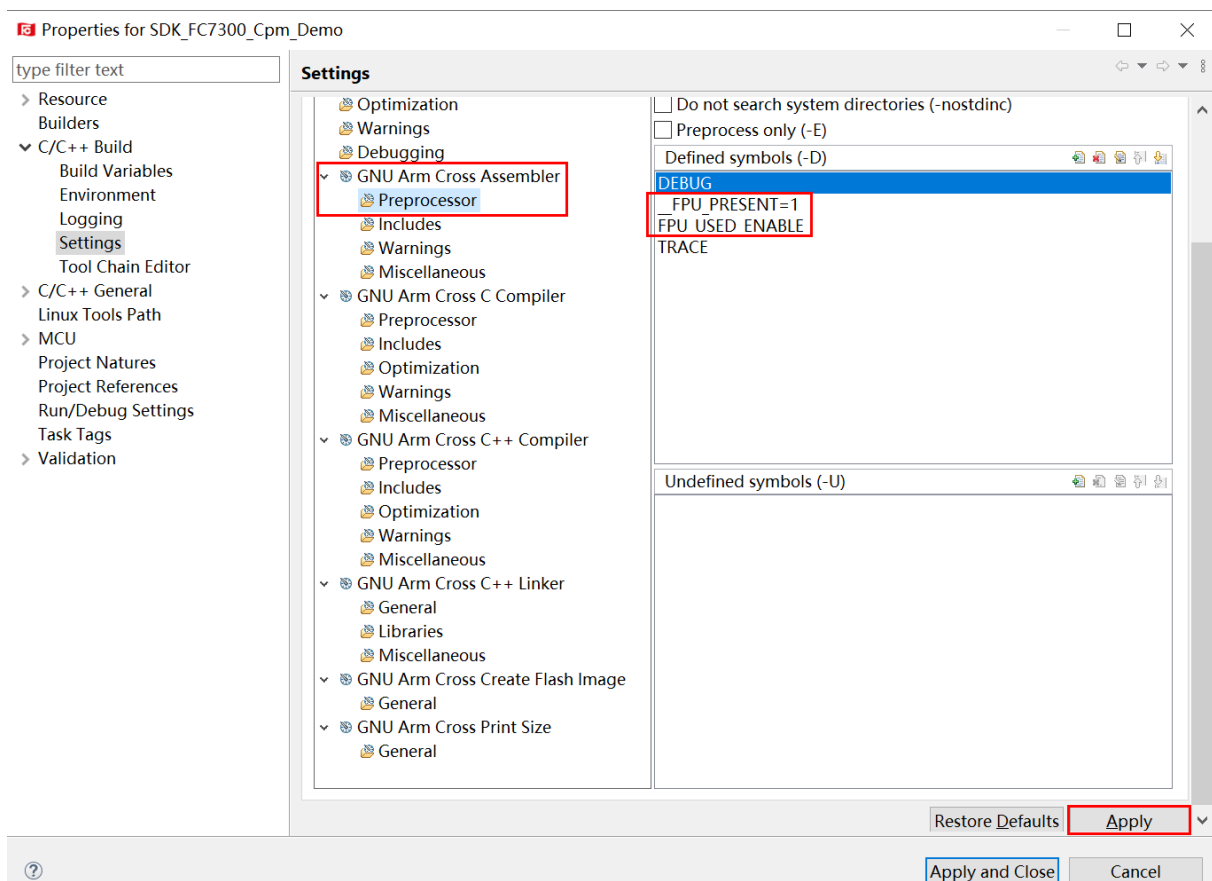
2.7.4 Steps to Run

To monitor an FPU error, the user needs to configure the FC_IDE as follows before running the example.

- 1) Right-click on the project name, and select **Properties** from the pop-up menu.
- 2) In the pop-up dialog, select **C/C++ Build > Settings > Tool Settings > Target Processor**, set **Float ABI (FP instructions (hard))** and **FPU Type (fpv5-sp-d16)**, and click **Apply**.



- 3) Select **GNU Arm Cross Assembler > Preprocessor**, add **__FPU_PRESENT=1** and **FPU_USED_ENABLE**, and then click **Apply**.



- 4) Click the **Apply and Close** button.

- 5) Build the project and download it.

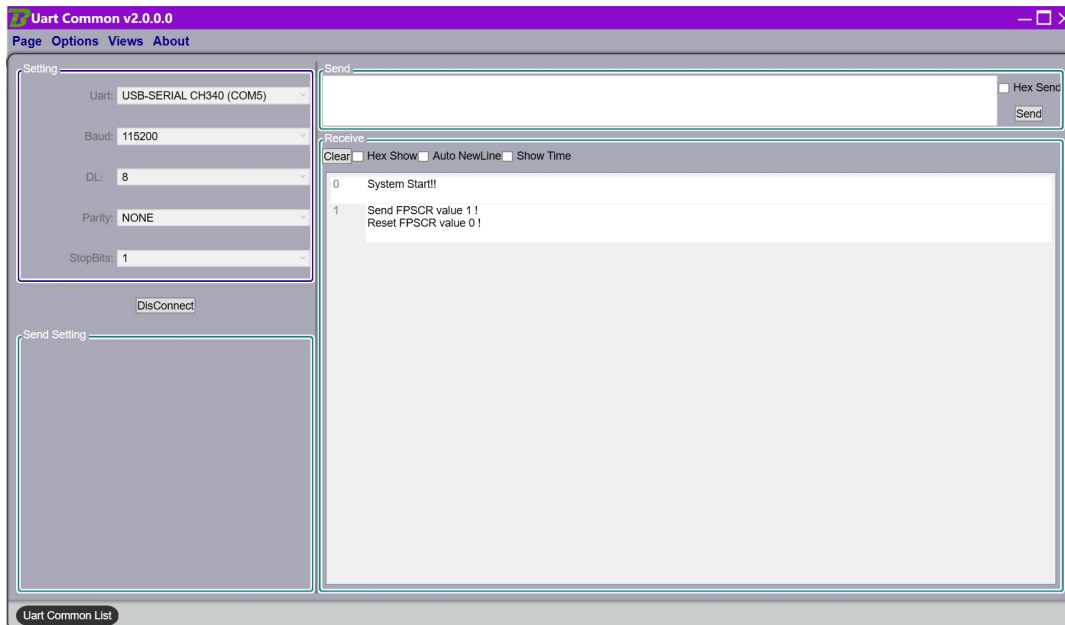
The steps to run the example are as follows:

- 1) Plug in the USB Type-C cable and open the up-machine (UART2 115200/8/N/1).
- 2) Import the **Cpm_Example** project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 3) When the MCU detects an FPU error, it will send the message below.



2.8 CRC

The CRC module generates 16/32-bit CRC code for error detection that may occur after data transmission or storage. The user can select 16-bit or 32-bit code to be calculated for 32 bits of data at a time. This demo provides four kinds CRC16: CCITT, CCITT-False, ModBus, and USB.

The user can compute the 16-bit or 32-bit CRC by performing the following steps:

- 1) Set CRC_CR[TCRC] to enable 16-bit/32-bit CRC mode. (**Note:** this step must be configured firstly, otherwise the high 16-bit register cannot be set in 32-bit CRC mode.)
- 2) Program the data swap and complement options in the CTRL register as required for the CRC calculation.
- 3) Write a polynomial to the CRC_POLY field, for 16-bit CRC mode, the user should write a 16-bit polynomial and the CRC[HIGH] is unable.
- 4) Set CRC_CR[WAS] to program the seed value.
- 5) Write seed data to CRC_DATA, for 16-bit CRC mode, CRC_DATA[HU:HL] are not used.
- 6) Clear the CRC_CR[WAS] to start writing data values.
- 7) Write data values into CRC_DATA[HU:HL:LU:LL]. A CRC is computed on every data value write, and the intermediate CRC result is stored back into CRC_DATA, for 16-bit CRC mode, the CRC results will only be stored into CRC_DATA[LU:LL].

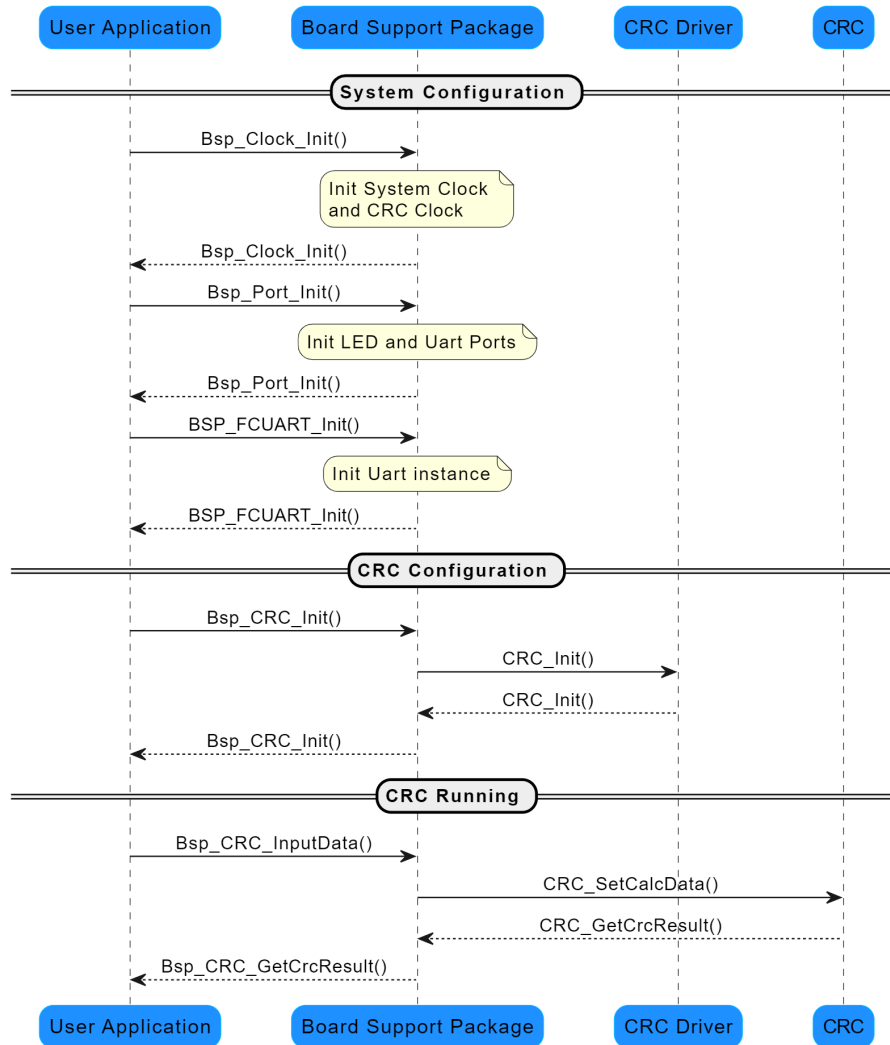
Note: If byte swap is configured, the CRC result should be read in CRC_DATA[HU:HL].

2.8.1 CRC_Demo

2.8.1.1 Description

This demo is used in the CRC module with test data.

The sequence diagram of the demo is as follows:



When the input data is written successfully, the CRC result can be read from DATA register immediately. After CRC calculation, the seed value must be reset before the next CRC calculation.

2.8.1.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.8.1.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

2.8.1.4 Steps to Run

2.8.1.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: Make sure "**Debug_Flash**" is selected as the active build configuration.

After building and debugging the project, the results will be shown as below:

- The result of CRC_16_USB:

```

Rx: System Start!!
CRC reference result:      20DD
CRC hard calculation result: 20DD
CRC hard calculate right!
Running tick:0!

Rx: Running tick:1!

Rx: Running tick:2!

Rx: Running tick:3!

Rx: Running tick:4!

Rx: Running tick:5!

Rx: Running tick:6!

Rx: Running tick:7!

Rx: Running tick:8!

Rx: Running tick:9!

Rx: Running tick:10!

```

- The result of CRC_16_CCITT:

```

Rx: System Start!!
CRC reference result:      4FC0
CRC hard calculation result: 4FC0
CRC hard calculate right!
Running tick:0!

Rx: Running tick:1!

Rx: Running tick:2!

Rx: Running tick:3!

Rx: Running tick:4!

Rx: Running tick:5!

Rx: Running tick:6!

Rx: Running tick:7!

Rx: Running tick:8!

Rx: Running tick:9!

Rx: Running tick:10!

```

- The result of CRC_16_CCITT_FALSE:

```

Rx: System Start!!
CRC reference result:      F9B9
CRC hard calculation result: F9B9
CRC hard calculate right!

Rx: Running tick:0!

Rx: Running tick:1!

Rx: Running tick:2!

Rx: Running tick:3!

Rx: Running tick:4!

Rx: Running tick:5!

Rx: Running tick:6!

Rx: Running tick:7!

Rx: Running tick:8!

Rx: Running tick:9!

Rx: Running tick:10!

```

- The result of CRC_16_MODBUS:

```

Rx: System Start!!
CRC reference result:      DF22
CRC hard calculation result: DF22
CRC hard calculate right!
R
Rx: unning tick:0!

Rx: Running tick:1!

Rx: Running tick:2!

Rx: Running tick:3!

Rx: Running tick:4!

Rx: Running tick:5!

Rx: Running tick:6!

Rx: Running tick:7!

Rx: Running tick:8!

Rx: Running tick:9!

Rx: Running tick:10!

```

2.8.1.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\CRC\CRC_Demo_Project\IAR_Project, and double-click the **SDK_FC7300_Demo_CRC** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.8.1.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.8.2 CRC_Demo_Hard_Soft_Compare

2.8.2.1 Description

This demo shows how to use software and hardware CRC to calculate CRC8, CRC16 and CRC32. Each CRC protocol also provides two calculation methods: blocking and non-blocking. Blocking means that the data is calculated once to get the final result, and non-blocking can calculate the data separate multiple times. The final CRC results of two methods are the same.

2.8.2.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.8.2.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A26: MUX = ALT1, GPIO Output	LED1
Port D31: MUX = ALT1, GPIO Output	LED2
Port A14: MUX = ALT1, GPIO Output	LED3

2.8.2.4 Steps to Run

2.8.2.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: Make sure "**Debug_Flash**" is selected as the active build configuration.

After building and debugging the project, the result will be shown as below:

```

System Start!
Total Data Length: 6192!
Crc8ROMC block software:          1
Crc8ROMC block hardware:          1
Crc8ROMC non-block software:      1
Crc8ROMC non-block hardware:      1
Crc16CCITT block software:        32C0
Crc16CCITT block hardware:        32C0
Crc16CCITT non-block software:    32C0
Crc16CCITT non-block hardware:    32C0
Crc16CCITT_FALSE block software:  F9D3
Crc16CCITT_FALSE block hardware:  F9D3
Crc16CCITT_FALSE non-block software: F9D3
Crc16CCITT_FALSE non-block hardware: F9D3
Crc16MODBUS block software:       B7B9
Crc16MODBUS block hardware:       B7B9
Crc16MODBUS non-block software:   B7B9
Crc16MODBUS non-block hardware:   B7B9
Crc16USB block software:          4846
Crc16USB block hardware:          4846
Crc16USB non-block software:      4846
Crc16USB non-block hardware:      4846
Crc32 software:                   C4D5C4DF
Crc32 software less time:         C4D5C4DF

Crc32 non-block software less time: C4D5C4DF
Crc32 hardware:                   C4D5C4DF
Crc32 non-block hardware:         C4D5C4DF

```

2.8.2.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\CRC\CRC_Demo_Hard_Soft_Compare_Project\IAR_Project, and double-click the **CRC_Demo_Hard_Soft_Compare** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.8.2.4.3 For Keil

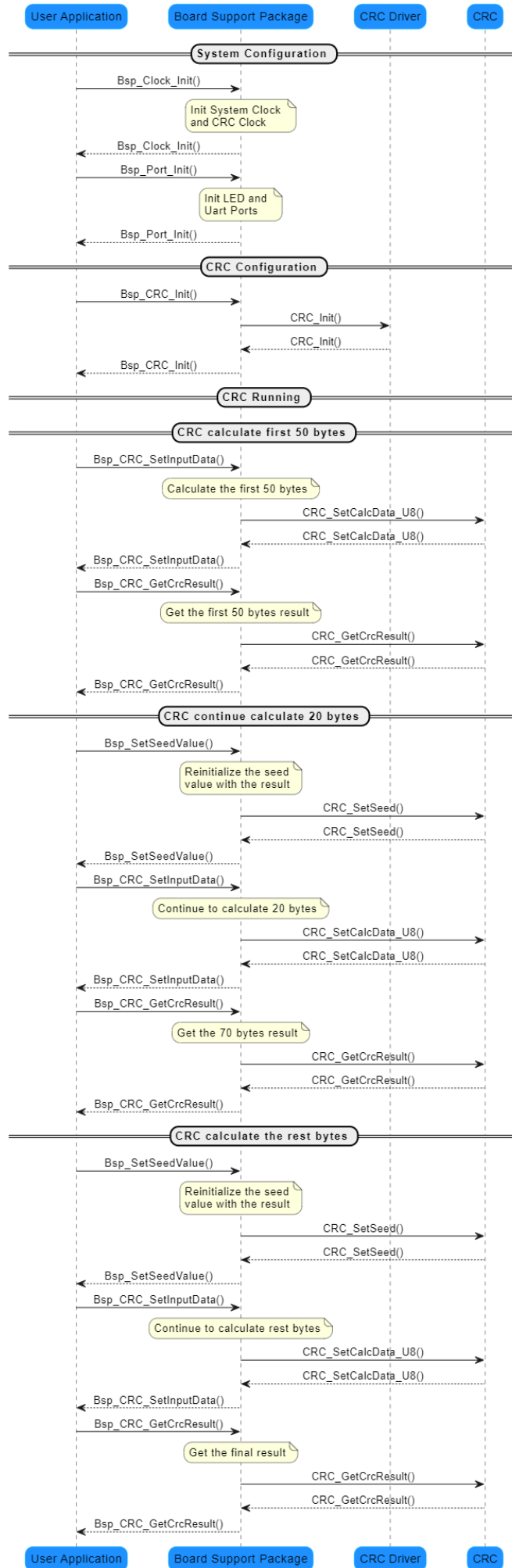
Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.8.3 CRC_Demo_Multi_Step

2.8.3.1 Description

This demo is used to calculate CRC separated multiple times, and the result of each calculation is used as the initial value for the next calculation. After all data are calculated, the CRC result of all data is obtained. The sequence diagram of the demo is as follows:

FC7300F8MDQ SDK Quick Start



When the input data is written successfully, the CRC result can be read from DATA register immediately. After CRC calculation, the seed value must be reset before the next CRC calculation.

2.8.3.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.8.3.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A26: MUX = ALT1, GPIO Output	LED1
Port D31: MUX = ALT1, GPIO Output	LED2
Port A14: MUX = ALT1, GPIO Output	LED3

2.8.3.4 Steps to Run

2.8.3.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: Make sure "**Debug_Flash**" is selected as the active build configuration.

After building and debugging the project, the results will be shown as below:

- The result of CRC_16_USB:

```
System Start!!
CRC hardware calculates the first 50 bytes: D38E
CRC hardware calculates the first 70 bytes: 2FC7
CRC hardware calculates the all bytes: 20DD
Running tick:0!
Running tick:1!
Running tick:2!
Running tick:3!
Running tick:4!
Running tick:5!
Running tick:6!
Running tick:7!
Running tick:8!
Running tick:9!
```

2.8.3.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\CRC\CRC_Demo_Multi_Step_Project\IAR_Project, and double-click the **CRC_Demo_Multi_Step** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.8.3.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.9 DMA

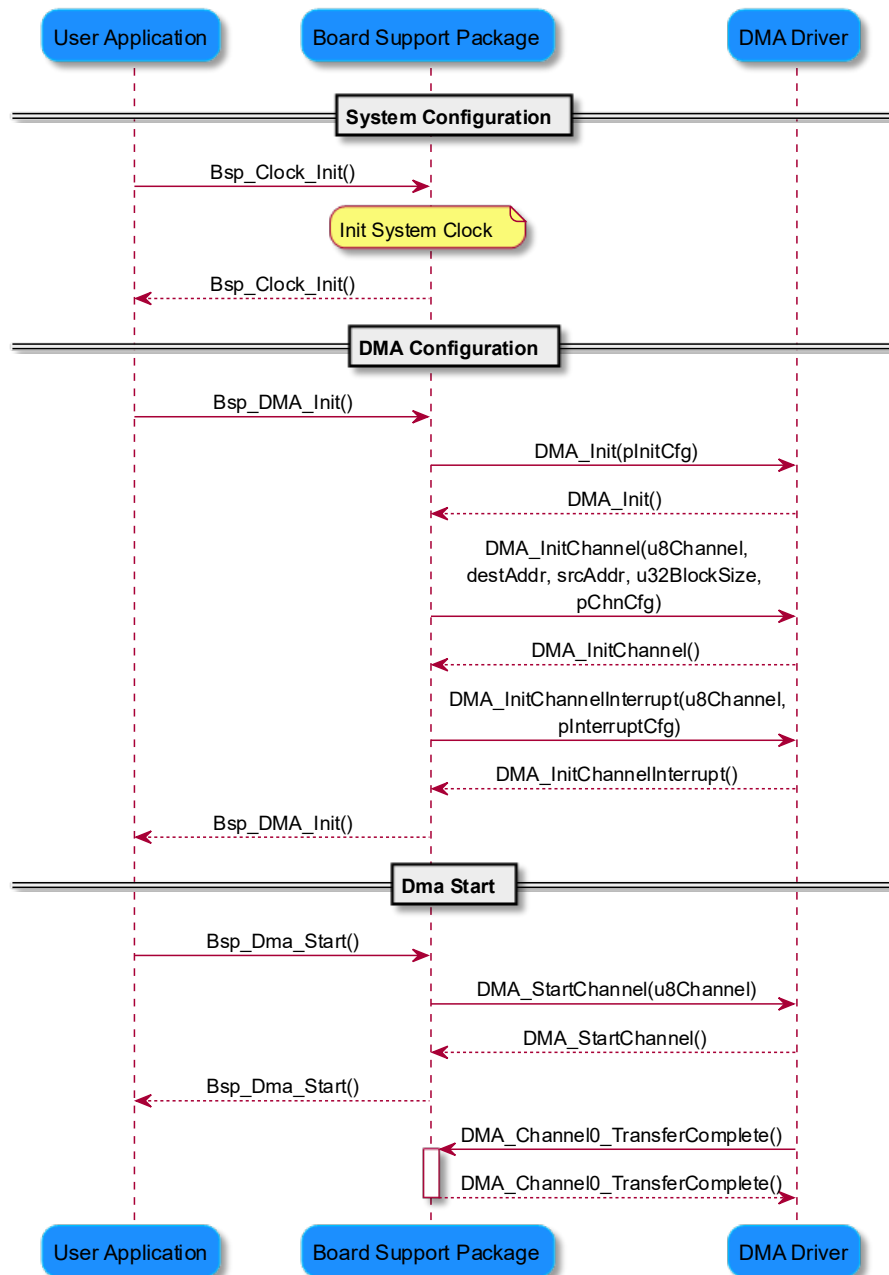
There are two DMA demos: DMA_Memcpy and DMA_CircularBuffer.

2.9.1 DMA_Memcpy

2.9.1.1 Description

This demo shows the user the general configuration process to use the DMA to transfer data.

The sequence diagram of the demo is as follows:



2.9.1.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board

- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.9.1.3 MCU Ports

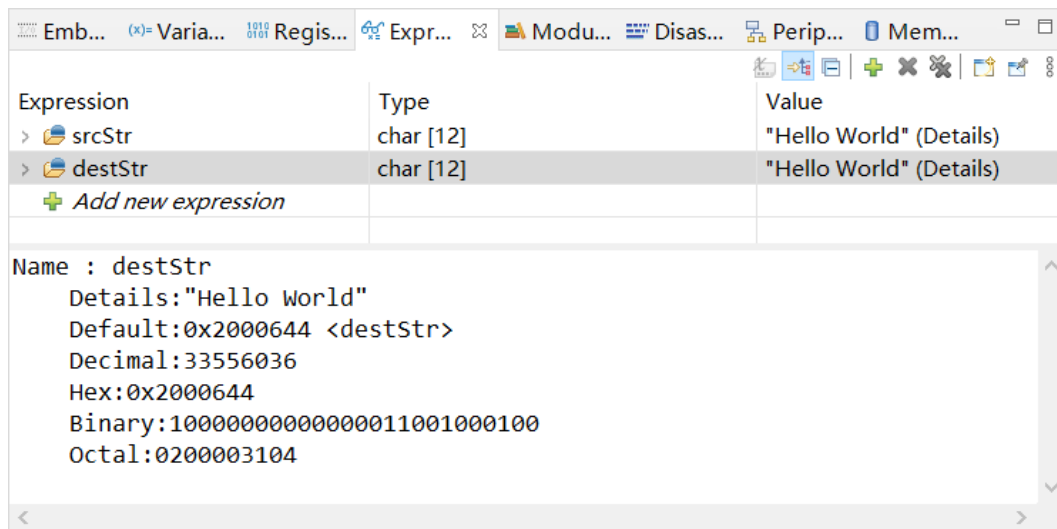
N/A.

2.9.1.4 Steps to Run

2.9.1.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

After building and debugging, you can see the srcStr and destStr variables in the **Expressions** tab. Before the DMA transfer, the srcStr is "Hello World" and the destStr is "\0". While after DMA transfer completed, the srcStr and destStr are both "Hello World".



2.9.1.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\DMA\DMA_Memcpy_Project\IAR_Project, and double-click the **DMA_Memcpy** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.9.1.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.9.2 DMA_CircularBuffer

2.9.2.1 Description

This demo shows the user how to use the DMA to implement a circular buffer.

When using the circular buffer feature of the DMA, there are some limitations to the data address. If the source buffer is used as a DMA circular buffer, the source address should be power of 2 aligned to the buffer size. For example, if the source circular

buffer is 32 bytes, then the source data address should be 32 byte aligned. And if the source circular buffer is 100 bytes, then the source data address should be 128 byte aligned. And this rule applies to the destination address, too.

2.9.2.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.9.2.3 MCU Ports

N/A.

2.9.2.4 Steps to Run

2.9.2.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

After building and debugging, you can see the srcStr and destStr variables in the **Expressions** tab. Before the DMA transfer, the srcStr is "Hello World" and the destStr is "\0". In the program, we have three DMA transfer loops. After the first two loops, the beginning 24 bytes the of destination buffer are written with the source buffer.

Expression	Type	Value
> g_strSrc	char [12]	"Hello World" (Details)
▼ g_strDest	char [25]	"Hello World\0Hello World\0" (Details)
g_strDest[0]	char	0x48
g_strDest[1]	char	0x65
g_strDest[2]	char	0x6c
g_strDest[3]	char	0x6c
g_strDest[4]	char	0x6f
g_strDest[5]	char	0x20
g_strDest[6]	char	0x57
g_strDest[7]	char	0x6f
g_strDest[8]	char	0x72
g_strDest[9]	char	0x6c
g_strDest[10]	char	0x64
g_strDest[11]	char	0x0
g_strDest[12]	char	0x48
g_strDest[13]	char	0x65
g_strDest[14]	char	0x6c
g_strDest[15]	char	0x6c
g_strDest[16]	char	0x6f
g_strDest[17]	char	0x20
g_strDest[18]	char	0x57
g_strDest[19]	char	0x6f
g_strDest[20]	char	0x72
g_strDest[21]	char	0x6c
g_strDest[22]	char	0x64
g_strDest[23]	char	0x0
g_strDest[24]	char	0x0
+	Add new expression	

After the third transfer, the last byte of the destination buffer is written with the source buffer. Then, the continuous source buffer is written to the beginning of the destination buffer.

FC7300F8MDQ SDK Quick Start

Expression	Type	Value
> g_strSrc	char [12]	"Hello World" (Details)
▼ g_strDest	char [25]	"ello World\0\0Hello World\0H" (Details)
g_strDest[0]	char	0x65
g_strDest[1]	char	0x6c
g_strDest[2]	char	0x6c
g_strDest[3]	char	0x6f
g_strDest[4]	char	0x20
g_strDest[5]	char	0x57
g_strDest[6]	char	0x6f
g_strDest[7]	char	0x72
g_strDest[8]	char	0x6c
g_strDest[9]	char	0x64
g_strDest[10]	char	0x0
g_strDest[11]	char	0x0
g_strDest[12]	char	0x48
g_strDest[13]	char	0x65
g_strDest[14]	char	0x6c
g_strDest[15]	char	0x6c
g_strDest[16]	char	0x6f
g_strDest[17]	char	0x20
g_strDest[18]	char	0x57
g_strDest[19]	char	0x6f
g_strDest[20]	char	0x72
g_strDest[21]	char	0x6c
g_strDest[22]	char	0x64
g_strDest[23]	char	0x0
g_strDest[24]	char	0x48
✚ Add new expression		

2.9.2.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\DMA\DMA_CircularBuffer_Project\IAR_Project, and double-click the **DMA_CircularBuffer** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.9.2.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.10 EFTU

There are four EFTU demos: Eftu_SignalDetect, Eftu_PwmMeasurement, Eftu_Timer, and Eftu_Pwm.

2.10.1 Eftu_SignalDetect

2.10.1.1 Description

This example shows how to use the EFTU module as an input capture. When the user adds the PWM waveform on the pin, the Interrupt callback function will be called and the LED will toggle.

2.10.1.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable

- 1 Personal computer
- 1 Digital Oscilloscope
- 1 J-Link debugger

2.10.1.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port D28: MUX = ALT6, EFTU0_TIM5	Eftu0 Tim channel 5 input pin
Port A26: MUX = ALT1, GPIO Output	LED1
Port D31: MUX = ALT1, GPIO Output	LED2

2.10.1.4 Steps to Run

The steps to run the example are as follows:

- 1) Import the EFTU project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) Measure the PWM waveform on the pin.

2.10.2 Eftu_PwmMeasurement

2.10.2.1 Description

This example shows how to use the EFTU module as signal measurement. When the user adds the PWM waveform on the pin, The Interrupt callback function will be called and the LED will toggle. The user call get the period and duty cycle in the callback function.

2.10.2.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 Digital Oscilloscope
- 1 J-Link debugger

2.10.2.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port D28: MUX = ALT6, EFTU0_TIM5	Eftu0 Tim channel 5 input pin

Pin Function	FC7300F8MDQ Demo Board
Port A26: MUX = ALT1, GPIO Output	LED1
Port D31: MUX = ALT1, GPIO Output	LED2

2.10.2.4 Steps to Run

The steps to run the example are as follows:

- 1) Import the EFTU project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) Measure the PWM waveform on the pin.

2.10.3 Eftu_Timer

2.10.3.1 Description

This example shows how to use the EFTU module as timer. The Interrupt callback function will be called and the LED will toggle.

2.10.3.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 Digital Oscilloscope
- 1 J-Link debugger

2.10.3.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A26: MUX = ALT1, GPIO Output	LED1
Port D31: MUX = ALT1, GPIO Output	LED2

2.10.3.4 Steps to Run

The steps to run the example are as follows:

- 1) Import the EFTU project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

2.10.4 Eftu_Pwm

2.10.4.1 Description

This example shows how to use the EFTU module to generate Pwm wave. you can send "a" to increase the duty cycle. You can send "m" to increase the duty cycle, send "f" to generate the fault, or can send "c" to clear the fault.

2.10.4.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 Digital Oscilloscope
- 1 J-Link debugger

2.10.4.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A26: MUX = ALT1, GPIO Output	LED1
Port D31: MUX = ALT1, GPIO Output	LED2
Port C19: MUX = ALT6, EFTU2_OUT7	PWM output pin
Port D19: MUX = ALT4, EFTU2_OUT23	PWM output pin
Port C9: MUX = ALT3, FTU_FLT10	Fault input pin
Port A1: MUX = ALT1, GPIO Output	Fault generation pin

2.10.4.4 Steps to Run

The steps to run the example are as follows:

- 1) Import the EFTU project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) Connect PTC9 and PTA1.
- 3) Measure the PWM period and duty cycle.

2.11 EIM/ERM

2.11.1 Description

The demo shows how to check errors when an interrupt occurs to ERM. Users need to code the function according to their requirements.

2.11.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.11.3 MCU Ports

N/A

2.11.4 Steps to Run

The steps to run the example are as follows:

- 1) Plug in the USB Type-C cable and open the up-machine (UART1 115200/8/N/1).
- 2) Import the EIM/ERM project, then build and download it.

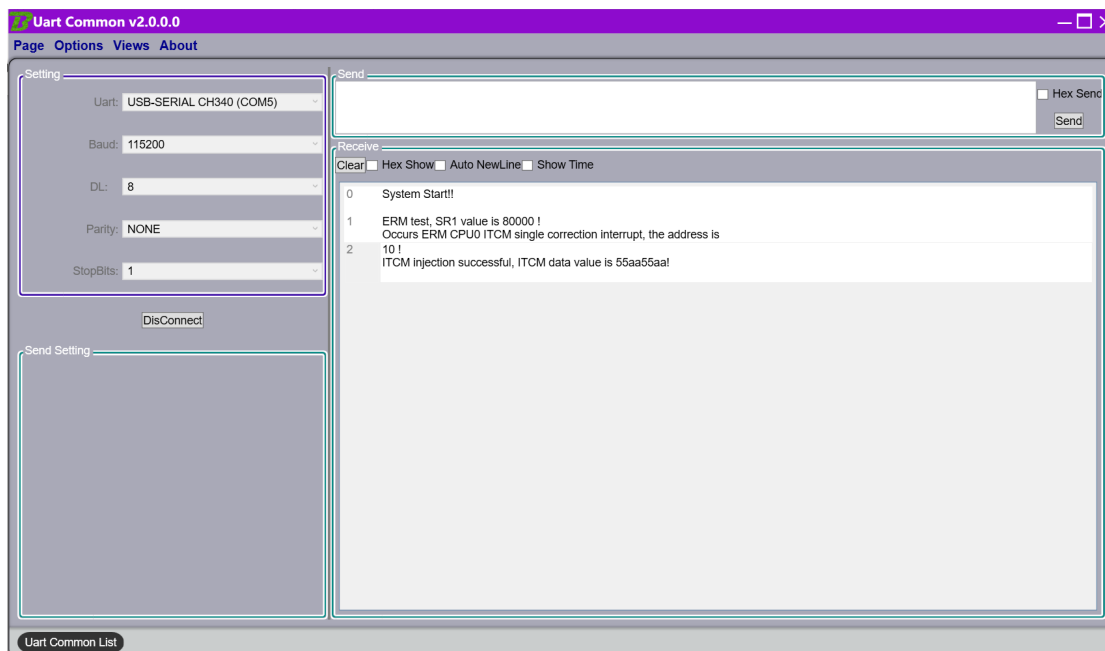
Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 3) The MCU sends EIM/ERM messages for test.

Note: ERM_UserIRQHandler () is called when a MCM interrupt occurs. The demo tells users how to check the errors when an interrupt occurs to ERM. Users need to code the function according to their requirements.

The demo relates to injection errors for CPU0 ITCM. To inject DTCM0/1 or the region used by process, users need to update the link file, put interrupts_ram, and heap and stack to the region not to use.



2.12 ENET

There are four ENET demos: ENET_lwIP_FreeRTOS, ENET_lwIP_NoSys, RT_THREAD, and RT_THREAD_AMP.

2.12.1 ENET_lwIP_FreeRTOS

2.12.1.1 Description

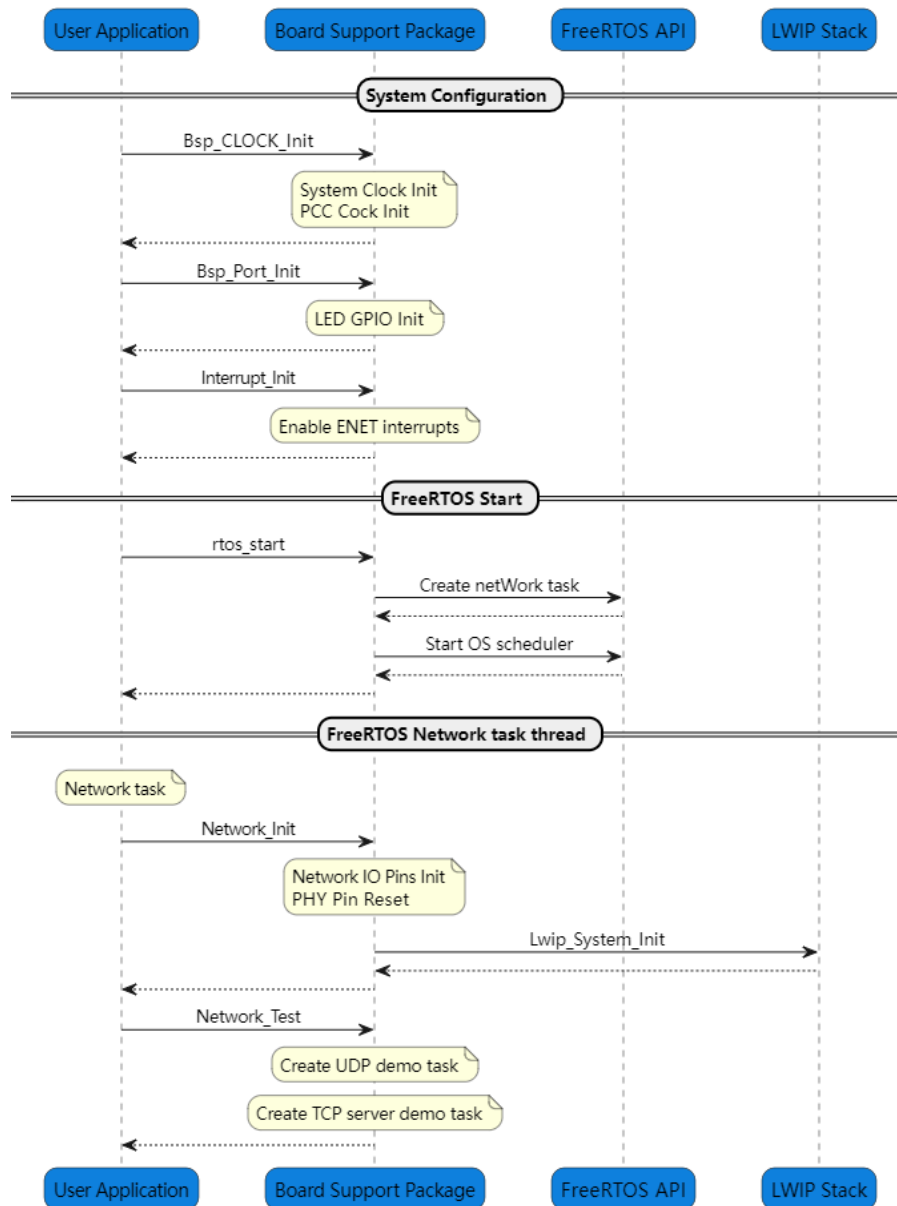
This demo shows how to use LWIP and ENET driver with no operation system.

- Board-level configurations such as system clock and port IO are initialized in `prvSetupHardware` function.
- Network module starts from `NetworkInit` function, and the TCP and UDP demo are started in `Network_Test` function.

PHY selection: FC7300F8MDQ Demo Board support 1000BaseTx PHY and 1000BaseT1 PHY, the mode is selected by `PHY_TYPE` in `enet_config.h`. `PHY_JL311X` and `PHY_JL311X_YT8531` are supported on FC7300F8MDQ Demo Board.
`PHY_JL311X`: 1000BaseT1 PHY JL311X is connected to MCU via RGMII interface.

`PHY_JL311X_YT8531`: JL311X is connected to MCU via RGMII interface and YT8531 is connected to JL311X via SGMII interface.

The sequence diagram of the demo is as follows:



2.12.1.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board

- 1 Power adapter 12 V
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.12.1.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A26: MUX = ALT1, GPIO Output	LED1 – wired on board
Port D31: MUX = ALT1, GPIO Output	LED2 – wired on board
Port E20: MUX = ALT1, GPIO Output	LED3 – wired on board
Port B26: MUX = ALT1, GPIOOUT, 1000BaseT1 ETH_EN	ETH EN – wired on board
Port E26: GPIOOUT, 1000BaseT1 ETH_WAKE	ETH WAKE – wired on board
Port B19: MUX = ALT1, GPIOIN, 1000BaseT1 ETH_INT_N	ETH INH – wired on board
Port B4: MUX = ALT5, ENET_MDIO	MDIO Select – wired on board
Port E8: MUX = ALT1: GPIOOUT, ETH_RST	ETH_RST – wired on board
Port B4: MUX = ALT5, ENET_MDIO	ENET MDIO – wired on board
Port B5: MUX = ALT5, ENET_MDC	ENET MDC – wired on board
Port D11: MUX = ALT5, ENET_TX_CLK	ENET TX CLK – wired on board
Port D12: MUX = ALT5, ENET_TX_EN_CTRL	ENET TX EN – wired on board
Port C2: MUX = ALT5, ENET_TXD0	ENET TXD0 – wired on board
Port D7: MUX = ALT5, ENET_TXD1	ENET TXD1 – wired on board
Port D6: MUX = ALT5, ENET_TXD2	ENET TXD2 – wired on board
Port D5: MUX = ALT5, ENET_TXD3	ENET TXD3 – wired on board
Port D10: MUX = ALT5, ENET_RX_CLK	ENET RX CLK – wired on board
Port D8: MUX = ALT5, ENET_RX_DV_CTRL	ENET RX DV – wired on board
Port C1: MUX = ALT5, ENET_RXD0	ENET RXD0 – wired on board
Port C0: MUX = ALT5, ENET_RXD1	ENET RXD1 – wired on board
Port D9: MUX = ALT5, ENET_RXD2	ENET RXD2 – wired on board
Port C15: MUX = ALT5, ENET_RXD3	ENET RXD3 – wired on board
Port G8: MUX = ALT5, ENET_REF_CLK	ENET REFCLK – wired on board
Port B20: MUX = ALT5, ENET_PPS0	ENET PPS0 –output pin on board

2.12.1.4 Steps to Run

The steps to run the example are as follows:

- 1) Import the ENET_lwIP_FreeRTOS project, then build and download it.

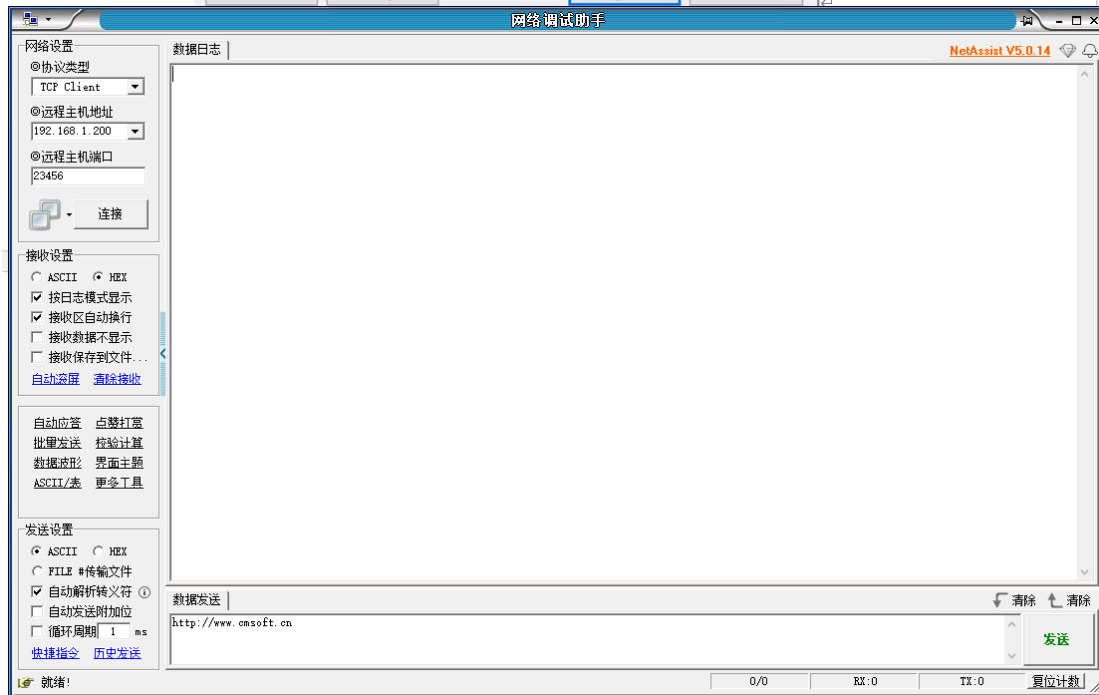
Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) The ethernet driver and lwip stack are implemented in this project. You can use the lwip raw interface or socket interface for network applications. The default MAC address and IP address (192.168.1.200) are defined in enet_config.h. We have implemented a TCP server demo and a UDP echo demo as network applications examples.

a) Connect to TCP server with net tools (netassist):

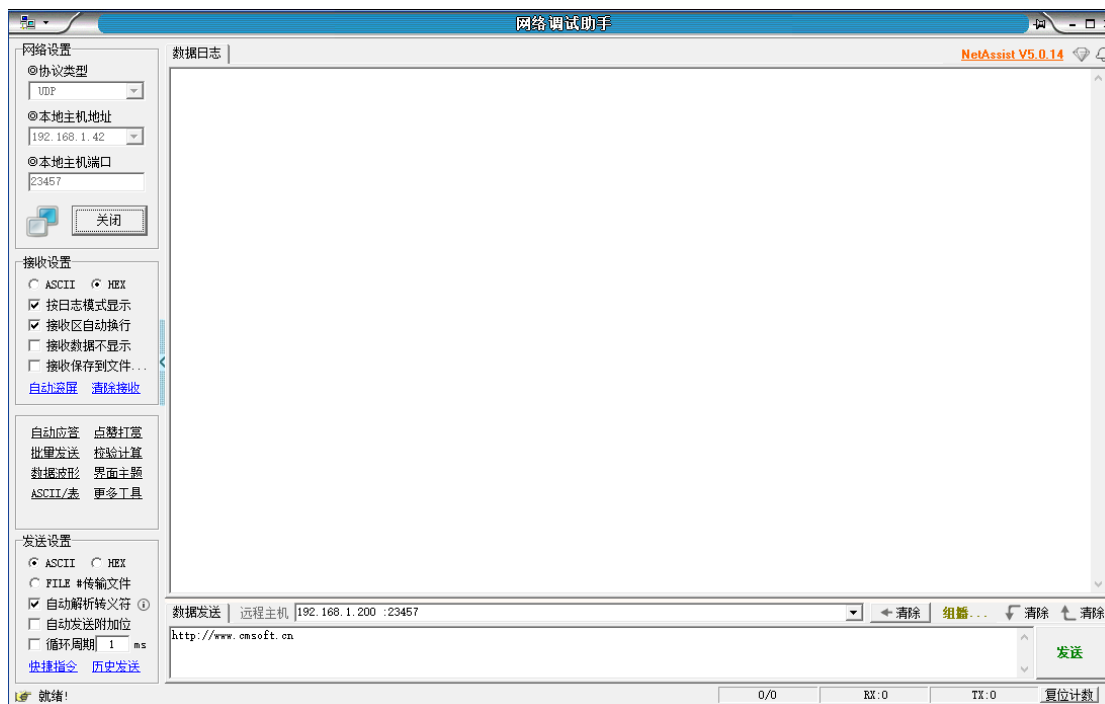
Start a TCP client on PC, select the IP address (192.168.1.200) of demo board as the remote host address, and select the port 23456 (Ports used in TCP demo). Connect the server, send packets, and the demo board will send the packets back to PC.



b) Test UDP demo use net tools (netassist):

i. Start a UDP communication on PC, select the local IP address (192.168.1.42) of your PC as the local host address, and select the port 23457 (Ports used in UDP demo). Click open and configure the IP and port of remote host (192.168.1.200:23457).

ii. Send packets to demo board, the demo board will send the packets back to PC.



2.12.2 ENET_lwIP_NoSys

2.12.2.1 Description

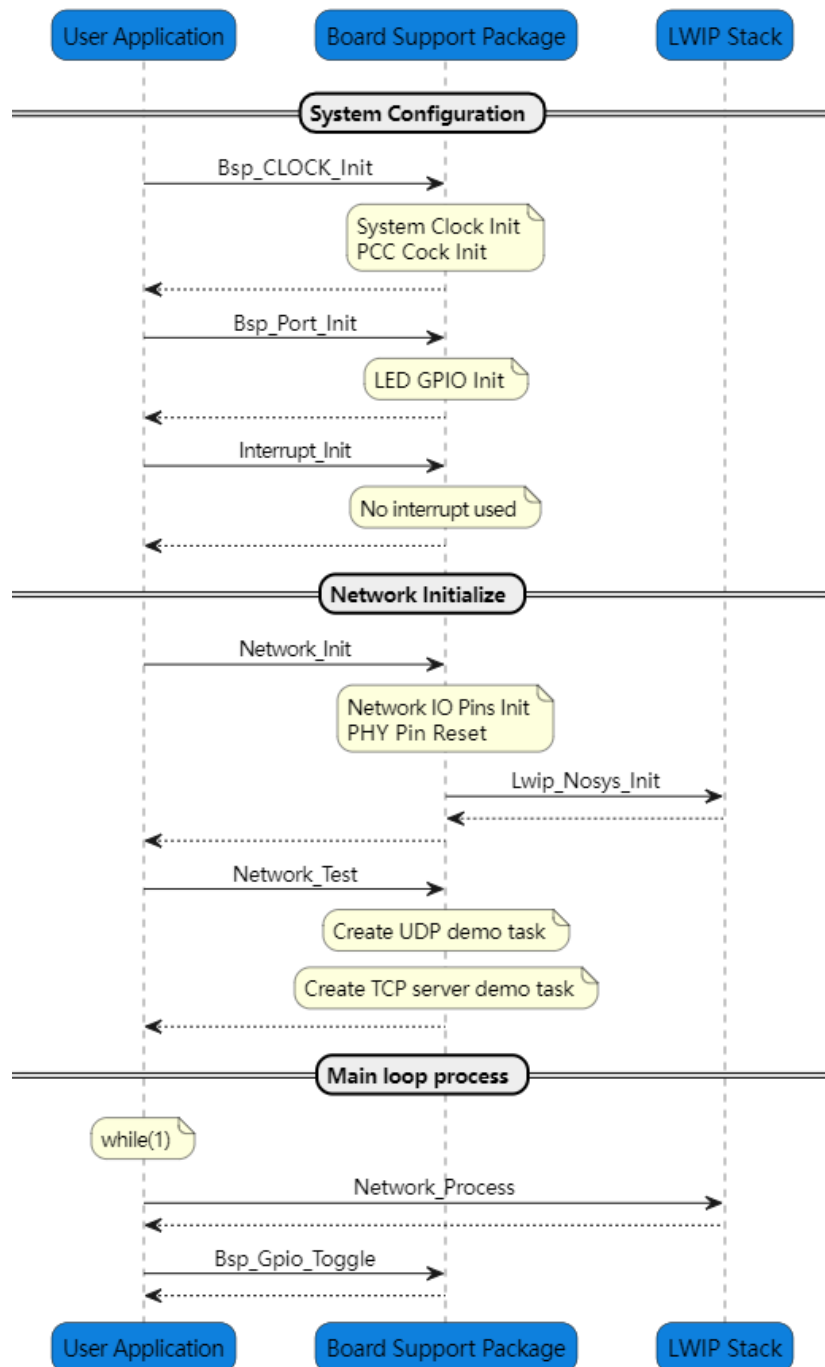
This demo shows how to use LWIP and ENET driver with no operation system.

- Board-level configurations such as system clock and port IO are initialized in `prvSetupHardware` function.
- Network module starts from `NetworkInit` function, and the TCP and UDP demo are started in `Network_Test` function.

PHY selection: FC7300F8MDQ Demo Board support 1000BaseTx PHY and 1000BaseT1 PHY, the mode is selected by `PHY_TYPE` in `enet_config.h`. `PHY_JL311X` and `PHY_JL311X_YT8531` are supported on FC7300F8MDQ Demo Board.
`PHY_JL311X`: 1000BaseT1 PHY JL311X is connected to MCU via RGMII interface.

`PHY_JL311X_YT8531`: JL311X is connected to MCU via RGMII interface and YT8531 is connected to JL311X via SGMII interface.

The sequence diagram of the demo is as follows:



2.12.2.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.12.2.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A26: MUX = ALT1, GPIO Output	LED1 – wired on board
Port D31: MUX = ALT1, GPIO Output	LED2 – wired on board
Port E20: MUX = ALT1, GPIO Output	LED3 – wired on board
Port B26: MUX = ALT1, GPIOOUT, 1000BaseT1 ETH_EN	ETH EN – wired on board
Port E26: GPIOOUT, 1000BaseT1 ETH_WAKE	ETH WAKE – wired on board
Port B19: MUX = ALT1, GPIOIN, 1000BaseT1 ETH_INT_N	ETH INH – wired on board
Port B4: MUX = ALT5, ENET_MDIO	MDIO Select – wired on board
Port E8: MUX = ALT1: GPIOOUT, ETH_RST	ETH_RST – wired on board
Port B4: MUX = ALT5, ENET_MDIO	ENET MDIO – wired on board
Port B5: MUX = ALT5, ENET_MDC	ENET MDC – wired on board
Port D11: MUX = ALT5, ENET_TX_CLK	ENET TX CLK – wired on board
Port D12: MUX = ALT5, ENET_TX_EN_CTRL	ENET TX EN – wired on board
Port C2: MUX = ALT5, ENET_TXD0	ENET TXD0 – wired on board
Port D7: MUX = ALT5, ENET_TXD1	ENET TXD1 – wired on board
Port D6: MUX = ALT5, ENET_TXD2	ENET TXD2 – wired on board
Port D5: MUX = ALT5, ENET_TXD3	ENET TXD3 – wired on board
Port D10: MUX = ALT5, ENET_RX_CLK	ENET RX CLK – wired on board
Port D8: MUX = ALT5, ENET_RX_DV_CTRL	ENET RX DV – wired on board
Port C1: MUX = ALT5, ENET_RXD0	ENET RXD0 – wired on board
Port C0: MUX = ALT5, ENET_RXD1	ENET RXD1 – wired on board
Port D9: MUX = ALT5, ENET_RXD2	ENET RXD2 – wired on board
Port C15: MUX = ALT5, ENET_RXD3	ENET RXD3 – wired on board
Port G8: MUX = ALT5, ENET_REF_CLK	ENET REFCLK – wired on board
Port B20: MUX = ALT5, ENET_PPS0	ENET PPS0 –output pin on board

2.12.2.4 Steps to Run

The steps to run the example are as follows:

- 1) Import the ENET_lwIP_NoSys project, then build and download it.

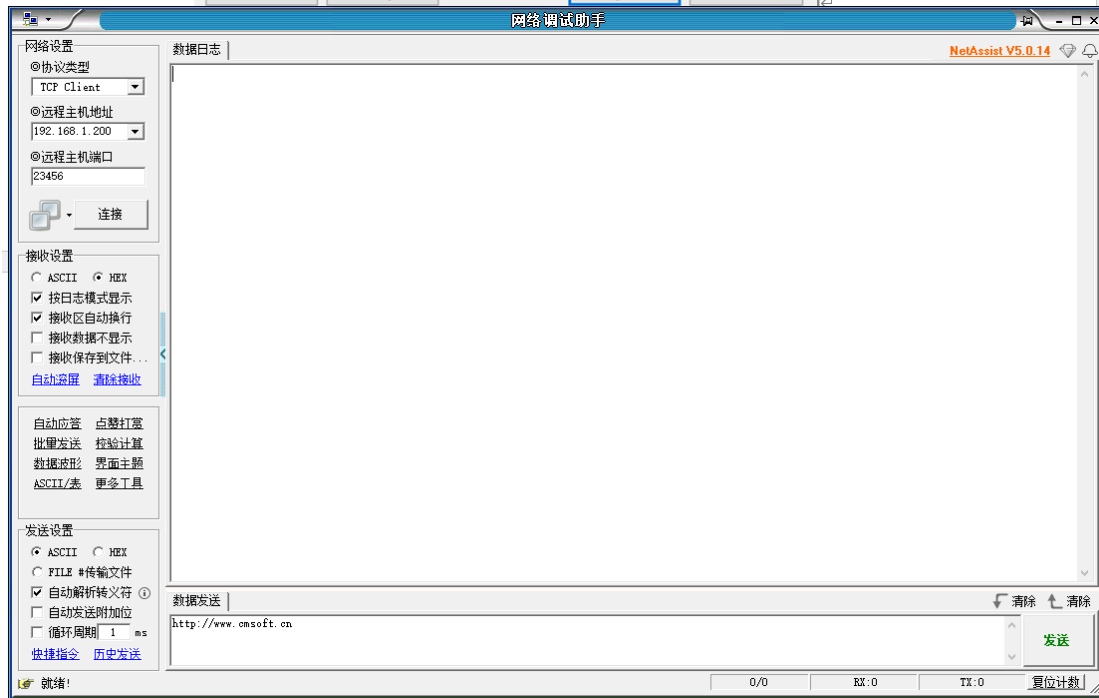
Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) The ethernet driver and lwip stack are implemented in this project. You can use the lwip raw interface or socket interface for network applications. The default MAC address and IP address (192.168.1.200) are defined in `enet_config.h`. We have implemented a TCP server demo and a UDP echo demo as network applications examples.

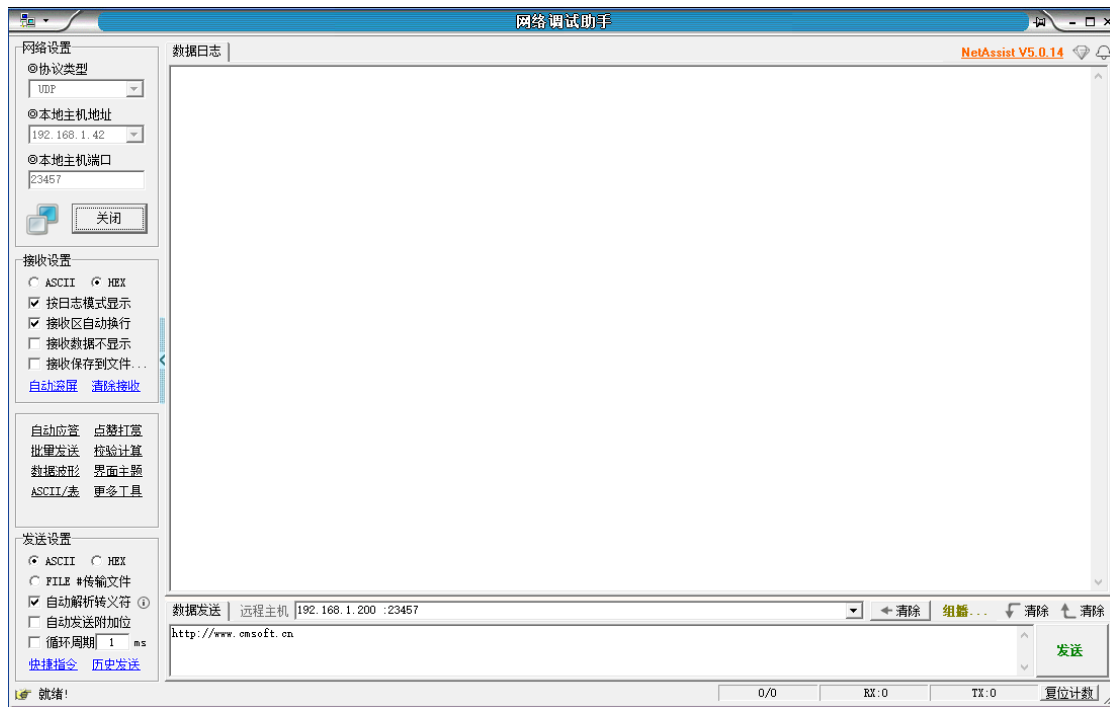
a) Connect to TCP server with net tools (netassist):

Start a TCP client on PC, select the IP address (192.168.1.200) of demo board as the remote host address, and select the port 23456 (Ports used in TCP demo). Connect the server, send packets, and the demo board will send the packets back to PC.



b) Test UDP demo use net tools (netassist):

- i. Start a UDP communication on PC, select the local IP address (192.168.1.42) of your PC as the local host address, and select the port 23457 (Ports used in UDP demo). Click open and configure the IP and port of remote host (192.168.1.200:23457).
- ii. Send packets to demo board, the demo board will send the packets back to PC.



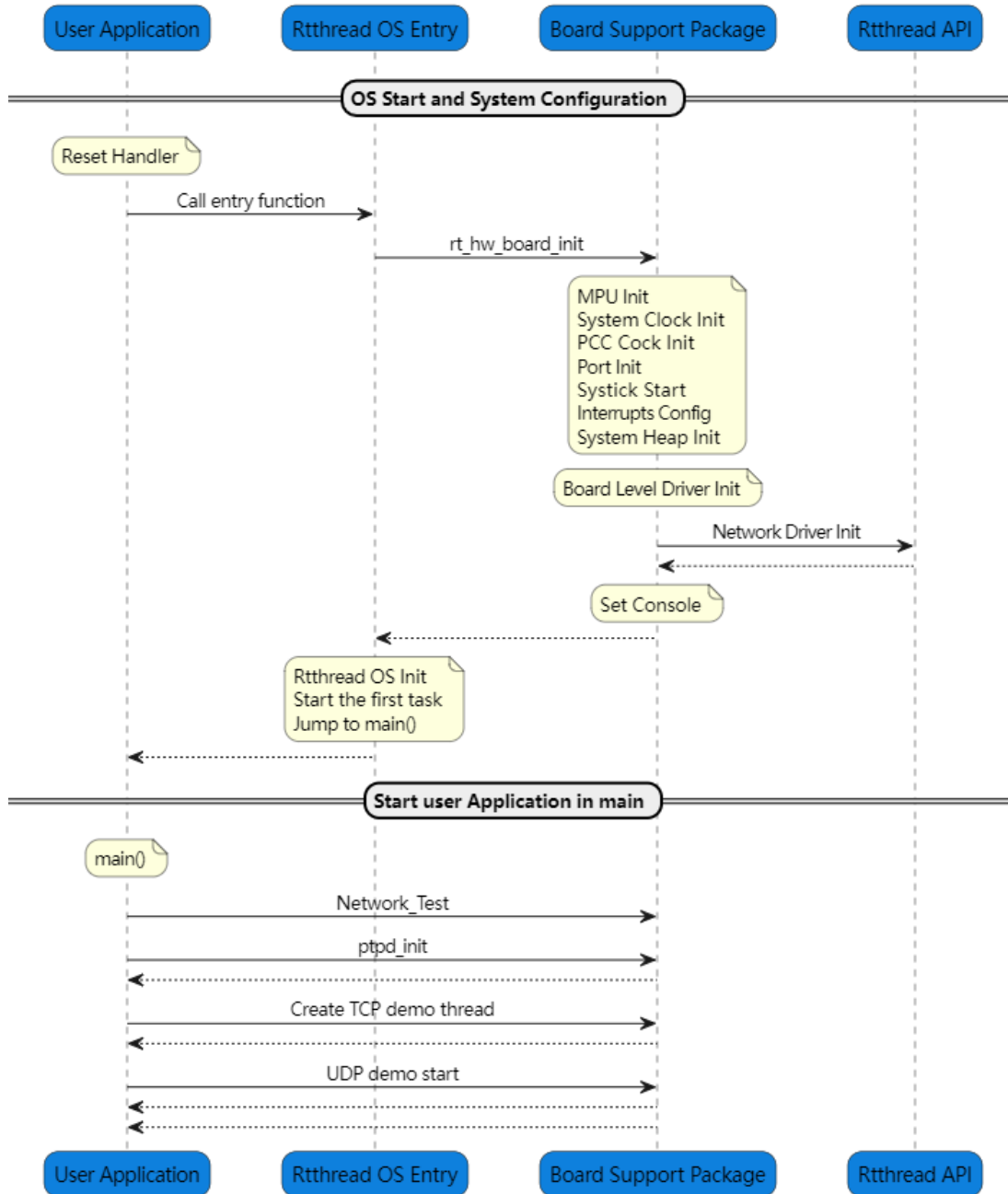
2.12.3 RT_THREAD

2.12.3.1 Description

The purpose of this demo is to show you the basic usage of RTTHREAD operation system and ENET SDK driver.

- Board-level configurations such as system clock system tick, port io, heap and console are initialized in board.c.
- User command line is supported on the UART console via USB Type-C.
- PHY selection: FC7300F8MDQ Demo Board support 1000BaseTx PHY and 1000BaseT1 PHY, the mode is selected by PHY_TYPE in rtconfig.h, PHY_JL311X and PHY_JL311X_YT8531 are supported on FC7300F8MDQ Demo Board.
PHY_JL311X: 1000BaseT1 PHY JL311X is connected to MCU via RGMII interface.
PHY_JL311X_YT8531: JL311X is connected to MCU via RGMII interface and YT8531 is connected to JL311X via SGMII interface.

The sequence diagram of the demo is as follows:



2.12.3.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.12.3.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A26: MUX = ALT1, GPIO Output	LED1 – wired on board
Port D31: MUX = ALT1, GPIO Output	LED2 – wired on board
Port E20: MUX = ALT1, GPIO Output	LED3 – wired on board
Port B26: MUX = ALT1, GPIOOUT, 1000BaseT1 ETH_EN	ETH EN – wired on board

Pin Function	FC7300F8MDQ Demo Board
Port E26: GPIOOUT, 1000BaseT1 ETH_WAKE	ETH WAKE – wired on board
Port B19: MUX = ALT1, GPIOIN, 1000BaseT1 ETH_INT_N	ETH INH – wired on board
Port B4: MUX = ALT5, ENET_MDIO	MDIO Select – wired on board
Port E8: MUX = ALT1: GPIOOUT, ETH_RST	ETH_RST – wired on board
Port B4: MUX = ALT5, ENET_MDIO	ENET MDIO – wired on board
Port B5: MUX = ALT5, ENET_MDC	ENET MDC – wired on board
Port D11: MUX = ALT5, ENET_TX_CLK	ENET TX CLK – wired on board
Port D12: MUX = ALT5, ENET_TX_EN_CTRL	ENET TX EN – wired on board
Port C2: MUX = ALT5, ENET_TXD0	ENET TXD0 – wired on board
Port D7: MUX = ALT5, ENET_TXD1	ENET TXD1 – wired on board
Port D6: MUX = ALT5, ENET_TXD2	ENET TXD2 – wired on board
Port D5: MUX = ALT5, ENET_TXD3	ENET TXD3 – wired on board
Port D10: MUX = ALT5, ENET_RX_CLK	ENET RX CLK – wired on board
Port D8: MUX = ALT5, ENET_RX_DV_CTRL	ENET RX DV – wired on board
Port C1: MUX = ALT5, ENET_RXD0	ENET RXD0 – wired on board
Port C0: MUX = ALT5, ENET_RXD1	ENET RXD1 – wired on board
Port D9: MUX = ALT5, ENET_RXD2	ENET RXD2 – wired on board
Port C15: MUX = ALT5, ENET_RXD3	ENET RXD3 – wired on board
Port G8: MUX = ALT5, ENET_REF_CLK	ENET REFCLK – wired on board
Port B20: MUX = ALT5, ENET_PPS0	ENET PPS0 –output pin on board

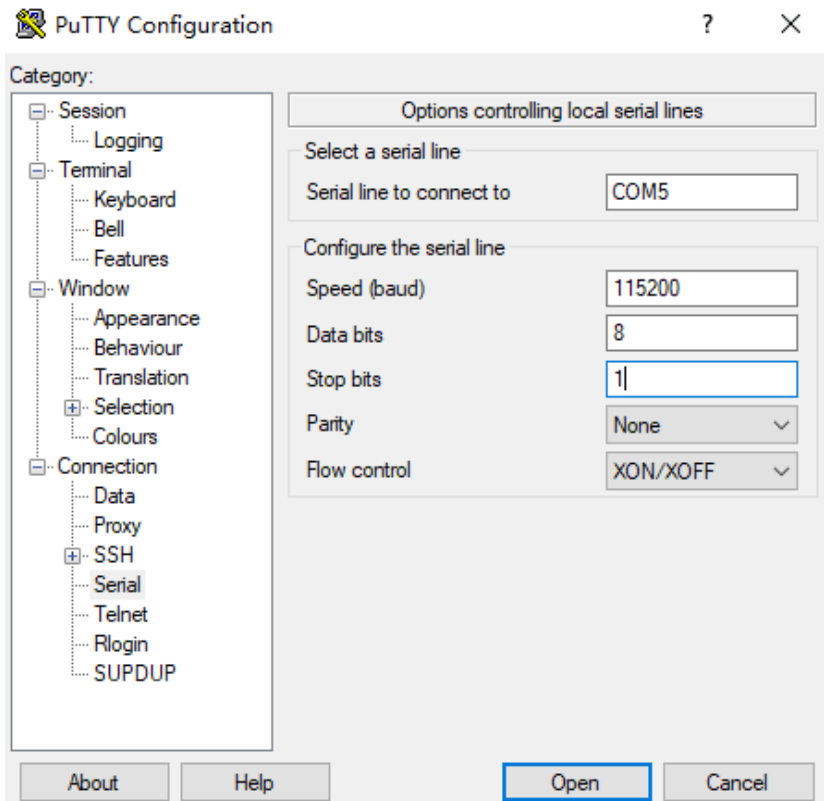
2.12.3.4 Steps to Run

- 1) Import the RT_Thread project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) Open the console tool on your personal computer (**PuTTY** or **SecureCRT**).



3) Console.

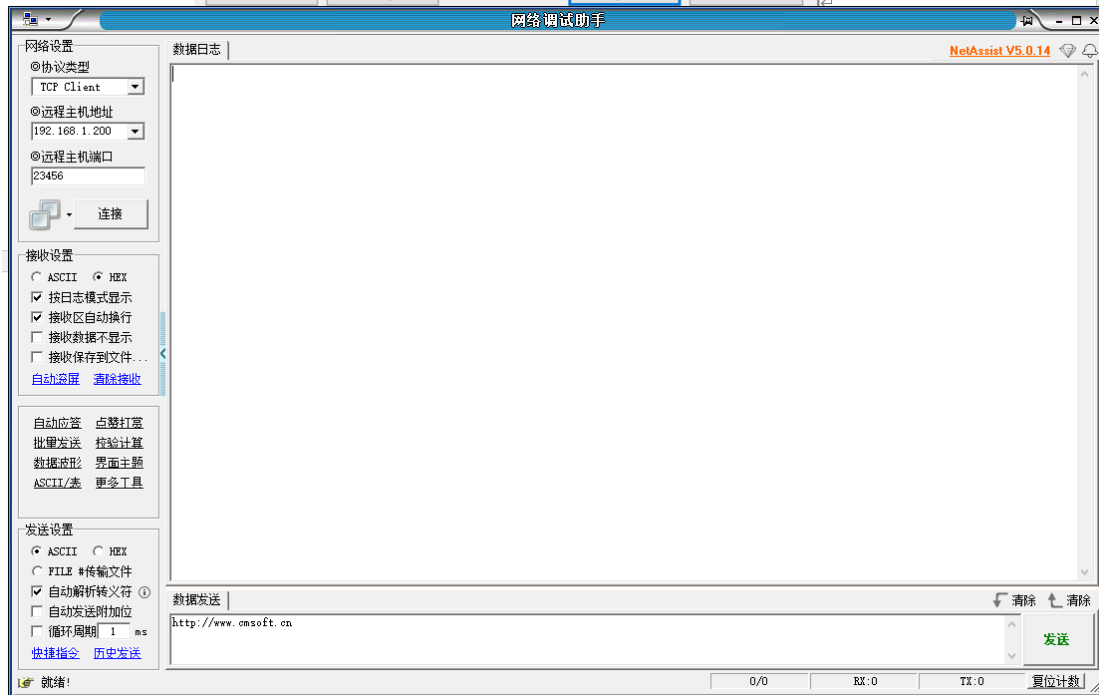
Enter help on the console, all implemented command will be listed.

The ping command is available on the console, you can use ping command to ping other devices.

4) The ethernet driver and lwip stack are implemented in this project. You can use the lwip raw interface or socket interface for network applications. The default MAC address and IP address (192.168.1.200) are defined in rtconfig.h. We have implemented a TCP server demo and a UDP echo demo as network applications examples.

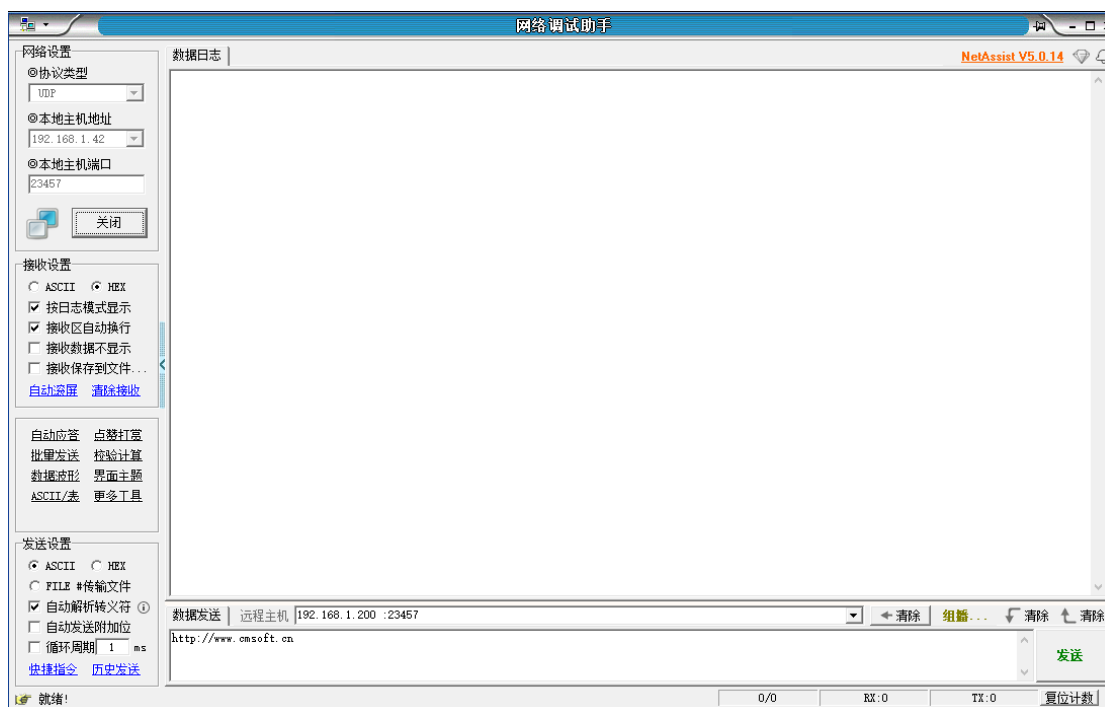
a) Connect to TCP server with net tools (netassist):

Start a TCP client on PC, select the IP address (192.168.1.200) of demo board as the remote host address, and select the port 23456 (Ports used in TCP demo). Connect the server, send packets, and the demo board will send the packets back to PC.



b) Test UDP demo with net tools (netassist):

- i. Start a UDP communication on PC, select the local IP address (192.168.1.42) of your PC as the local host address, and select the port 23457 (Ports used in UDP demo). Click open and configure the IP and port of remote host (192.168.1.200:23457).
- ii. Send packets to demo board, the demo board will send the packets back to PC.



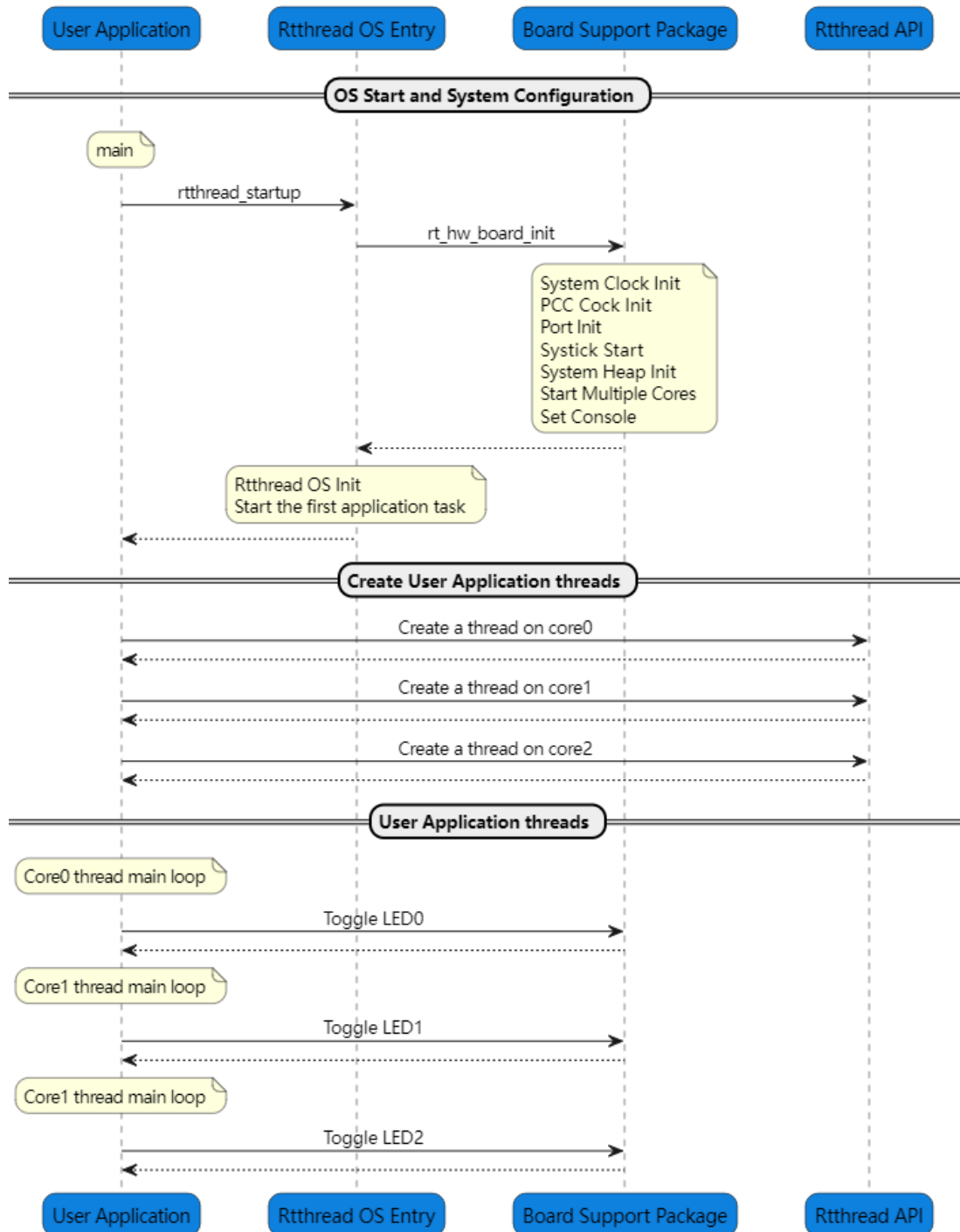
2.12.4 RT_THREAD_AMP

2.12.4.1 Description

The purpose of this demo is to show you the basic usage of RTTHREAD operation system and ENET SDK driver.

- Board-level configurations such as system clock system tick, port io, heap and console are initialized in board.c. All data, bss, heap, stack, rtthread system heap are allocated to the separate DTCM for each core.
- User command line is supported on the UART console via USB Type-C.
- Each core creates a thread, and LED1, LED2, LED3 are toggled by core0, core1, core2 respectively.

The sequence diagram of the demo is as follows:



2.12.4.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.12.4.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A26: MUX = ALT1, GPIO Output	LED1 – wired on board
Port D31: MUX = ALT1, GPIO Output	LED2 – wired on board
Port E20: MUX = ALT1, GPIO Output	LED3 – wired on board

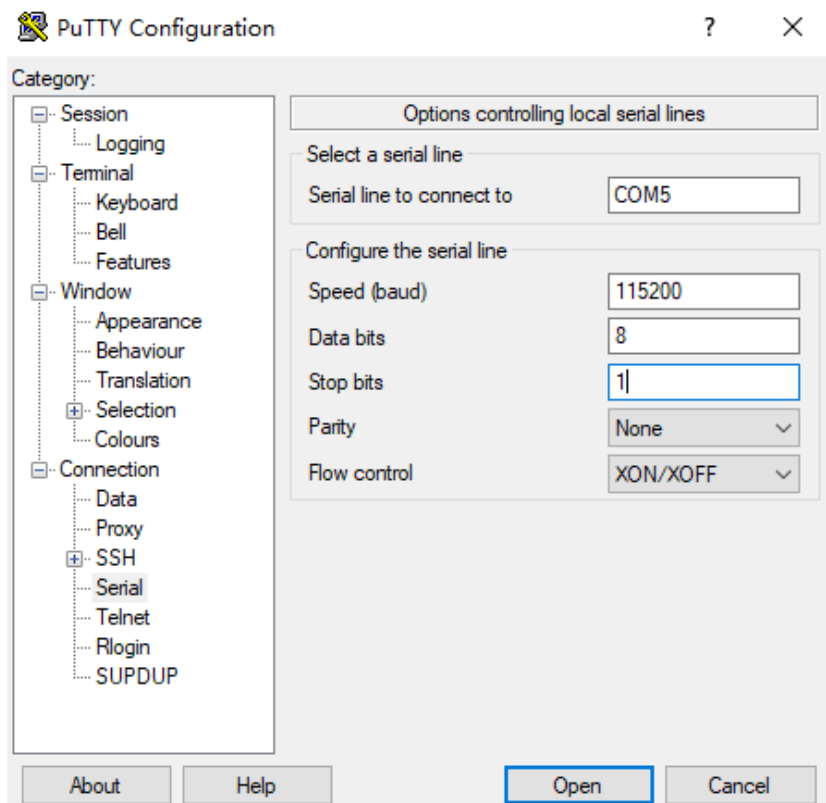
2.12.4.4 Steps to Run

- 1) Import the RT_Thread_AMP project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) Open the console tool on your personal computer (**PuTTY** or **SecureCRT**).

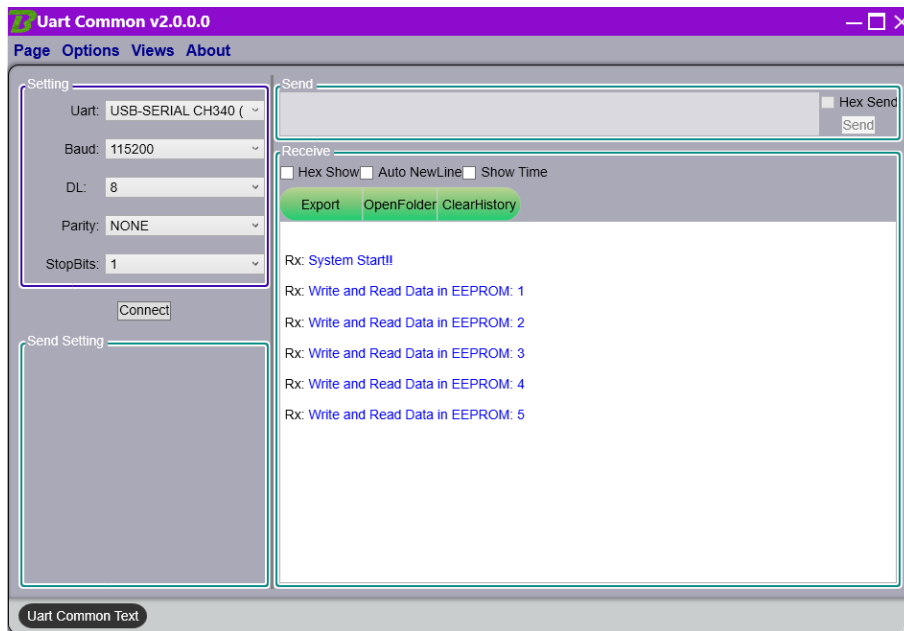


- 3) Console.

Enter help on the console, all implemented command will be listed.

- 4) The LEDs on the evaluation board will start to toggle.

- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
 - To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).
- 3) Normally, the MCU will write data to EEPROM and read the written data, and then upload the data to the PC via UART.



2.14 FCPIT

There are three FCPIT demos: FCPIT_TimeCounter, FCPIT_ChainModeCounter, and FCPIT_Trgsel.

2.14.1 FCPIT_TimeCounter

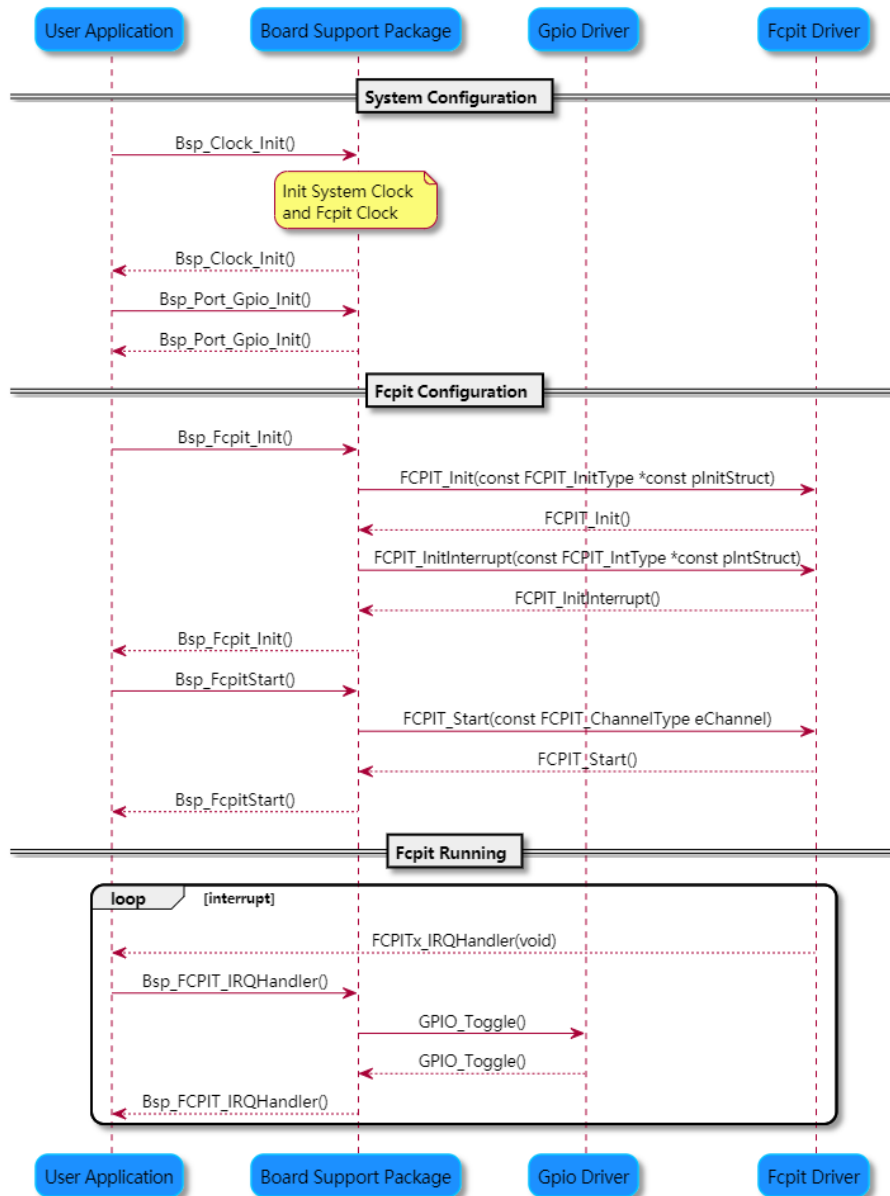
2.14.1.1 Description

This demo shows how to use one channel of the FCPIT to generate 1-second period interrupts through the SDK.

This demo project will toggle LED every second.

The sequence diagram of the demo is as follows:

FC7300F8MDQ SDK Quick Start



2.14.1.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 Personal computer
- 1 J-Link debugger

2.14.1.3 MCU Ports

N/A.

2.14.1.4 Steps to Run

2.14.1.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

2.14.1.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\FCPIT\FCPIT_TimeCounter_Project\IAR_Project, and double-click the **FCPIT_TimeCounter** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.14.1.4.3 For Keil

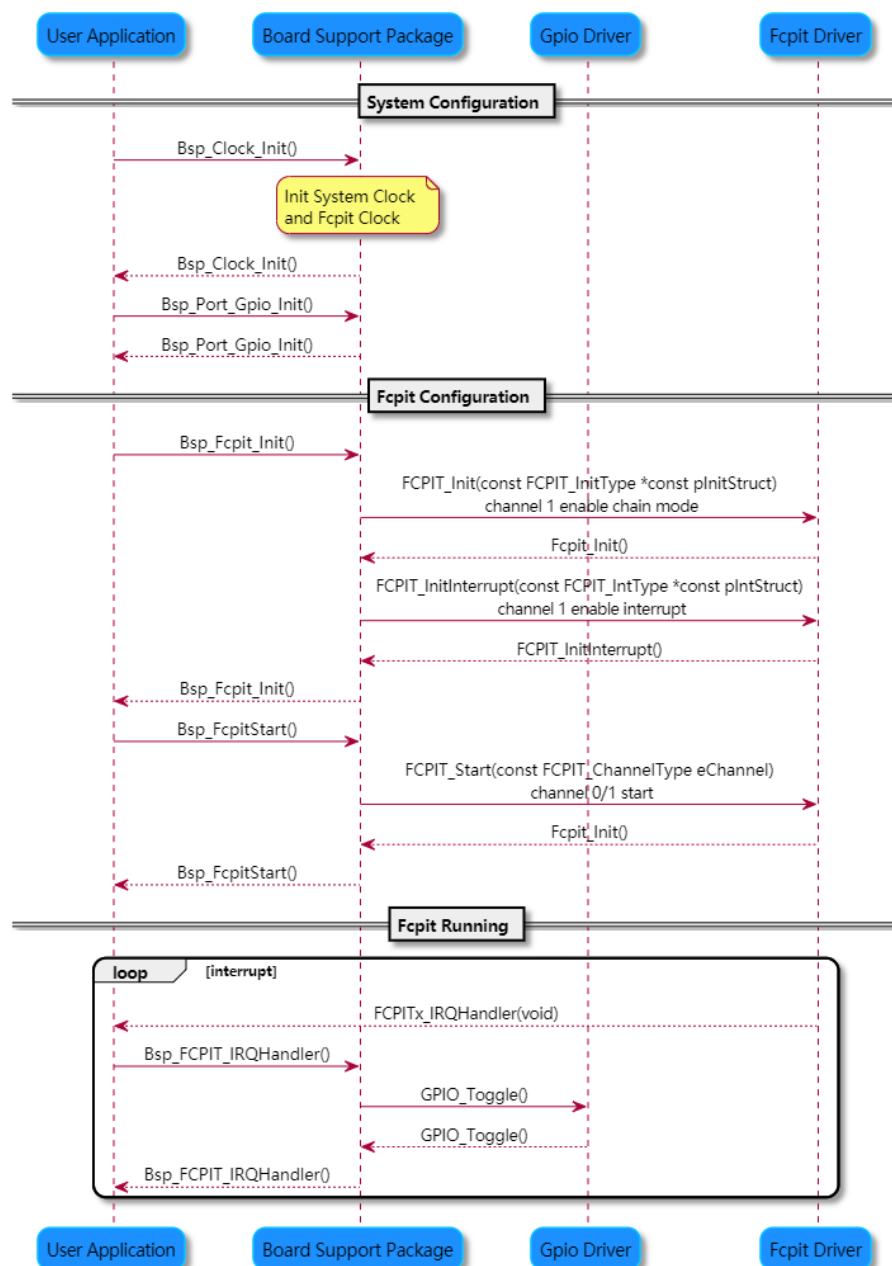
Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.14.2 FCPIT_ChainModeCounter

2.14.2.1 Description

This demo shows how to use two channels of the FCPIT to generate 2-second period interrupts through the SDK. The two channels of the FCPIT are cascaded in ChainMode. This demo project will toggle LED every 2 seconds.

The sequence diagram of the demo is as follows:



2.14.2.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 Personal computer
- 1 J-Link debugger

2.14.2.3 MCU Ports

N/A.

2.14.2.4 Steps to Run

2.14.2.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

2.14.2.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\FCPIT\FCPIT_ChainModeCounter_Project\IAR_Project, and double-click the **FCPIT_ChainModeCounter** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.14.2.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.14.3 FCPIT_Trgsel

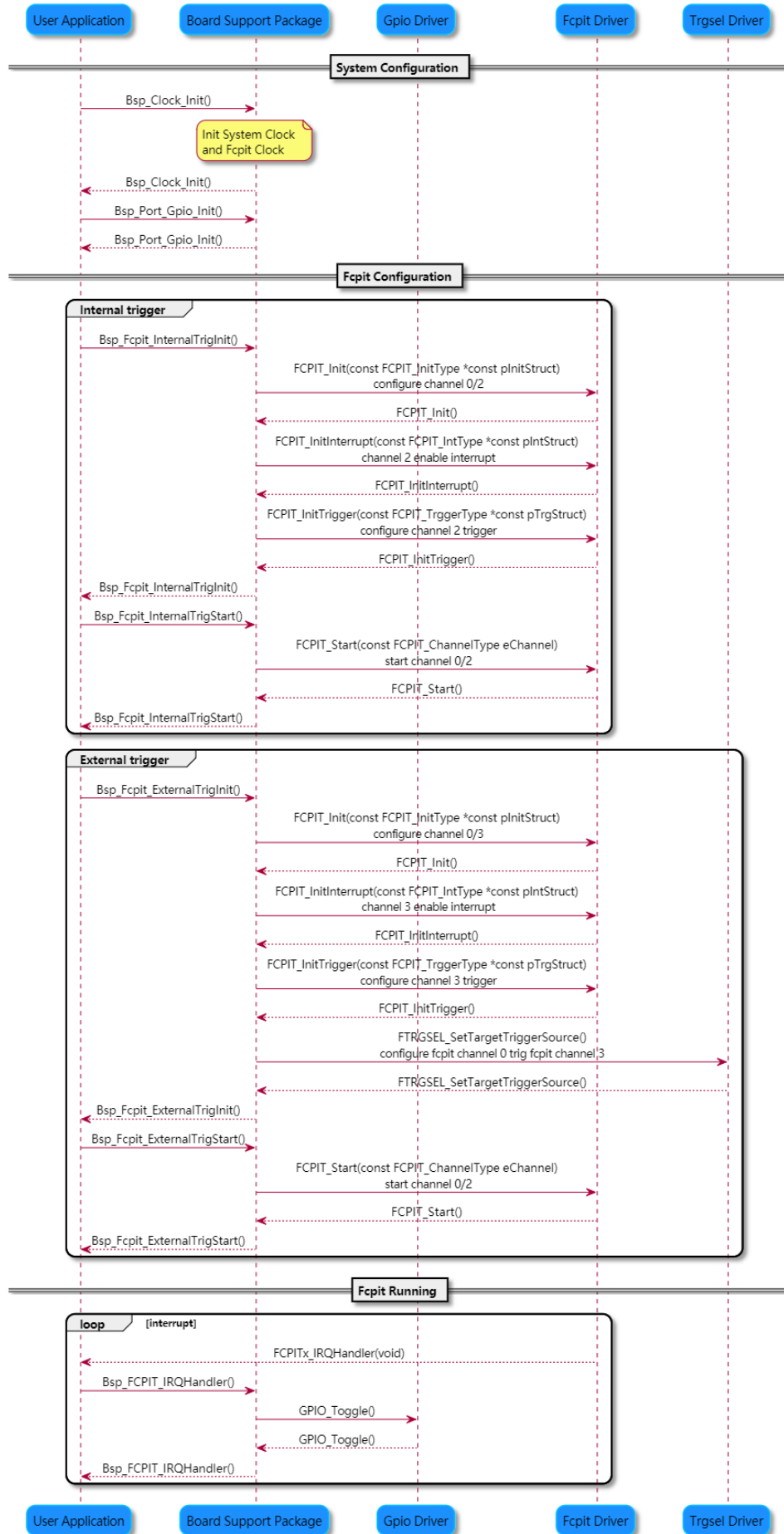
2.14.3.1 Description

This demo shows how to use two channels of the FCPIT to generate 2-second period interrupts through the SDK. The two channels of the FCPIT in external trigger or internal trigger.

This demo project will toggle LED every 2 seconds.

The sequence diagram of the demo is as follows:

FC7300F8MDQ SDK Quick Start



2.14.3.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 Personal computer
- 1 J-Link debugger

2.14.3.3 MCU Ports

N/A.

2.14.3.4 Steps to Run

2.14.3.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

2.14.3.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\FCPIT\FCPIT_Trgsel_Project\IAR_Project, and double-click the **FCPIT_Trgsel** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.14.3.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.15 FCSMU

2.15.1 Description

This demo is just for FCSMU test.

2.15.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.15.3 MCU Ports

N/A

2.15.4 Steps to Run

The steps to run the example are as follows:

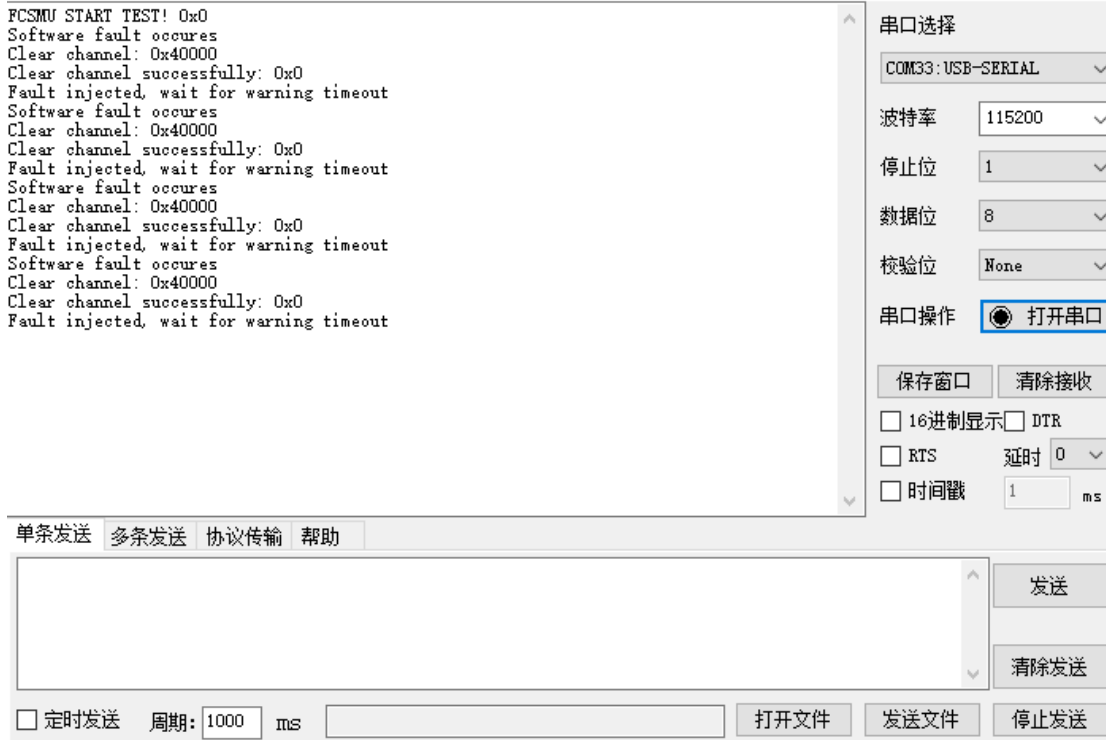
- 1) Plug in the USB Type-C cable and open the up-machine (115200/8/N/1).

- 2) Download the FCSMU_Demo program to the MCU by JTAG.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 3) Normally, LED will blink when warning interrupt is handled.
- 4) MCU will print texts as below.



2.16 FCSPI

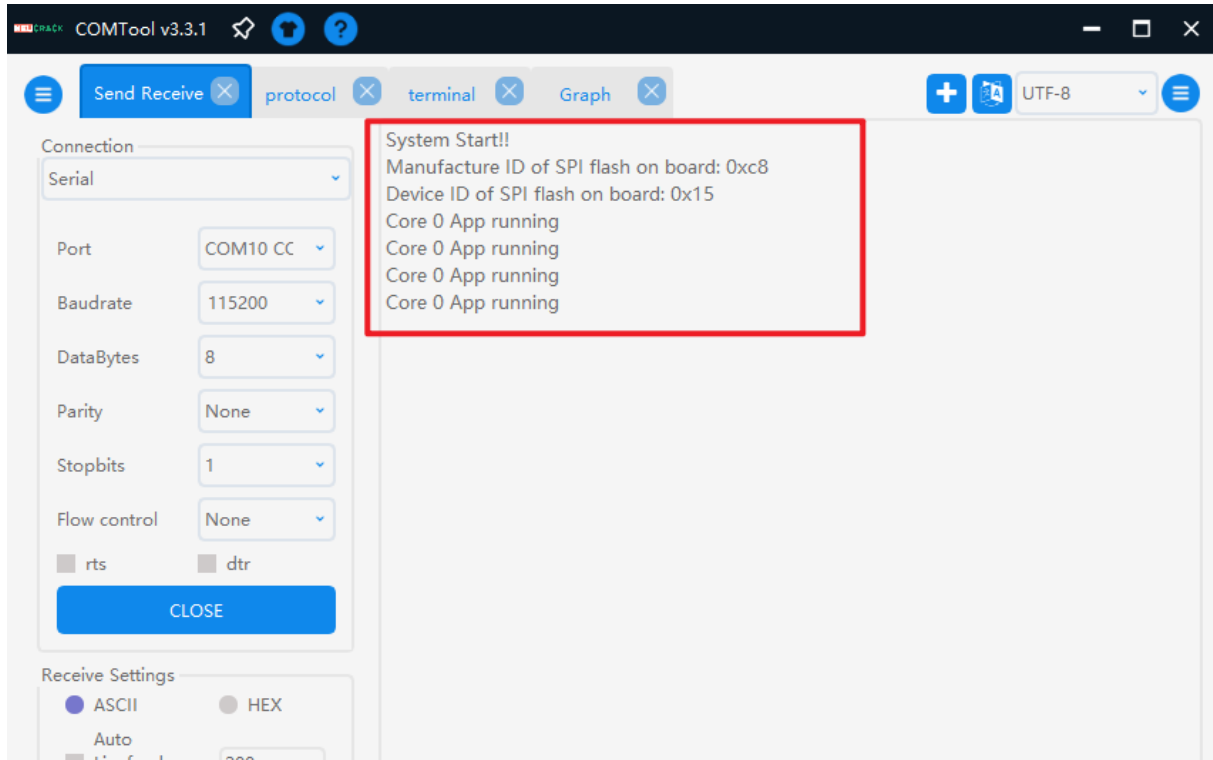
2.16.1 FCSPI_Master_Polling

2.16.1.1 Description

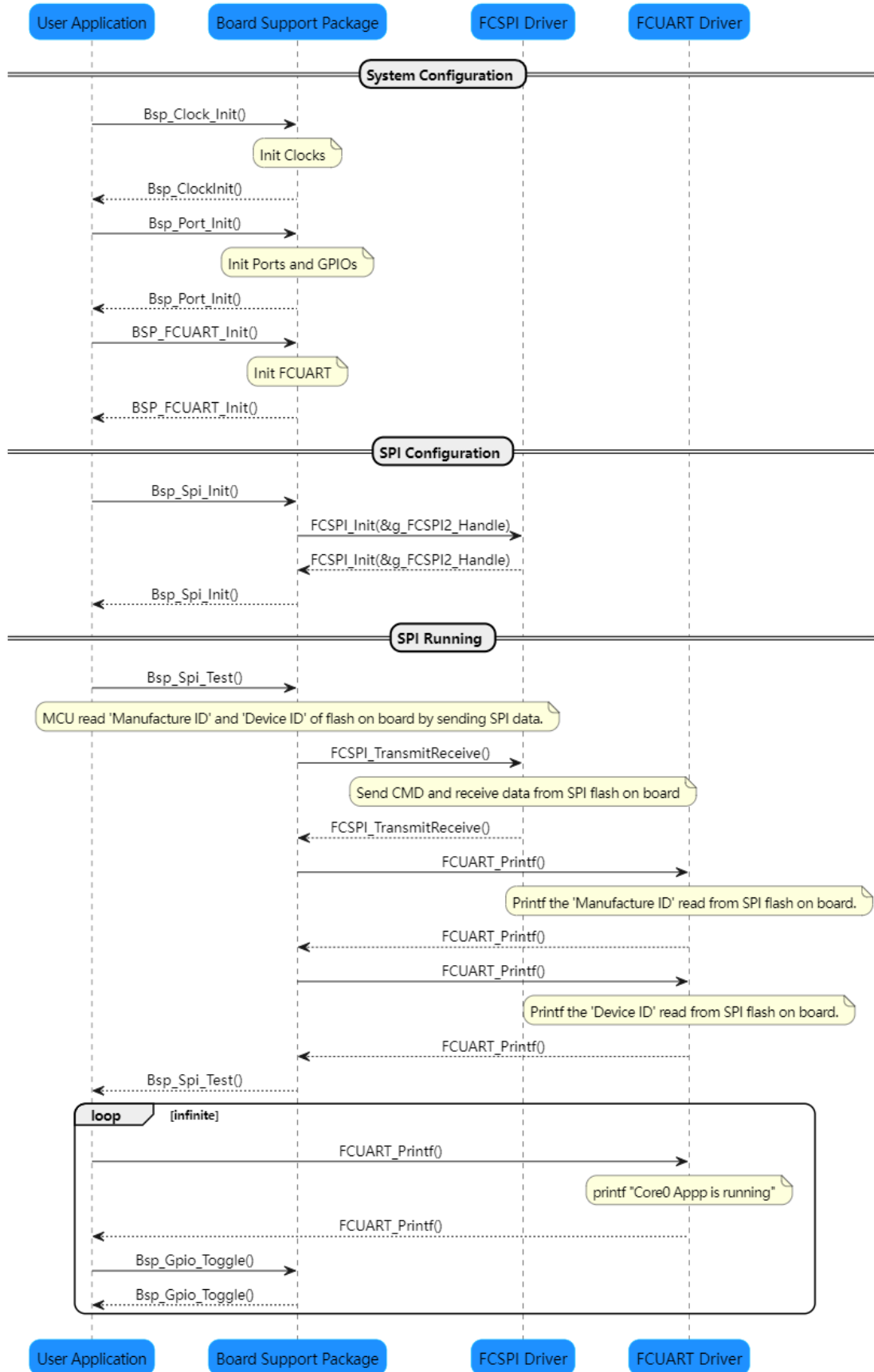
The purpose of the demo is to show you the usage of api "FCSPI_TransmitReceive", which is a blocking api that no interrupt participates. The demo configures SPI2 as a Master with GD25Q32(SPI flash) as its slave device. SPI2 sends CMD data to GD25Q32 and reads the data returned by GD25Q32 than stores it in Rx buffer. The process of sending and receiving data at the same time is accomplished by calling this one api "FCSPI_TransmitReceive".

The sequence diagram of the demo is as follows:

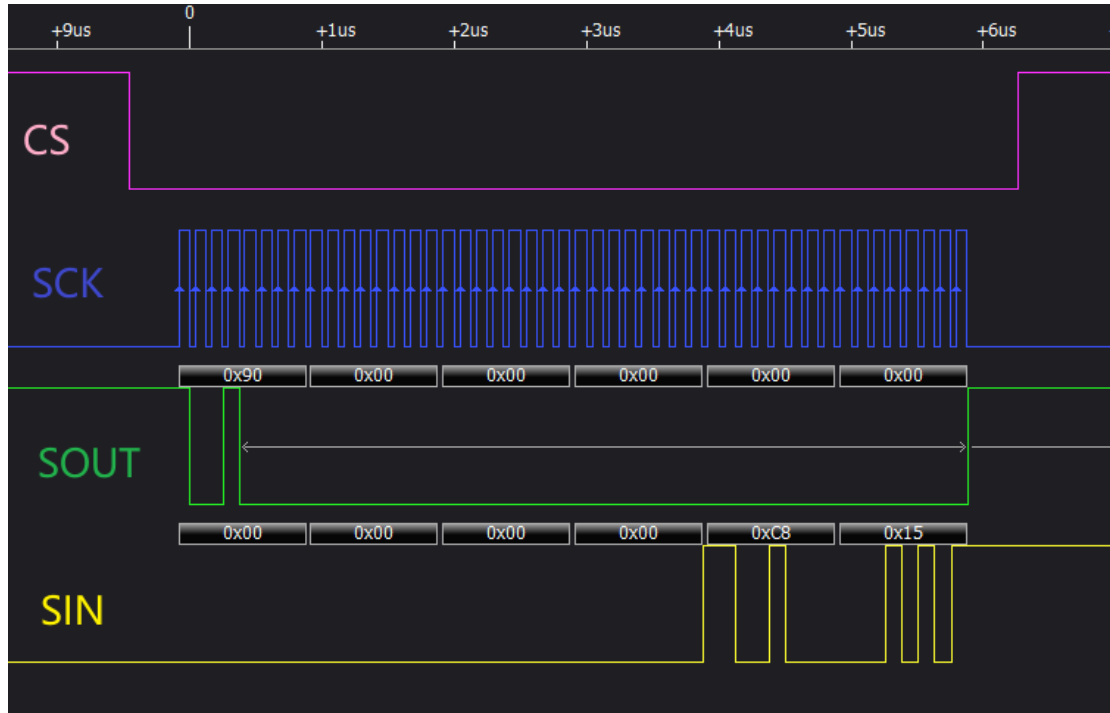
- 1) Bsp_Clock_Init : Initialize the system and peripheral clock(FCSPI2、FCUART1).
- 2) Bsp_Port_Init : Initialize port pins used by FCSPI2 and FCUART1.
- 3) BSP_FCUART_Init: Initialize FCUART1.
- 4) Bsp_Spi_Init: Initialize FCSPI2.
- 5) Bsp_Spi_Test: Start SPI transfer, after the "FCSPI_TransmitReceive" return, you can check the Manufacture ID and Device ID of GD25Q32 in array "RxBuffer". And these two ID will be printed out by FCUART too.



The sequence diagram of the demo is as follows:



The actual waveform of this demo is shown in the figure below:



2.16.1.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.16.1.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
PTA26: MUX = ALT1, GPIO	LED1
PTC3: MUX = ALT1, GPIO	WP#(refer to GD25Q32 data sheet)
PTC7: MUX = ALT1, GPIO	HOLD#(refer to GD25Q32 data sheet)
PTB23: MUX = ALT3, FCSPI2_PCS0	Chip select pin
PTC16: MUX = ALT3, FCSPI2_SOUT	Serial output pin
PTC18: MUX = ALT3, FCSPI2_SIN	Serial input pin
PTB31: MUX = ALT3, FCSPI2_SCK	Serial clock pin
PTA19: MUX = ALT3, FCUART1_RX	UART receive pin
PTA18: MUX = ALT3, FCUART1_TX	UART transmit pin

2.16.1.4 Steps to Run

The steps to run the example are as follows:

- 1) Plug in the USB Type-C cable and open the up-machine (115200/8/N/1).
- 2) Download the demo program to the MCU.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

2.16.2 FCSPI_Master_Interrupt

2.16.2.1 Description

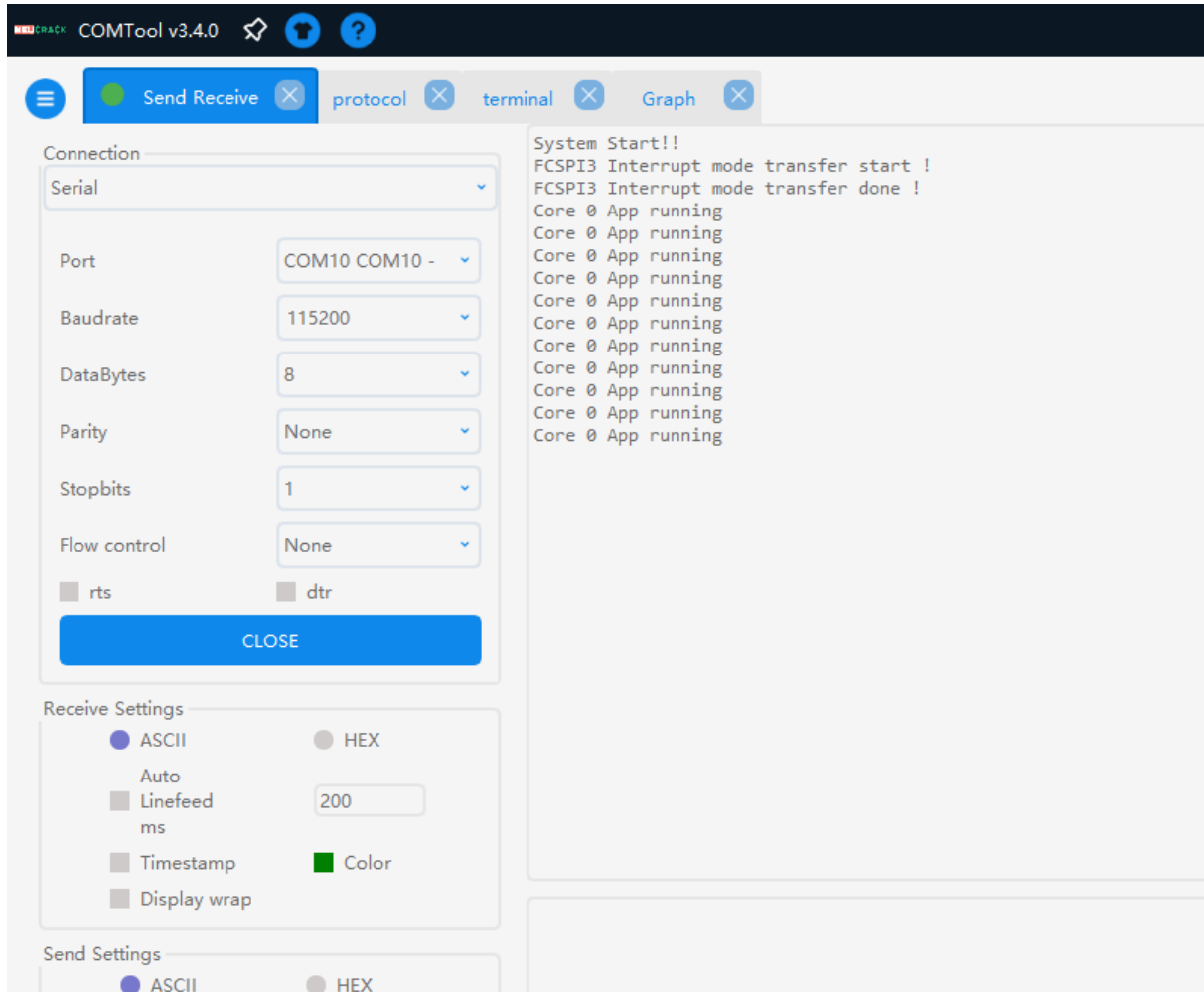
The purpose of the demo is to show you how to start SPI transfer in interrupt mode, which is a kind of non-blocking mode.

The demo call "FCSPI_TransmitReceive_IT()" to start a non-blocking transfer in interrupt mode. When "FCSPI_TransmitReceive_IT()" executes end and return, does not indicate that the SPI transfer completes. User should check the member ".tStatus.state" of FCSPI_Handle struct to determine if the transfer completes.

In FCSPI_TransmitReceive_IT(), the "TansferStartNotification()" callback function will be called to indicate SPI transfer will start soon. After calling "FCSPI_TransmitReceive_IT()", SPI interrupt will be triggered. In the ISR of SPI interrupt, driver function "FCSPI_In_IRQHandler()" shall be called. And in "FCSPI_In_IRQHandler()", Tx data will be written into TXFIFO, Rx data will be read from RXFIFO. During the entire transmission, interrupts are continuously triggered. The more data is transmitted, the more interrupts are triggered. In the last ISR, "FCSPI_In_IRQHandler()" will call the "TansferEndNotification()" to indicate the transfer has done.

The sequence diagram of the demo is as follows:

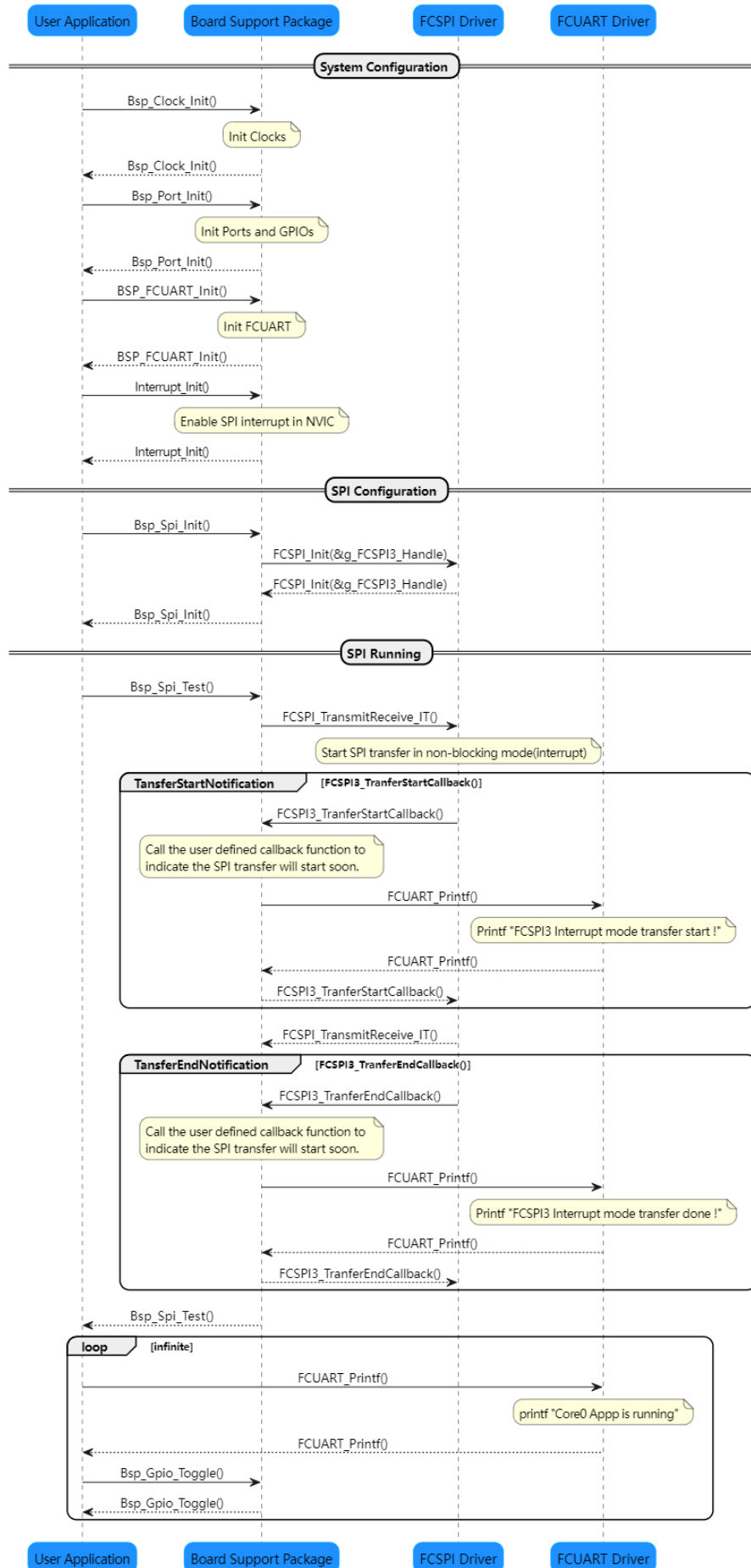
- 1) Bsp_Clock_Init : Initialize the system and peripheral clock(FCSPI3、FCUART1).
- 2) Bsp_Port_Init : Initialize port pins used by FCSPI3 and FCUART1.
- 3) BSP_FCUART_Init: Initialize FCUART1.
- 4) Interrup_Init: Enable interrupt of SPI3 in NVIC.
- 5) Bsp_Spi_Init: Initialize FCSPI3.
- 6) Bsp_Spi_Test: Start SPI transfer. In the transfer start callback function, FCUART1 will printf "FCSPI3 Interrupt mode transfer start ! ". In the transfer end callback function, FCUART1 will printf " FCSPI3 Interrupt mode transfer end! ".



SPI3 has a set of pin-outs on the board, you can use a logic analyzer to connect these pins to observe the waveforms.

The sequence diagram of the demo is as follows:

FC7300F8MDQ SDK Quick Start



2.16.2.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger
- PC com tool (UART mode: 115200, 8N1)

2.16.2.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
PTA26: MUX = ALT1, GPIO	LED1
PTD31: MUX = ALT1, GPIO	LED2
PTA14: MUX = ALT1, GPIO	LED3
PTG7: MUX = ALT4, FCSPI3_PCS0	Chip select pin
PTG6: MUX = ALT4, FCSPI3_SOUT	Serial output pin
PTF8: MUX = ALT4, FCSPI3_SIN	Serial input pin
PTH21: MUX = ALT4, FCSPI3_SCK	Serial clock pin
PTA19: MUX = ALT3, FCUART1_RX	UART receive pin
PTA18: MUX = ALT3, FCUART1_TX	UART transmit pin

2.16.2.4 Steps to Run

Same as FCSPI_Master_Polling.

2.16.3 FCSPI_Master_DMA

2.16.3.1 Description

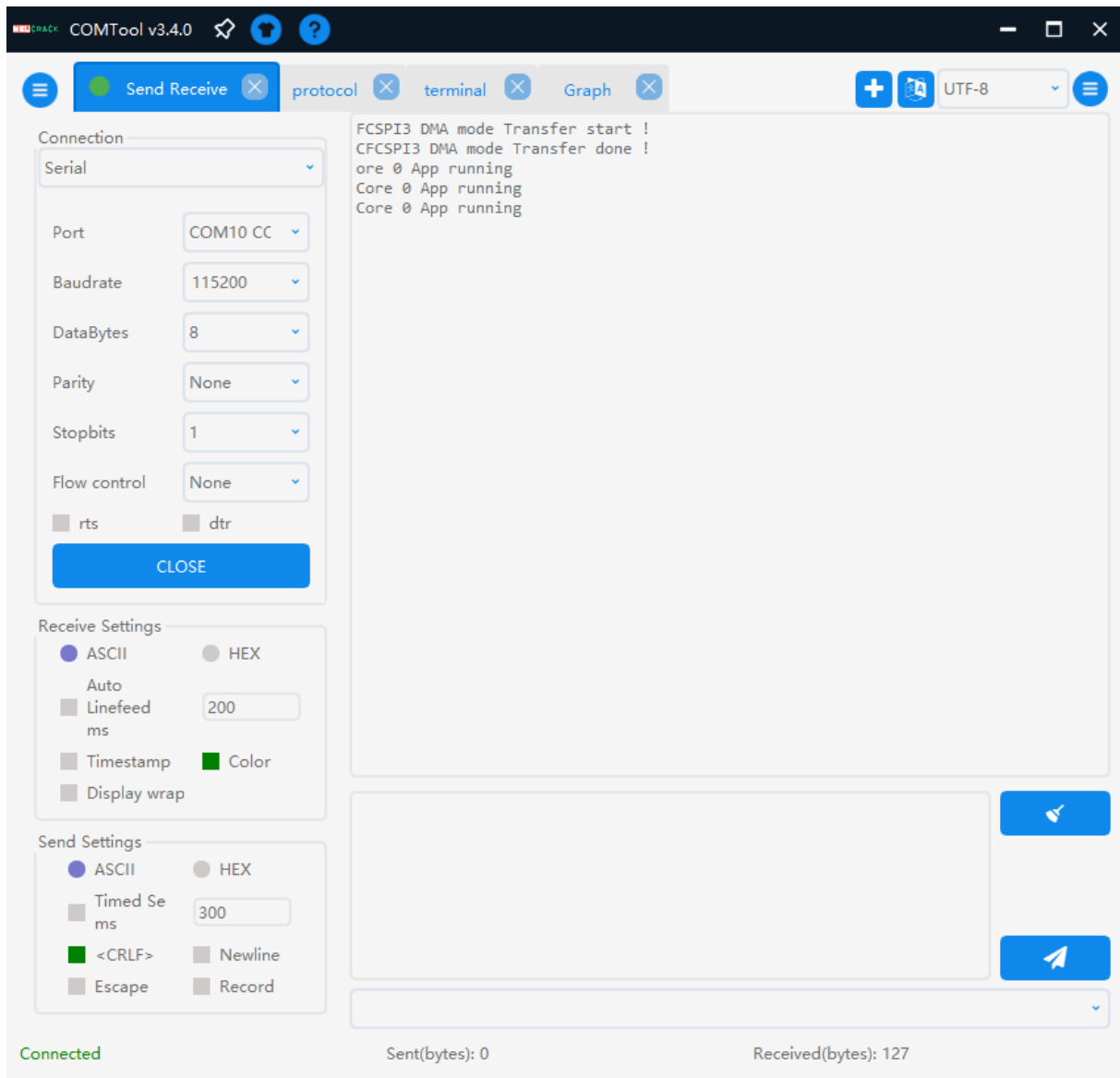
This demo is to show how to use another non-blocking mode of SPI: DMA mode.

In this mode, after the user configures the DMA well and correctly, once the "FCSPI_TransmitReceive_DMA" is called, the DMA will automatically carry the data from the user's TxBuffer to the SPI's TXFIFO, and carry the received data, from the SPI's RXFIFO to the user's RxBuffer without the participation of MCU.

In "FCSPI_TransmitReceive_DMA()", the callback function "TansferStartNotification()" will be called to indicate SPI transfer will start soon. After the DMA channel used to carry all the Rx data into RX buffer done its work, the interrupt of this DMA Rx channel will be triggered. In the ISR, callback function "FCSPIIn_DmaDoneCallback_Rx()" will be called. And "FCSPIIn_DmaDoneCallback_Rx()" will call the "TansferEndNotification()" to indicate the transfer has done.

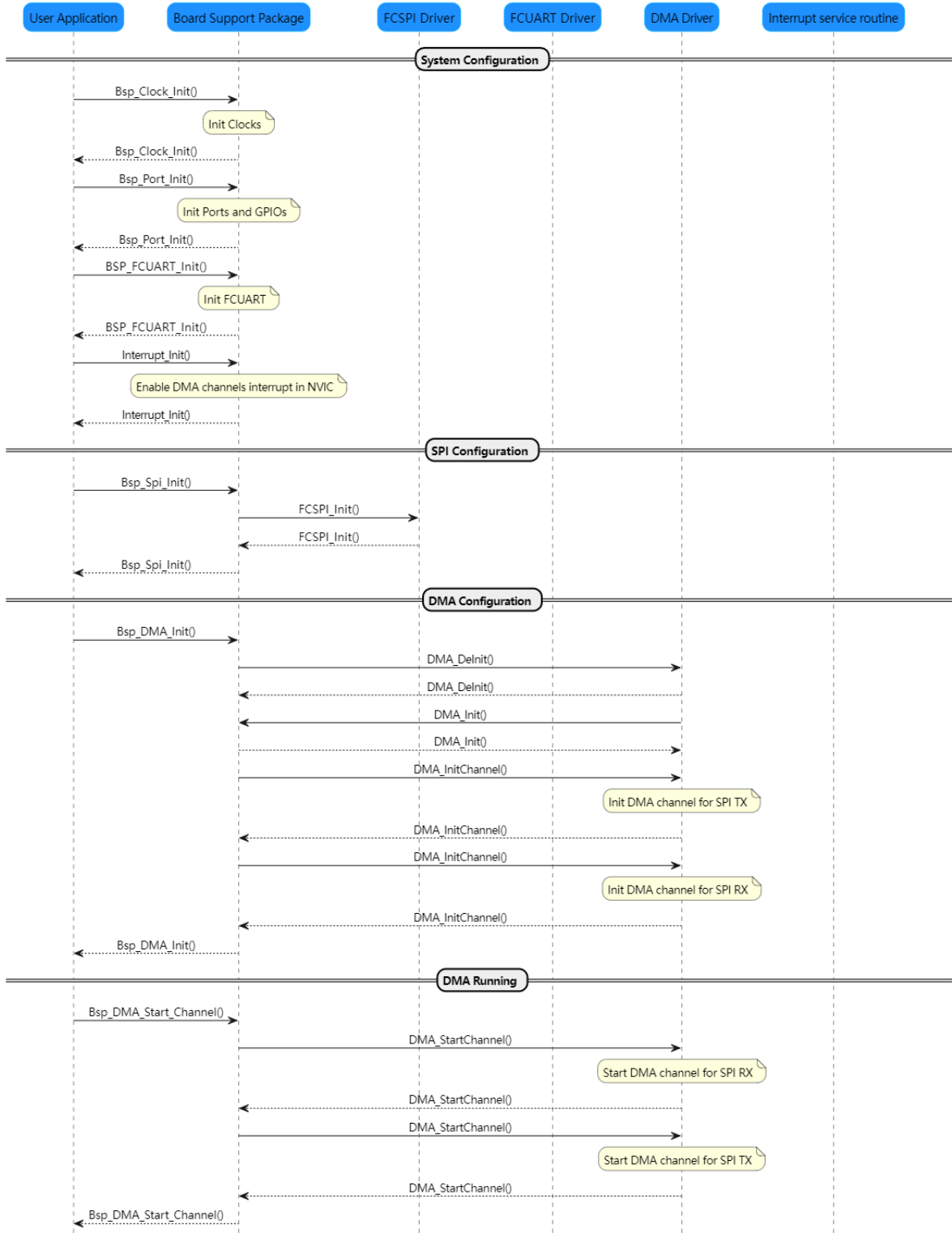
The sequence diagram of the demo is as follows:

- 1) Bsp_Clock_Init : Initialize the system and peripheral clock(FCSPI3 / DMA0 / DMAMUX0 / FCUART1).
- 2) Bsp_Port_Init : Initialize port pins used by FCSPI3 and FCUART1.
- 3) BSP_FCUART_Init: Initialize FCUART1.
- 4) Interrup_Init: Enable interrupt of DMA channels in NVIC.
- 5) Bsp_Spi_Init: Initialize FCSPI3.
- 6) Bsp_DMA_Init: Initialize DMA instance and DMA channels.
- 7) Bsp_Spi_Test: Start SPI transfer. In the transfer start callback function, FCUART1 will printf "FCSPI3 DMA mode transfer start !". In the transfer end callback function, FCUART1 will printf "FCSPI3 DMA mode transfer end!" .



The sequence diagram of the demo is as follows:

FC7300F8MDQ SDK Quick Start



[illegible]

2.16.3.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger
- PC com tool (UART mode: 115200, 8N1)

2.16.3.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
PTA26: MUX = ALT1, GPIO	LED1
PTD31: MUX = ALT1, GPIO	LED2
PTA14: MUX = ALT1, GPIO	LED3
PTG7: MUX = ALT4, FCSPI3_PCS0	Chip select pin
PTG6: MUX = ALT4, FCSPI3_SOUT	Serial output pin
PTF8: MUX = ALT4, FCSPI3_SIN	Serial input pin
PTH21: MUX = ALT4, FCSPI3_SCK	Serial clock pin
PTA19: MUX = ALT3, FCUART1_RX	UART receive pin
PTA18: MUX = ALT3, FCUART1_TX	UART transmit pin

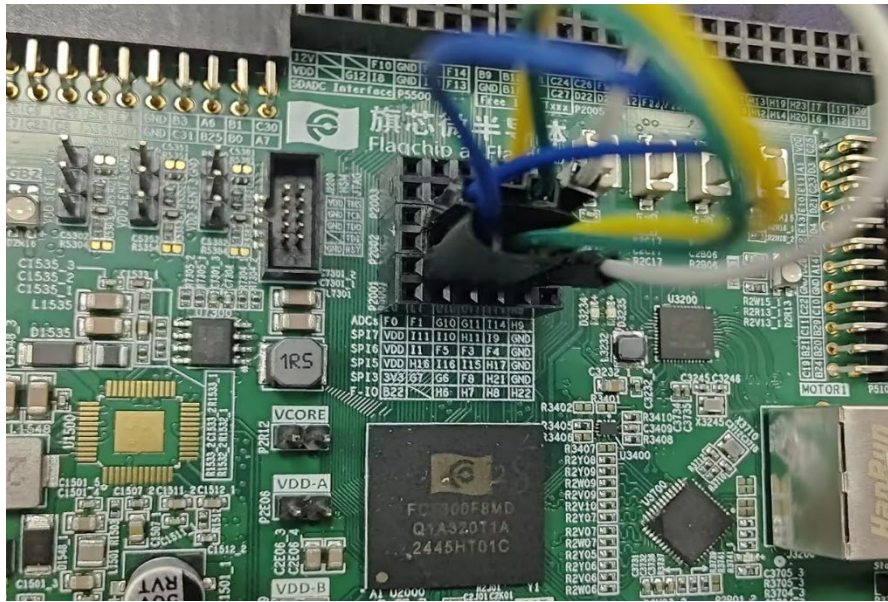
2.16.3.4 Steps to Run

Same as FCSPI_Master_Polling

2.16.4 FCSPI_Slave_Interrupt

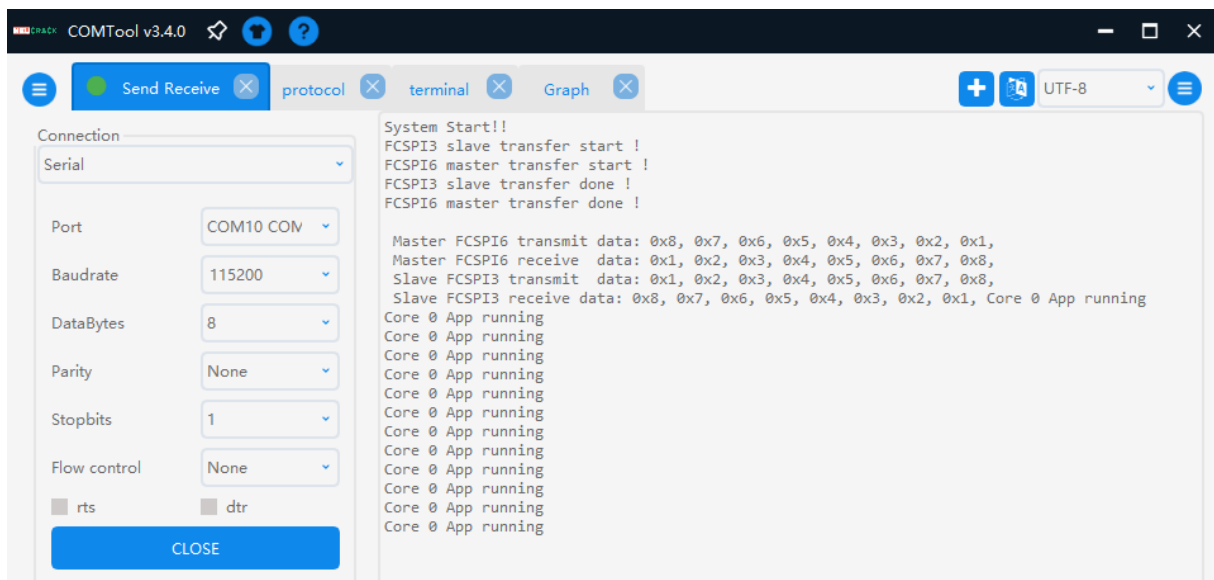
2.16.4.1 Description

This demo is to show how to use SPI in slave mode. In this demo, FCSPI3 is configured to slave mode and FCSPI6 is configured to master mode. Users need to refer to the schematic diagram and connect the pins of SPI3 and SPI6 on the board together, as shown in the figure below:

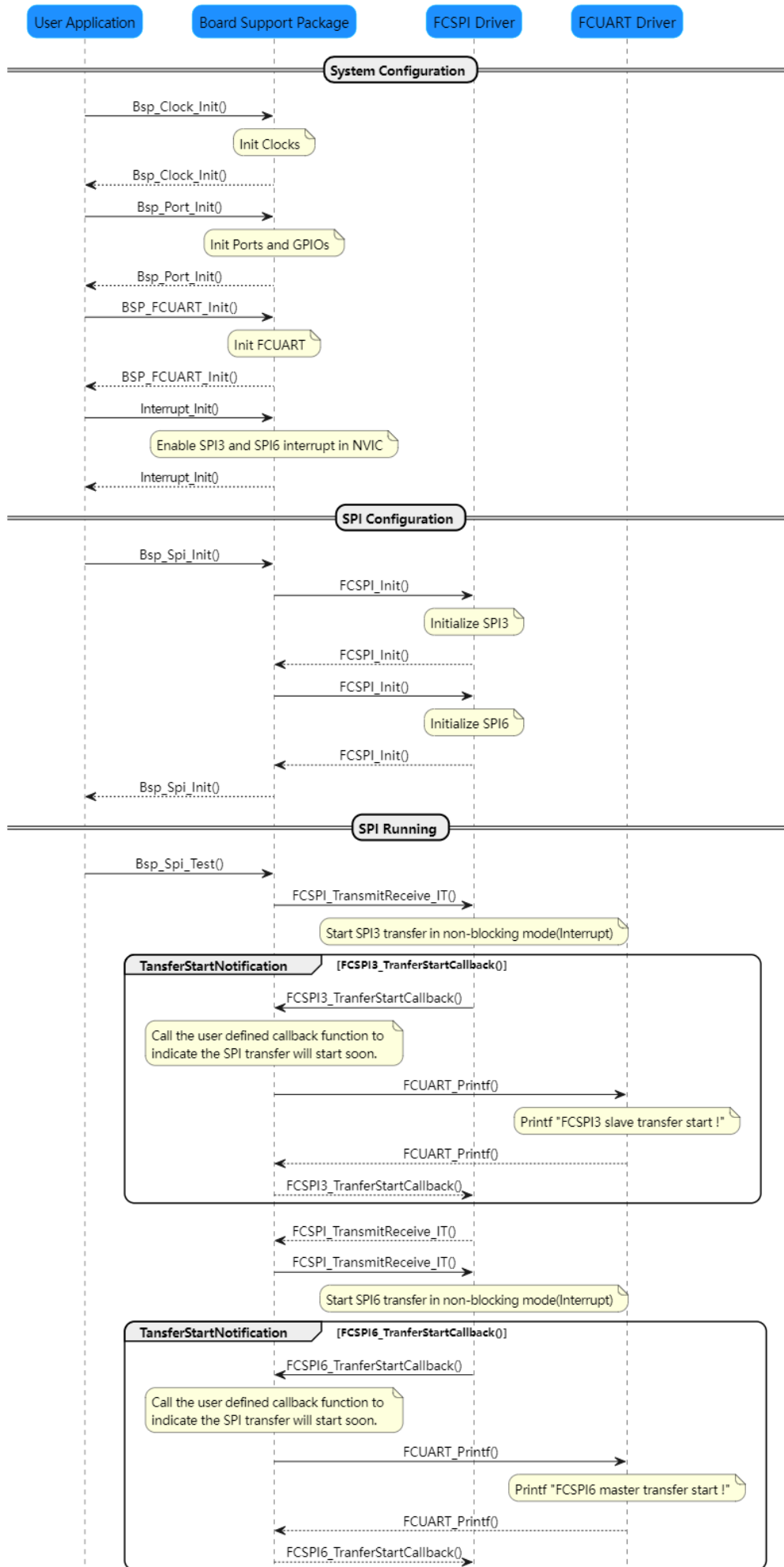


The sequence diagram of the demo is as follows:

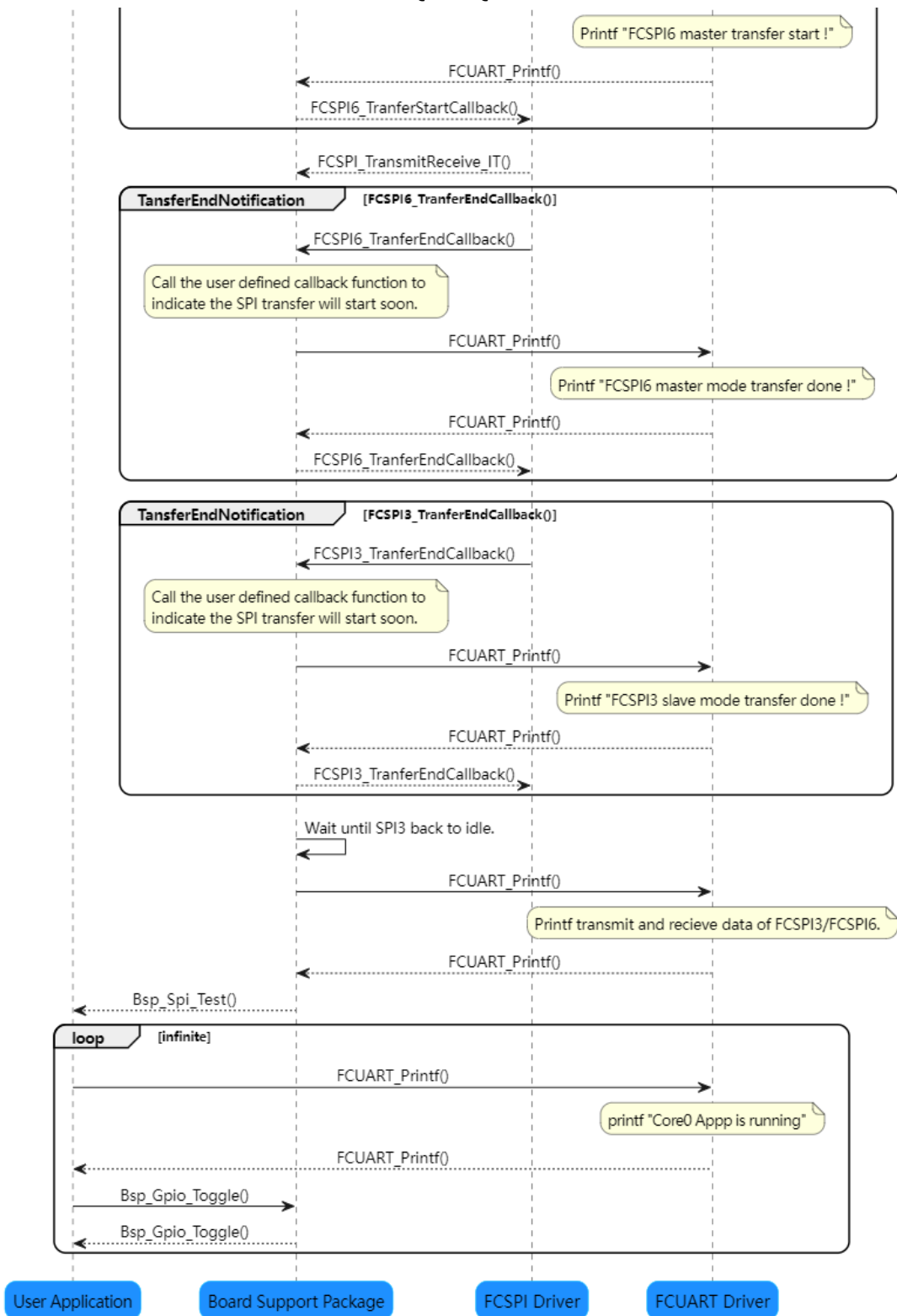
- 1) Bsp_Clock_Init : Initialize the system and peripheral clock(FCSPI3 / FCSPI6 / FCUART1).
- 2) Bsp_Port_Init : Initialize port pins used by FCSPI3/FCSPI6 and FCUART1.
- 3) BSP_FCUART_Init: Initialize FCUART1.
- 4) Interrup_Init: Enable interrupt of SPI3 and SPI6 in NVIC.
- 5) Bsp_Spi_Init: Initialize FCSPI3 and FCSPI6 with configuration user defined.
- 6) Bsp_Spi_Test: call 'FCSPI_TransmitReceive_IT' twice to start slave transfer of FCSPI3 and master transfer of FCSPI6 in interrupt mode. Wait until FCSPI3 back to idle state, than printf all the transmit/receive data of FCSPI3 and FCSPI6. Users can determine whether the transmission is success by checking whether the received data of FCSPI3 and FCSPI6 is the send data of each other in the printed log.



The sequence diagram of the demo is as follows:



FC7300F8MDQ SDK Quick Start



2.16.4.2 Prerequisites

Same as FCSPI_Master_Polling

2.16.4.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
PTA26: MUX = ALT1, GPIO	LED1
PTD31: MUX = ALT1, GPIO	LED2
PTA14: MUX = ALT1, GPIO	LED3
PTG7: MUX = ALT4, FCSPI3_PCS0	Chip select pin
PTG6: MUX = ALT4, FCSPI3_SOUT	Serial output pin
PTF8: MUX = ALT4, FCSPI3_SIN	Serial input pin
PTH21: MUX = ALT4, FCSPI3_SCK	Serial clock pin
PTA19: MUX = ALT3, FCUART1_RX	UART receive pin
PTA18: MUX = ALT3, FCUART1_TX	UART transmit pin

2.16.4.4 Steps to Run

Same as FCSPI_Master_Polling

2.17 FCUART

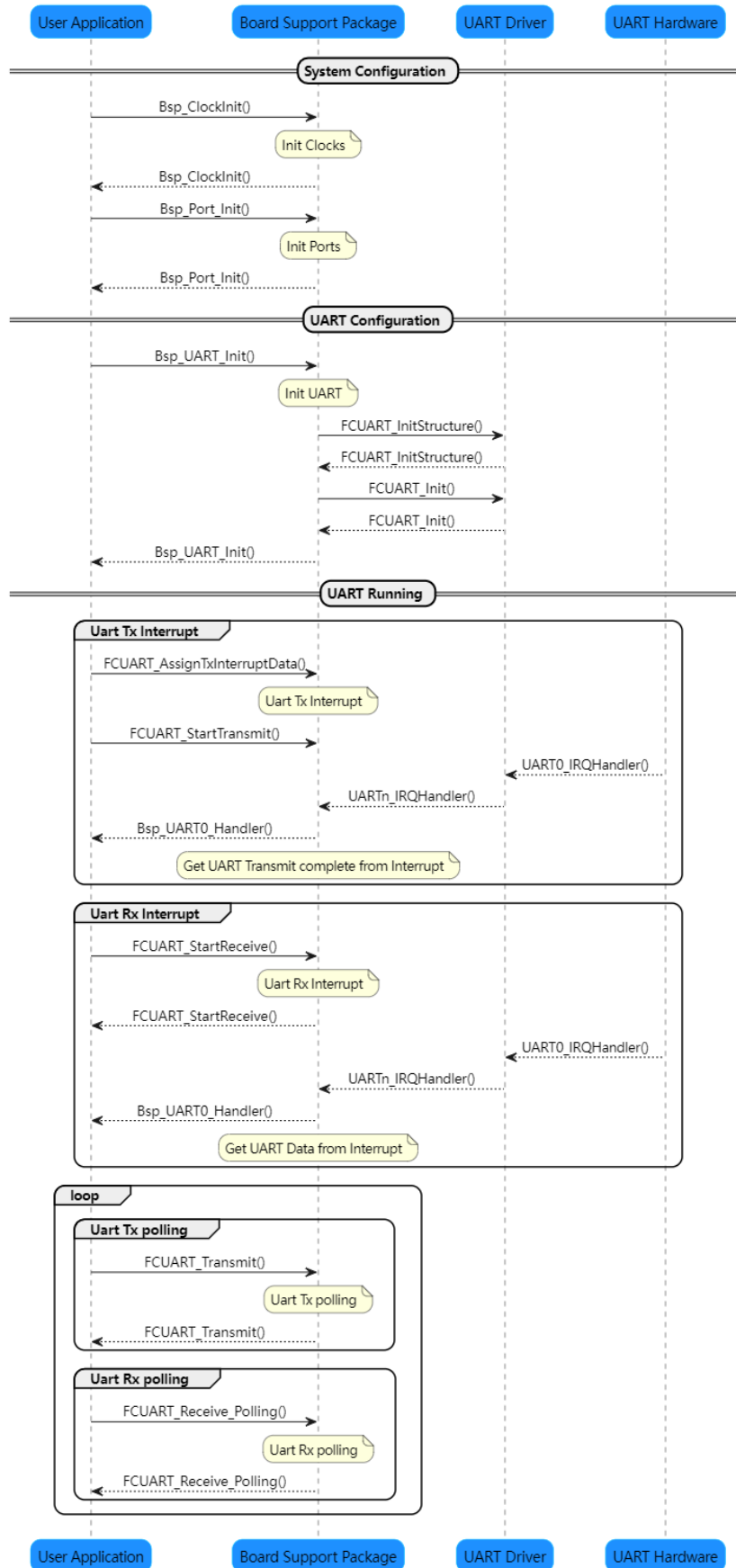
2.17.1 Description

There are six FCUART demos: Uart_DMA, Uart Idle DMA, Uart RxInterrupt, Uart TxInterrupt, Uart_Polling, and Uart Printf.

- **Uart_DMA_Transmit:** Demo for using DMA to transport data from UUART register to RAM buffer.
- **Uart Idle DMA:** Demo for using uart idle interrupt and DMA to transport data from uart register to ram buffer
- **Uart RxInterrupt:** Demo for using interrupt to receive data.
- **Uart TxInterrupt:** Demo for using interrupt to transmit data.
- **Uart_Polling:** Demo for using polling to transfer data.
- **Uart_Printf:** Demo for using UART to redirect the system printf function.

The sequence diagram of the demo is as follows:

FC7300F8MDQ SDK Quick Start



2.17.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable

- 1 USB-2-UART cable
- 1 Personal computer
- 1 J-Link debugger

2.17.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

For other ports, reference to the *FC7300F8MDQ Pinout* attached to *FC7300F8MDQ Reference Manual*.

2.17.4 Steps to Run

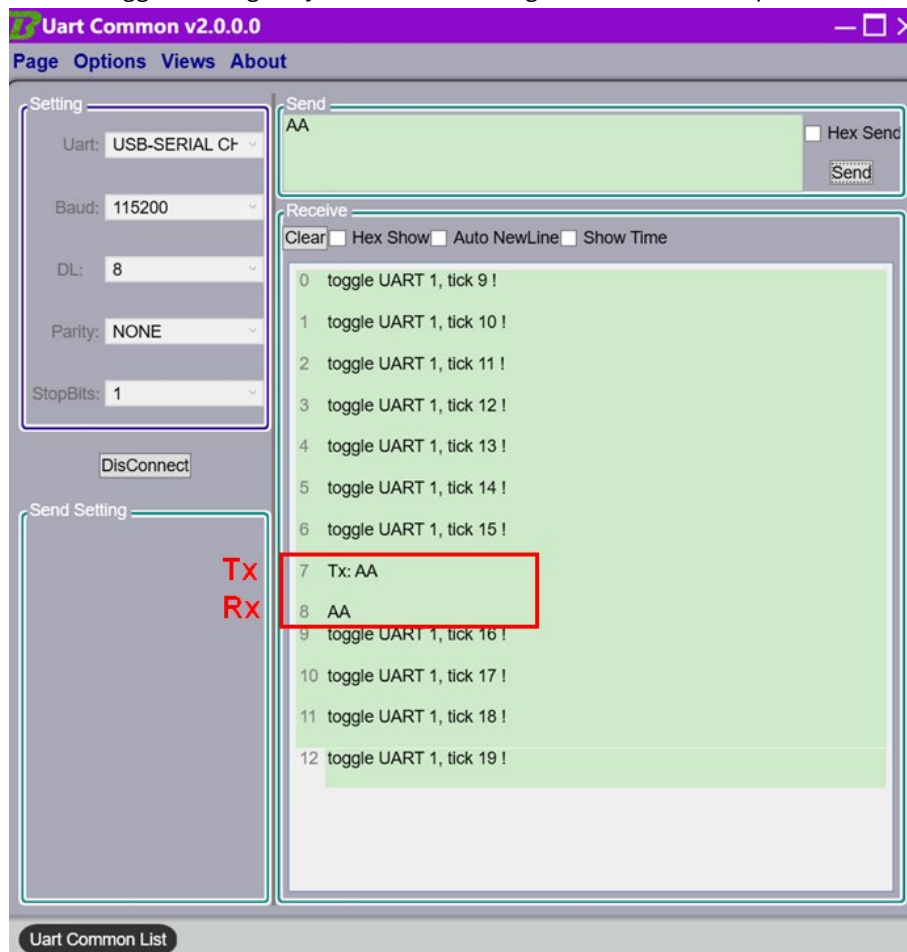
2.17.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: Make sure "**Debug_Flash**" is selected as the active build configuration.

After building and debugging the project, take the following steps:

- 1) Plug in the USB-2-UART cable and open the up-machine (UART1 115200/8/N/1).
- 2) Plug in the USB Type-C cable and connect inner chip CH340 to UART1 pin (UART1 115200/8/N/1).
- 3) MCU will upload the toggle message. If you transmit message to it and it will respond same message.



2.17.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

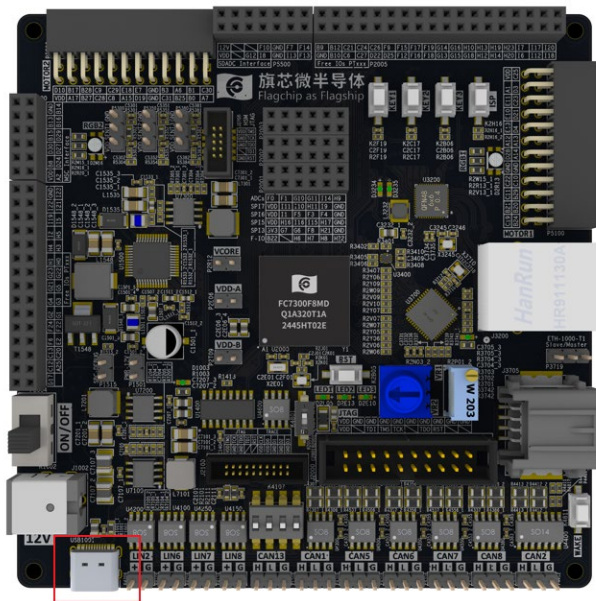
- 1) Open the directory: SDK\Example\UART\ **Uart_DMA_Transmit** (**Uart_Idle_DMA**, **Uart_RxInterrupt**, **Uart_TxInterrupt**, **Uart_Polling**, or **Uart_Printf**)_Project\IAR_Project, and double-click the Uart IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.17.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.18 Flash

There are three Flash demos: Flash_Async_Project, Flash_INT_Project, and Flash_Async_Project.



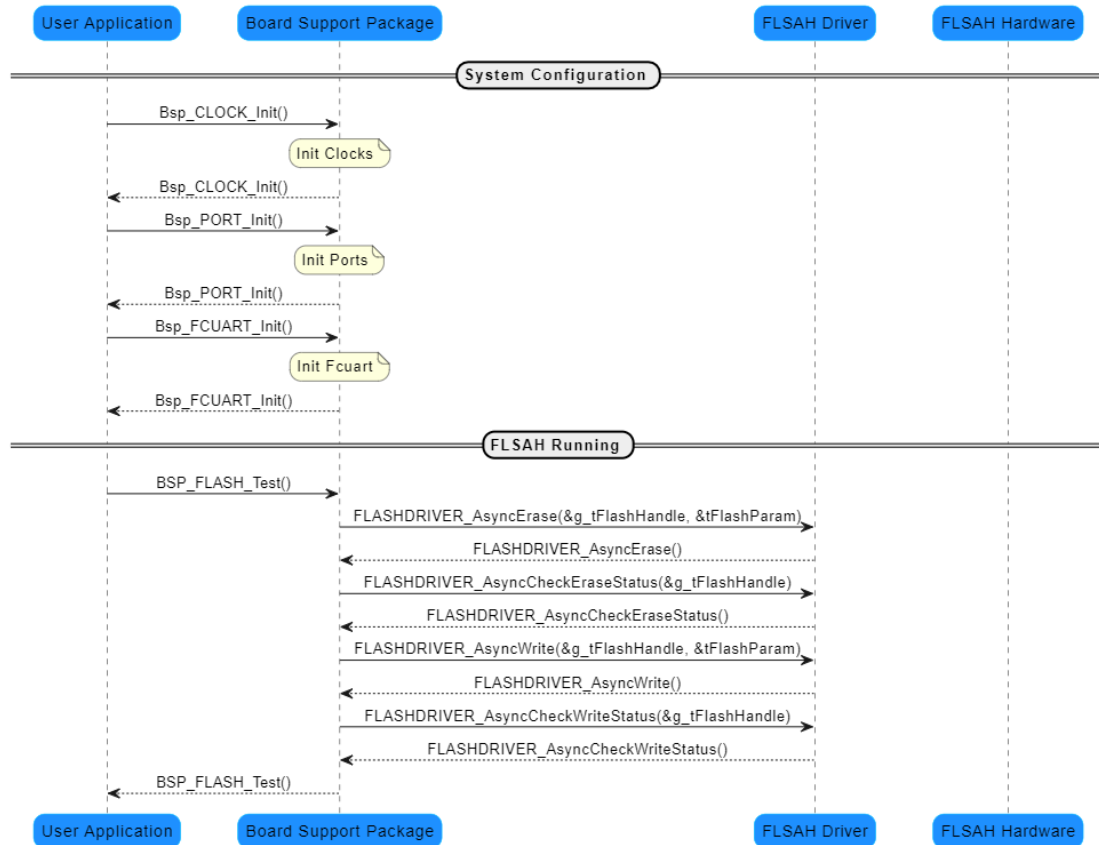
2.18.1 Flash_Async_Project

2.18.1.1 Description

This demo shows the asynchronous erase and program features of Flash.

The sequence diagram of the demo is as follows:

FC7300F8MDQ SDK Quick Start



2.18.1.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V or Micro USB line
- 1 Personal computer
- 1 J-Link debugger

2.18.1.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

2.18.1.4 Steps to Run

The steps to run the example are as follows:

- 1) Import the Flash project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) After building and debugging, the result will be shown as below.

FC7300F8MDQ SDK Quick Start

The screenshot shows the IDE with the main.c file open. The function `BSP_FLASH_Test` is defined, which initializes the flash driver and performs an erase operation. The Memory window on the right shows the address range 0x00FC00 to 0x00FD20, with the value 0x00000000 displayed for the first few addresses.

```

27 }
28
29
30 void BSP_FLASH_Test(void)
31 {
32     FLASHDRIVER_Init();
33     /* erase write function test */
34     Async_Return_t async_status;
35
36     FLASH_DRIVER_ParamType tFlashParam = {.pData = fls_test_buff_w};
37
38     Buff_Init(fls_test_buff_w, TEST_BUFF_LEN);
39     FLASHDRIVER_Init();
40
41     /* Async function test */
42     tFlashParam.u32Address = PFLASH_SIZE - PFLASH_ERASE_SECTOR_SIZE;
43     FLASHDRIVER_AsyncErase(&g_tFlashHandle, &tFlashParam);
44     while (ASYNC_ERR_BUSY == FLASHDRIVER_AsyncCheckEraseStatus(&g_tFlashHandle))
45     {
46         delay(10);
47     }
48     async_status = FLASHDRIVER_AsyncCheckEraseStatus(&g_tFlashHandle);
49     if (ASYNC_ERR_OK == async_status)
50     {
51         tFlashParam.u32Length = TEST_BUFF_LEN;
52         FLASHDRIVER_AsyncWrite(&g_tFlashHandle, &tFlashParam);
53         while (ASYNC_ERR_BUSY == FLASHDRIVER_AsyncCheckWriteStatus(&g_tFlashHandle))
54         {
55             delay(2);
56         }
57         async_status = FLASHDRIVER_AsyncCheckWriteStatus(&g_tFlashHandle);
58         if (ASYNC_ERR_OK == async_status)
59         {
60             /* Success */
61         }
62         else
63         {
64             DEV_ASSERT(0);
65         }
66     }
67 }

```

Address	0 - 3	4 - 7	8 - B	C - F
000FC00	01020304	05060708	090A0B0C	0D0E0F10
000FC10	11121314	15161718	191A1B1C	1D1E1F20
000FC20	21222324	25262728	292A2B2C	2D2E2F30
000FC30	31323334	35363738	393A3B3C	3D3E3F40
000FC40	41424344	45464748	494A4B4C	4D4E4F50
000FC50	51525354	55565758	595A5B5C	5D5E5F60
000FC60	61626364	65666768	696A6B6C	6D6E6F70
000FC70	71727374	75767778	797A7B7C	7D7E7F80
000FC80	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
000FC90	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
000FCA0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
000FCB0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
000FCC0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
000FCD0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
000FCE0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
000FCF0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
000FD00	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
000FD10	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
000FD20	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF

The screenshot shows the IDE with the main.c file open. The function `BSP_FLASH_Test` is defined, which initializes the flash driver and performs an erase operation. The Memory window on the right shows the address range 0x1000000 to 0x1000120, with the value 0x00000000 displayed for the first few addresses.

```

57
58     async_status = FLASHDRIVER_AsyncCheckWriteStatus(&g_tFlashHandle);
59     if (ASYNC_ERR_OK == async_status)
60     {
61         /* Success */
62     }
63     else
64     {
65         DEV_ASSERT(0);
66     }
67 }
68 {
69     DEV_ASSERT(0);
70 }
71
72 #if (DFLASH_SUPPORT == STD_ON)
73
74 /* Async function test */
75 tFlashParam.u32Address = DFLASH_BANK0_START;
76 FLASHDRIVER_AsyncErase(&g_tFlashHandle, &tFlashParam);
77 while (ASYNC_ERR_BUSY == FLASHDRIVER_AsyncCheckEraseStatus(&g_tFlashHandle))
78 {
79     delay(10);
80 }
81 async_status = FLASHDRIVER_AsyncCheckEraseStatus(&g_tFlashHandle);
82 if (ASYNC_ERR_OK == async_status)
83 {
84     tFlashParam.u32Length = TEST_BUFF_LEN;
85     FLASHDRIVER_AsyncWrite(&g_tFlashHandle, &tFlashParam);
86     while (ASYNC_ERR_BUSY == FLASHDRIVER_AsyncCheckWriteStatus(&g_tFlashHandle))
87     {
88         delay(2);
89     }
90     async_status = FLASHDRIVER_AsyncCheckWriteStatus(&g_tFlashHandle);
91     if (ASYNC_ERR_OK == async_status)
92     {
93         /* Success */
94     }
95     else
96     {
97         DEV_ASSERT(0);
98     }
99 }

```

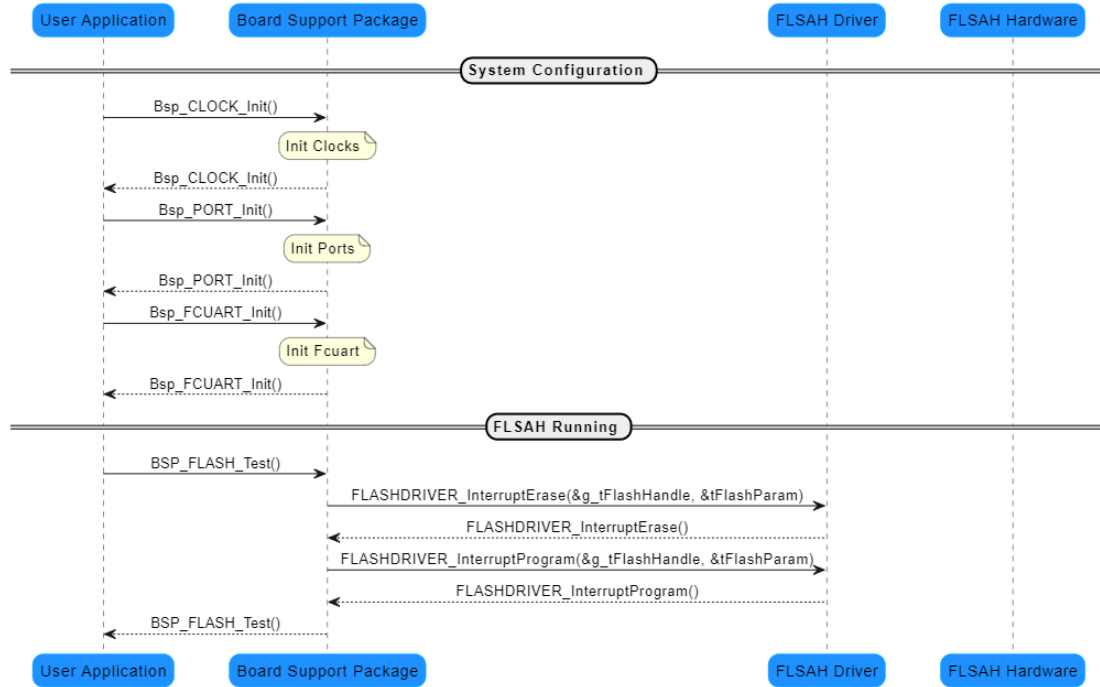
Address	0 - 3	4 - 7	8 - B	C - F
01100000	01020304	05060708	090A0B0C	0D0E0F10
01100010	11121314	15161718	191A1B1C	1D1E1F20
01100020	21222324	25262728	292A2B2C	2D2E2F30
01100030	31323334	35363738	393A3B3C	3D3E3F40
01100040	41424344	45464748	494A4B4C	4D4E4F50
01100050	51525354	55565758	595A5B5C	5D5E5F60
01100060	61626364	65666768	696A6B6C	6D6E6F70
01100070	71727374	75767778	797A7B7C	7D7E7F80
01100080	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
01100090	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
011000A0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
011000B0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
011000C0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
011000D0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
011000E0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
011000F0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
01100100	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
01100110	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
01100120	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF

2.18.2 Flash_INT_Project

2.18.2.1 Description

This demo shows the interrupt erase and program features of Flash.

The sequence diagram of the demo is as follows:



2.18.2.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V or Micro USB line
- 1 Personal computer
- 1 J-Link debugger

2.18.2.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

2.18.2.4 Steps to Run

The steps to run the example are as follows:

- 1) Import the Flash project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) After building and debugging, the result will be shown as below.

The top screenshot shows the main.c file with the following code:

```

5 uint8_t fls_test_buff_w[TEST_BUFF_LEN];
6
7 FLASH_HandleType g_tFlashHandle = {.Instance = FLASH_INSTANCE_0};
8
9 static void Buff_Init(uint8_t *ptr, uint32_t len)
10 {
11     for (uint32_t u32Index = 0U; u32Index < len; u32Index++)
12     {
13         ptr[u32Index] = (uint8_t)(u32Index + 1U);
14     }
15 }
16
17
18 void BSP_FLASH_Test(void)
19 {
20     FLASHDRIVER_Init();
21     /** erase write function test */
22
23     FLASH_DRIVER_ParamType tFlashParam = {.pData = fls_test_buff_w};
24
25     Buff_Init(fls_test_buff_w, TEST_BUFF_LEN);
26
27     /** Async function test */
28     tFlashParam.u32Address = PFLASH_SIZE - 3 * PFLASH_ERASE_SECTOR_SIZE;
29     tFlashParam.u32Length = 3 * PFLASH_ERASE_SECTOR_SIZE;
30     FLASHDRIVER_InterruptErase(&g_tFlashHandle, &tFlashParam);
31
32     while(g_tFlashHandle.tStatus.eFlsStatus != FLS_IDLE);
33     tFlashParam.u32Length = TEST_BUFF_LEN;
34     tFlashParam.u32Address = PFLASH_SIZE - PFLASH_ERASE_SECTOR_SIZE;
35     FLASHDRIVER_InterruptProgram(&g_tFlashHandle, &tFlashParam);
36
37     #if (DFLASH_SUPPORT == STD_ON)
38
39     while(g_tFlashHandle.tStatus.eFlsStatus != FLS_IDLE);
40     tFlashParam.u32Address = DFLASH_ADDR_START;
41     tFlashParam.u32Length = DFLASH_ERASE_SECTOR_SIZE;

```

The bottom screenshot shows the same code with the following changes:

```

4 #define TEST_BUFF_LEN 0x100
5 uint8_t fls_test_buff_w[TEST_BUFF_LEN];
6
7 FLASH_HandleType g_tFlashHandle = {.Instance = FLASH_INSTANCE_0};
8
9 static void Buff_Init(uint8_t *ptr, uint32_t len)
10 {
11     for (uint32_t u32Index = 0U; u32Index < len; u32Index++)
12     {
13         ptr[u32Index] = (uint8_t)(u32Index + 1U);
14     }
15 }
16
17
18 void BSP_FLASH_Test(void)
19 {
20     FLASHDRIVER_Init();
21     /** erase write function test */
22
23     FLASH_DRIVER_ParamType tFlashParam = {.pData = fls_test_buff_w};
24
25     Buff_Init(fls_test_buff_w, TEST_BUFF_LEN);
26
27     /** Async function test */
28     tFlashParam.u32Address = PFLASH_SIZE - 3 * PFLASH_ERASE_SECTOR_SIZE;
29     tFlashParam.u32Length = 3 * PFLASH_ERASE_SECTOR_SIZE;
30     FLASHDRIVER_InterruptErase(&g_tFlashHandle, &tFlashParam);
31
32     while(g_tFlashHandle.tStatus.eFlsStatus != FLS_IDLE);
33     tFlashParam.u32Length = TEST_BUFF_LEN;
34     tFlashParam.u32Address = PFLASH_SIZE - PFLASH_ERASE_SECTOR_SIZE;
35     FLASHDRIVER_InterruptProgram(&g_tFlashHandle, &tFlashParam);
36
37     #if (DFLASH_SUPPORT == STD_ON)
38
39     while(g_tFlashHandle.tStatus.eFlsStatus != FLS_IDLE);
40     tFlashParam.u32Address = DFLASH_ADDR_START;
41     tFlashParam.u32Length = DFLASH_ERASE_SECTOR_SIZE;
42     FLASHDRIVER_InterruptErase(&g_tFlashHandle, &tFlashParam);
43
44     while(g_tFlashHandle.tStatus.eFlsStatus != FLS_IDLE);
45     tFlashParam.u32Length = TEST_BUFF_LEN;
46     tFlashParam.u32Address = DFLASH_ADDR_START;
47     FLASHDRIVER_InterruptProgram(&g_tFlashHandle, &tFlashParam);
48
49     #endif
50 }

```

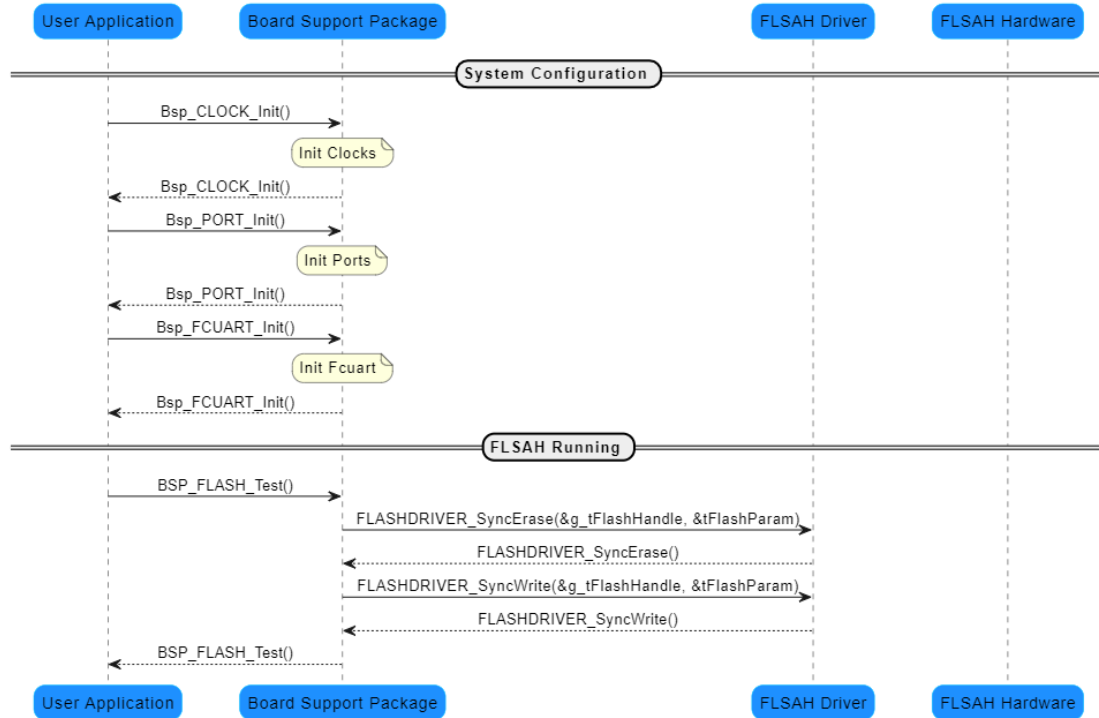
The Memory window in both screenshots shows the 0x100000 address range. The bottom screenshot also shows the 0x1000000 address range.

2.18.3 Flash_Sync_Project

2.18.3.1 Description

This demo project shows the synchronous erase and program features of Flash.

The sequence diagram of the demo is as follows:



2.18.3.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V or Micro USB line
- 1 Personal computer
- 1 J-Link debugger

2.18.3.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

2.18.3.4 Steps to Run

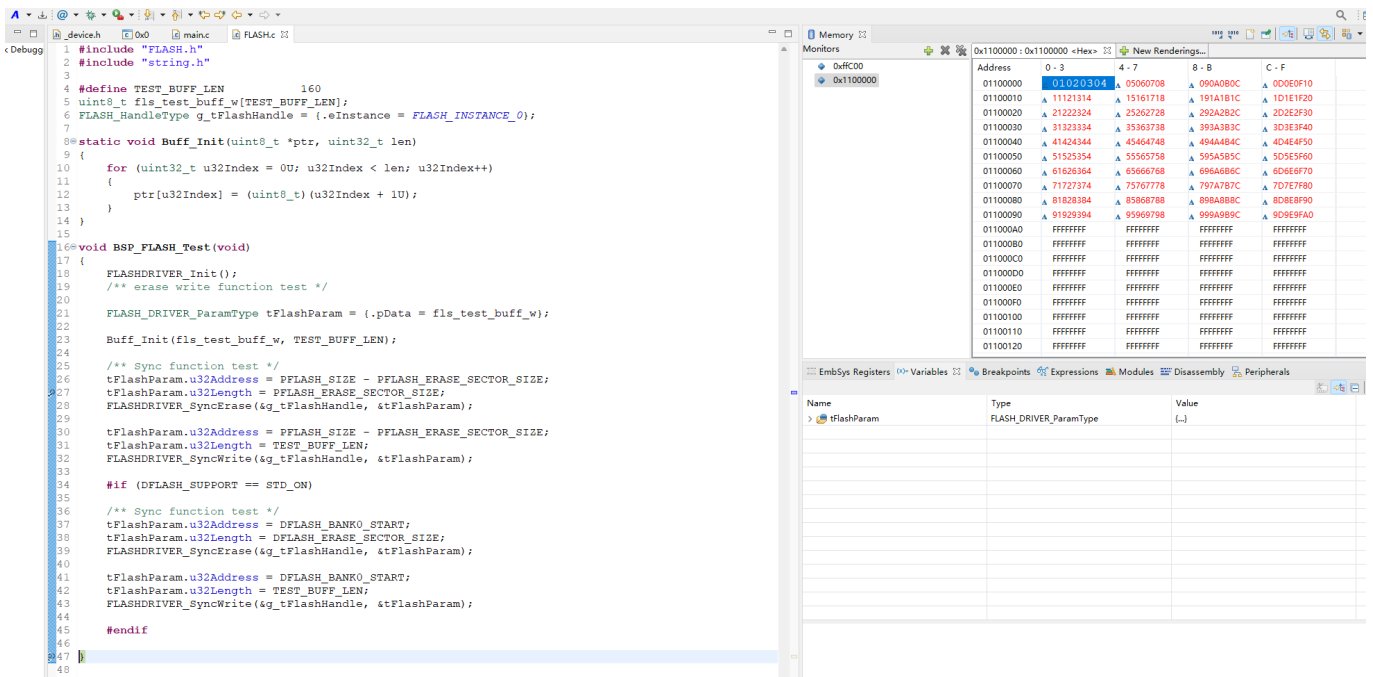
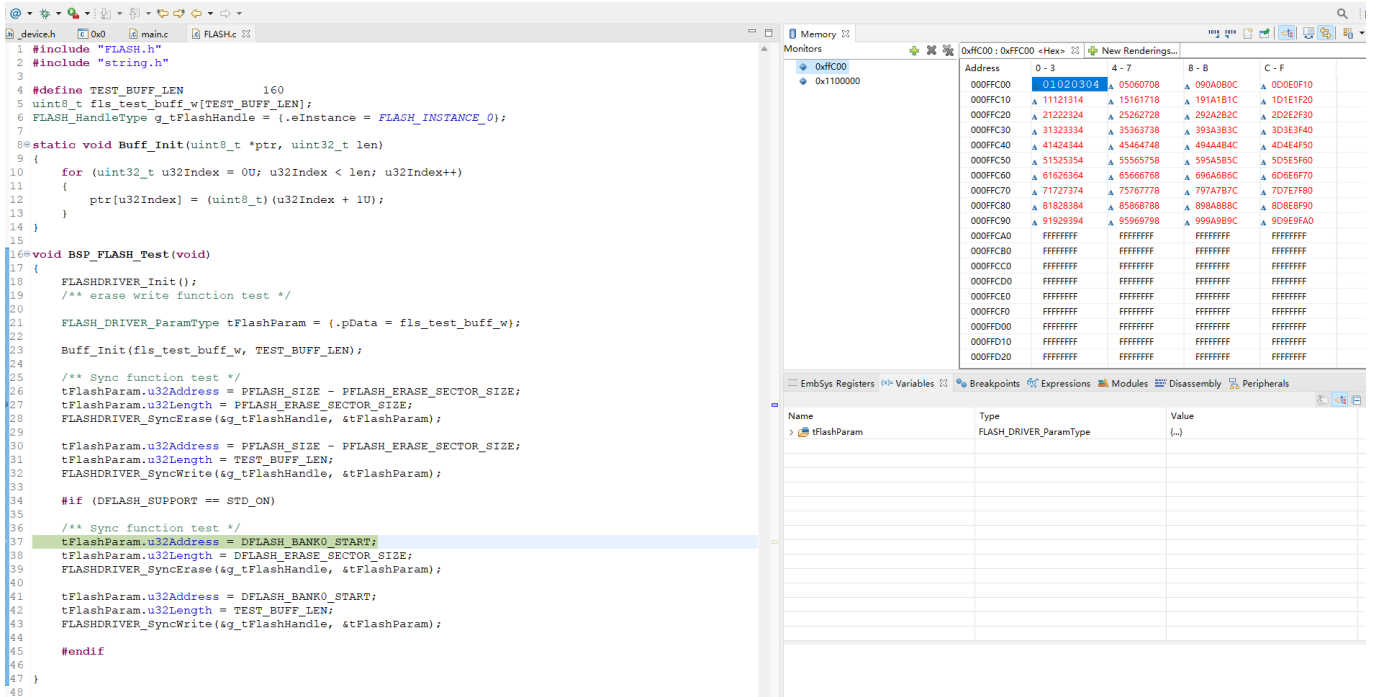
The steps to run the example are as follows:

- 1) Import the Flash project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) After building and debugging, the result will be shown as below.



2.19 FLEXCAN

2.19.1 CAN_DMA_Transmit

2.19.1.1 Description

This demo illustrates how to use Direct Memory Access (DMA) for CAN test, transmitting data on the CAN bus.

- 1) System and demo content contains can Peripheral Clock Initialization: Begin by initializing the system clock and canfd the peripheral clock for the CAN module to ensure the system is set up to provide the necessary timing signals.
- 2) CAN Module Initialization: Initialize the CAN module by setting up the baud rate, bit timing, and other necessary configurations. This includes selecting the clock source, prescaler, and segments for the CAN bit rate.

- 3) **DMA Initialization:** Set up the Direct Memory Access (DMA) controller to handle the transmission of CAN messages. This involves configuring the DMA channels, setting the source and destination addresses, and defining the data transfer size and priority.
- 4) **DMA Transfer Configuration:** Configure the DMA to transfer the CAN message buffers to the CAN module's transmit buffers. This includes setting up the block size, block count, and any necessary loopback or circular buffer settings.
- 5) **Interrupt Configuration:** If interrupt-driven transmission is desired, configure the interrupt system for the CAN module and the DMA. This involves setting up interrupt callback functions for transfer completion and error handling.
- 6) **Enable CAN and DMA:** Enable the CAN module and the DMA channels to start the transmission process.
- 7) **Start DMA Transfer:** Initiate the DMA transfer to start sending the CAN messages. The DMA will automatically handle the movement of data from the message buffers to the CAN transmit buffers.

2.19.1.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB-To-CAN device
- 1 Personal computer
- 1 Power supply 12V
- 1 J-Link debugger

2.19.1.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port C7: MUX = GPIO output, Function = CAN1_STB	-
Port E23: MUX = ALT3, Function = CAN1_RX	-
Port E24: MUX = ALT3, Function = CAN1_TX	-
Port A31: MUX = GPIO output, Function = CAN2_EN	-
Port B18: MUX = GPIO output, Function = CAN2_STBn	-
Port D16: MUX = ALT3, Function = CAN2_TX	-
Port D15: MUX = ALT3, Function = CAN2_RX	-
Port A22: MUX = GPIO output, Function = CAN5_SLT	-
Port E15: MUX = ALT3, Function = CAN5_RX	-
Port E16: MUX = ALT3, Function = CAN5_TX	-
Port G9: MUX = ALT2, Function = CAN6_RX	-
Port F11: MUX = ALT2, Function = CAN6_TX	-
Port H15: MUX = ALT3, Function = CAN7_RX	-
Port G17: MUX = ALT3, Function = CAN7_TX	-
Port I4: MUX = ALT4, Function = CAN8_RX	-
Port I3: MUX = ALT4, Function = CAN8_TX	-

2.19.1.4 Steps to Run

The steps to run the example are as follows:

- 1) Download the CAN_DMA_Transmit program to the MCU.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
Make sure "Debug_Flash" is selected as the active build configuration.
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

2) Then you can see the data sent by the MCU from the busmaster.

Time	Tx/Rx	Channel	Msg Type	ID	Message	DLC	Count	Data Byte(s)
14:54:19:8145	Rx	1	s	0x100	0x100	8	1	11 22 00 00 22 33 44 55
14:54:19:8148	Rx	1	s	0x101	0x101	8	1	11 22 00 01 22 33 44 55
14:54:19:8150	Rx	1	s	0x102	0x102	8	1	11 22 00 02 22 33 44 55
14:54:19:8152	Rx	1	s	0x103	0x103	8	1	11 22 00 03 22 33 44 55
14:54:19:8155	Rx	1	s	0x104	0x104	8	1	11 22 00 04 22 33 44 55
14:54:19:8157	Rx	1	s	0x105	0x105	8	1	11 22 00 05 22 33 44 55
14:54:19:8159	Rx	1	s	0x106	0x106	8	1	11 22 00 06 22 33 44 55
14:54:19:8162	Rx	1	s	0x107	0x107	8	1	11 22 00 07 22 33 44 55
14:54:19:8164	Rx	1	s	0x108	0x108	8	1	11 22 00 08 22 33 44 55
14:54:19:8166	Rx	1	s	0x109	0x109	8	1	11 22 00 09 22 33 44 55
14:54:19:8169	Rx	1	s	0x110	0x110	8	1	11 22 00 10 22 33 44 55
14:54:19:8171	Rx	1	s	0x111	0x111	8	1	11 22 00 11 22 33 44 55
14:54:19:8173	Rx	1	s	0x112	0x112	8	1	11 22 00 12 22 33 44 55
14:54:19:8175	Rx	1	s	0x113	0x113	8	1	11 22 00 13 22 33 44 55
14:54:19:8178	Rx	1	s	0x114	0x114	8	1	11 22 00 14 22 33 44 55
14:54:19:8180	Rx	1	s	0x115	0x115	8	1	11 22 00 15 22 33 44 55
14:54:19:8182	Rx	1	s	0x116	0x116	8	1	11 22 00 16 22 33 44 55
14:54:19:8185	Rx	1	s	0x117	0x117	8	1	11 22 00 17 22 33 44 55
14:54:19:8187	Rx	1	s	0x118	0x118	8	1	11 22 00 18 22 33 44 55
14:54:19:8189	Rx	1	s	0x119	0x119	8	1	11 22 00 19 22 33 44 55
14:54:19:8191	Rx	1	s	0x120	0x120	8	1	11 22 00 20 22 33 44 55
14:54:19:8194	Rx	1	s	0x121	0x121	8	1	11 22 00 21 22 33 44 55
14:54:19:8196	Rx	1	s	0x122	0x122	8	1	11 22 00 22 22 33 44 55
14:54:19:8198	Rx	1	s	0x123	0x123	8	1	11 22 00 23 22 33 44 55
14:54:19:8201	Rx	1	s	0x124	0x124	8	1	11 22 00 24 22 33 44 55

2.19.2 CAN_EnhanceFIFO_DMA

2.19.2.1 Description

The CAN_EnhanceFIFO_DMA demo illustrates the process of initializing the CAN module with Enhanced FIFO, setting up Direct Memory Access (DMA) for efficient message handling, and transmitting and receiving CAN messages using DMA for improved performance without CPU intervention.

- System and Peripheral Clock Initialization:** Start by initializing the system clock and the peripheral clock for the CAN module to ensure the system is set up to provide the necessary timing signals.
- CAN Module Initialization:** Initialize the CAN module by configuring the baud rate, bit timing, and other necessary settings. This includes selecting the clock source, prescaler, and segments for the CAN bit rate.
- DMA Controller Initialization:** Set up the Direct Memory Access (DMA) controller to handle the reception of CAN messages. This involves configuring the DMA channels, setting the source and destination addresses, and defining the data transfer size and priority.
- Interrupt Configuration:** If interrupt-driven reception is desired, configure the interrupt system for the CAN module and the DMA. This involves setting up interrupt callback functions for transfer completion and error handling.
- Enable CAN and DMA:** Enable the CAN module and the DMA channels to start the reception process.
- Start DMA Transfer:** Initiate the DMA transfer to begin receiving CAN messages. The DMA will automatically handle the movement of data from the CAN receive buffers to the application's message buffers.

2.19.2.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB-To-CAN device
- 1 Personal computer
- 1 Power supply 12V
- 1 J-Link debugger

2.19.2.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port C7: MUX = GPIO output, Function = CAN1_STB	-
Port E23: MUX = ALT3, Function = CAN1_RX	-
Port E24: MUX = ALT3, Function = CAN1_TX	-
Port A31: MUX = GPIO output, Function = CAN2_EN	-
Port B18: MUX = GPIO output, Function = CAN2_STBn	-
Port D16: MUX = ALT3, Function = CAN2_TX	-
Port D15: MUX = ALT3, Function = CAN2_RX	-
Port A22: MUX = GPIO output, Function = CAN5_SLT	-
Port E15: MUX = ALT3, Function = CAN5_RX	-
Port E16: MUX = ALT3, Function = CAN5_TX	-
Port G9: MUX = ALT2, Function = CAN6_RX	-
Port F11: MUX = ALT2, Function = CAN6_TX	-
Port H15: MUX = ALT3, Function = CAN7_RX	-
Port G17: MUX = ALT3, Function = CAN7_TX	-
Port I4: MUX = ALT4, Function = CAN8_RX	-
Port I3: MUX = ALT4, Function = CAN8_TX	-

2.19.2.4 Steps to Run

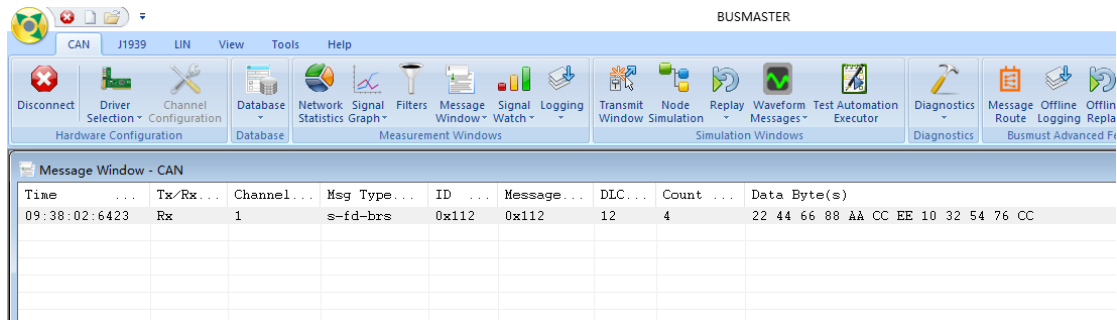
The steps to run the example are as follows:

- 1) Download the CAN_EnhanceFIFO_DMA program to the MCU.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
Make sure "Debug_Flash" is selected as the active build configuration.
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) Then you can see the data sent by the MCU from the busmaster.



2.19.3 CAN_LegacyFIFO_DMA

2.19.3.1 Description

This demo illustrates how to use Direct Memory Access (DMA) for CAN test, transmitting data on the CAN bus.

The CAN_LegacyFIFO_DMA demo illustrates the process of initializing the CAN module with LegacyFIFO FIFO, setting up Direct Memory Access (DMA) for efficient message handling, and transmitting and receiving CAN messages using DMA for improved performance without CPU intervention.

- 1) System and Peripheral Clock Initialization: Start by initializing the system clock and the peripheral clock for the CAN module to ensure the system is set up to provide the necessary timing signals.
- 2) CAN Module Initialization: Initialize the CAN module by configuring the baud rate, bit timing, and other necessary settings. This includes selecting the clock source, prescaler, and segments for the CAN bit rate.
- 3) DMA Controller Initialization: Set up the Direct Memory Access (DMA) controller to handle the reception of CAN messages. This involves configuring the DMA channels, setting the source and destination addresses, and defining the data transfer size and priority.
- 4) Interrupt Configuration: If interrupt-driven reception is desired, configure the interrupt system for the CAN module and the DMA. This involves setting up interrupt callback functions for transfer completion and error handling.
- 5) Enable CAN and DMA: Enable the CAN module and the DMA channels to start the reception process.
- 6) Start DMA Transfer: Initiate the DMA transfer to begin receiving CAN messages. The DMA will automatically handle the movement of data from the CAN receive buffers to the application's message buffers.

2.19.3.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB-To-CAN device
- 1 Personal computer
- 1 Power supply 12V
- 1 J-Link debugger

2.19.3.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port C7: MUX = GPIO output, Function = CAN1_STB	-
Port E23: MUX = ALT3, Function = CAN1_RX	-
Port E24: MUX = ALT3, Function = CAN1_TX	-
Port A31: MUX = GPIO output, Function = CAN2_EN	-
Port B18: MUX = GPIO output, Function = CAN2_STBn	-

Pin Function	FC7300F8MDQ Demo Board
Port D16: MUX = ALT3, Function = CAN2_TX	-
Port D15: MUX = ALT3, Function = CAN2_RX	-
Port A22: MUX = GPIO output, Function = CAN5_SLT	-
Port E15: MUX = ALT3, Function = CAN5_RX	-
Port E16: MUX = ALT3, Function = CAN5_TX	-
Port G9: MUX = ALT2, Function = CAN6_RX	-
Port F11: MUX = ALT2, Function = CAN6_TX	-
Port H15: MUX = ALT3, Function = CAN7_RX	-
Port G17: MUX = ALT3, Function = CAN7_TX	-
Port I4: MUX = ALT4, Function = CAN8_RX	-
Port I3: MUX = ALT4, Function = CAN8_TX	-

2.19.3.4 Steps to Run

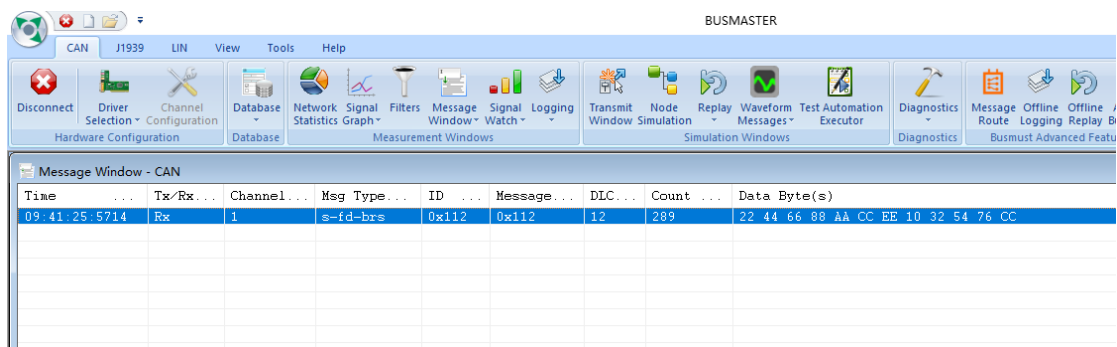
The steps to run the example are as follows:

- 1) Download the CAN_LegacyFIFO_DMA program to the MCU.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
Make sure "Debug_Flash" is selected as the active build configuration.
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) Then you can see the data sent by the MCU from the busmaster.



2.19.4 CAN_MB_EnhancedFIFO

2.19.4.1 Description

The CAN_MB_EnhancedFIFO demo demonstrates how to initialize the CAN module, configure Message Buffers (MBs) and Enhanced FIFO, and handle CAN message reception and transmission via interrupt processing.

- 1) System and Peripheral Clock Initialization: Begin by initializing the system clock and the peripheral clock for the CAN module to ensure the system is set up to provide the necessary timing signals.
- 2) CAN Module Initialization: Initialize the CAN module by setting up the baud rate, bit timing, and other necessary configurations. This includes selecting the clock source, prescaler, and segments for the CAN bit rate.
- 3) Interrupt Configuration: If interrupt-driven message handling is desired, configure the interrupt system for the CAN module. This involves setting up interrupt callback functions for error handling, FIFO issues, transmission

completion, and reception.

- 4) Enable CAN Module: Enable the CAN module to start the communication process.
- 5) Start Message Transmission: Initiate the transmission of CAN messages. If using Enhanced FIFO with interrupts, the transmission will trigger interrupts upon completion.

2.19.4.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB-To-CAN device
- 1 Personal computer
- 1 Power supply 12V
- 1 J-Link debugger

2.19.4.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port C7: MUX = GPIO output, Function = CAN1_STB	-
Port E23: MUX = ALT3, Function = CAN1_RX	-
Port E24: MUX = ALT3, Function = CAN1_TX	-
Port A31: MUX = GPIO output, Function = CAN2_EN	-
Port B18: MUX = GPIO output, Function = CAN2_STBn	-
Port D16: MUX = ALT3, Function = CAN2_TX	-
Port D15: MUX = ALT3, Function = CAN2_RX	-
Port A22: MUX = GPIO output, Function = CAN5_SLT	-
Port E15: MUX = ALT3, Function = CAN5_RX	-
Port E16: MUX = ALT3, Function = CAN5_TX	-
Port G9: MUX = ALT2, Function = CAN6_RX	-
Port F11: MUX = ALT2, Function = CAN6_TX	-
Port H15: MUX = ALT3, Function = CAN7_RX	-
Port G17: MUX = ALT3, Function = CAN7_TX	-
Port I4: MUX = ALT4, Function = CAN8_RX	-
Port I3: MUX = ALT4, Function = CAN8_TX	-

2.19.4.4 Steps to Run

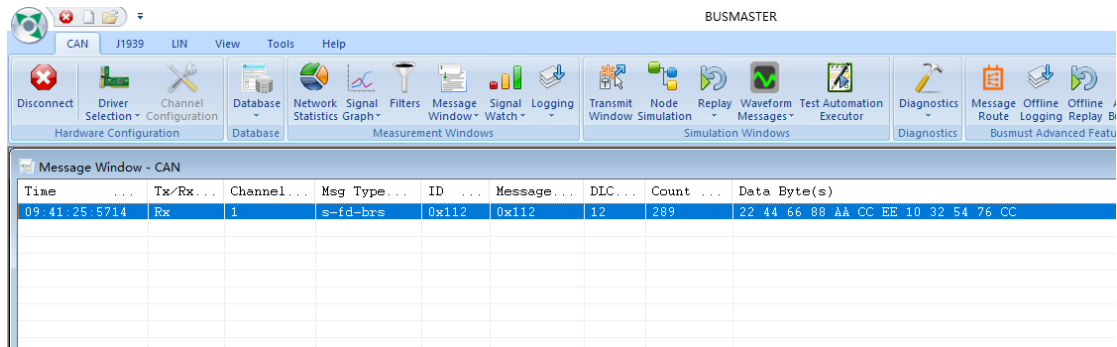
The steps to run the example are as follows:

- 1) Download the CAN_MB_EnhancedFIFO program to the MCU.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
Make sure "Debug_Flash" is selected as the active build configuration.
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) Then you can see the data sent by the MCU from the busmaster.



2.19.5 CAN_MB_LegacyFIFO

2.19.5.1 Description

The CAN_MB_LegacyFIFO demo demonstrates how to initialize the CAN module, configure Message Buffers (MBs) and Legacy FIFO, and handle CAN message reception and transmission via interrupt processing.

- 1) System and Peripheral Clock Initialization: Begin by initializing the system clock and the peripheral clock for the CAN module to ensure the system is set up to provide the necessary timing signals.
- 2) CAN Module Initialization: Initialize the CAN module by setting up the baud rate, bit timing, and other necessary configurations. This includes selecting the clock source, prescaler, and segments for the CAN bit rate.
- 3) Interrupt Configuration: If interrupt-driven message handling is desired, configure the interrupt system for the CAN module. This involves setting up interrupt callback functions for error handling, FIFO issues, transmission completion, and reception.
- 4) Enable CAN Module: Enable the CAN module to start the communication process.
- 5) Start Message Transmission: Initiate the transmission of CAN messages. If using Enhanced FIFO with interrupts, the transmission will trigger interrupts upon completion.

2.19.5.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB-To-CAN device
- 1 Personal computer
- 1 Power supply 12V
- 1 J-Link debugger

2.19.5.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port C7: MUX = GPIO output, Function = CAN1_STB	-
Port E23: MUX = ALT3, Function = CAN1_RX	-
Port E24: MUX = ALT3, Function = CAN1_TX	-
Port A31: MUX = GPIO output, Function = CAN2_EN	-
Port B18: MUX = GPIO output, Function = CAN2_STBn	-
Port D16: MUX = ALT3, Function = CAN2_TX	-
Port D15: MUX = ALT3, Function = CAN2_RX	-
Port A22: MUX = GPIO output, Function = CAN5_SLT	-
Port E15: MUX = ALT3, Function = CAN5_RX	-

Pin Function	FC7300F8MDQ Demo Board
Port E16: MUX = ALT3, Function = CAN5_TX	-
Port G9: MUX = ALT2, Function = CAN6_RX	-
Port F11: MUX = ALT2, Function = CAN6_TX	-
Port H15: MUX = ALT3, Function = CAN7_RX	-
Port G17: MUX = ALT3, Function = CAN7_TX	-
Port I4: MUX = ALT4, Function = CAN8_RX	-
Port I3: MUX = ALT4, Function = CAN8_TX	-

2.19.5.4 Steps to Run

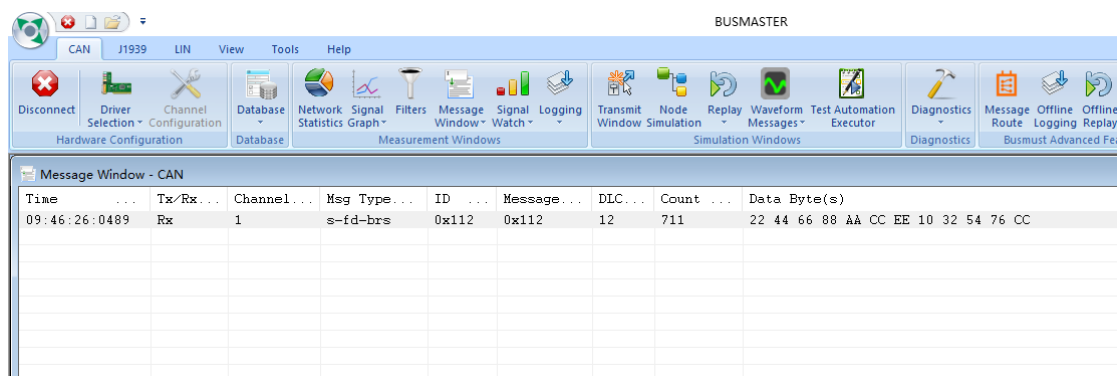
The steps to run the example are as follows:

- 1) Download the CAN_MB_LegacyFIFO program to the MCU.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
Make sure "Debug_Flash" is selected as the active build configuration.
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) Then you can see the data sent by the MCU from the busmaster.



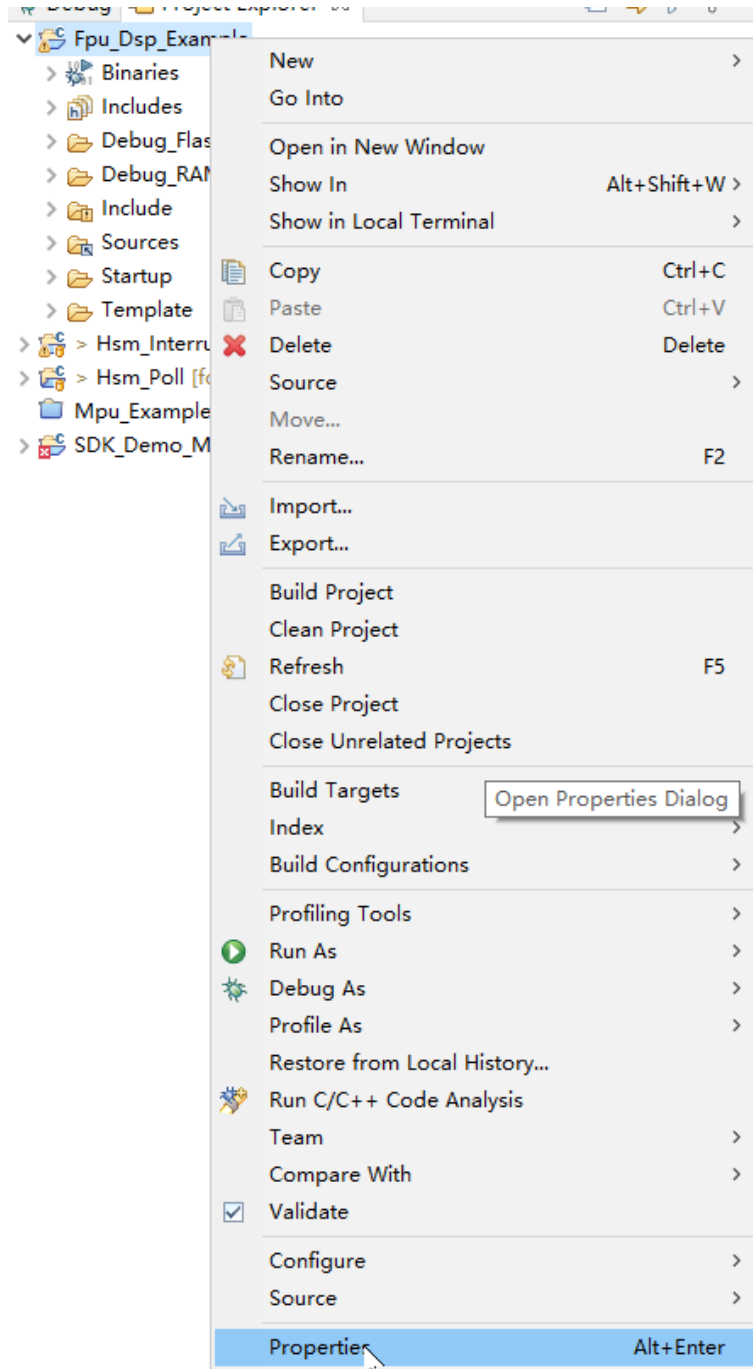
2.20 FPU&DSP

2.20.1 Description

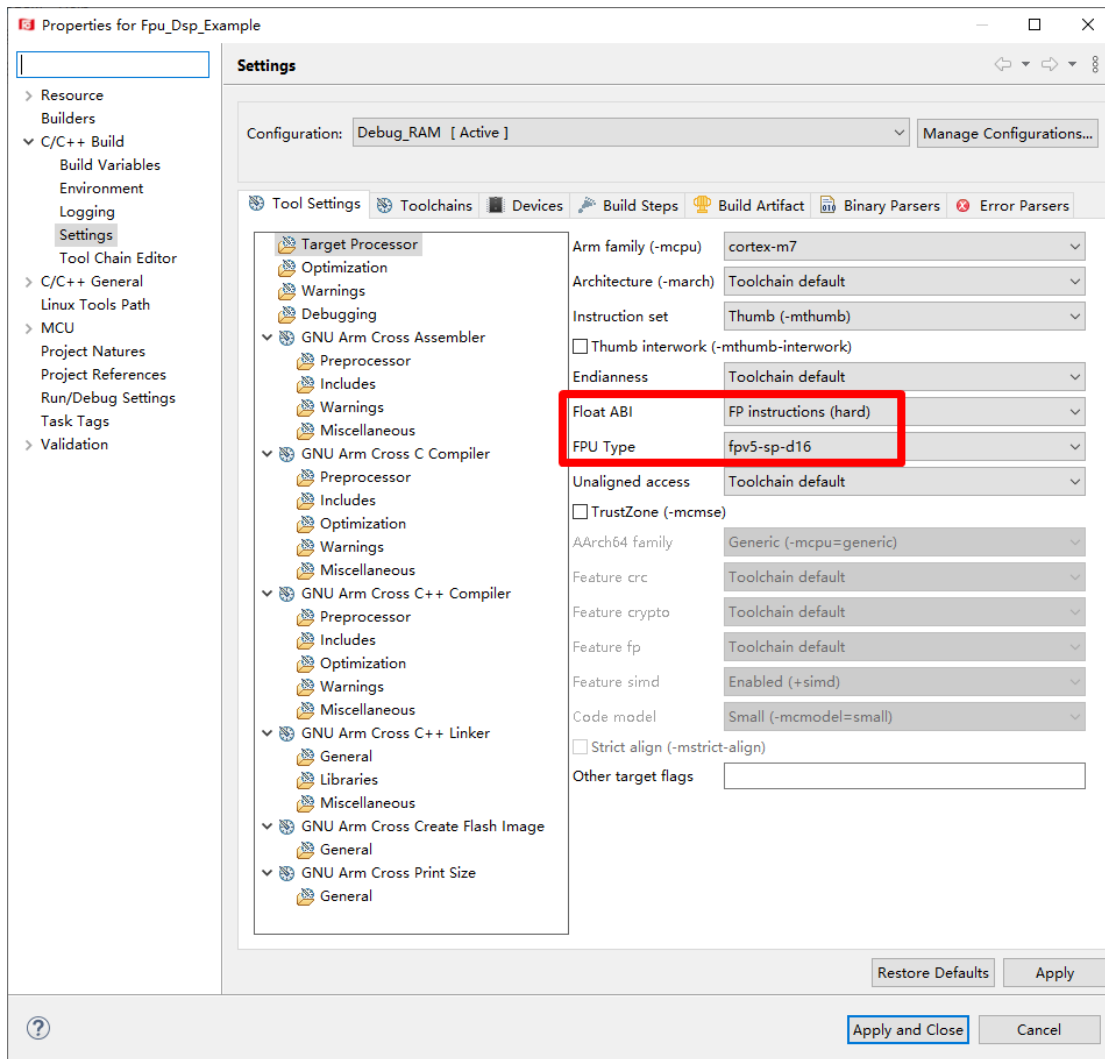
Realized by the ARM Cortex M7 kernel, the FPU only depends on the CPU. It implements the FPv5-sp-d16 floating-point extension, and supports 32-bit floating-point computation functionality. This chip driver only provides FPU_Enable/FPU_Disable API to enable/disable the FPU. To use the FPU, the user needs to open the IDE compiler to support it, and call FPU_Enable at the beginning of program before main function.

Besides, the CMSIS library is provided for the user who wants to do some digital signal processing. The user should add the CMSIS header path into the IDE configuration and add the compiler macro "Driver_CM7_DSP_ENABLE" to the IDE to enable the DSP C source file to the compiler.

The following figure shows how to enable the FPU instruction.



FC7300F8MDQ SDK Quick Start



Enable FPU based on whether hardware is enabled or not in the system_init function.

```
void system_init(void)
{
    ... /* only errata in debug mode */
    ... if (0 != (0x01 & (*(volatile uint32*)0xE00EDF0))) ...

    ... /* only power on reset or low power reset need to clear all ram */
    ... // if (0U != (*(volatile uint32*)0x40046008) & 0x82) ...
    ... { ...

    ... /* disable wdog 0 */
    ... *(volatile uint32*)0x40022004 = 0x08181982;
    ... while (0U == (0x800u & *(volatile uint32*)0x40022000));
    ... *(volatile uint32*)0x40022000 = 0x2920;
    ... *(volatile uint32*)0x40022008 = 0xF000;
    ... while (0U == (0x400u & *(volatile uint32*)0x40022000));

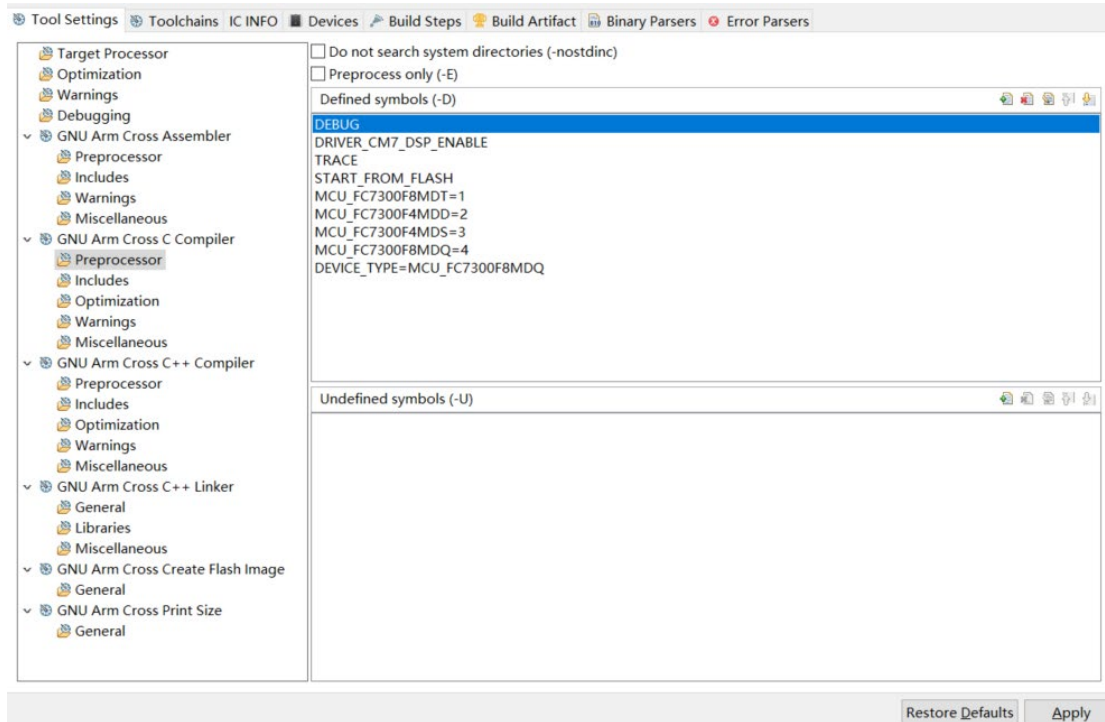
    ... /* disable wdog 1 */
    ... *(volatile uint32*)0x40433004 = 0x08181982;
    ... while (0U == (0x800u & *(volatile uint32*)0x40433000));
    ... *(volatile uint32*)0x40433000 = 0x2920;
    ... *(volatile uint32*)0x40433008 = 0xF000;
    ... while (0U == (0x400u & *(volatile uint32*)0x40433000));

    #if ((__FPU_PRESENT == 1) && (__FPU_USED == 1))
    ... *(volatile uint32*)0xE00ED88 = 0x0F000000u; ... /* set CP10, CP11 Full Access */
    #endif /* ((__FPU_PRESENT == 1) && (__FPU_USED == 1)) */

    ... asm volatile("mov pc, lr");
}
```

The following demonstrates how to add DSP support.

- Add the following macro: `Driver_CM7_DSP_ENABLE`



2.20.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V or micro-USB cable
- 1 Personal computer
- 1 J-Link debugger

2.20.3 MCU Ports

N/A.

2.20.4 Steps to Run

2.20.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

2.20.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: `SDK\Example\FPU_DSP\Fpu_Dsp_Sample\_Project\IAR_Project`, and double-click the **Fpu_Dsp_Sample** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.20.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.21 FreeRTOS

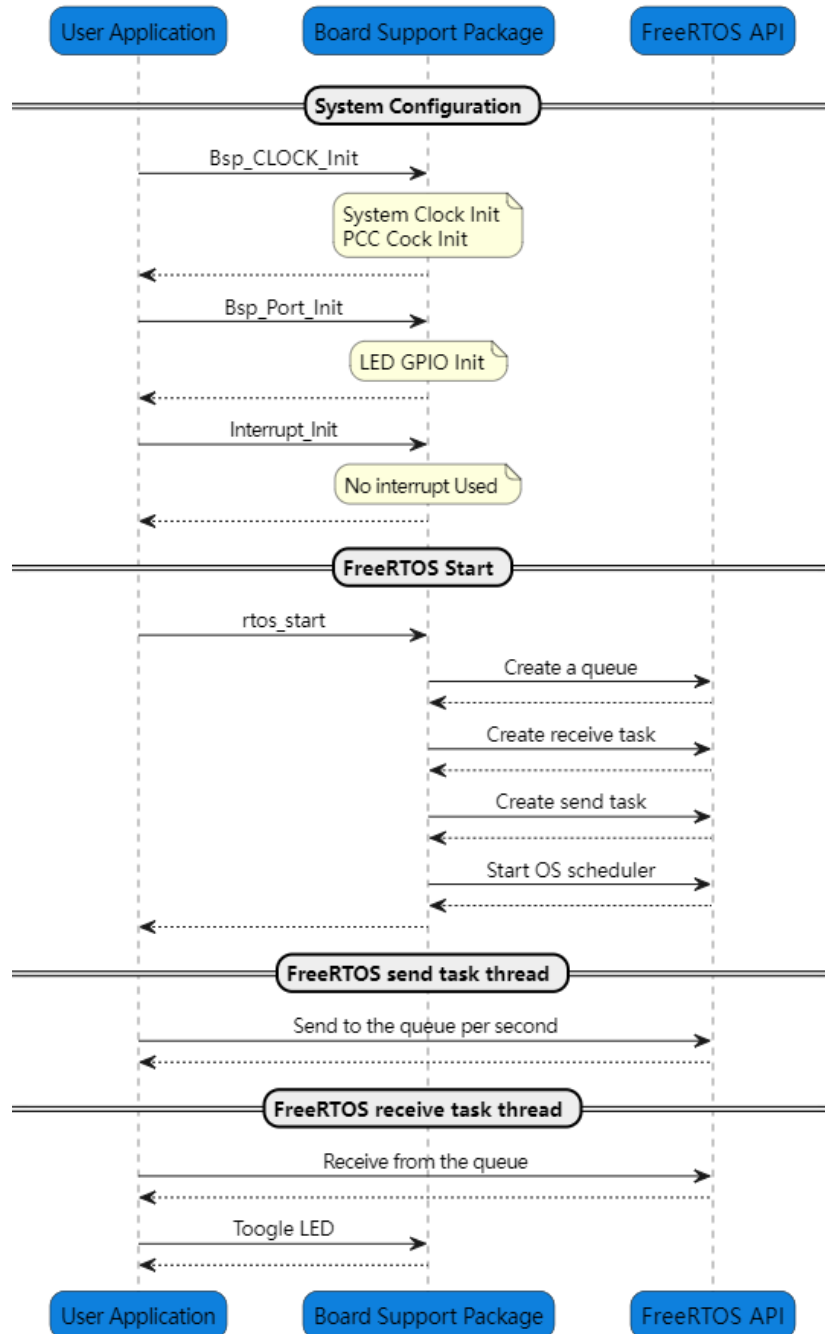
2.21.1 Description

This demo shows how to use the FreeRTOS.

For the main function, we first initialize the system clock and GPIOs, and then get the clock frequency to a global variable `SystemCoreClock`, which will be used by FreeRTOS to initialize the SYSTICK clock.

After that, we create two tasks with one queue, wherein the sender task will send a number every 1 second and the receiver task will toggle the LED when it receives the number.

The sequence diagram of the demo is as follows:



2.21.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board

- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.21.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A26: MUX = ALT1, GPIO Output	LED1 – wired on board
Port D31: MUX = ALT1, GPIO Output	LED2 – wired on board
Port E20: MUX = ALT1, GPIO Output	LED3 – wired on board

2.21.4 Steps to Run

2.21.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

2.21.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\FreeRTOS\OS_ToggleLED_Project\IAR_Project, and double-click the **FreeRTOS** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.21.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

After building and debugging, the LEDs on the demo board will start to toggle.

2.22 FREQM

2.22.1 Description

The example is just for testing FREQM function. In the example, the bus clock is used as the reference clock to measure FOSC DIVH clock. Test procedures in this example are as below:

- 1) Initialize System clock and PCC;
- 2) Initialize Port;
- 3) Initialize Uart for debug log;
- 4) Initialize FREQM;
- 5) Start frequency measurement;
- 6) Print measure result when the measurement stops.

2.22.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V or micro-USB cable
- 1 Personal computer

- 1 J-Link debugger

2.22.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

2.22.4 Steps to Run

The steps to run the example are as follows:

- 1) Plug in the USB Type-C cable and open the up-machine (115200/8/N/1).
- 2) Download the **FREQM_Demo** program to the MCU by JTAG.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 3) Click Run. You will see the test result via the UART debug terminal.

```
FREQM Example.
Bus clk freq is 150000000.
Measure clk divider value is 8.
N_Start is 130, N_Stop is 3276880
MesClkFreq 24000000, FoscDivhClkFreq is 24000000
```

2.23 FS26

2.23.1 Description

This demo is just for FS26 with FCSMU test.

2.23.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V or micro-USB cable
- 1 Personal computer
- 1 J-Link debugger

2.23.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port E17: MUX = ALT3, FCSMU_PIN1	-
Port A16: MUX = ALT3, FCSMU_PIN0	-

Pin Function	FC7300F8MDQ Demo Board
Port A28: MUX = ALT5, FCSPI1_SCK	Serial clock pin
Port A27: MUX = ALT5, FCSPI1_PCS0	Chip select pin
Port E12: MUX = ALT5, FCSPI1_SOUT	Serial output pin
Port A29: MUX = ALT5, FCSPI1_SIN	Serial input pin

2.23.4 Steps to Run

The steps to run the example are as follows:

- 1) Plug in the USB Type-C cable and open the up-machine (115200/8/N/1).
- 2) Make sure the debug pin jumps and plugs in.
- 3) Download the FS26 Demo program to MCU within 12s.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 4) Power off the board, then remove the debug pin jumps.
- 5) Power on the board, then MCU will print the following text.

```
System Start!!
Init FCSMU
Init FS26
FS26 current mode is Init FS
FS26 collect error counter is [1] when power up
Referesh watchdog 1 times to clear FS26 error counter
FS26 current mode is Normal
Enter while refresh watchdog and then wait 12s for fcsmu fault injection
```

- 6) Wait for 12s for FCSMU fault injection to cause system reset.

```
System Start!!
Init FCSMU
Init FS26
FS26 current mode is Init FS
FS26 collect error counter is [1] when power up
Referesh watchdog 1 times to clear FS26 error counter
FS26 current mode is Normal
Enter while refresh watchdog and then wait 12s for fcsmu fault injection
=====
Fault injected, wait for warning timeout
System Start!!
Init FCSMU
Init FS26
FS26 current mode is Init FS
FS26 collect error counter is [1] when power up
Referesh watchdog 1 times to clear FS26 error counter
FS26 current mode is Normal
Enter while refresh watchdog and then wait 12s for fcsmu fault injection
```

- 7) If you want to update the program, you need to restart from Step 2.

2.24 FTU

There are five FTU demos: InputCapture, TimeCounter, OutputCompare, PWM, and QuadratureDecoder.

2.24.1 InputCapture

2.24.1.1 Description

This example shows how to use the FTU module as an input capture. You can send "I" from the serial port to generate a rising edge. And the serial port will output the timestamp.

2.24.1.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.24.1.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port D31: MUX=ALT1, GPIO Output	LED(D2E13) control pin
Port A12: MUX=ALT2, FTU0_CH7	FTU input capture pin
Port A13: MUX=ALT1, GPIO Output	Output to FTU input capture pin

2.24.1.4 Steps to Run

The steps to run the example are as follows:

- 1) Connect the Type-C cable to the demo board and PC. And open UART debug terminal (USB-SERIAL CH340/115200/8/N/1).
- 2) Connect PTA30 to MOTOR1 I-W.
- 3) Download the InputCapture program to the MCU and debug it.

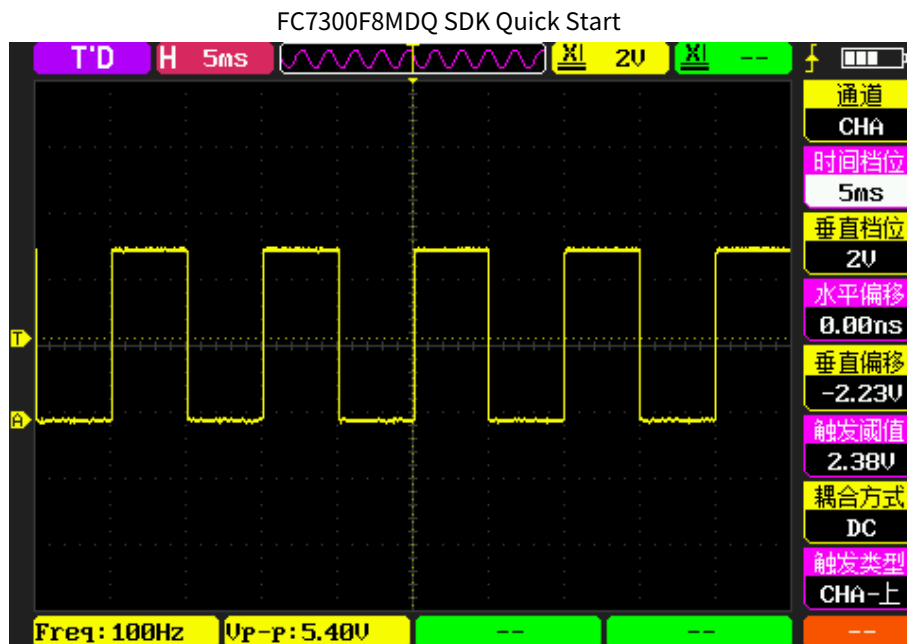
Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

2.24.2 TimeCounter

2.24.2.1 Description

This example shows how to use the FTU module as a timer. You will see that PTA30 generates a 100Hz signal as shown below. And the LED will toggle every 0.5s.



2.24.2.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger
- 1 Oscilloscope

2.24.2.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port D31: MUX=ALT1, GPIO Output	LED control pin
Port A25: MUX=ALT1, GPIO Output	Output a 100Hz signal

2.24.2.4 Steps to Run

The steps to run the example are as follows:

- 1) Connect the Type-C cable to the demo board and PC. And open UART debug terminal (USB-SERIAL CH340/115200/8/N/1).
- 2) Connect PTA30 to the Oscilloscope.
- 3) Download the TimeCounter program to the MCU and debug it.

Note:

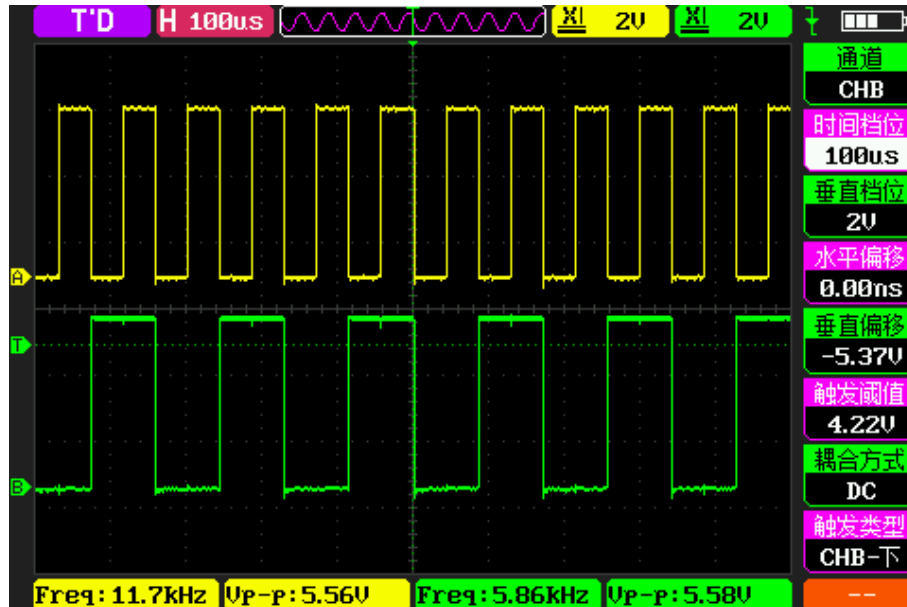
- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

2.24.3 OutputCompare

2.24.3.1 Description

This example shows how to generate two different PWM signals with different periods by the FTU module. You will see MOTOR1 I-W generates a 7.81KHz signal and MOTOR1 E-W generates a 3.91KHz signal. And the LED will toggle.

The waveform is shown below:



2.24.3.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger
- 1 Oscilloscope

2.24.3.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port D31: MUX = ALT1, GPIO Output	LED control pin
Port A13: MUX = ALT2, FTU0_CH3	Output a 7.81KHz signal
Port A12: MUX = ALT2, FTU0_CH7	Output a 3.91KHz signal

2.24.3.4 Steps to Run

The steps to run the example are as follows:

- 1) Connect the Type-C cable to the demo board and PC. And open UART debug terminal (USB-SERIAL CH340/115200/8/N/1).
- 2) Connect MOTOR1 I-W and MOTOR1 E-W connected to the Oscilloscope.

- 3) Download the OutputCompare program to the MCU and debug it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

2.24.4 PWM

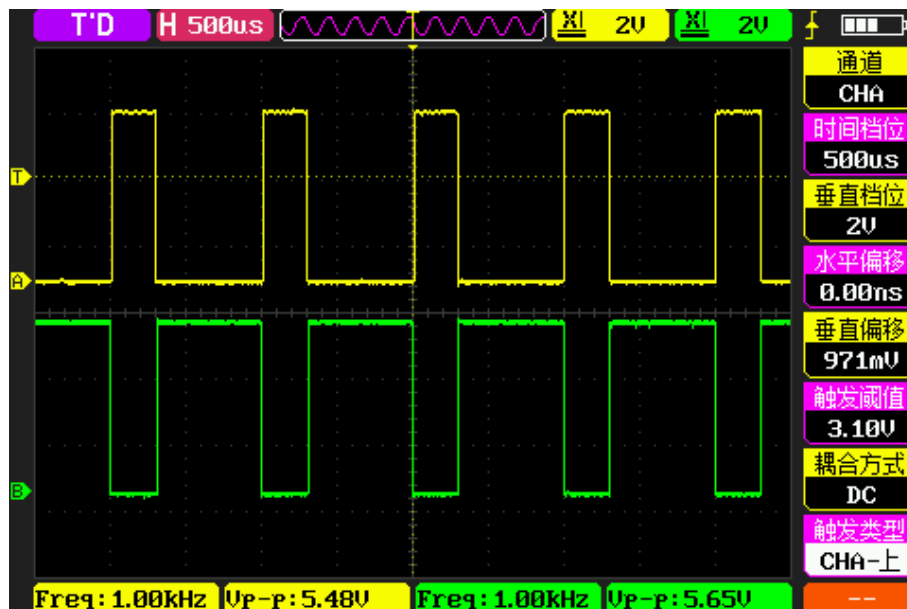
2.24.4.1 Description

This example shows how to generate PWM signals by the FTU module. You will see MOTOR1 I-W generates a PWM signal with period of 1ms and a duty cycle of 300us. Send "a" will increase the duty cycle by 50us, send "m" will reduce the duty cycle by 50us.

You can switch to USE_LINK_MODE. And you can get a complementary signal on MOTOR1 E-W. And LED will toggle every 0.5s.

If switching to USE_FAULT, you can use the fault input. And then sending "f" will generate a fault input, and the PWM will stop. Sending "c" to clear the fault, and the PWM will recover.

The waveform is shown below:



2.24.4.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger
- 1 Oscilloscope

2.24.4.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port D31: MUX=ALT1, GPIO Output	LED control pin
Port D4: MUX=ALT2, FTU3_CH6	Output a variable PWM signal
Port D3: MUX=ALT2, FTU3_CH7	Output a complement signal
Port A14: MUX=ALT2, FTU_FLT17	Fault input pin
Port E13: MUX=ALT1, GPIO Output	Output to fault input

2.24.4.4 Steps to Run

The steps to run the example are as follows:

- 1) Connect the Type-C cable to the demo board and PC. And open UART debug terminal (USB-SERIAL CH340/115200/8/N/1).
- 2) Connect MOTOR1 I-W and MOTOR1 E-W to the Oscilloscope, and connect PTA30 to MOTOR1 FLT.
- 3) Download the PWM program to the MCU and debug it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

2.24.5 QuadratureDecoder

2.24.5.1 Description

This example shows how to use the FTU module as a quadrature decoder. And the serial port will output the counter value. You can send "I" from the serial port to increase counter, and send "d" to decrease counter.

2.24.5.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.24.5.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port D31: MUX=ALT1, GPIO Output	LED control pin
Port G22: MUX=ALT6, FTU1_QD_PHA	Phase A input
Port G23: MUX=ALT6, FTU1_QD_PHB	Phase B input
Port A12: MUX=ALT1, GPIO Output	Output to Phase A
Port A13: MUX=ALT1, GPIO Output	Output to Phase B

2.24.5.4 Steps to Run

The steps to run the example are as follows:

- 1) Connect the Type-C cable to the demo board and PC. And open UART debug terminal (USB-SERIAL CH340/115200/8/N/1).
- 2) Connect PTA30 to MOTOR1 CHA, and connect PTA3 to MOTOR1 CHB.
- 3) Download the QuadratureDecoder program to the MCU and debug it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

2.25 GPIO

There are three GPIO demos: GPIO_Toggle, GPIO_EdgeDetect, and GPIO_DigitalFilter.

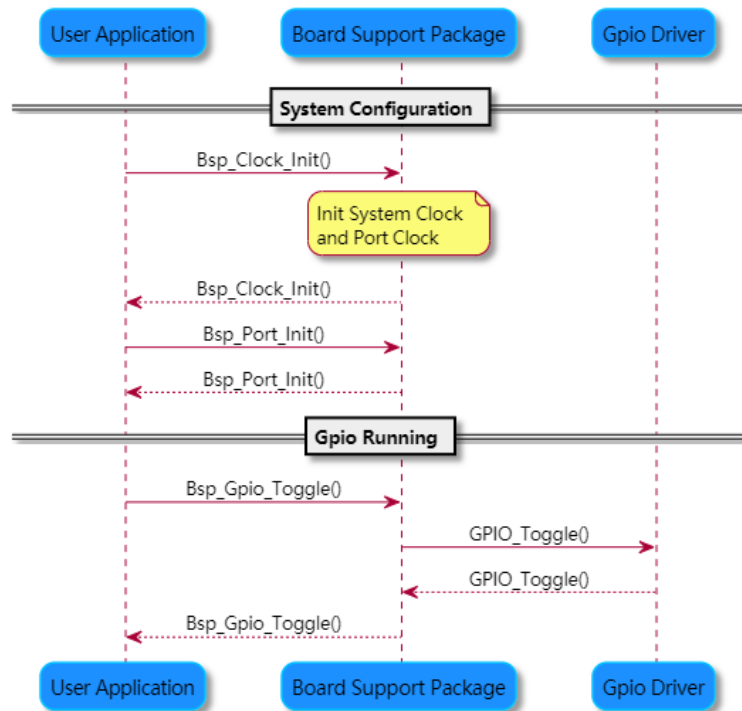
2.25.1 GPIO_Toggle

2.25.1.1 Description

This demo shows how to use PORT and GPIO to toggle pins through the SDK.

This demo project will toggle LED1, LED2 and LED3 every second.

The sequence diagram of the demo is as follows:



2.25.1.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 Personal computer
- 1 J-Link debugger

2.25.1.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A26: MUX = ALT1, GPIO Output	LED1 – wired on board
Port D31: MUX = ALT1, GPIO Output	LED2 – wired on board
Port E20: MUX = ALT1, GPIO Output	LED3 – wired on board

2.25.1.4 Steps to Run

2.25.1.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

2.25.1.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\GPIO\GPIO_Toggle_Project\IAR_Project, and double-click the **GPIO_TOGGLE** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.25.1.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

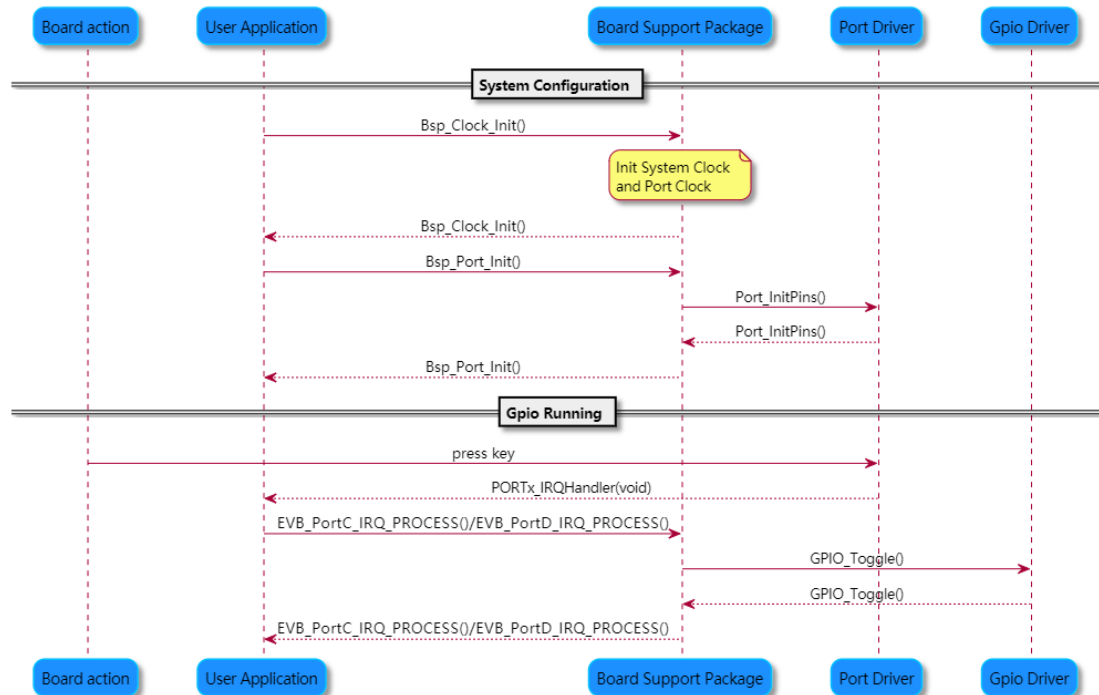
2.25.2 GPIO_EdgeDetect

2.25.2.1 Description

This demo shows how to use PORT and GPIO to detect the input rising edge and toggle pins through the SDK.

This demo project will separately toggle LED1/LED2/LED3 when detecting input falling/rising/falling and rising edge at the KEY1 (PTD20)/ KEY2(PTD26)/ KEY3 (PTA11).

The sequence diagram of the demo is as follows:



2.25.2.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 Personal computer
- 1 J-Link debugger

2.25.2.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A26: MUX = ALT1, GPIO Output	LED1 – wired on board
Port D31: MUX = ALT1, GPIO Output	LED2 – wired on board
Port E20: MUX = ALT1, GPIO Output	LED3 – wired on board
Port D20: MUX = ALT1, GPIO IN	KEY1
Port D26: MUX = ALT1, GPIO IN	KEY2
Port A11: MUX = ALT7, NMI	KEY3

2.25.2.4 Steps to Run

2.25.2.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

2.25.2.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\GPIO\GPIO_EdgeDetect_Project\IAR_Project, and double-click the **GPIO_EdgeDetect** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.25.2.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

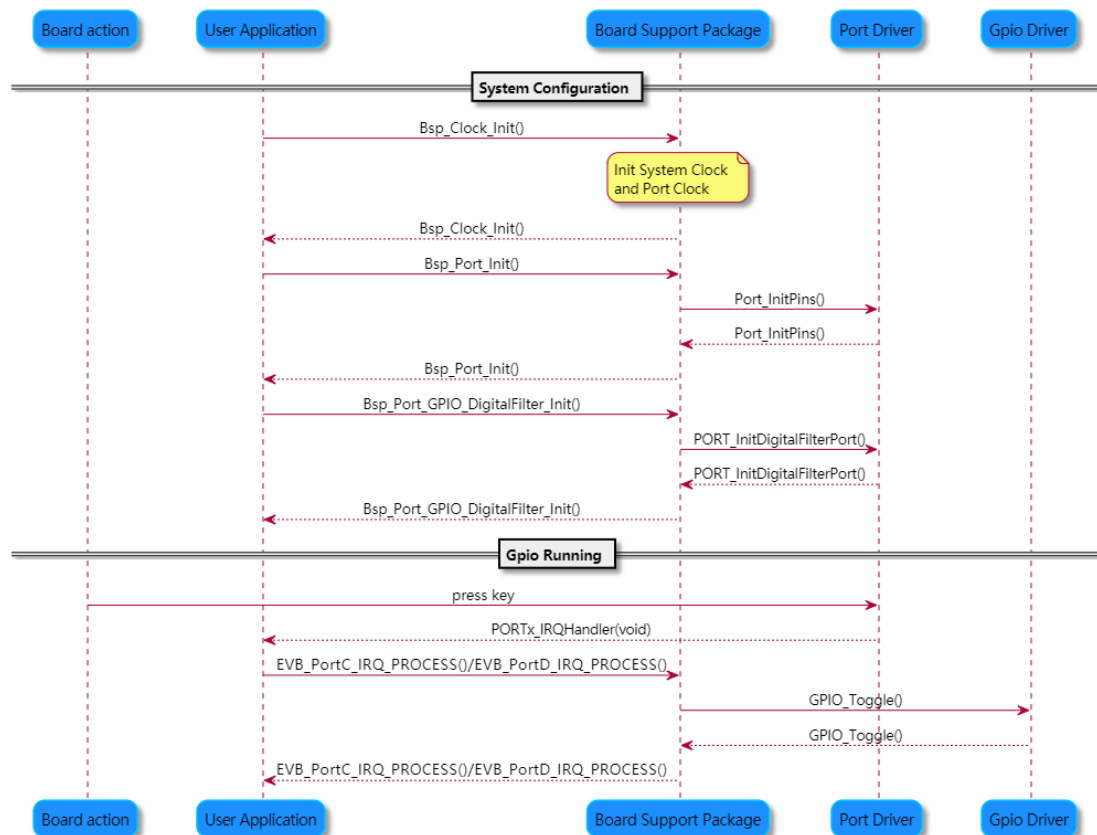
2.25.3 GPIO_DigitalFilter

2.25.3.1 Description

This demo shows how to use PORT and GPIO to detect the input rising edge and toggle pins through the SDK. In this demo, the port uses a digital filter, so the input rising edge will be detected after a specific MCU clock period.

This demo project will separately toggle LED1/LED2/LED3 when detect input falling/rising/falling and rising edge at the KEY1 (PTD20)/ KEY2(PTD26)/ KEY3 (PTA11).

The sequence diagram of the demo is as follows:



2.25.3.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V

- 1 Personal computer
- 1 J-Link debugger

2.25.3.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A26: MUX = ALT1, GPIO Output	LED1 – wired on board
Port D31: MUX = ALT1, GPIO Output	LED2 – wired on board
Port E20: MUX = ALT1, GPIO Output	LED3 – wired on board
Port D20: MUX = ALT1, GPIO IN	KEY1
Port D26: MUX = ALT1, GPIO IN	KEY2
Port A11: MUX = ALT7, NMI	KEY3

2.25.3.4 Steps to Run

2.25.3.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

2.25.3.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\GPIO\GPIO_DigitalFilter_Project\IAR_Project, and double-click the **GPIO_DigitalFilter** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.25.3.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.26 HRPWM

2.26.1 Description

This example shows how to use the HRPWM module to generate Pwm wave. You can measure the PWM wave on the pin.

2.26.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 Digital Oscilloscope
- 1 J-Link debugger

2.26.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1_TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A26: MUX = ALT1, GPIO Output	LED1
Port D31: MUX = ALT1, GPIO Output	LED2
Port E13: MUX = ALT4, HRPWM0_CH5	HRPWM output pin

2.26.4 Steps to Run

The steps to run the example are as follows:

- 1) Import the HRPWM project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) Measure the PWM period and duty cycle.

2.27 HSADC

The HSADC demos monitors two external HSADC channels. They are:

Channel	Type	Description
HSADC0_CHANNEL_2	External	Connected to the free pin
HSADC0_CHANNEL_3	External	Connected to the free pin

In the HSADC demos, we use the internal voltage references and assume the voltage is 5.0V.

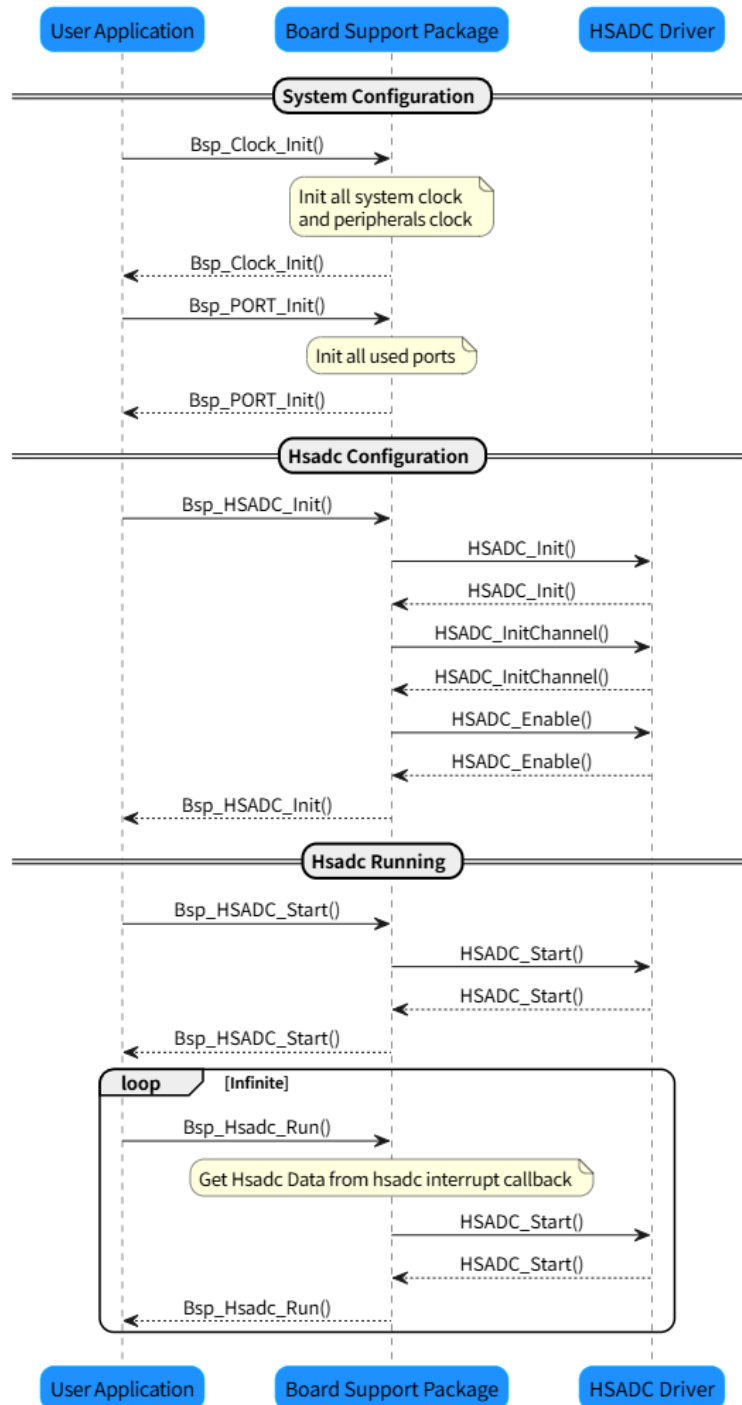


You may want to watch HSADC result by UART, then you can connect a type-c USB cable to the USB-to-UART interface on demo board.

There are four HSADC demos:

2.27.1.1 Description

The sequence diagram of the demo is as follows:



First you need to initialize the system and peripheral clock, and then the HSADC ports.

In the HSADC initialization function, you should specify the HSADC resolution, trigger mode and sequence mode, you could also configure the HSADC sample time and average mode, etc.

After initialization of HSADC instance, you need to configure the HSADC channels, in which you should specify which HSADC channels you want to use.

If you want the HSADC result to be transferred with interrupt, you should then configure the HSADC interrupt, in which you should specify the result buffer and the interrupt callback functions.

After the HSADC instance is properly configured, you could enable the HSADC instance.

After that, you could start the HSADC instance, and the HSADC conversion result will return in the interrupt callback function.

2.27.1.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 Personal computer
- 1 J-Link debugger
- 1 type-c USB cable (optional)

2.27.1.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
HSADC0_SE2 (PTH0)	Free pin – wired on board
HSADC0_SE3 (PTH1)	Free pin – wired on board

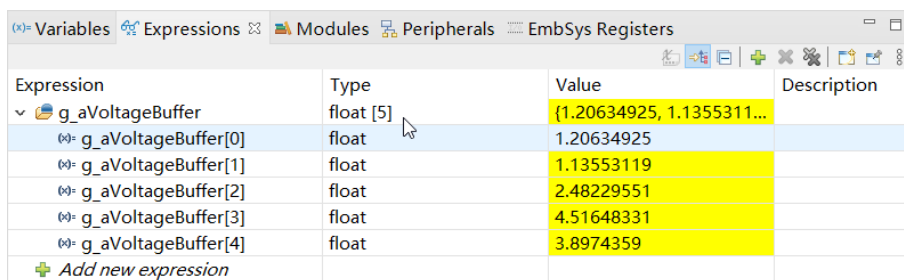
2.27.1.4 Steps to Run

2.27.1.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: Make sure "**Debug_Flash**" is selected as the active build configuration.

After building and debugging, you can see the g_aVoltageBuffer variable in the **Expressions** tab.



Expression	Type	Value	Description
g_aVoltageBuffer	float [5]	{1.20634925, 1.1355311...	
g_aVoltageBuffer[0]	float	1.20634925	
g_aVoltageBuffer[1]	float	1.13553119	
g_aVoltageBuffer[2]	float	2.48229551	
g_aVoltageBuffer[3]	float	4.51648331	
g_aVoltageBuffer[4]	float	3.8974359	
+ Add new expression			

2.27.1.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\HSADC\HSADC_Single_Project\IAR_Project, and double-click the **HSADC_Single** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

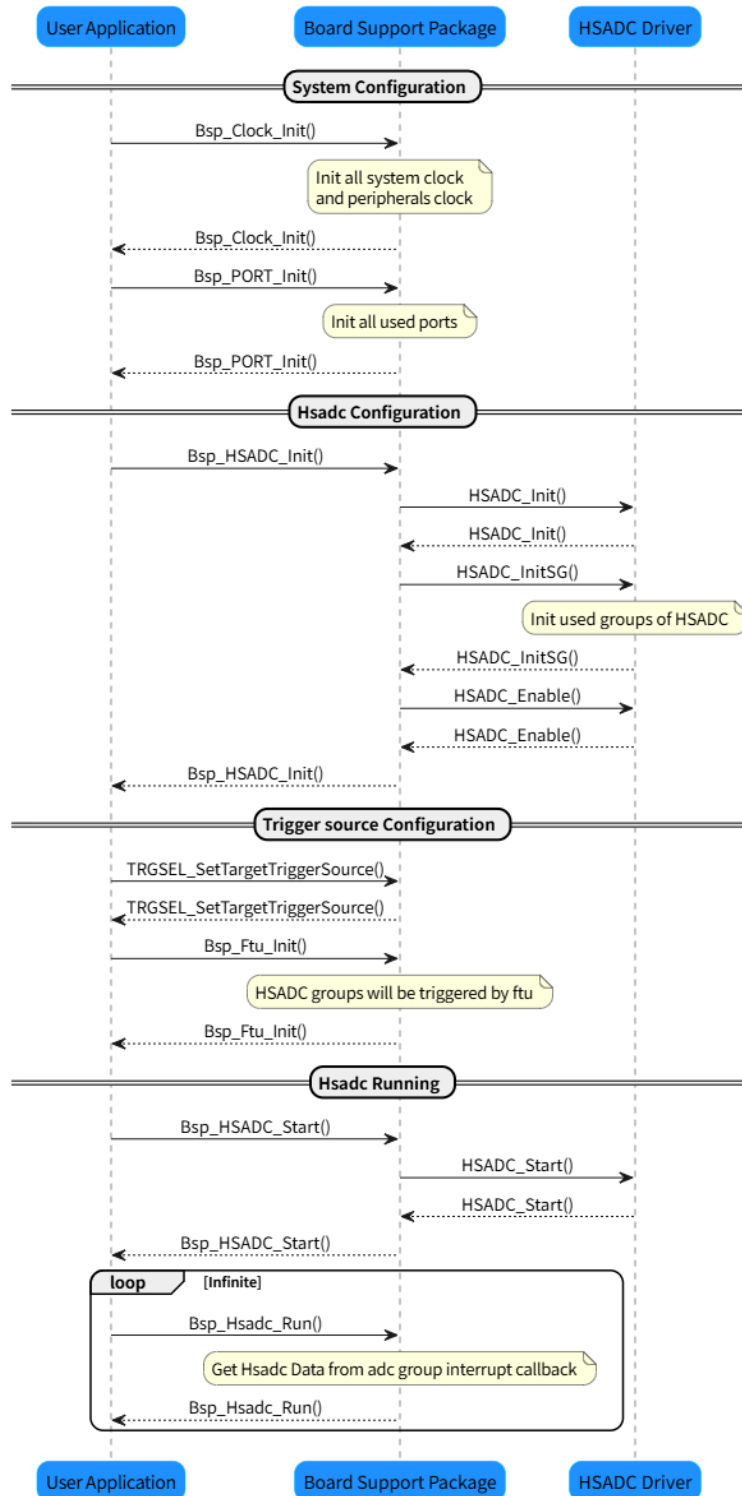
2.27.1.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.27.2 HSADC_Sequence_Group

The purpose of this demo is to show you the usage of HSADC sequence group mode. The HSADC supports a Sequence Group mode. In this mode, an HSADC instance can be configured with four hardware groups, which can be individually triggered, as if they are converting in parallel.

In this demo, we initialize two groups of HSADC, and they both be triggered by FTU reload trigger event.



2.27.2.1 Prerequisites

Same as HSADC_Single.

2.27.2.2 MCU Ports

Same as HSADC_Single.

2.27.2.3 Steps to Run

Same as HSADC_Single.

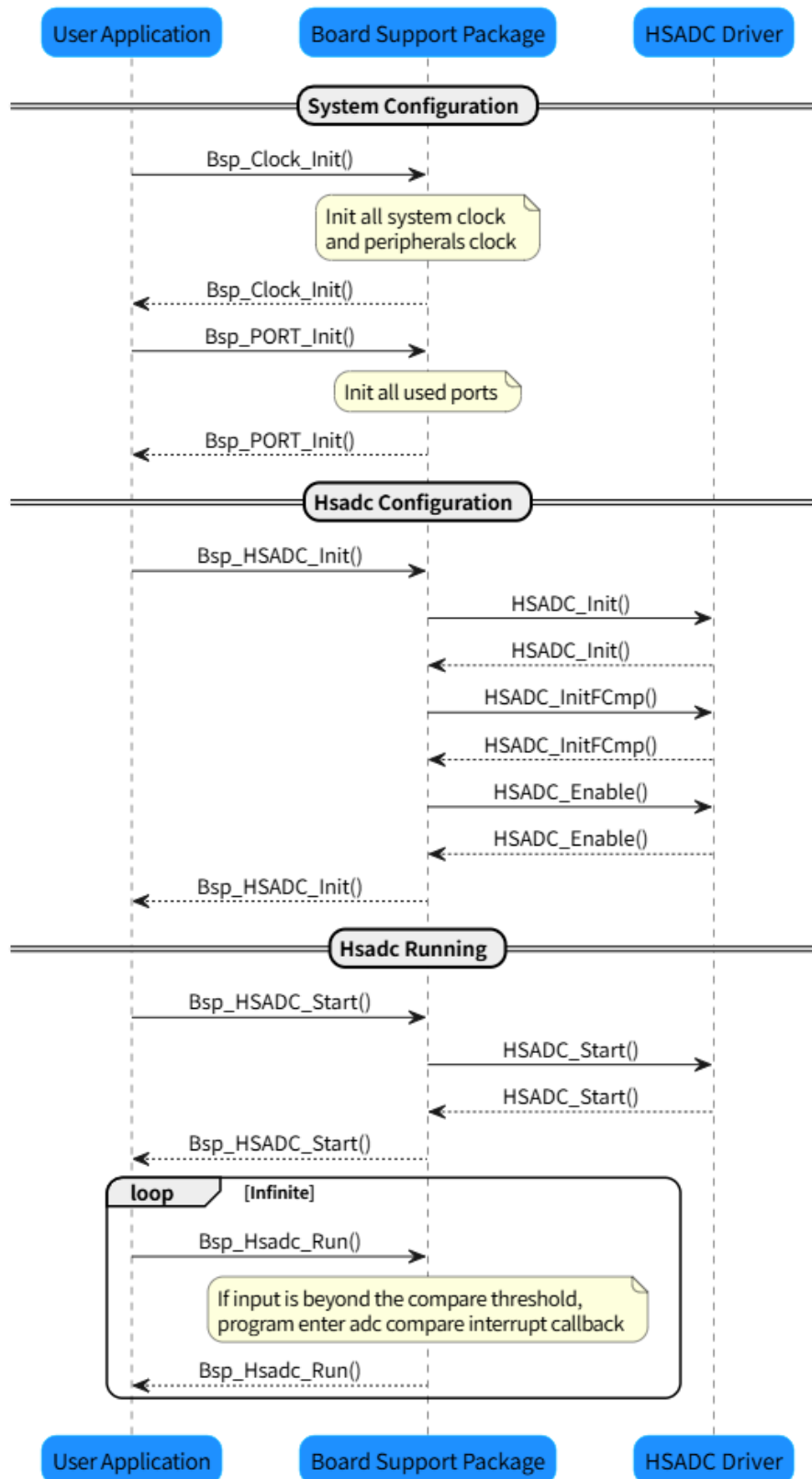
2.27.3 HSADC_Single_Internal_Trigger

This demo is quite like HSADC_Single demo, the only difference is that the HSADC conversion is triggered by an internal timer.

2.27.4 HSADC_Fast_Compare

The purpose of this demo is to show you the usage of HSADC fast compare feature mode.

When HSADC works in fast compare function, it will compare the sampled input signal with a reference value defined in HSADC_FCREF register. Total fast compare conversion time is less than normal conversion time.



2.27.4.1 Prerequisites

Same as HSADC_Single.

2.27.4.2 MCU Ports

Same as HSADC_Single.

2.27.4.3 Steps to Run

The main steps are same as HSADC_Single. If you adjust the adjustable resistor on the board, the program will enter the interrupt callback function of the comparison function.

2.28 HSM

2.28.1 Description

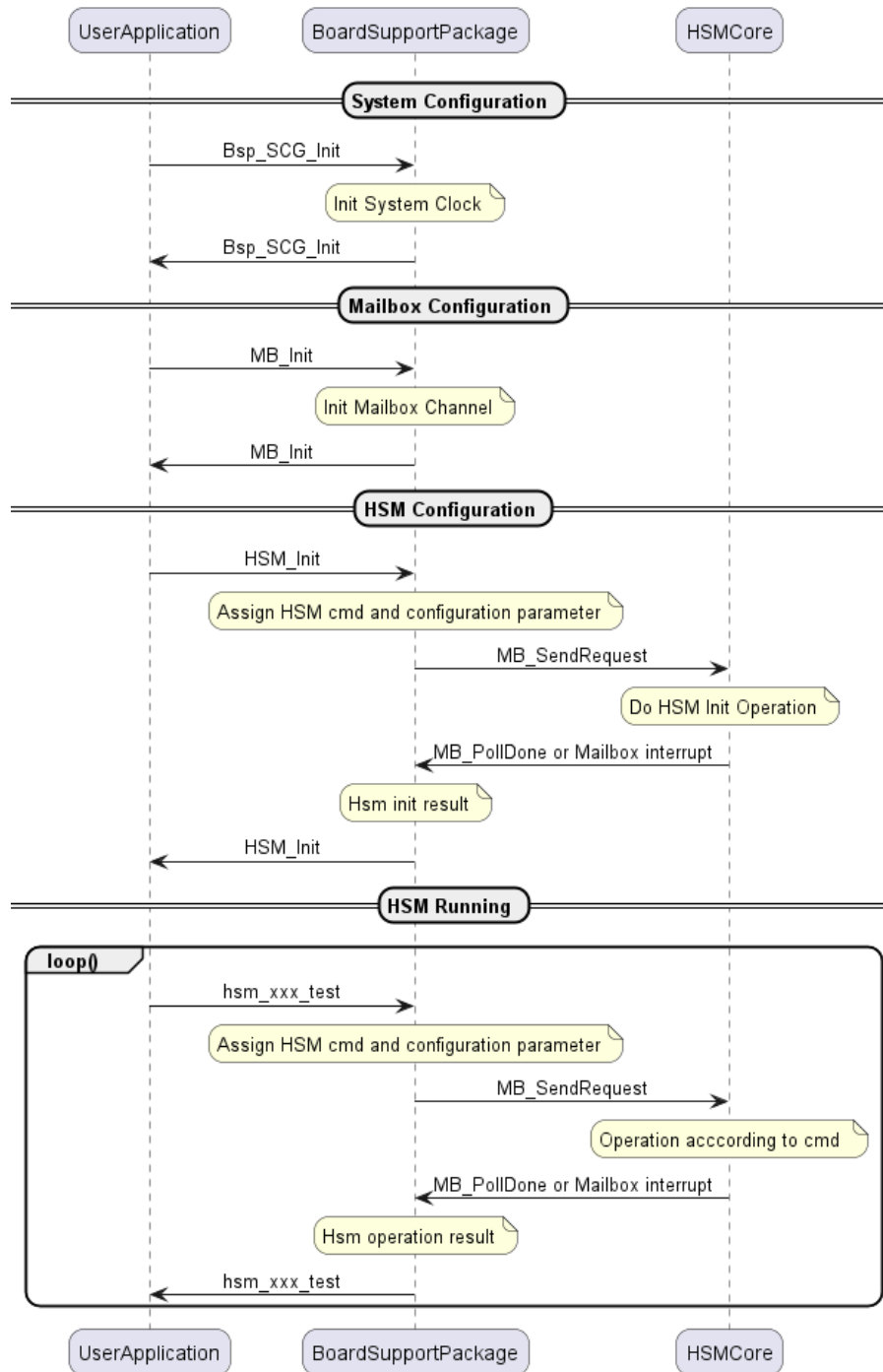
HSM driver only contains simple APIs that fill the command structure variable. To trigger the HSM core to process the command, the user should send the command structure by mailbox driver APIs.

The HSM driver works in two modes: ISR mode and poll mode. These two modes are implemented by mailbox driver.

At first, the user should call "HSM_Init" to initialize the command structure variable and send it. This command will trigger the HSM to update internal random poll. If not updated, other commands will not work. Then send other command.

To ensure the driver run successfully, the HSM's clock should be enabled.

The sequence diagram of the demo is as follows:



2.28.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V or micro-USB cable
- 1 Personal computer
- 1 J-Link debugger

2.28.3 MCU Ports

N/A.

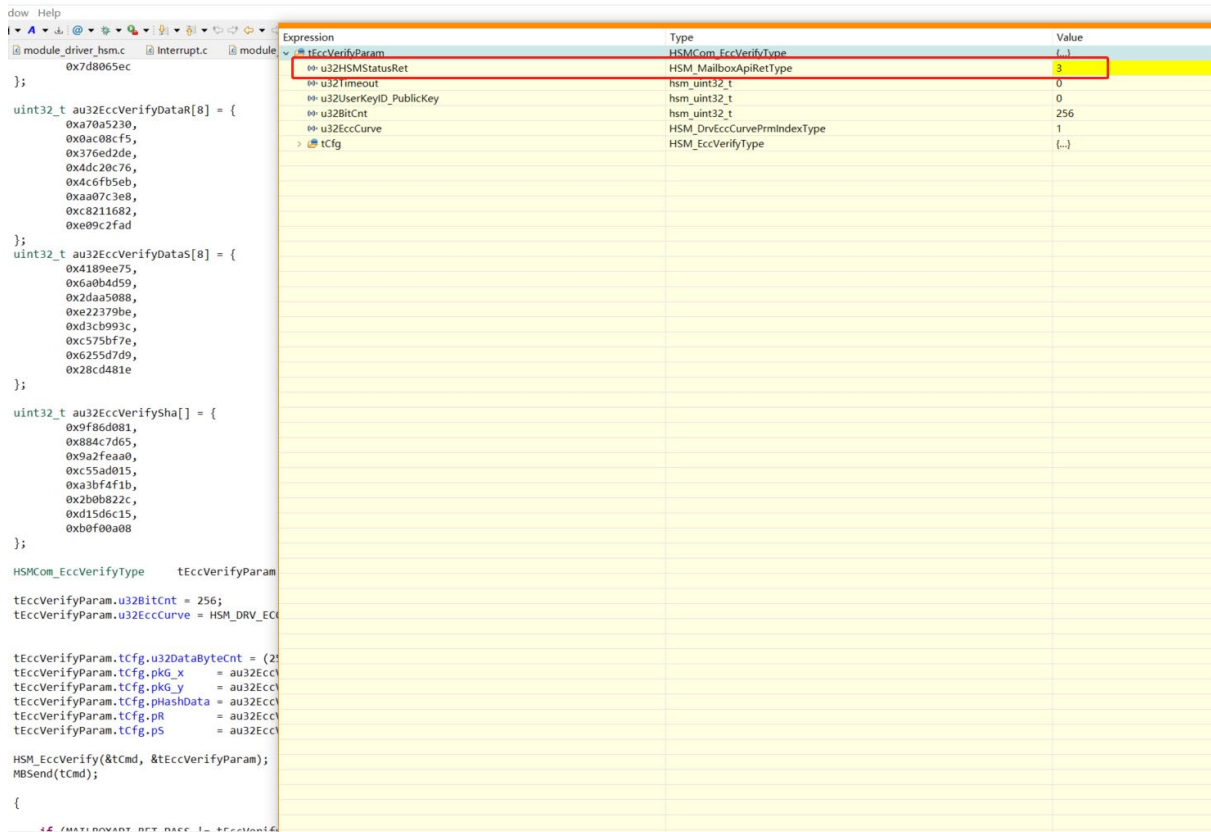
2.28.4 Steps to Run

2.28.4.1.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: Make sure "**Debug_Flash**" is selected as the active build configuration.

After building and debugging, you can get the operation result by u32HSMStatusRet of the configuration struct.



2.28.4.1.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\HSM\Hsm_Interrupt (or Hsm_Poll)_Project\IAR_Project, and double-click the **Hsm_Interrupt** (or **Hsm_Poll**) IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

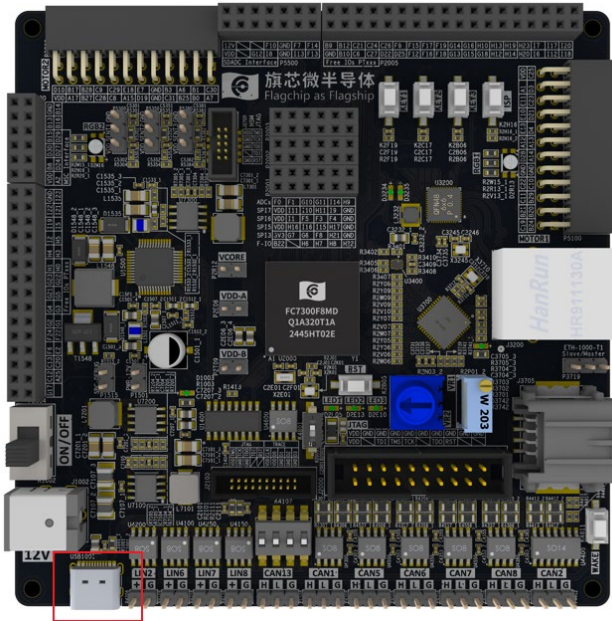
2.28.4.1.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

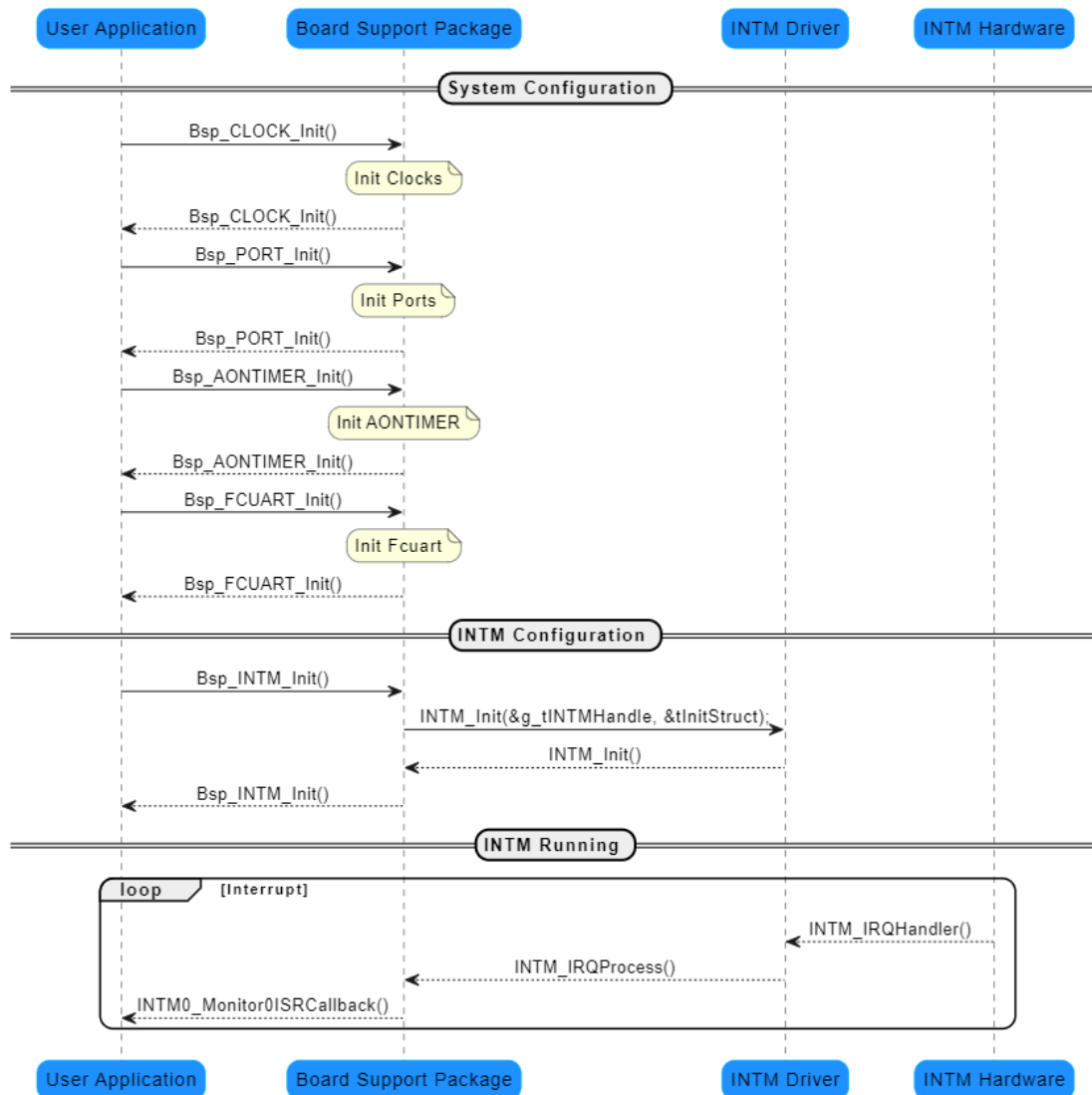
2.29 INTM

2.29.1 Description

This demo is just for INTM test.



The sequence diagram of the demo is as follows:



2.29.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board

- 1 Power adapter 12 V
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.29.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

2.29.4 Steps to Run

The steps to run the example are as follows:

- 1) Plug in the USB Type-C cable, and open the up-machine (115200/8/N/1).
- 2) Download the INTM_Demo program to the MCU by JTAG.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 3) MCU will print the below texts, which means the interrupt is delayed.

```
INTM DEMO START
INTMO Monitor1 interrupt happend
INTMO Monitor1 interrupt happend
```

2.30 ISM

2.30.1 Description

This demo is used for verifying ISM functions. It will monitor these cases:

- 1) Pulse duty shorter than threshold.
- 2) Pulse duty longer than threshold.
- 3) PWM frequency faster than threshold. If period is shorter than expected for 15 times, interrupt will happen.

2.30.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 USB-2-UART cable
- 1 Personal computer
- 1 J-Link debugger

2.30.3 MCU Ports

N/A.

2.30.4 Steps to Run

The steps to run the example are as follows:

- 1) Plug in the USB Type-C cable and open the up-machine (115200/8/N/1).
- 2) Download the ISM_Demo program to the MCU by JTAG.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

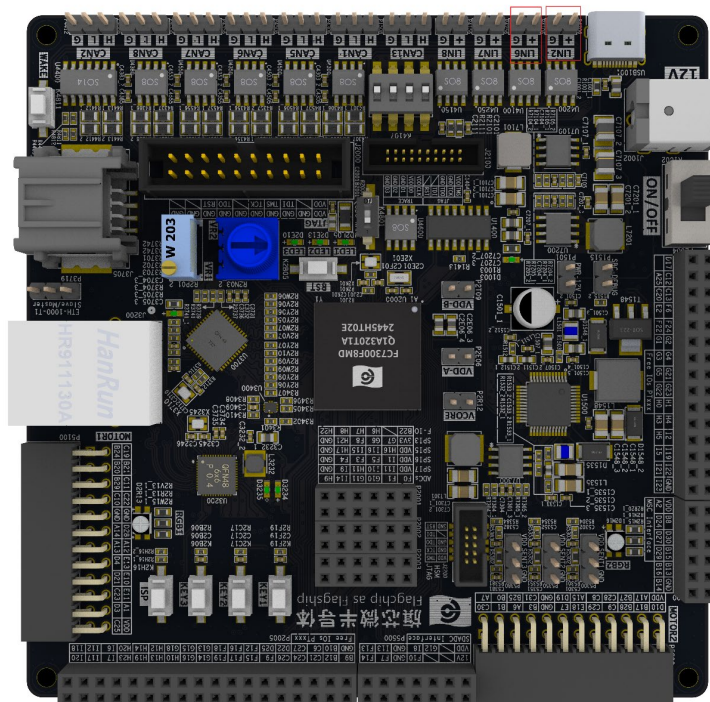
- 3) Normally, LED will blink when program is running.
- 4) MCU will print texts as below.

```
ISM detected pwm frequency too fast 0x74FF < 0x249F0 for 15 times
ISM detected duty too long 0x19F338 > 0x249F0
ISM detected pwm frequency too fast 0x74FF < 0x249F0 for 15 times
ISM detected duty too short 0xCFA0B < 0x16E36D
ISM detected duty too long 0x19F338 > 0x249F0
ISM detected pwm frequency too fast 0x74FF < 0x249F0 for 15 times
ISM detected duty too short 0xCFA0B < 0x16E36D
ISM detected duty too long 0x19F338 > 0x249F0
ISM detected pwm frequency too fast 0x74FF < 0x249F0 for 15 times
```

2.31 LIN

2.31.1 Description

The demos are just for testing LIN Master and Slave node functions. In these demos, we connect LIN2 and LIN6 on board as master node and slave node.



Totally 3 cases are tested in this example:

- 1) Master sending data, slave receiving data;
- 2) Master receiving data, Slave sending data;
- 3) Master sending a wakeup signal to wakeup slave node;

2.31.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board

- 1 USB-2-UART cable
- 1 Personal computer
- 1 J-Link debugger

2.31.3 MCU Ports

We will use LIN2 and LIN6 as the test instance, please do below connection on the board:

- 1) Connect LIN2 data line with LIN6 data line.
- 2) Program example LIN_Master.

2.31.4 Steps to Run

The steps to run the example are as follows:

- 1) Plugin the USB-2-UART cable and open UART debug terminal (UART1 115200/8/N/1) for FC7300F8MDQ Demo Board;
- 2) Import the LIN_Master project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
 - To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
 - To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).
- 3) You will find the log information can be read from the UART debug terminal.

```
This is LIN slave node!  
This is LIN master node!  
PID: c1  
Slave response send  
Master response received  
c1 11 ee  
PID: 42  
Slave response send  
Master response received  
42 11 ac  
PID: 3  
Slave response send  
Master response received  
3 11 22 cc  
PID: c4  
Slave response send  
Master response received  
c4 11 22 8  
PID: 85  
Slave response send  
Master response received  
85 11 22 33 99  
PID: 6  
Slave response send  
Master response received  
6 11 22 33 93  
PID: 47  
Slave response send  
Master response received  
47 11 22 33 44 55  
PID: 8  
Slave response send  
Master response received  
8 11 22 33 44 4d  
PID: 49  
Slave response send  
Master response received  
49 11 22 33 44 55 0  
PID: ca  
Slave response send  
Master response received  
ca 11 22 33 44 55 35  
PID: 8b  
Slave response send  
Master response received  
8b 11 22 33 44 55 66 99  
PID: 4c  
Slave response send  
Master response received  
4c 11 22 33 44 55 66 4d  
PID: d  
Slave response send  
Master response received  
d 11 22 33 44 55 66 77 22  
PID: 8e  
Slave response send  
Master response received  
8e 11 22 33 44 55 66 77 93  
PID: cf  
Slave response send  
Master response received  
cf 11 22 33 44 55 66 77 88 99  
PID: 50  
Slave response send  
Master response received  
50 11 22 33 44 55 66 77 88 49  
PID: 11  
Master response send  
Slave response received  
11 11 ee  
PID: 92  
Master response send  
Slave response received  
92 11 5c  
PID: d3  
Master response send  
Slave response received  
d3 11 22 cc  
PID: 14  
Master response send  
Slave response received  
14 11 22 b8
```

2.32 LU

2.32.1 Description

This demo project shows how to use LU module to implement GPIO trigger FCPIT through the SDK.

This demo project will toggle LED after 1-second when PortC11 has an input signal.

2.32.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.32.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
TRGMUX IN10(PTC11)	C11 (P5100) --- connect key or other to VCC

Note: Use UART by USB-Type C port and connect 12V power.

2.32.4 Steps to Run

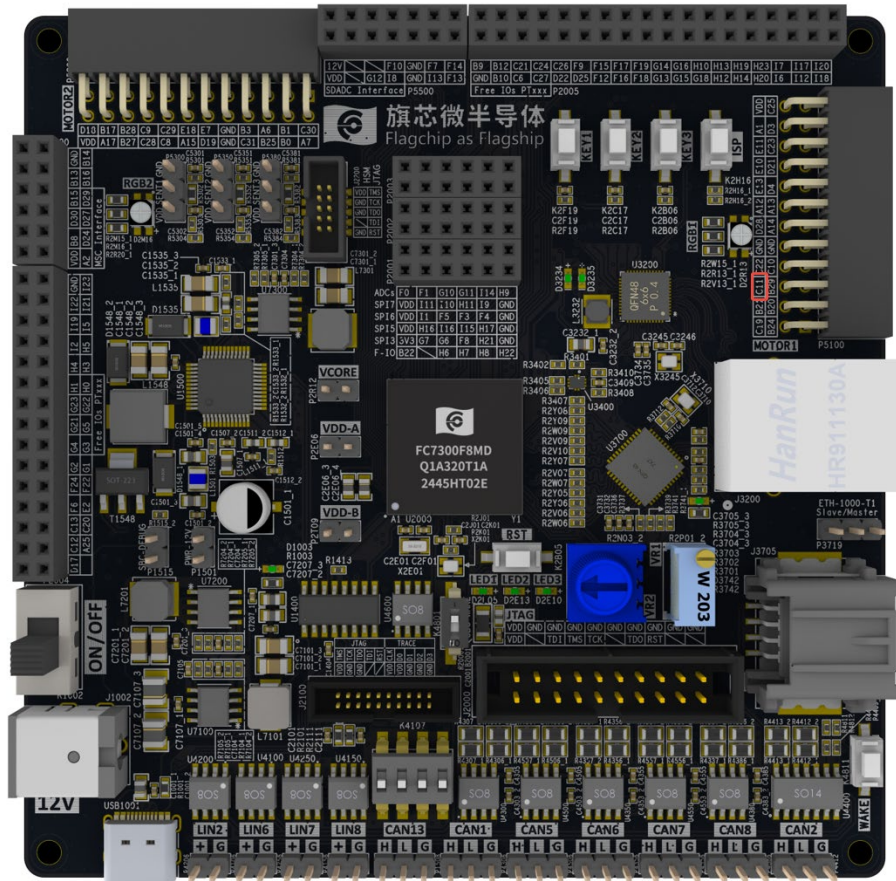
The steps to run the example are as follows:

- 1) Import the LU_Demo project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) Connect PTC11 with VCC, the LED will toggle after 1 second.



2.33 Mailbox

2.33.1 Description

There are three demos: Mutex, ShortMessage, and LongMessage.

- **Mutex:** This example shows how to use the MB module as a mutex between multiple cores.
- **ShortMessage:** This example shows how to use the MB module to send short message between multiple cores.
- **LongMessage:** This example shows how to use the MB module to send long message between multiple cores.

2.33.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger
- 1 Serial port adapter board

2.33.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	Output information of Core 0
Port A19: MUX = ALT3, UART1_RX	-
Port D, MUX = 1, GPIO Output	LED(D2E13) control pin
Port I18, MUX = 4, FCUART2_TX	Output information of Core 1

2.33.4 Steps to Run

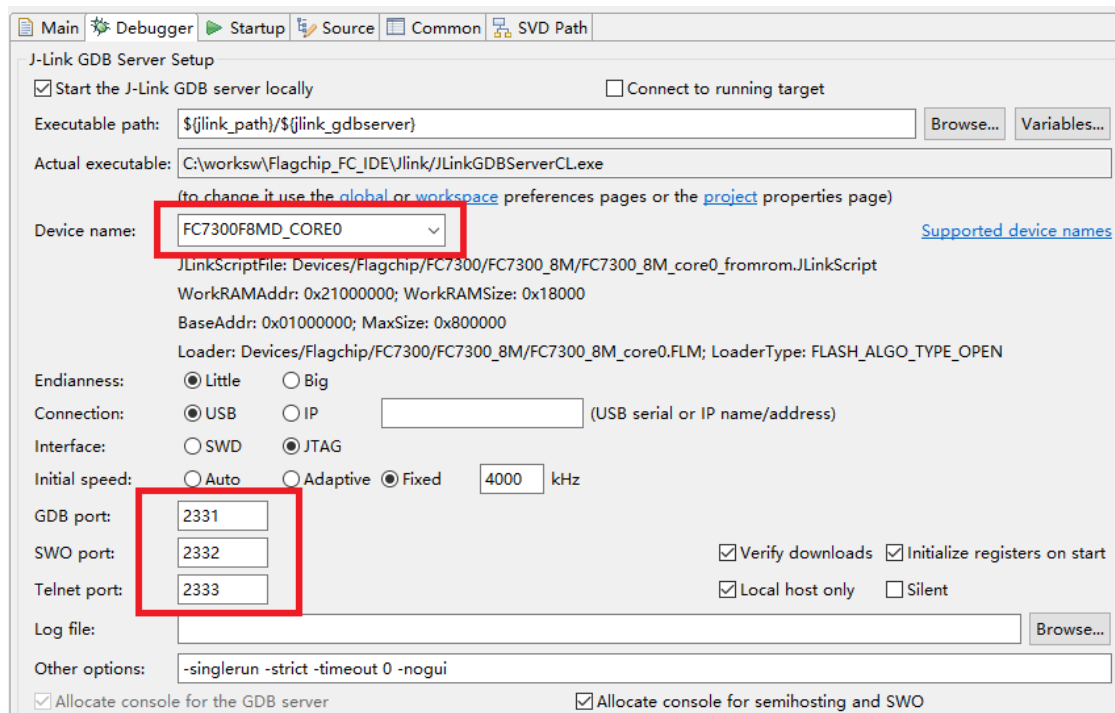
First, connect the USB Type-C cable to the demo board and PC. And open the UART debug terminal (USB-SERIAL CH340/115200/8/N/1).

Then, connect PTI18 to a serial TXD port(115200/8/n/1).

2.33.4.1 For FC_IDE

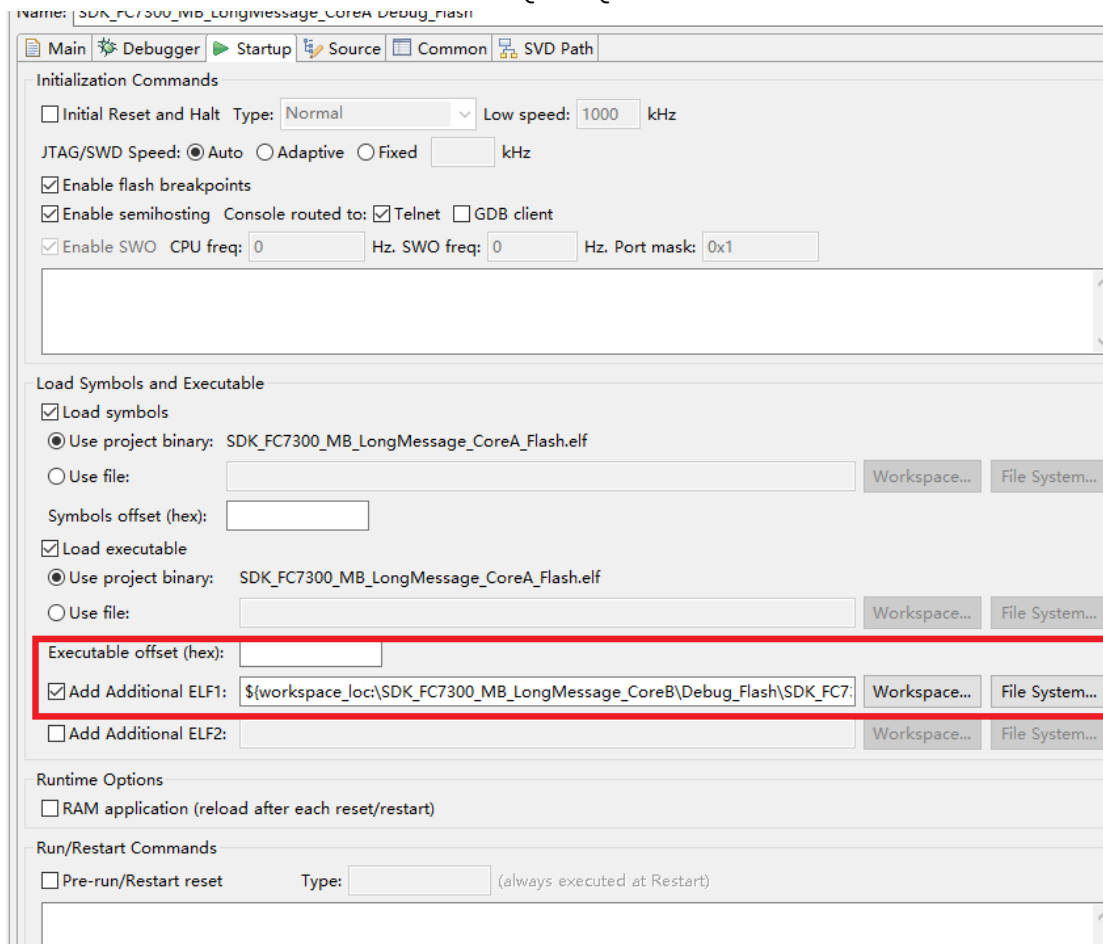
To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

- 1) Open two workspaces to debug CoreA and CoreB.
- 2) Debug Settings for CoreA:
 - a) **Device Name:** FC7300F8MD_CORE0.
 - b) **GDB port:** 2331.
 - c) **SWO port:** 2332.
 - d) **Telnet port:** 2333.



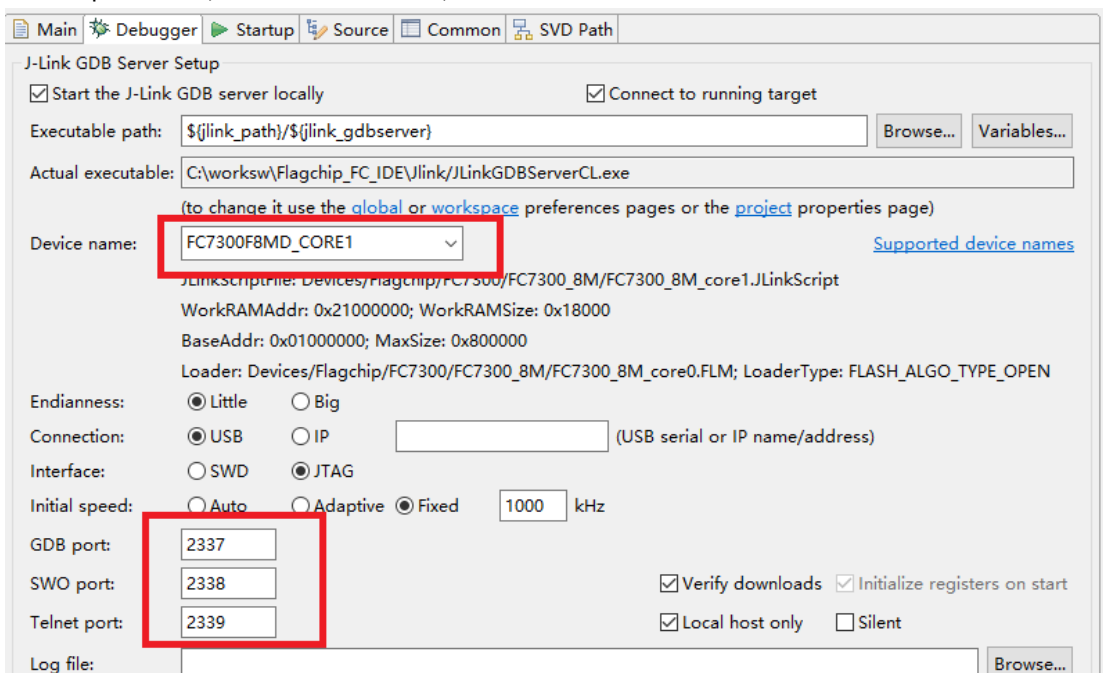
- e) **Add Additional ELF1:** select the Core B image.

FC7300F8MDQ SDK Quick Start

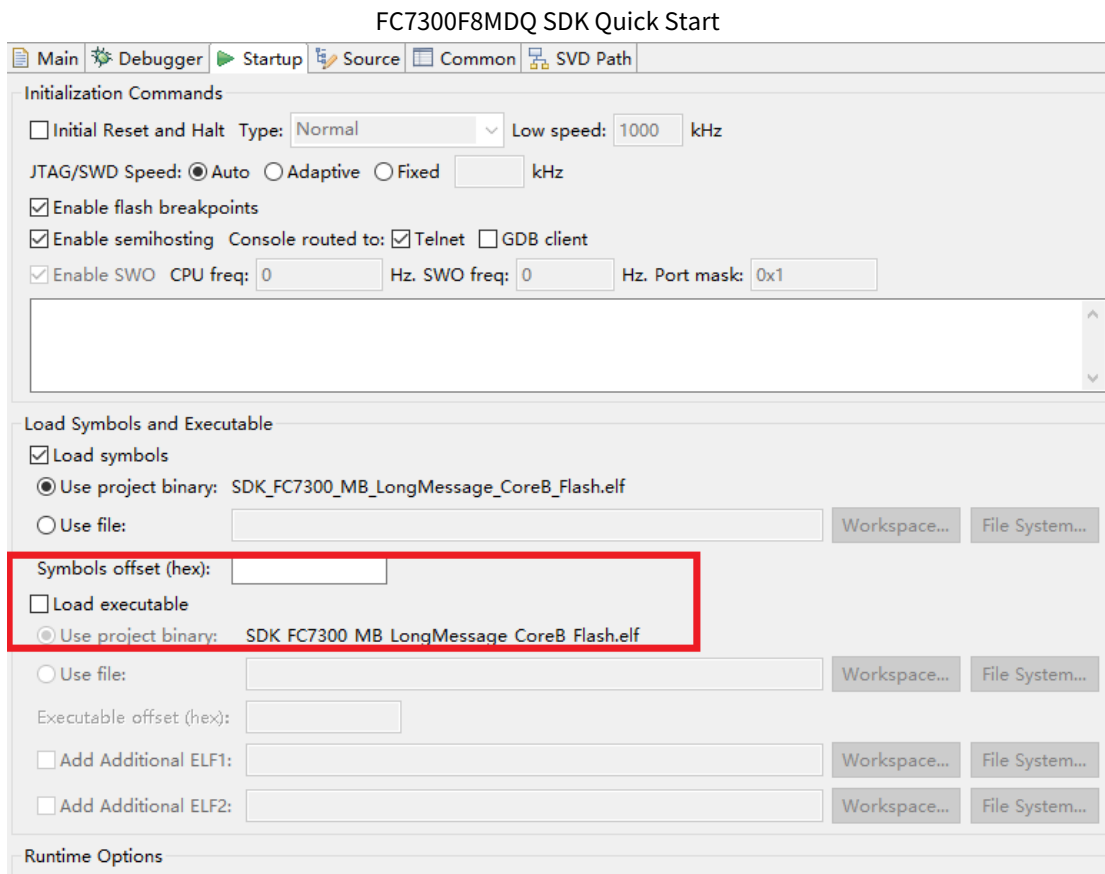


3) Debug Settings for CoreB:

- Device Name: FC7300F8MD_CORE1.
- GDB port: 2337(Different from CoreA).
- SWO port: 2338(Different from CoreA).
- Telnet port: 2339(Different from CoreA).



- Switch off **Load executable**.



- 4) Debug two Cores.

2.33.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\Mailbox\LongMessage (or Mutex)_Project\IAR_Project, and double-click the IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.33.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.34 MAM

2.34.1 Description

This demo is just for mam drive test. The MAM module provides an integrated, scalable architecture framework for access control, memory protection, and peripheral isolation. It allows software to configure Master (including core and Non-core) to access the access rights of each block of Slave (memories or peripherals), namely read, write, execute, and supervisor access. At the same time, it supports locking the access policy of each block to prevent malicious programs from tampering. In addition, each Master on the MAM has an independent watchdog, which is used to prevent the slave from continuing to occupy the bus if it does not respond for a long time.

2.34.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.34.3 MCU Ports

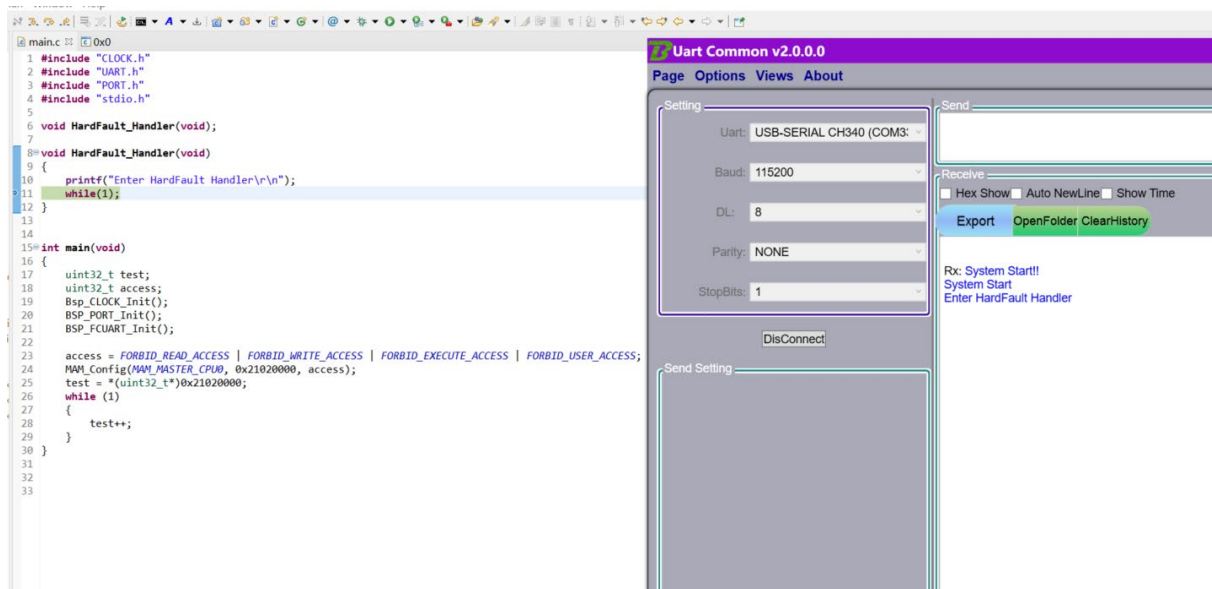
N/A

2.34.4 Steps to Run

2.34.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

- 1) Plug in the USB Type-C cable and open the up-machine (UART1 115200/8/N/1).
- 2) Import the **MAM_Demo** project, then build and download it.
- 3) Click run and then click suspend, you will see the print information, as shown in the figure below.



Note: Because in MAM_Config, the MAM_MASTER_CPU0 has been configured with block (start at 0x21020000) permission of 4'b1111. A hard fault will be generated when it has read access, when it has write access, when it fetches instructions from the block, or when it is accessed in user mode. At this time, the Master cannot have any access to the block.

2.34.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\MAM\MAM_Demo_Project\IAR_Project, and double-click the **MAM_Demo** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.34.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.35 MPU

2.35.1 Description

MPU is realized by the ARM Cortex M7 kernel. The driver just provides APIs to enable/disable the MPU and MPU region. For the details, refer to the related documents of the ARM Corporation.

2.35.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V or micro-USB cable
- 1 Personal computer
- 1 J-Link debugger

2.35.3 MCU Ports

N/A.

2.35.4 Steps to Run

2.35.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

After building and debugging the project, take the following steps:

- 1) Break the point before line 27 `dwData = *((uint32_t *)MPU_TEST_ADDR);` and line 44 `dwFlagVar=3U`, as shown in the figure below.

```

43 int main(void)
44 {
45     uint32_t dwData;
46     uint32_t dwFlagVar;
47
48     dwData = *((uint32_t *)MPU_TEST_ADDR);
49     (void)dwData;
50     dwFlagVar = 2U;
51     (void)dwFlagVar;
52
53     Mpu_Config();
54     /* illegal operation, exception occurred */
55     dwData = *((uint32_t *)MPU_TEST_ADDR);
56     (void)dwData;
57     dwFlagVar = 3U;
58     (void)dwFlagVar;
59
60     while (1)
61     {
62         delay(1000U);
63     }
64 }
```

- 2) Click **Run at Full Speed** to successfully access the 0x21010000 address in line 48.
 - a) Perform MPU Config in line 36. The config code is shown in Figure below.
 - b) Set the MPU starting address as 0x21001000, the size as 4KB, and the permission as inaccessible.
 - c) After config, 0x21010000 is not accessible.
- 3) Continue to click **Run at Full Speed**. Due to 0x21001000 access in the line 42, a memory error will occur, and the memory management error will be interrupted (as shown in the figure below). The next and 54 lines of code will not be executed.

```
66 void MemManage_Handler(void)
67 {
68     while(1);
69 }
70
```

2.35.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\MPU\Mpu_Sample_Project\IAR_Project, and double-click the **Mpu_Example** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.35.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

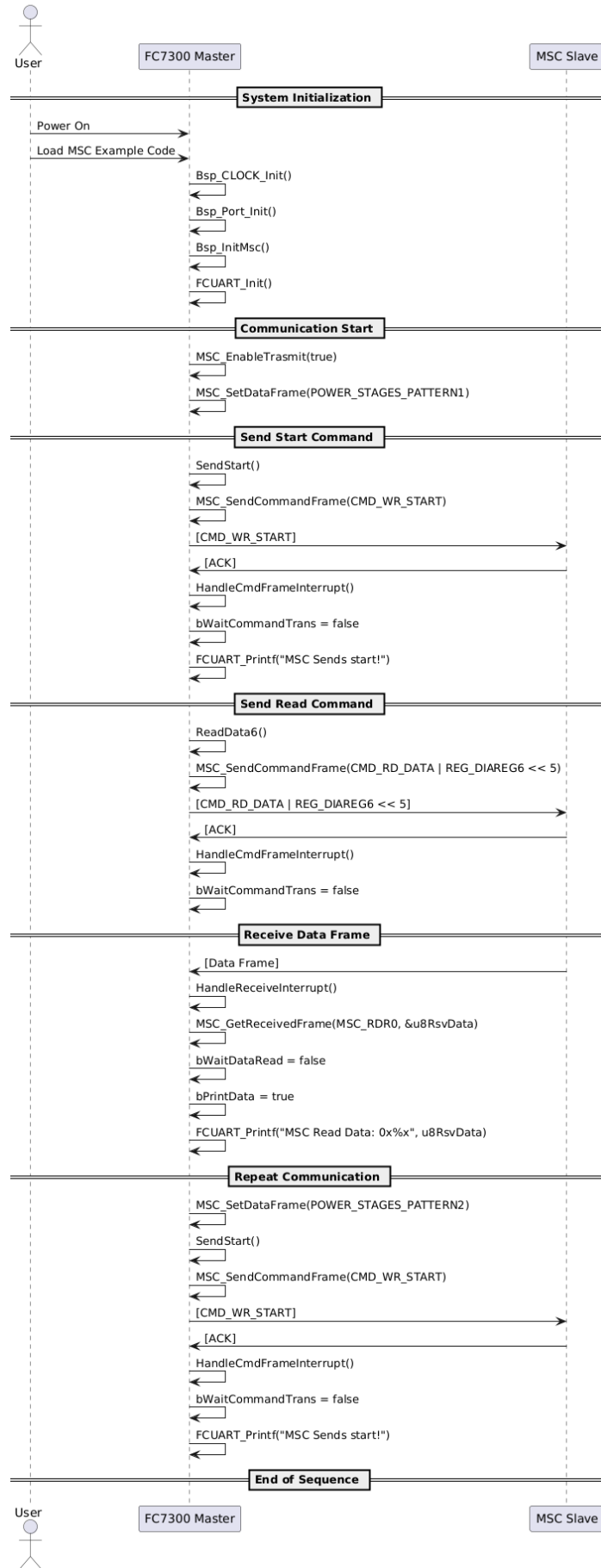
2.36 MSC

This guide provides a step-by-step quick start for testing the MSC (Master Serial Communication) function using the demo board as the master and an MSC slave board (TLE8718) as the slave. The example demonstrates how the master board sends data frames and command frames repeatedly to query the output stage of the slave board.

Timing Diagram:

FC7300F8MDQ SDK Quick Start

MSC Example Sequence Diagram



2.36.1 Description

This example demonstrates the functionality of the MSC module on the demo board. The master board sends data frames and command frames to the MSC slave board (TLE8718) to control and query its output stages. The communication results are logged via a UART interface.

2.36.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 MSC slave board (TLE8718)
- 1 USB-2-UART cable
- 1 Personal computer
- 1 J-Link debugger
- UART debug terminal software (e.g., Tera Term, Putty)

2.36.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A2: MUX = ALT4, MSC1 FCLP	-
Port D24: MUX = ALT4, MSC1 SOP	-
Port B8: MUX = ALT5, MSC1 SDI0	-
Port D27: MUX = ALT5, MSC1 EN0	-

Make the following connections between the demo board and the MSC slave board:

- 1) Power Supply:
 - a) Switch the demo board to 3.3V power supply.
 - b) Connect the Vref pin on the MSC slave board to the 3.3V pin on the demo board.
 - c) Connect the GND pins between the two boards.
- 2) Signal Connections:
 - a) Connect FCLP (FC7300) to FCL-SE (MSC slave).
 - b) Connect SOP (FC7300) to SO-SE (MSC slave).
 - c) Connect SDI0 (FC7300) to SDI (MSC slave).
 - d) Connect EN0 (FC7300) to SSY1 (MSC slave).

2.36.4 Steps to Run

The steps to run the example are as follows:

- 1) Power on both the FC7300F8MDQ Demo Board and the MSC slave board.
- 2) Plug in the USB-2-UART cable and open UART debug terminal (UART1 115200/8/N/1) for the FC7300F8MDQ Demo Board.
- 3) Import the MSC_Demo project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 4) The LEDs on the MSC slave board will blink to indicate active communication, and the UART terminal will print the following text.

```
[2023-02-08 04:57:05.975]RX: MSC Demo start.
[2023-02-08 04:57:06.144]RX: MSC Sends start!
MSC Read Data: 0xec
[2023-02-08 04:57:06.499]RX: MSC Read Data: 0x55
[2023-02-08 04:57:06.834]RX: MSC Read Data: 0xaa
[2023-02-08 04:57:07.170]RX: MSC Read Data: 0xaa
[2023-02-08 04:57:07.505]RX: MSC Read Data: 0xaa
[2023-02-08 04:57:07.840]RX: MSC Read Data: 0x55
[2023-02-08 04:57:08.176]RX: MSC Read Data: 0x55
[2023-02-08 04:57:08.510]RX: MSC Read Data: 0x55
[2023-02-08 04:57:08.845]RX: MSC Read Data: 0xaa
[2023-02-08 04:57:09.181]RX: MSC Read Data: 0xaa
[2023-02-08 04:57:09.516]RX: MSC Read Data: 0xaa
[2023-02-08 04:57:09.851]RX: MSC Read Data: 0x55
[2023-02-08 04:57:10.186]RX: MSC Read Data: 0x55
[2023-02-08 04:57:10.522]RX: MSC Read Data: 0x55
[2023-02-08 04:57:10.857]RX: MSC Read Data: 0xaa
[2023-02-08 04:57:11.195]RX: MSC Read Data: 0xaa
[2023-02-08 04:57:11.528]RX: MSC Read Data: 0xaa
[2023-02-08 04:57:11.863]RX: MSC Read Data: 0x55
[2023-02-08 04:57:12.198]RX: MSC Read Data: 0x55
[2023-02-08 04:57:12.534]RX: MSC Read Data: 0x55
[2023-02-08 04:57:12.868]RX: MSC Read Data: 0xaa
[2023-02-08 04:57:13.204]RX: MSC Read Data: 0xaa
```

Notes:

- Ensure the power supply and signal connections are correct to avoid communication errors.
- Adjust the UART baud rate and other settings as needed for your specific setup.
- The example code includes a delay function to control the communication frequency. Modify this function if you need to adjust the timing.

2.37 Multicore

2.37.1 Description

This demo is just for MultiCore test, and demo content contains UART and GPIO.

2.37.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type C cable
- 1 Personal computer
- 1 Power supply

2.37.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A26, Core0 Led Toggle	LED1
Port D31, Core1 Led Toggle	LED2

Port A14, Core2 Led Toggle	LED3
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

2.37.4 Steps to Run

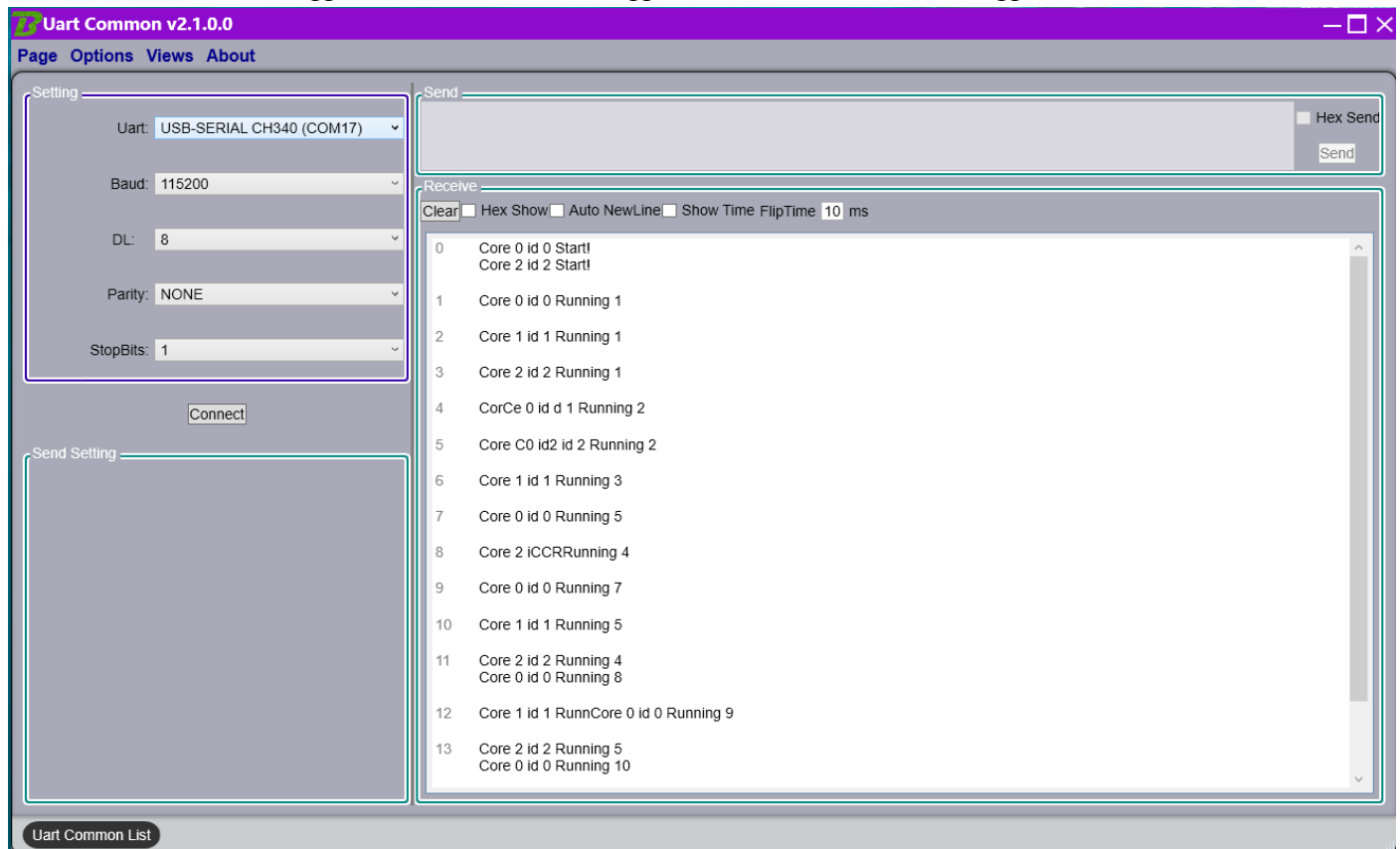
The steps to run the example are as follows:

- 1) Plug in the USB type c cable and open UART up-machine (115200Kbps).
- 2) Download the MultiCore Demo to MCU.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 3) Normally, Core 0 will upload the toggle message with core ID.
- 4) Besides, Core 0 toggles Port A26 LED, Core 1 toggles Port D31 LED, and Core 2 toggles Port A14 LED.



2.38 OTA

2.38.1 OTA_RegisterMode

2.38.1.1 Description

The purpose of this demo is to demonstrate the basic usage of chip OTA through direct register operations.

OTA_RegisterMode demonstrates how to implement flash bank swap by directly operating registers. This type of bank swap takes effect immediately after register configuration, meaning the specified swap range is exchanged right away.

This project consists of two parts: Bootloader code and APP code. The bootloader code checks the validity of the APP in the main flash area and the backup area. If both are invalid, it flashes the preserved APP code to the main area. Otherwise, it determines whether to configure the OTA register based on the version numbers in the main and backup areas, then jumps to the address of the main area APP.

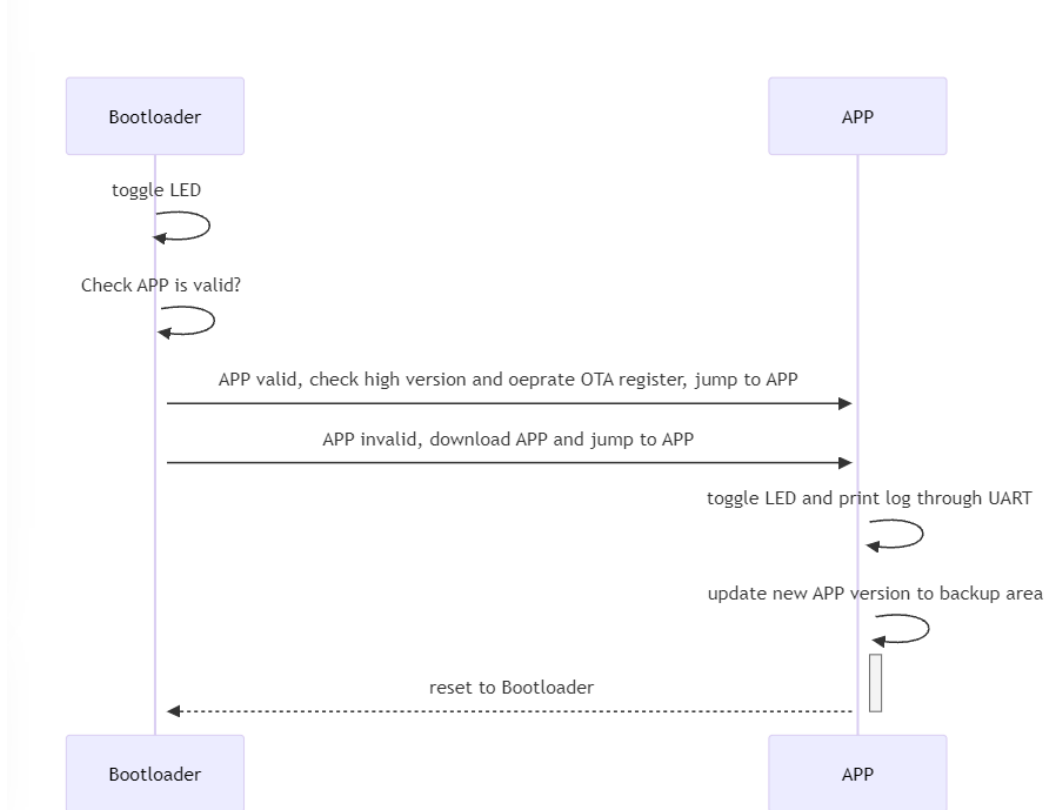
The project has two compilation options for building bootloader and APP code respectively. For FC7300F8MDQT1B project, the linker file shows BOOTLOADER code and APP code sections. In register-based bank swap mode, the FLASH_APP_VER area can also be defined in other areas such as dflash to store version information, or it can be omitted entirely.

The bootloader code toggles the LED state every 200 milliseconds, while the APP code toggles the LED state every 1 second and prints the value of the OTA register. If "a" is printed, no bank swap has occurred. If "2a" is printed, a bank swap has been performed.

2.38.1.2 Code Logic

- 1) The bootloader checks the APP version information in both bank areas. If both versions are valid, it performs bank swap by configuring FMC->OTA registers based on version numbers and jumps to the APP. If both versions are invalid, it flashes the APP code from AppCode.c to the APP area using the flash driver. After flashing, it jumps to the APP code.
- 2) The APP prints version information 5 times, then increments the current version number by 1 and updates it to bank B.
- 3) Both bootloader and APP code toggle LEDs, with the bootloader toggling at a faster rate.

The sequence diagram of the demo is as follows:



2.38.1.3 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.38.1.4 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A26: MUX = ALT1, GPIO Output	LED1 – wired on board
Port D31: MUX = ALT1, GPIO Output	LED2 – wired on board
Port A14: MUX = ALT1, GPIO Output	LED3 – wired on board

2.38.1.5 Steps to Run

The steps to run the example are as follows:

- 1) Download the OTA_DEMO program to the MCU by JTAG.

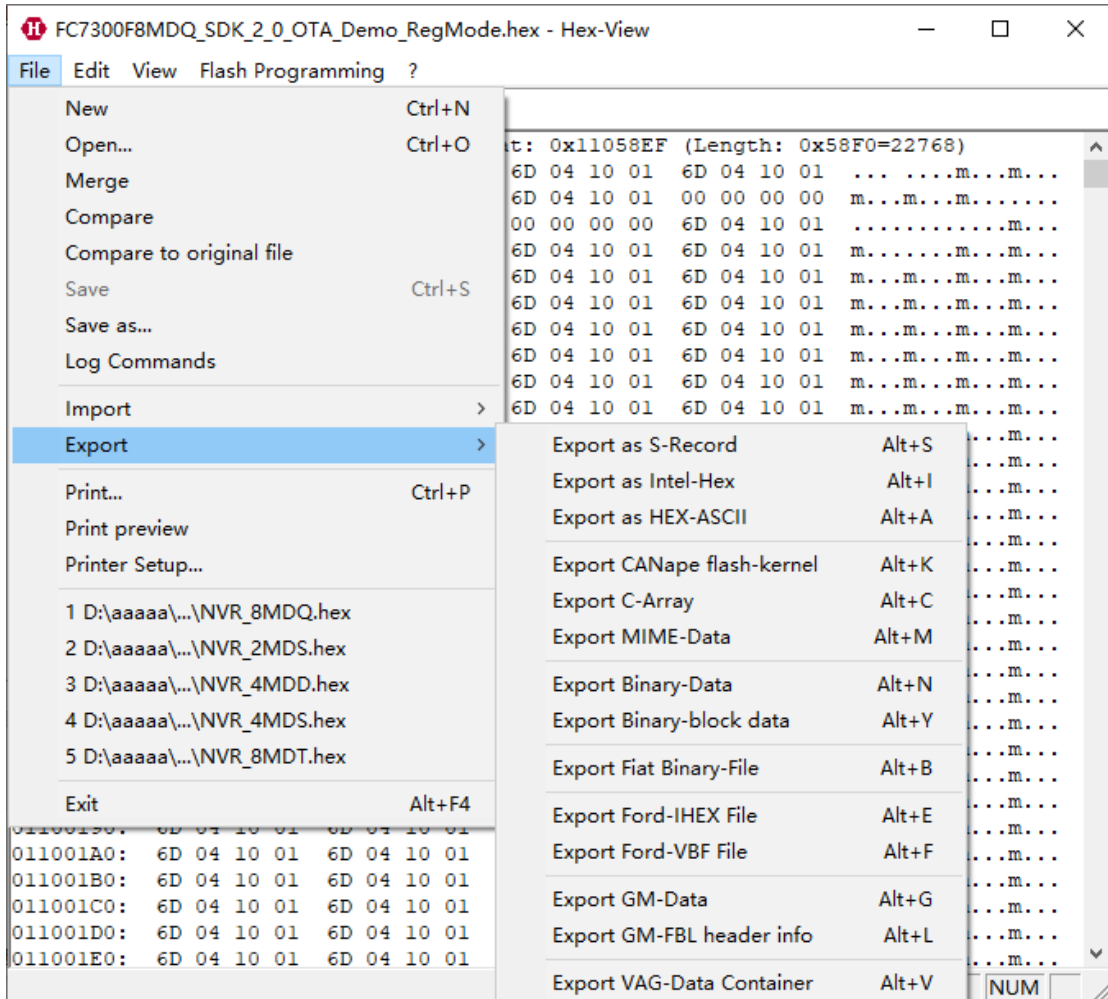
Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

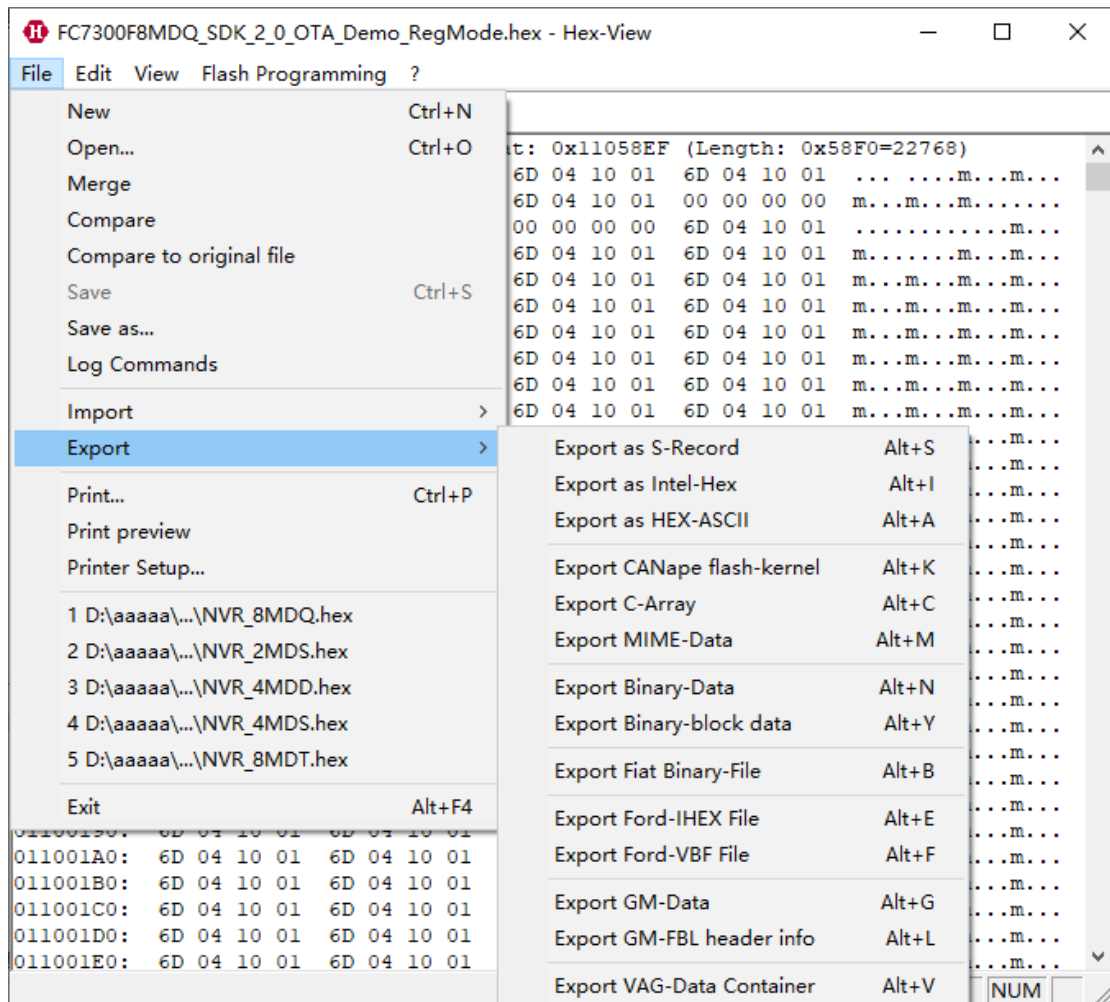
- 2) Then the LEDs on the board will start to toggle.

2.38.1.6 Compilation and Programming Steps

- 1) After compiling to generate two hex files, convert the APP text content into a byte array and add it to AppCode.c.
This can be done using the hexview tool to extract data from the hex file:
> Select Export C-Array



> Choose the output file path, then add the generated byte data and length to AppCode.c/h



- 2) Program the bootloader code to the device

2.38.1.7 Notes

- 1) FC7300F8MDQT1B chip has 4 banks and 8 blocks, each block is 1M. Block0/1 can be swapped with block4/5, and block2/3 can be swapped with block6/7. The swap range can be configured and must be 8K aligned.
- 2) As shown in the demo, for APP code, if you want to update the version, always pass the address of the higher bank to the flash driver. The actual physical address is automatically mapped by hardware. For example, if bank 2 and bank 3 have been swapped, and the current APP is running from physical bank 3, writing to bank 3 address through the flash driver will actually operate on the physical content of lower bank 2.
- 3) If third-party firmware configuration is enabled, NVR UFM disable is not 0xAAAA5555, and the chip starts from ROM, register-based OTA is not available.

2.38.2 OTA_NvrMode

2.38.2.1 Description

The purpose of this demo is to demonstrate the basic usage of chip OTA through NVR configuration.

OTA_NvrMode demonstrates how to implement flash bank swap through NVR area configuration. This type of bank swap takes effect immediately after reset, meaning it is active before the first instruction executes after reset.

This project consists of two parts: Bootloader code and APP code. The bootloader code checks the validity of the APP in the main flash area and the backup area. If both are invalid, it flashes the preserved APP code to the main area. Otherwise, it jumps to the address of the main area APP.

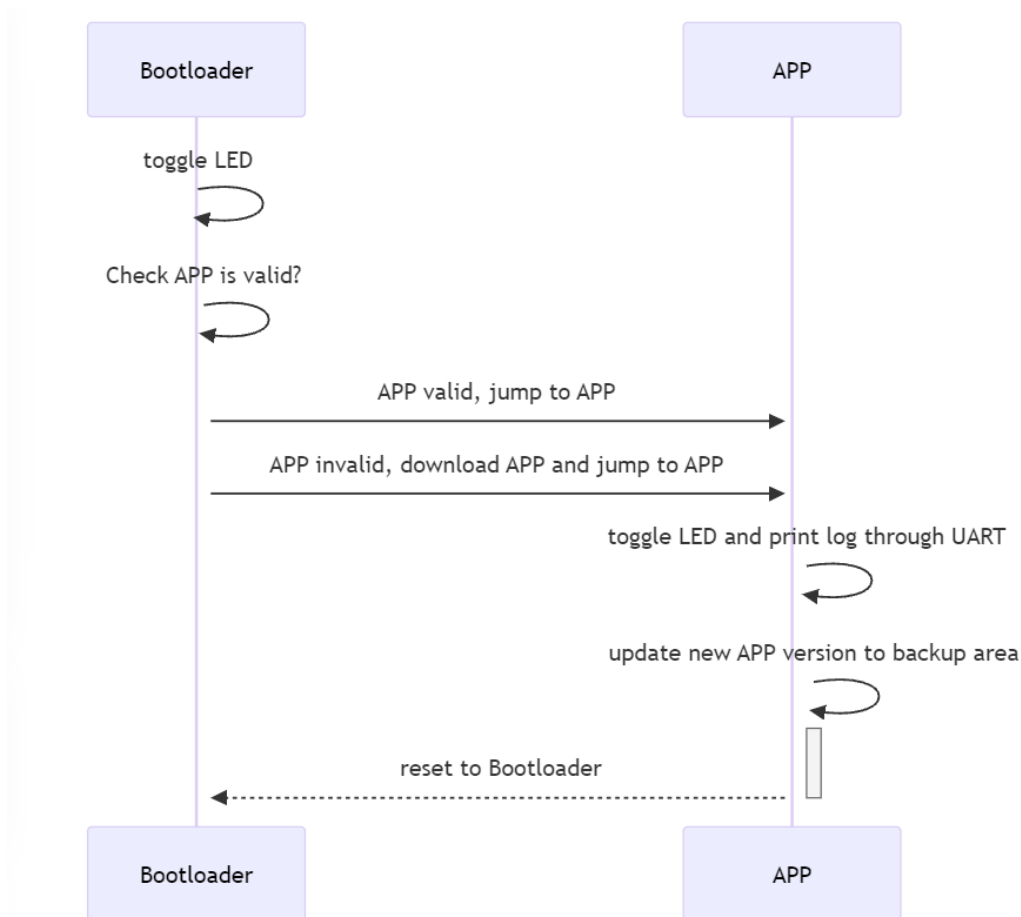
The project has two compilation options for building bootloader and APP code respectively. For FC7300F8MDQT1B project, the linker file shows BOOTLOADER code and APP code sections. In NVR-based bank swap mode, the FLASH_APP_VER area definition must match the offset configured in NVR. For example, if FLASH_APP_VER addresses are 0x015FF000 and 0x17FF000, the version location offset in NVR should be configured as 0xFF000, and the version number should be placed in the higher block.

The bootloader code toggles the LED state every 200 milliseconds, while the APP code toggles the LED state every 1 second and prints the value of the OTA register. If "a" is printed, no bank swap has occurred. If "2a" is printed, a bank swap has been performed.

2.38.2.2 Code Logic

- 1) The bootloader checks the APP version information in both bank areas. If both versions are valid, it jumps to the APP. If either version is invalid, it flashes the APP code from AppCode.c to the APP area using the flash driver and jumps to the APP.
- 2) The APP prints version information 5 times, then increments the current version number by 1 and updates it to bank B.
- 3) Both bootloader and APP code toggle LEDs, with the bootloader toggling at a faster rate.

The sequence diagram of the demo is as follows:



2.38.2.3 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer

- 1 J-Link debugger

2.38.2.4 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A26: MUX = ALT1, GPIO Output	LED1 – wired on board
Port D31: MUX = ALT1, GPIO Output	LED2 – wired on board
Port A14: MUX = ALT1, GPIO Output	LED3 – wired on board

2.38.2.5 Steps to Run

The steps to run the example are as follows:

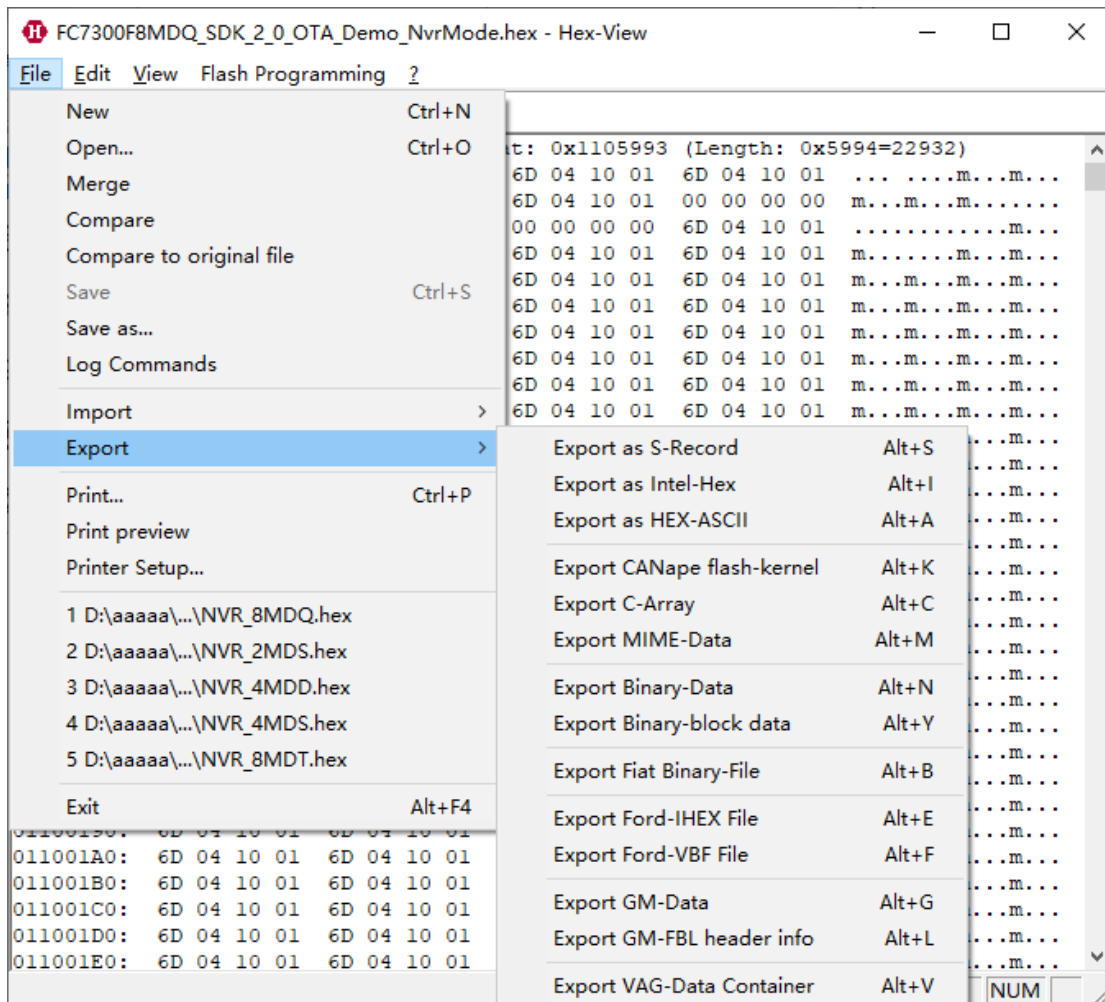
- 1) Run unlock script of specific chip first
- 2) Download NVR file which in the folder OTA_NvrMode\ Nvr_config
- 3) Download the OTA_DEMO program to the MCU by JTAG.

Note:

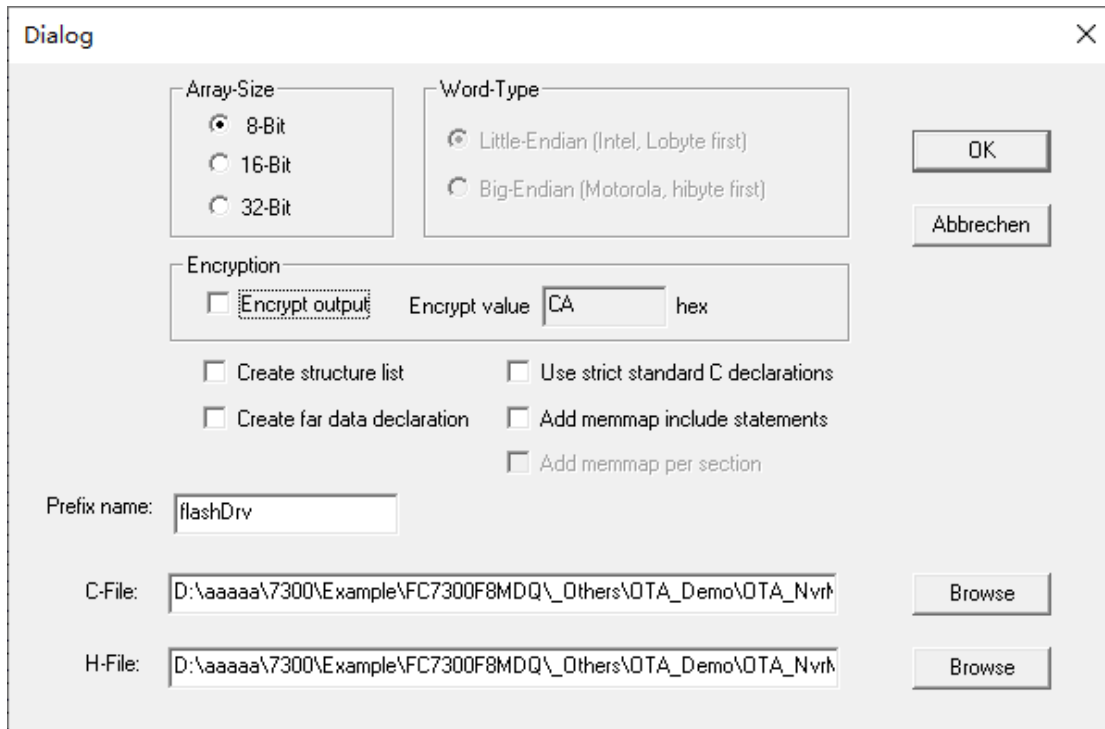
- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
 - To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
 - To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).
- 4) Then the LEDs on the board will start to toggle.

2.38.2.6 Compilation and Programming Steps

- 1) After compiling to generate two hex files, convert the APP text content into a byte array and add it to AppCode.c.
This can be done using the hexview tool to extract data from the hex file:
> Select Export C-Array



> Choose the output file path, then add the generated byte data and length to AppCode.c/h



- 2) Before programming, it is recommended to run the unlock script once, then download the corresponding chip's NVR configuration from OTA_NvrMode/Nvr_config, and finally program the bootloader code.

- 3) NVR configuration mainly targets OTAC0/OTAC_HIGH0/ OTAC1/OTAC1_HIGH0. OTAC0 controls banks 0/2 and OTAC1 controls banks 1/3. OTAC need to be configured with OTA enable, bank swap start address offset, end address offset, OTA version offset, and which block the OTA version is located in.

2.38.2.7 Notes

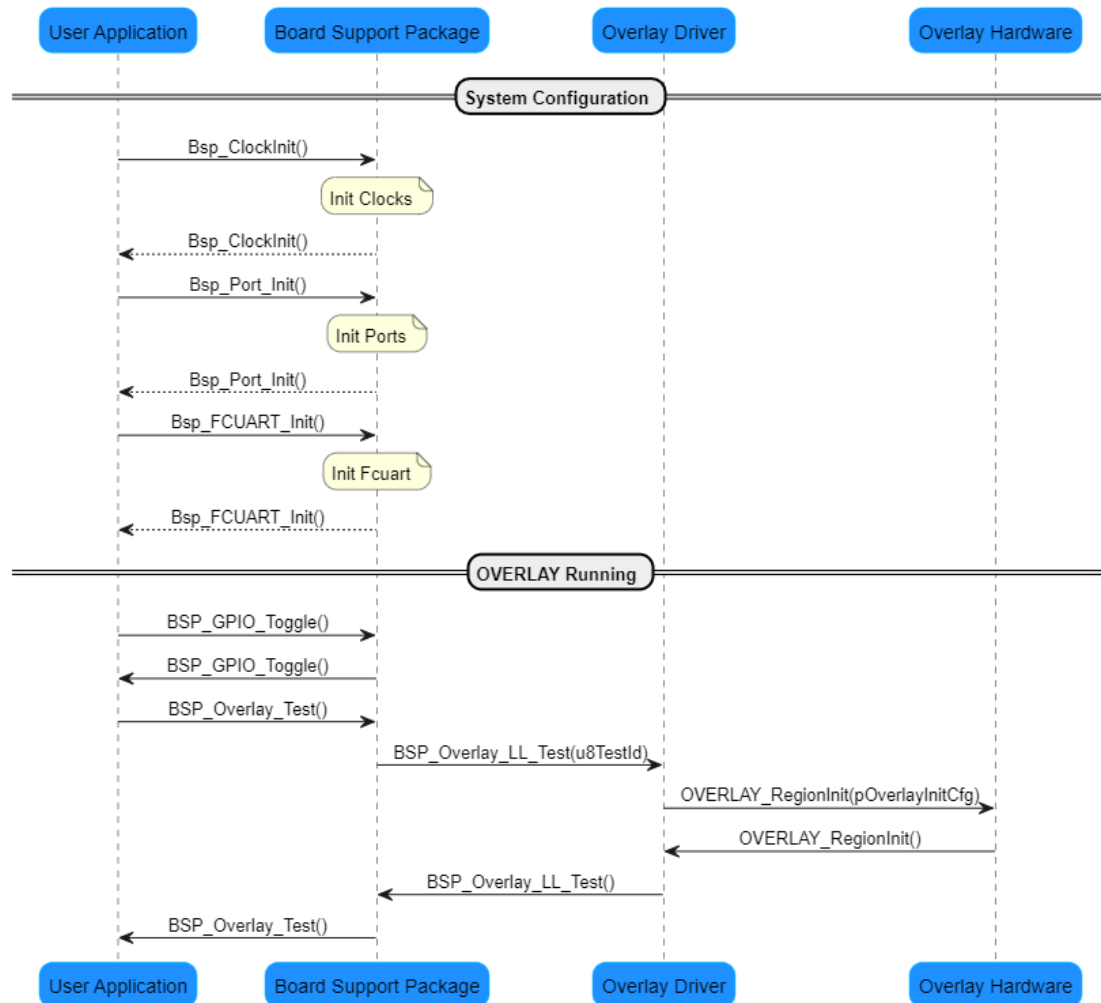
- 1) FC7300F8MDQT1B chip has 4 banks and 8 blocks, each block is 1M. Block0/1 can be swapped with block4/5, and block2/3 can be swapped with block6/7. The swap range can be configured and must be 8K aligned.
- 2) As shown in the demo, for APP code, if you want to update the version, always pass the address of the higher bank to the flash driver. The actual physical address is automatically mapped by hardware. For example, if bank 2 and bank 3 have been swapped, and the current APP is running from physical bank 3, writing to bank 3 address through the flash driver will actually operate on the physical content of lower bank 2.

2.39 Overlay

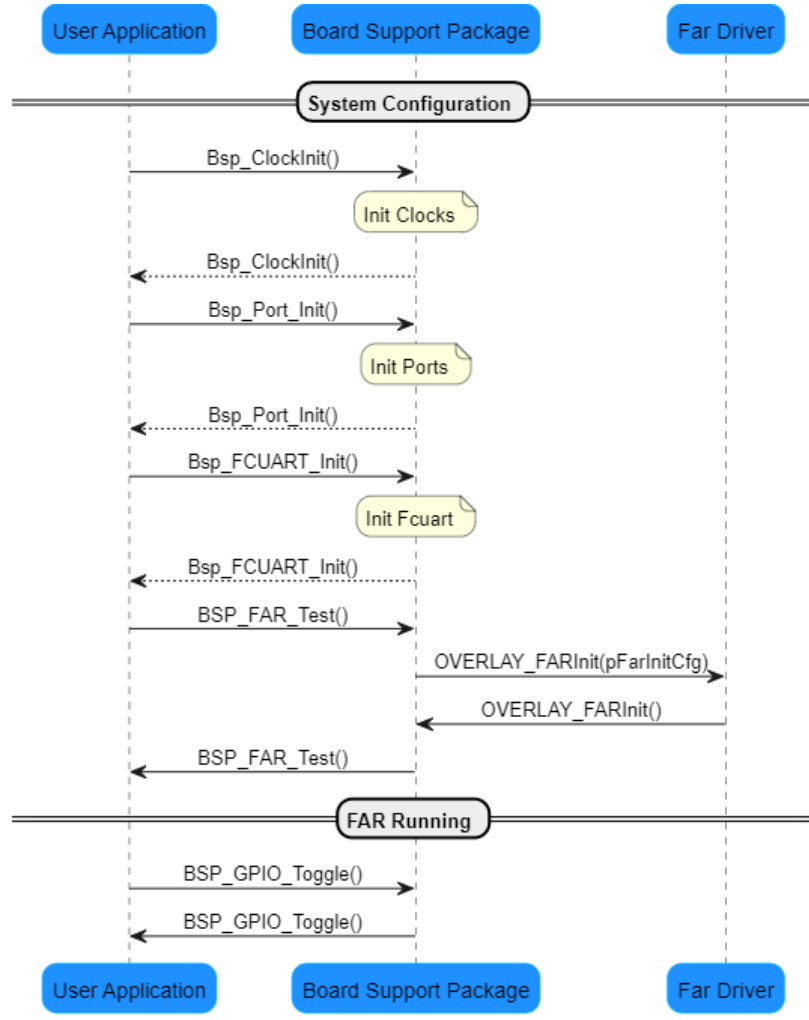
2.39.1 Description

This demo is just for Overlay and FAR test.

The sequence diagram of the overlay demo is as follows.



The sequence diagram of the far demo is as follows.



2.39.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.39.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A26: MUX = ALT1, GPIO OUT	GPIO Port
Port D31: MUX = ALT1, GPIO OUT	GPIO Port

2.39.4 Steps to Run

The steps to run the example are as follows:

- 1) Plug in the 12V power.
- 2) Plug in the USB Type-C cable and open the up-machine (115200/8/N/1).
- 3) Download the OVERLAY or FAR program to the MCU by JTAG.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 4) Normally, MCU will upload the data message for reading.
 - For OVERLAY, MCU always reads at the flash address, so there will be different data print for reading before overlaying and after overlaying. After overlaying, read flash will be remapped to RAM.
 - For FAR, MCU always reads at 0x08000000, but the real data is in flash.

```

[15:03:49.332]收←◆System Start
System Start r
Region 0 overlay

[15:03:55.959]收←◆Read In Flash 0 I am In RAM for Overlay Region 0

[15:03:57.299]收←◆Read In Flash 1 I am In Flash for Overlay Region 1

[15:04:00.135]收←◆Read In Flash 2 I am In Flash for Overlay Region 2

[15:04:03.502]收←◆Region 1 overlay
Read In Flash 0 I am In Flash for Overlay Region 0
Read In Flash 1 I am In RAM for Overlay Region 1
Read In Flash 2 I am In Flash for Overlay Region 2

[15:04:04.366]收←◆Region 2 overlay
Read In Flash 0 I am In Flash for Overlay Region 0
Read In Flash 1 I am In Flash for Overlay Region 1
Read In Flash 2 I am In RAM for Overlay Region 2

[15:04:05.230]收←◆Region all overlay
Read In Flash 0 I am In RAM for Overlay Region 0
Read In Flash 1 I am In RAM for Overlay Region 1
Read In Flash 2 I am In RAM for Overlay Region 2

[15:04:06.095]收←◆Read In Flash 0 I am In Flash for Overlay Region 0
Read In Flash 1 I am In Flash for Overlay Region 1
Read In Flash 2 I am In Flash for Overlay Region 2

[15:04:06.982]收←◆Region 0 overlay
Read In Flash 0 I am In RAM for Overlay Region 0
Read In Flash 1 I am In Flash for Overlay Region 1
Read In Flash 2 I am In Flash for Overlay Region 2

[15:04:07.847]收←◆Region 1 overlay
Read In Flash 0 I am In Flash for Overlay Region 0
Read In Flash 1 I am In RAM for Overlay Region 1
Read In Flash 2 I am In Flash for Overlay Region 2

[15:04:08.711]收←◆Region 2 overlay
Read In Flash 0 I am In Flash for Overlay Region 0
Read In Flash 1 I am In Flash for Overlay Region 1
Read In Flash 2 I am In RAM for Overlay Region 2

[15:04:09.576]收←◆Region all overlay
Read In Flash 0 I am In RAM for Overlay Region 0
Read In Flash 1 I am In RAM for Overlay Region 1
Read In Flash 2 I am In RAM for Overlay Region 2

[15:04:10.440]收←◆Read In Flash 0 I am In Flash for Overlay Region 0
Read In Flash 1 I am In Flash for Overlay Region 1
Read In Flash 2 I am In Flash for Overlay Region 2

```

Figure 1. Overlay Demo Print Message

```

[15:39:59.376] ←◆ System Start
System Start r
Region 0 overlay
Read In FAR SRC 0 I am In Flash for Overlay Region 0

```

Figure 2. FAR Demo Print Message

2.40 PMC

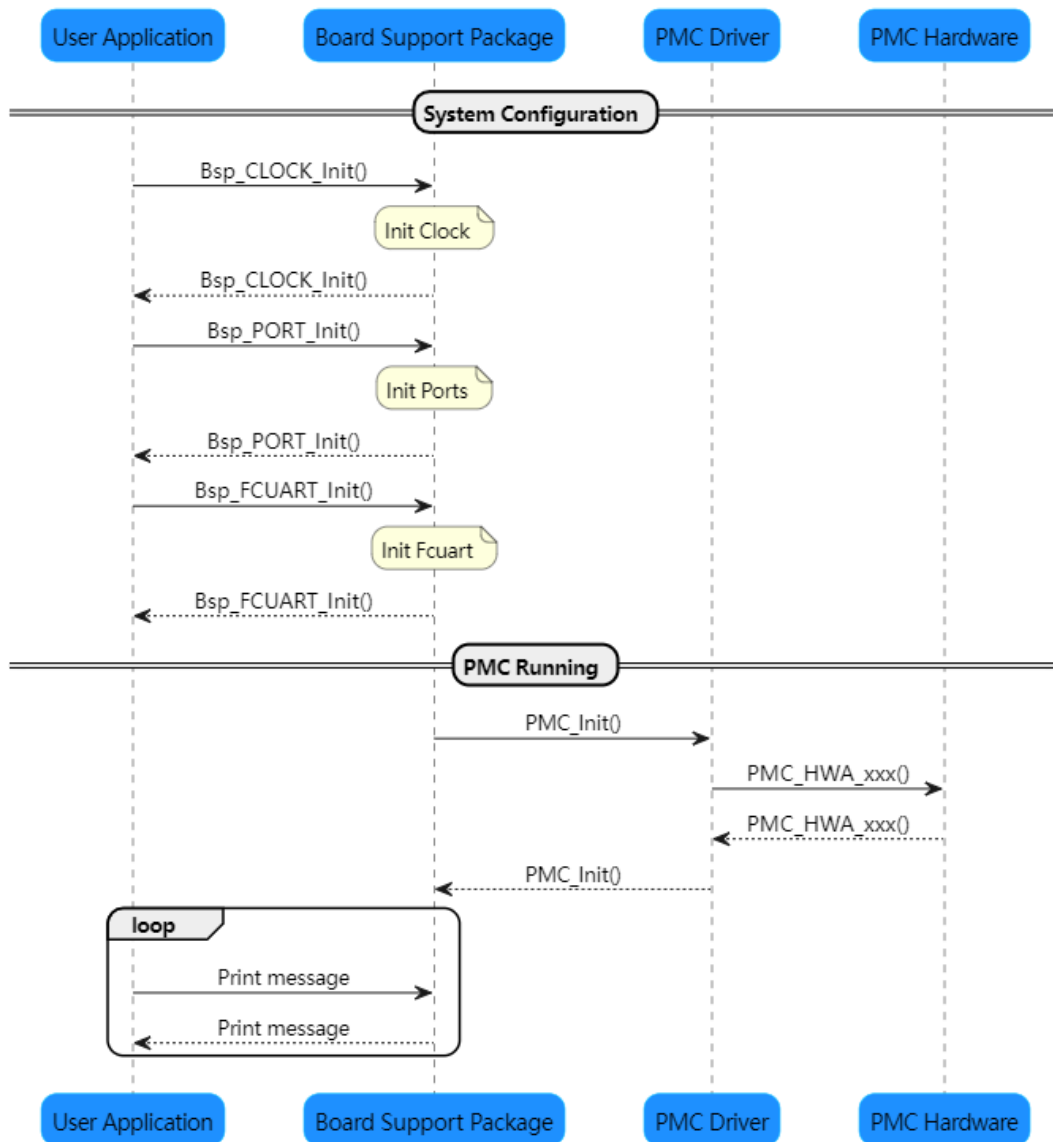
2.40.1 Description

This demo shows how to detect voltage anomalies. An interrupt will be requested when the voltage anomalies are detected.

PMC does not require special operation. It is always responsible for detecting the voltage flag.

Note: As an interrupt function of PMC, the `PMC0_IRQHandler()` will call `PMC_UserIRQHandler()`. The user can modify this function according to the project needs.

The sequence diagram of the demo is as follows:



2.40.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V or micro-USB cable
- 1 Personal computer
- 1 J-Link debugger

2.40.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
V11	Voltage on MCU: 1P1
V25	Voltage on MCU: 2P5V
VDD_HV_A	Voltage on MCU: 5V

2.40.4 Steps to Run

2.40.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

2.40.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\PMC\Pmc_Sample_Project\IAR_Project, and double-click the **Pmc_Sample** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.40.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

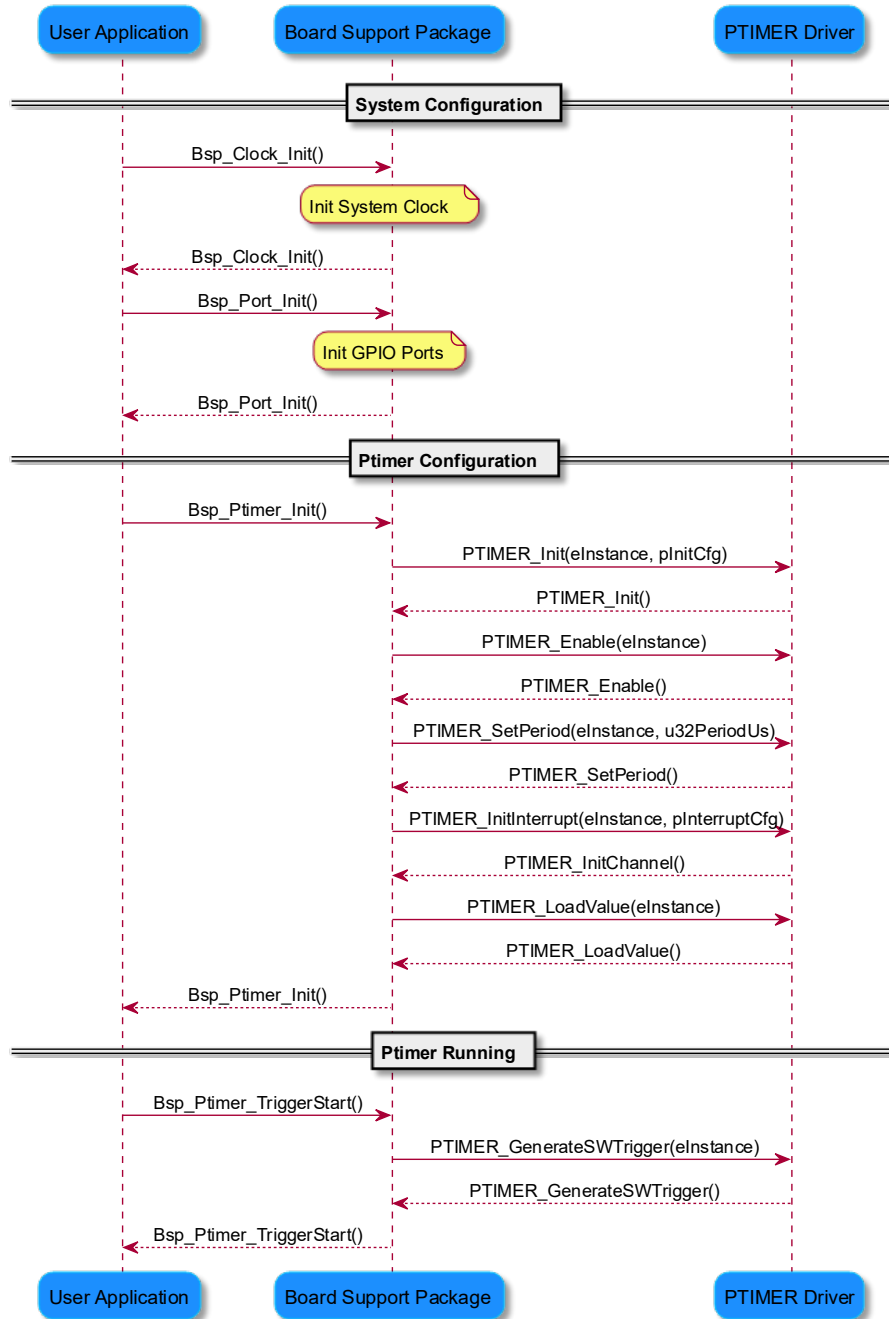
2.41 PTIMER

Ptimer is often used together with ADC, for the case which Ptimer uses with ADC, please refer to the ADC_Discontinuous demo.

2.41.1 Description

This demo shows the standalone usage of Ptimer, which uses Ptimer as a simple periodic timer that will generate interrupt periodically.

The sequence diagram of the demo is as follows:



2.41.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.41.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A26: MUX = ALT1, GPIO Output	LED1
Port D31: MUX = ALT1, GPIO Output	LED2
Port A14: MUX = ALT1, GPIO Output	LED3

2.41.4 Steps to Run

The steps to run the example are as follows:

- 1) Download the PTIMER_Demo program to the MCU by JTAG.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) The LED1, LED2 and LED3 will start to toggle every 50 ms.

2.42 QDT

There are six QDT demos: QDT_IC_Demo, QDT_ICDM_Demo, QDT_ICENM_Demo, QDT_ICEXPENM_Demo, QDT_ICPM_Demo, and QDT_QUAD_Demo.

2.42.1 QDT_IC_Demo

2.42.1.1 Description

This demo shows the general configuration process to use the QDT module with IC mode.

2.42.1.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.42.1.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port H4: MUX = ALT2, QDT0_PHA	-
Port H5: MUX = ALT2, QDT0_PHB	-
Port I2: MUX = ALT1, GPIO	-
Port H3: MUX = ALT1, GPIO	-

2.42.1.4 Steps to Run

2.42.1.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

- 1) Right click on the project name and make sure "Debug_Flash" is selected as the active build configuration.
- 2) Build the project.

- 3) Connect H4 port with I2 port, and connect H5 port with H3 port.
- 4) Create a "GDB SEGGER J-Link Debugging" configuration, and then click Debug.
- 5) Then you can see the below output via the UART tool:

```

System Start!!
-----QDT IC MODE DEMO Start-----
QDTO Channel1 Detect the Edge, CV value is 4537407
QDTO Channel0 Detect the Edge, CV value is 6055597
QDTO Channel1 Detect the Edge, CV value is 10588765
QDTO Channel0 Detect the Edge, CV value is 12107160
QDTO Channel1 Detect the Edge, CV value is 16640532
QDTO Channel0 Detect the Edge, CV value is 18158927

```

2.42.1.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\QDT\QDT_IC_Demo_Project\IAR_Project, and double-click the IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.42.1.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.42.2 QDT_ICDM_Demo

2.42.2.1 Description

This demo shows the general configuration process to use the QDT module with ICDM mode.

2.42.2.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.42.2.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port H4: MUX = ALT2, QDT0_PHA	-
Port H5: MUX = ALT2, QDT0_PHB	-
Port I2: MUX = ALT1, GPIO	-

2.42.2.4 Steps to Run

2.42.2.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

- 1) Right click on the project name and make sure "Debug_Flash" is selected as the active build configuration.
- 2) Build the project.
- 3) Connect H4 port with I2 port, and connect H5 port with H3 port.
- 4) Create a "GDB SEGGER J-Link Debugging" configuration, and then click Debug.
- 5) Then you can see the below output via the UART tool:

```
System Start!!
=====QDT ICDM MODE DEMO Start=====
=====Measure high-active duty cycle=====
High-active time is 201.24ms
Low-active time is 1618.56ms
High-active Duty-Cycle value is 11.05%
=====Measure low-active duty cycle=====
High-active time is 198.73ms
Low-active time is 1620.78ms
Low-active Duty-Cycle value is 89.07%
=====QDT ICDM MODE DEMO End=====
```

2.42.2.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\QDT\QDT_ICDM_Demo_Project\IAR_Project, and double-click the IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.42.2.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.42.3 QDT_ICENM_Demo

2.42.3.1 Description

This demo shows the general configuration process to use the QDT module with ICENM mode.

2.42.3.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.42.3.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1_RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1_TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port H4: MUX = ALT2, QDT0_PHA	-
Port H5: MUX = ALT2, QDT0_PHB	-
Port I2: MUX = ALT1, GPIO	-
Port H3: MUX = ALT1, GPIO	-

2.42.3.4 Steps to Run

2.42.3.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

- 1) Right click on the project name and make sure "Debug_Flash" is selected as the active build configuration.
- 2) Build the project.
- 3) Connect H4 port with I2 port, and connect H5 port with H3 port.
- 4) Create a "GDB SEGGER J-Link Debugging" configuration, and then click Debug.
- 5) Then you can see the below output via the UART tool:

```
System Start!!
=====QDT ICENM MODE DEMO Start=====
Detect the 8 number edges
=====QDT ICENM MODE DEMO End=====
```

2.42.3.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\QDT\QDT_ICENM_Demo_Project\IAR_Project, and double-click the IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.42.3.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.42.4 QDT_ICEXPENM_Demo

2.42.4.1 Description

This demo shows the general configuration process to use the QDT module with ICEXPENM mode.

2.42.4.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.42.4.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port H4: MUX = ALT2, QDT0_PHA	-
Port H5: MUX = ALT2, QDT0_PHB	-
Port I2: MUX = ALT1, GPIO	-
Port H3: MUX = ALT1, GPIO	-

2.42.4.4 Steps to Run

2.42.4.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

- 1) Right click on the project name and make sure "Debug_Flash" is selected as the active build configuration.
- 2) Build the project.
- 3) Connect H4 port with I2 port, and connect H5 port with H3 port.
- 4) Create a "GDB SEGGER J-Link Debugging" configuration, and then click Debug.
- 5) Then you can see the below output via the UART tool:

```
System Start!!
=====QDT ICEEXPMM MODE DEMO Start=====
The time from first edge to last edge is 5284.86ms
=====QDT ICEEXPMM MODE DEMO End=====
```

2.42.4.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\QDT\QDT_ICEXPENM_Demo_Project\IAR_Project, and double-click the IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.42.4.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.42.5 QDT_ICPM_Demo

2.42.5.1 Description

This demo shows the general configuration process to use the QDT module with IC mode.

2.42.5.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.42.5.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port H4: MUX = ALT2, QDT0_PHA	-
Port H5: MUX = ALT2, QDT0_PHB	-
Port I2: MUX = ALT1, GPIO	-
Port H3: MUX = ALT1, GPIO	-

2.42.5.4 Steps to Run

2.42.5.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

- 1) Right click on the project name and make sure "Debug_Flash" is selected as the active build configuration.
- 2) Build the project.
- 3) Connect H4 port with I2 port, and connect H5 port with H3 port.
- 4) Create a "GDB SEGGER J-Link Debugging" configuration, and then click Debug.
- 5) Then you can see the below output via the UART tool:

```
System Start!!
=====QDT ICPM MODE DEMO Start=====
period time is 1761.62ms
=====QDT ICDM MODE DEMO End=====
```

2.42.5.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\QDT\QDT_ICPM_Demo_Project\IAR_Project, and double-click the IAR project file with the suffix ".eww".

- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.42.5.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.42.6 QDT_QUAD_Demo

2.42.6.1 Description

This demo shows the general configuration process to use the QDT module with IC mode.

2.42.6.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.42.6.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port H4: MUX = ALT2, QDT0_PHA	-
Port H5: MUX = ALT2, QDT0_PHB	-
Port I2: MUX = ALT1, GPIO	-
Port H3: MUX = ALT1, GPIO	-

2.42.6.4 Steps to Run

2.42.6.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

- 1) Right click on the project name and make sure "Debug_Flash" is selected as the active build configuration.
- 2) Build the project.
- 3) Connect H4 port with I2 port, and connect H5 port with H3 port.
- 4) Create a "GDB SEGGER J-Link Debugging" configuration, and then click Debug.
- 5) Then you can see the below output via the UART tool:

```
System Start!!
-----QDT QUAD MODE DEMO Start-----
The Speed is 74770.7031RPM
The Speed is 74770.7031RPM
The Speed is 74770.7031RPM
The Speed is 74770.7031RPM
```

2.42.6.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\QDT\QDT_QUAD_Demo_Project\IAR_Project, and double-click the IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.42.6.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

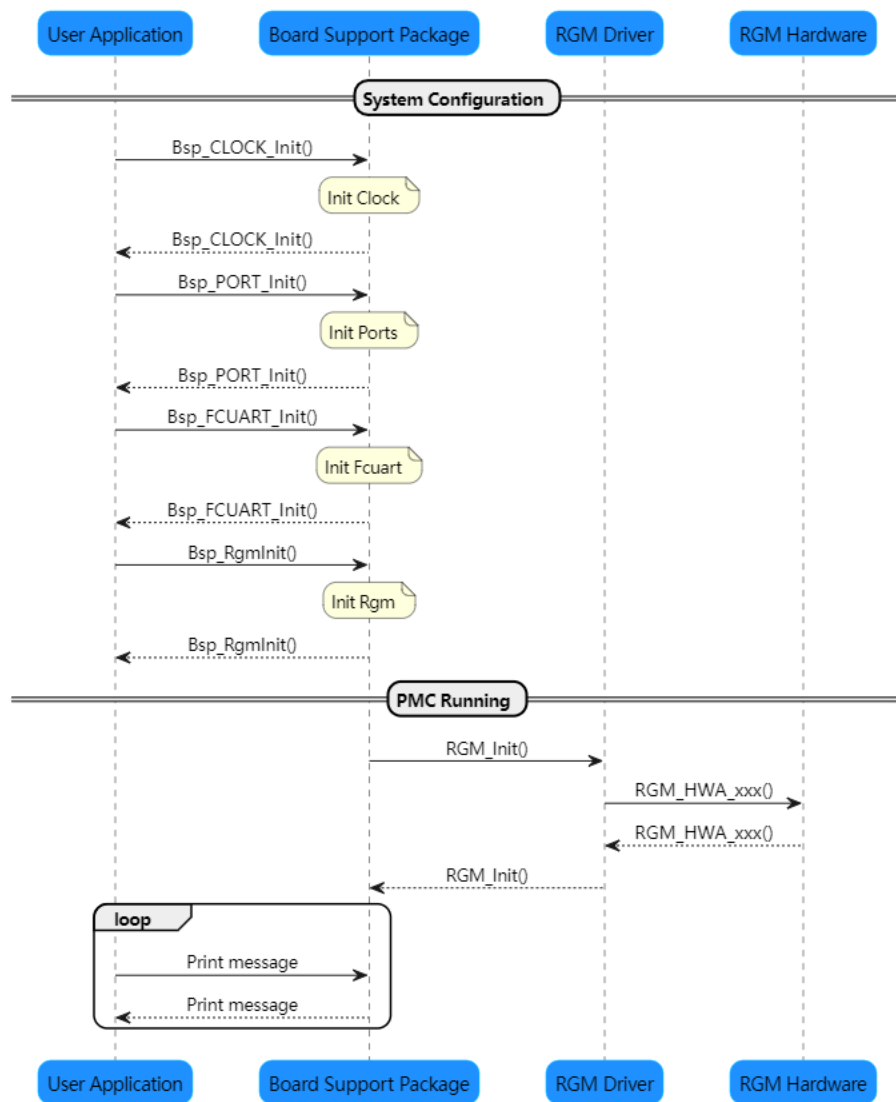
2.43 RGM

2.43.1 Description

This demo project shows how to use the RGM module to generate an interrupt before a reset event occurs through the SDK.

This demo project will toggle LED and use UART1 send message at regular intervals. If KEY1 pressed, the MCU will generate a SYSRST. If KEY2 pressed, the MCU will generate a SWRS, and RGM notification callback function will be called.

The sequence diagram of the demo is as follows:



2.43.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Power adapter 12 V
- 1 Personal computer
- 1 J-Link debugger

2.43.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1_RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1_TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

Users use the serial to display reset information

2.43.4 Steps to Run

The steps to run the example are as follows:

- 1) Plug in the USB Type-C cable and open the up-machine (UART1 115200/8/N/1).
- 2) Download the RGM_Demo program to the MCU by JTAG.

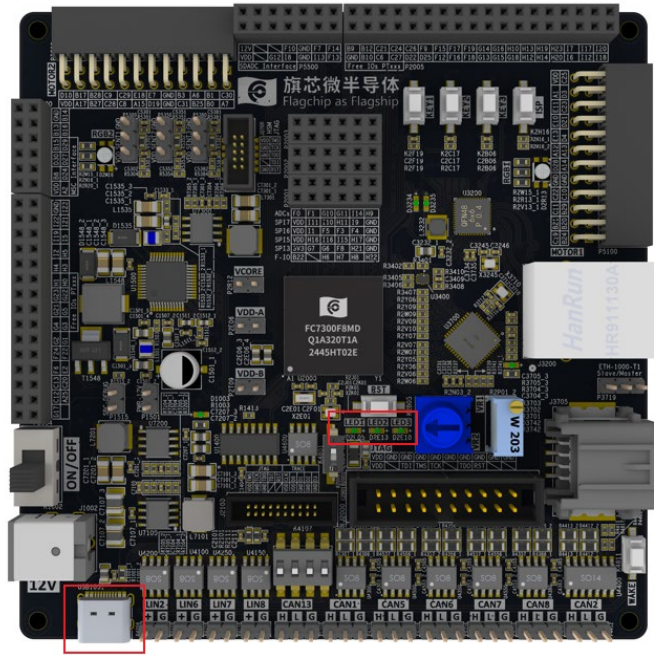
Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
 - To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
 - To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).
- 3) Normally, the MCU will print the value of registers RGM_SRS and RGM_C0_SRS at the beginning. You can learn the reset source of the last reset by referring to the register description in the RGM chapter of the *FC7300F8MDQ Reference Manual*.

2.44 RTC

There are four RTC demos: RTC_Alarm, RTC_SecondInterrupt, RTC_Overflow, and RTC_SetReadTime.

This example requires using a jumper cap to connect PTD31 and PTA26 on the demo board to enable the LED.



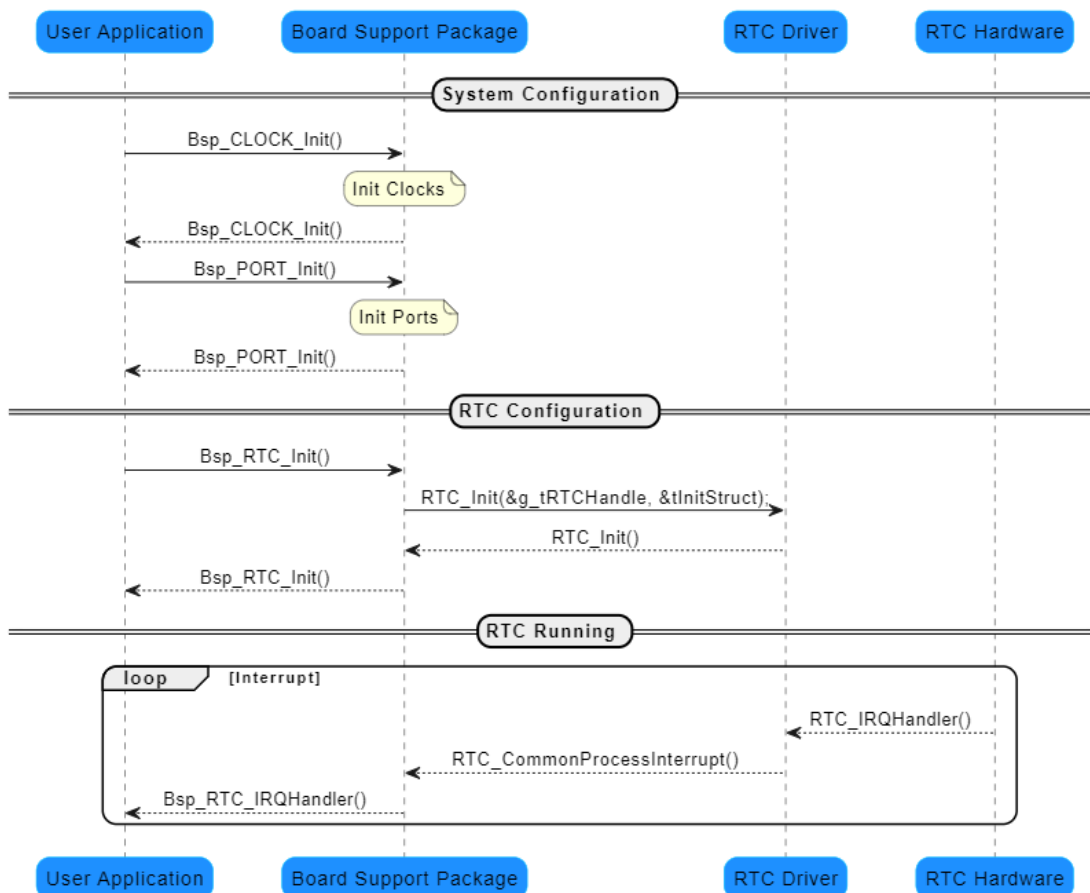
2.44.1 RTC_Alarm

2.44.1.1 Description

This demo shows how to use the RTC to generate 1-second period interrupts through the SDK.

This demo project will toggle LED every second. The interrupt source is the RTC alarm interrupt.

The sequence diagram of the demo is as follows:



2.44.1.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V or micro-USB cable
- 1 Personal computer
- 1 J-Link debugger

2.44.1.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A26: MUX = ALT2, GPIO Output	LED1
Port D31: MUX = ALT1, GPIO Output	LED2

2.44.1.4 Steps to Run

2.44.1.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

2.44.1.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\RTC\RTC_Alarm_Project\IAR_Project, and double-click the **RTC_Alarm** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.44.1.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

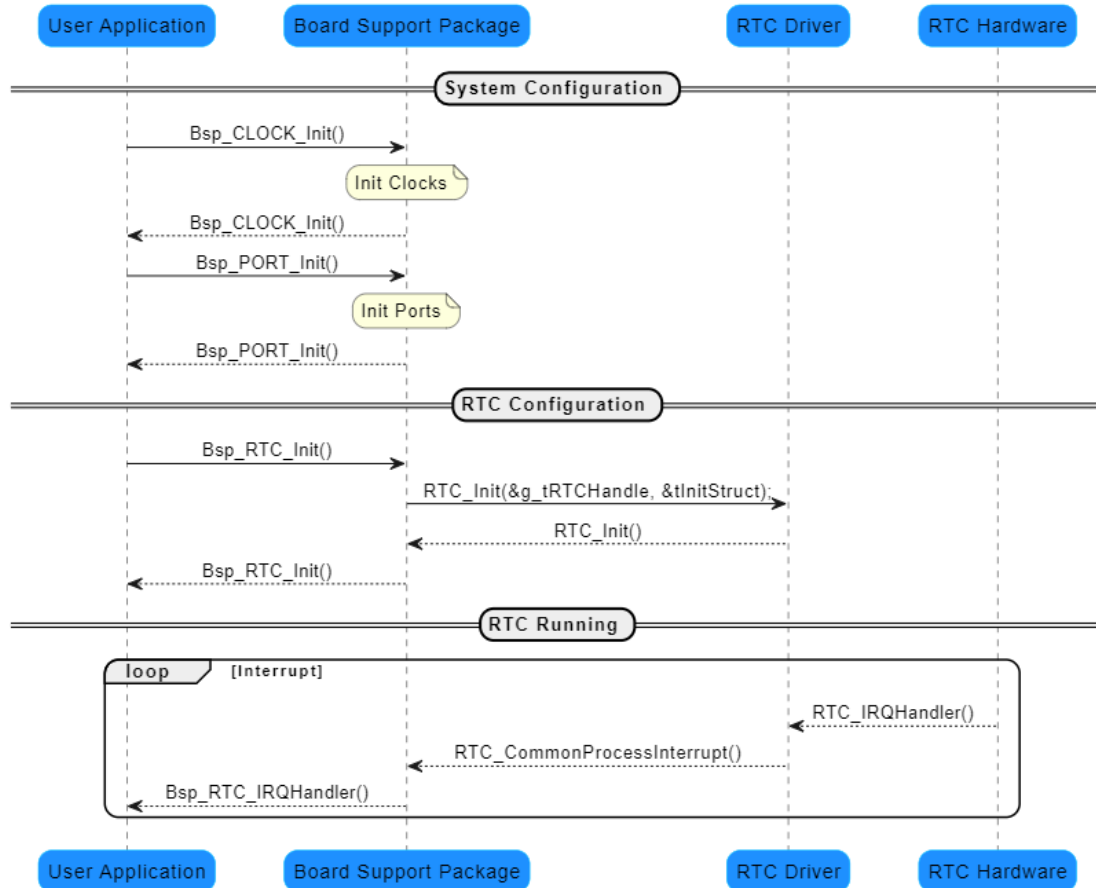
2.44.2 RTC_SecondInterrupt

2.44.2.1 Description

This demo shows how to use the RTC to generate 1 Hz period interrupts through the SDK.

This demo project will toggle LED at the frequency of 1 Hz. The interrupt source is the RTC second interrupt.

The sequence diagram of the demo is as follows:



2.44.2.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V or micro-USB cable
- 1 Personal computer
- 1 J-Link debugger

2.44.2.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A26: MUX = ALT2, GPIO Output	LED1
Port D31: MUX = ALT1, GPIO Output	LED2

2.44.2.4 Steps to Run

2.44.2.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

2.44.2.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\RTC\RTC_SecondInterrupt_Project\IAR_Project, and double-click the **RTC_SecondInterrupt** IAR project file with the suffix ".eww".

- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.44.2.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

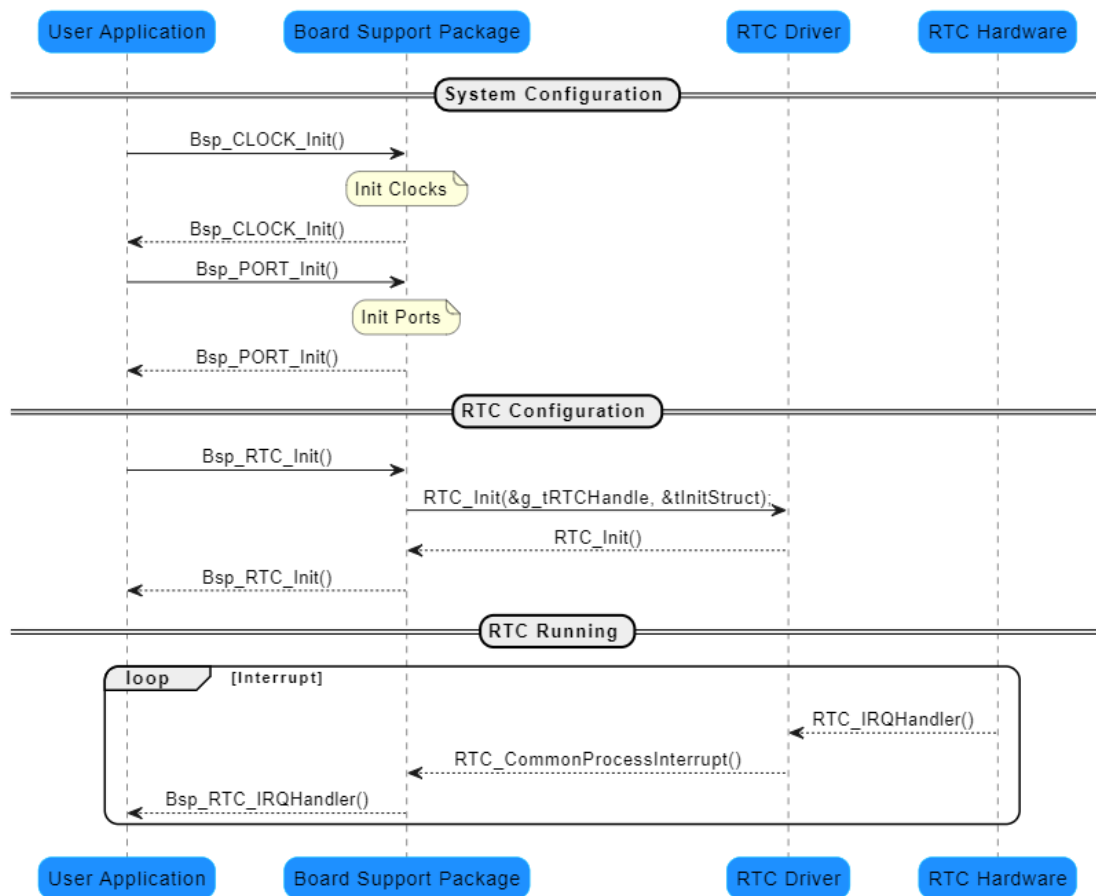
2.44.3 RTC_Overflow

2.44.3.1 Description

This demo shows how to use the RTC to generate an overflow interrupt through the SDK.

This demo project will toggle LED when detecting the overflow Interrupt.

The sequence diagram of the demo is as follows:



2.44.3.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V or micro-USB cable
- 1 Personal computer
- 1 J-Link debugger

2.44.3.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A26: MUX = ALT2, GPIO Output	LED1
Port D31: MUX = ALT1, GPIO Output	LED2

2.44.3.4 Steps to Run

2.44.3.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

2.44.3.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\RTC\RTC_Overflow_Project\IAR_Project, and double-click the **RTC_OverFlow** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.44.3.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

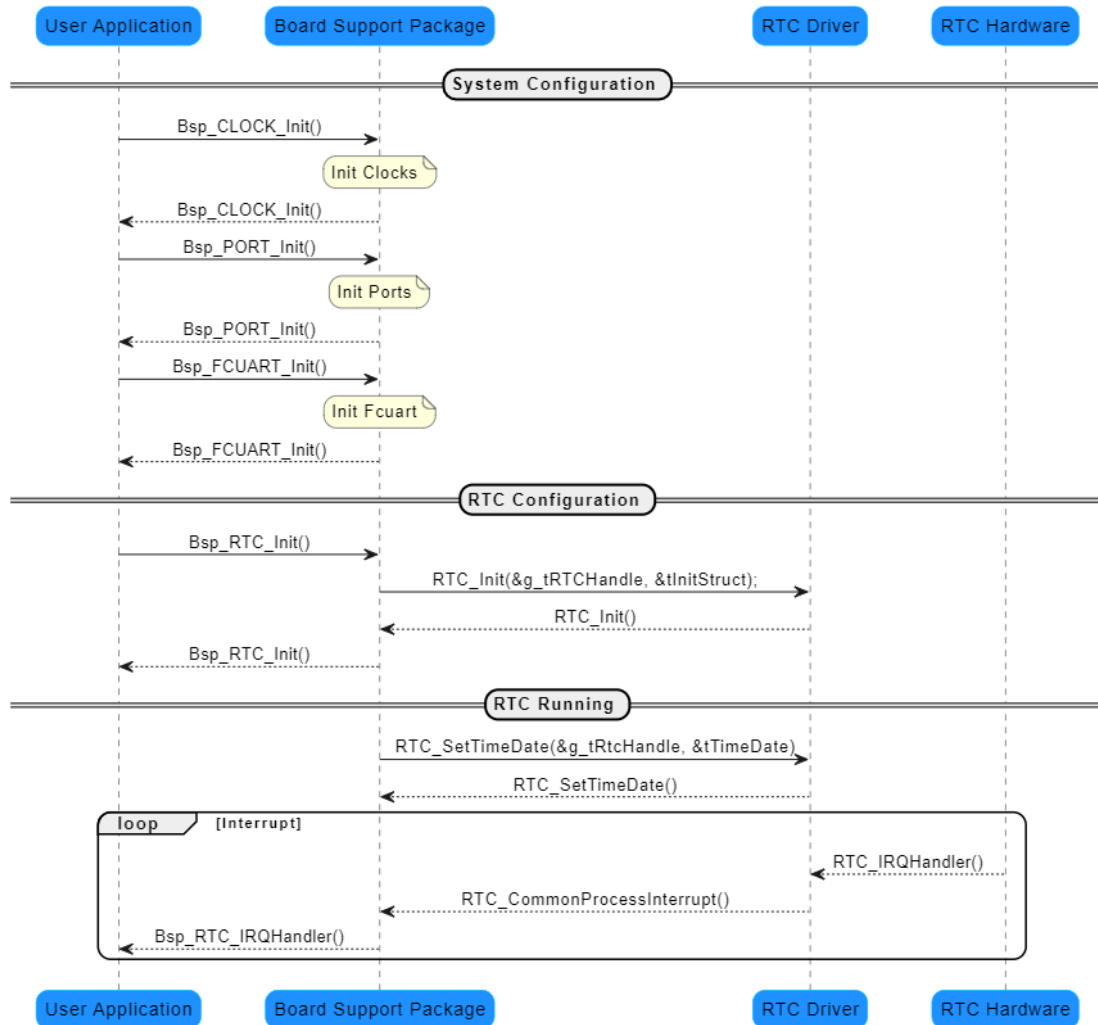
2.44.4 RTC_SetReadTime

2.44.4.1 Description

This demo shows how to set real time and get in real time.

This demo will toggle LED and print in real time.

The sequence diagram of the demo is as follows:



2.44.4.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V or micro-USB cable
- 1 Personal computer
- 1 J-Link debugger

2.44.4.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A26: MUX = ALT2, GPIO Output	LED1
Port D31: MUX = ALT1, GPIO Output	LED2

2.44.4.4 Steps to Run

2.44.4.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

2.44.4.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\RTC\RTC_SetReadTime_Project\IAR_Project, and double-click the **RTC_SetReadTime** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.44.4.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.45 SC6258XQ

2.45.1 Description

This demo is just for SC6258XQ with FCSMU test.

2.45.2 Prerequisites

To run the example, make sure you have the following materials:

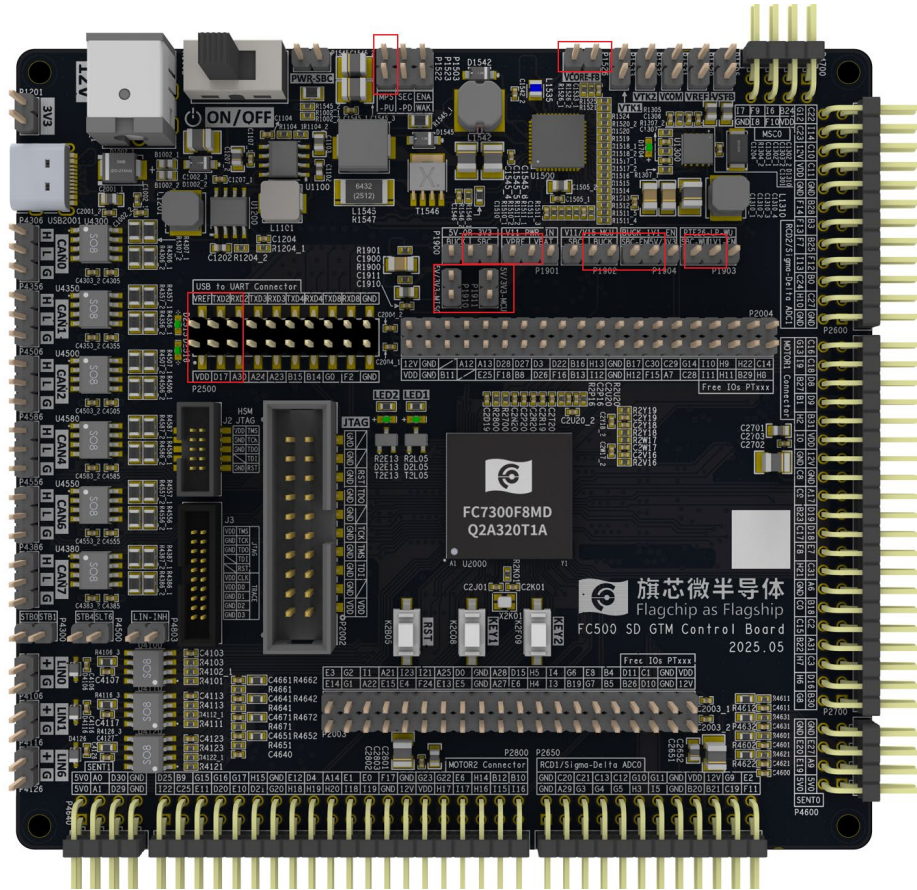
- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V or micro-USB cable
- 1 Personal computer
- 1 J-Link debugger

2.45.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A30: MUX = ALT3, UART2_RX	FCUART2_RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port D17: MUX = ALT3, UART2_TX	FCUART2_TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port E17: MUX = ALT3, FCSMU_PIN1	-
Port A16: MUX = ALT3, FCSMU_PIN0	-
Port F4: MUX = ALT4, FCSPI6_SCK	Serial clock pin
Port I2: MUX = ALT4, FCSPI6_PCS1	Chip select pin
Port F5: MUX = ALT4, FCSPI6_SOUT	Serial output pin
Port F3: MUX = ALT4, FCSPI6_SIN	Serial input pin

2.45.4 Steps to Run

- 1) Plugin the type-c and open the up-machine (115200/8/N/1).
- 2) Make sure the debug pin jumps and plugs in.



- 3) Import the SC6258XQ project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 3) Power off the board, then remove the debug pin jumps

- 4) Power on the board, mcu will print follow text.

```
System Start!!
Init FCSMU
1. Testing the comparator logic and the corresponding deglitching logic
===== ABist Success
Clear WWDSTAT[WWDENCT] to zero
Wait for error injection
```

- 5) Wait for 12s for FCSMU fault injection cause system reset.

```
System Start!!
Init FCSMU
1. Testing the comparator logic and the corresponding deglitching logic
===== ABist Success
Clear WWDSTAT[WWDENCT] to zero
Wait for error injection
=====
Fault injected, wait for warning timeout
Software fault occurs
Clear channel: 0x1
Clear channel successfully: 0x0
```

- 6) If you want to update the program, you need to restart from step 2.

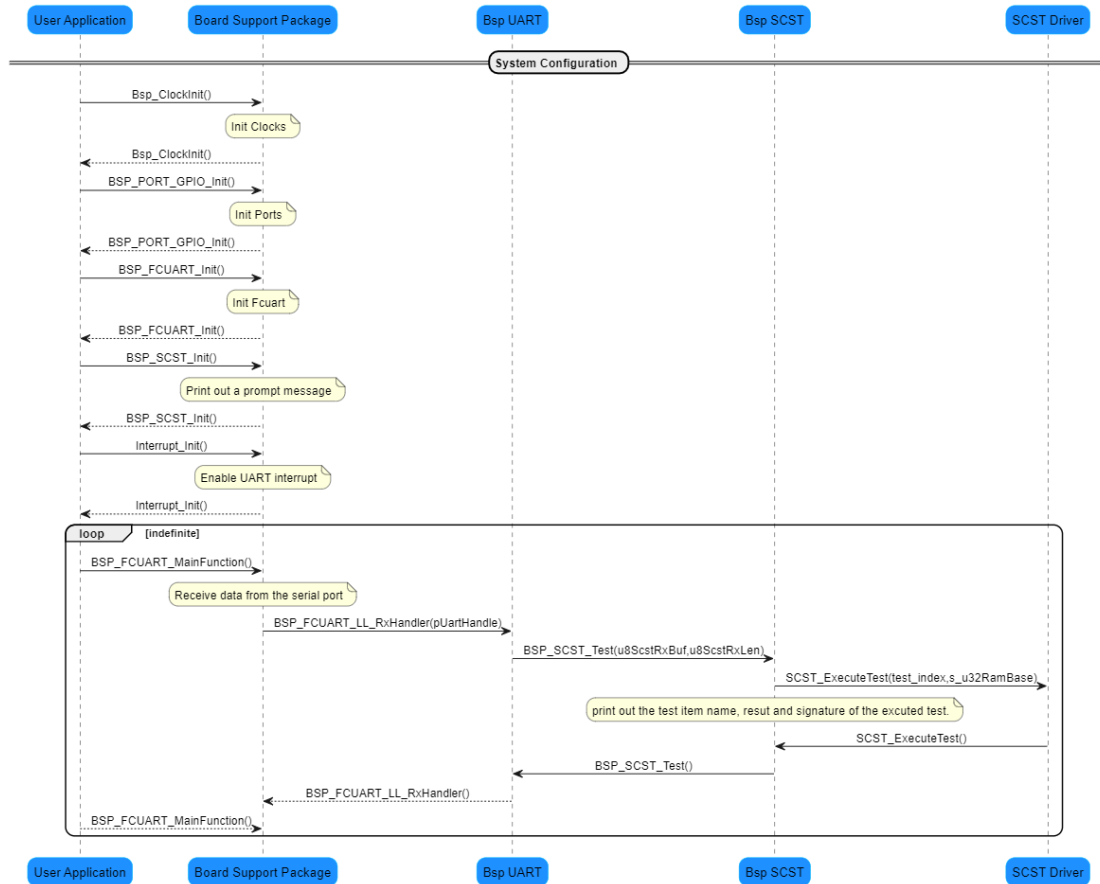
2.46 SCST

2.46.1 Description

The SCST demo uses M7ST API to provide diagnostic testing for the Cortex-M7 processor.

M7ST API contains a set of atomic tests to achieve the claimed fault coverage.

The sequence diagram of the demo is as follows:



2.46.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger
- 1 PC Uart tool

2.46.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

Users use serial communication to interact with test demo.

2.46.4 Steps to Run

The steps to run the example are as follows:

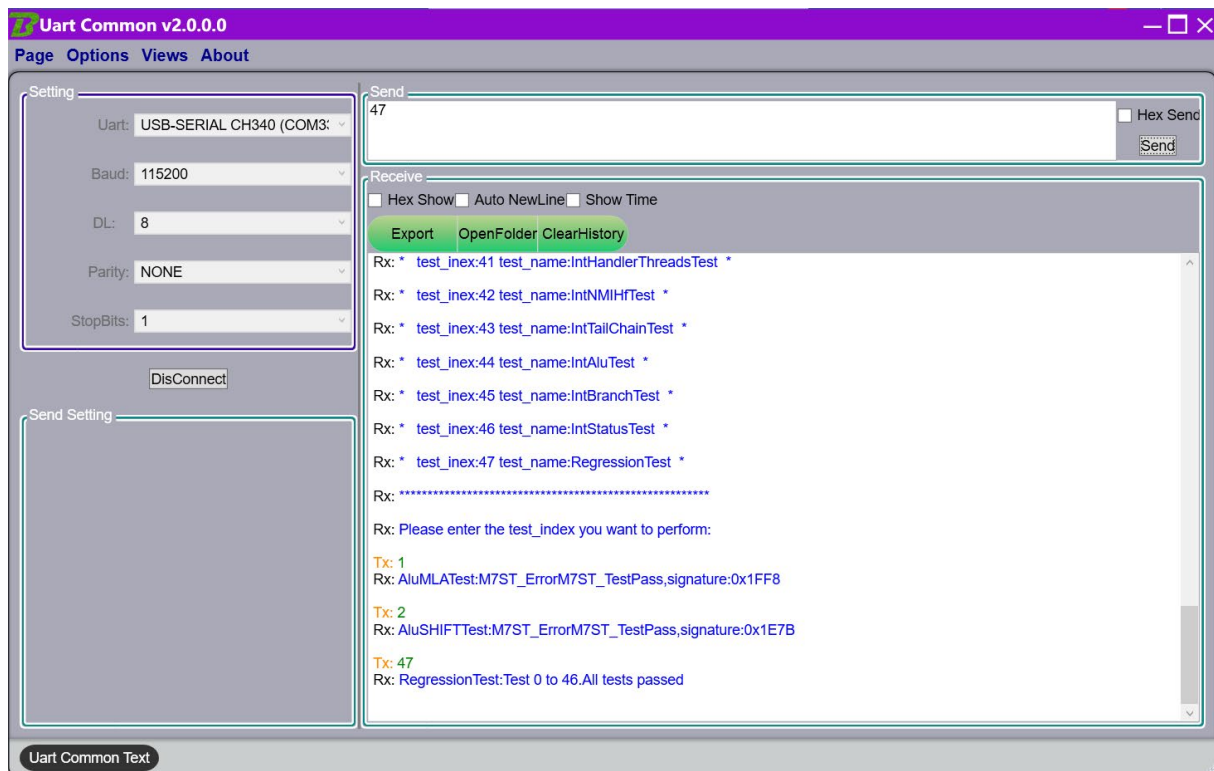
- 1) First, plug in the USB Type-C cable and open the up-machine (UART1 115200/8/N/1). Turn on the power, then demo board is powered on.
- 2) Import the SCST_Demo project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
Make sure "FC7300F8MDQ_Debug_Ram" is selected as the active build configuration.
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 3) After building and debugging, MCU will print out a prompt message normally. You need to enter the test_index of the test you want to execute, and then it will return the name, result and signature of the executed test.

Note: test_index 47 will perform all tests from beginning to end (0 to 47).



2.47 SDADC

There are three SDADC demos: SDADC_Single, SDADC_Continous, and SDADC_Calibration.

2.47.1 SDADC_Single

2.47.1.1 Description

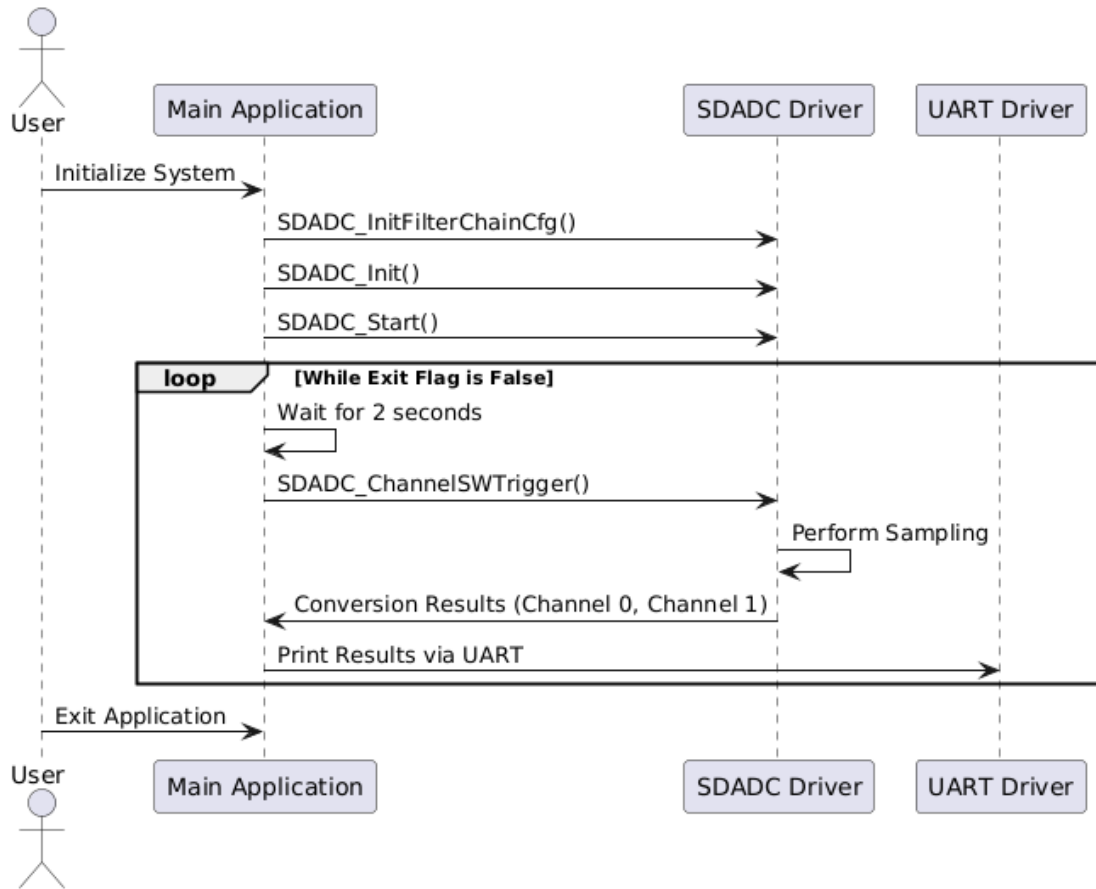
This demo demonstrates how to perform single-shot sampling using the SDADC. The code triggers sampling via software for two channels (Channel 0 and Channel 1), retrieves the results, and outputs them via UART.

- **Single-Shot Sampling:** Performs sampling on demand for two channels.
- **UART Output:** Prints the sampling results to the UART interface.

Note:

- 1) Ensure proper clock and UART configurations.
- 2) Adjust the sampling frequency based on application requirements.

The sequence diagram of the demo is as follows:



2.47.1.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 USB-2-UART cable
- 1 Personal computer
- 1 J-Link debugger

2.47.1.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

2.47.1.4 Steps to Run

The steps to run the example are as follows:

- 1) Import the SDADC_Single project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) Print the sampling results via UART.
- 3) Perform sampling every 2 seconds in the main loop and output the results.

2.47.2 SDADC_Continuous

2.47.2.1 Description

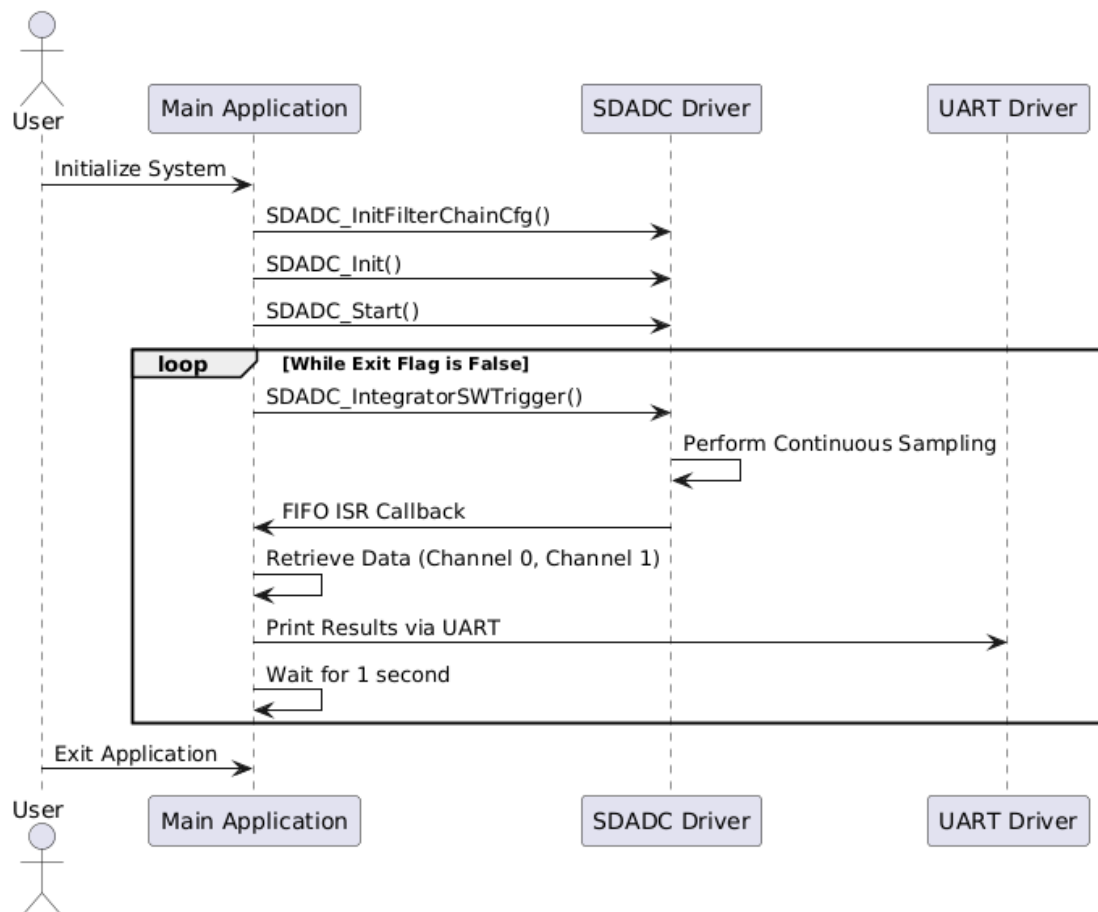
This demo demonstrates continuous sampling using the SDADC. The code configures the SDADC to continuously sample two channels and outputs the results via UART.

- **Continuous Sampling:** Samples data continuously using the SDADC FIFO.
- **Interrupt-Driven:** Uses interrupts to handle FIFO data ready events.
- **UART Output:** Prints the sampling results to the UART interface.

Note:

- 1) Ensure proper interrupt configuration for FIFO events.
- 2) Adjust the FIFO watermark and sampling frequency based on application requirements.

The sequence diagram of the demo is as follows:



2.47.2.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 USB-2-UART cable
- 1 Personal computer
- 1 J-Link debugger

2.47.2.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

2.47.2.4 Steps to Run

The steps to run the example are as follows:

- 1) Import the SDADC_Continuous project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 2) Print the sampling results via UART.

2.47.3 SDADC_Calibration

2.47.3.1 Description

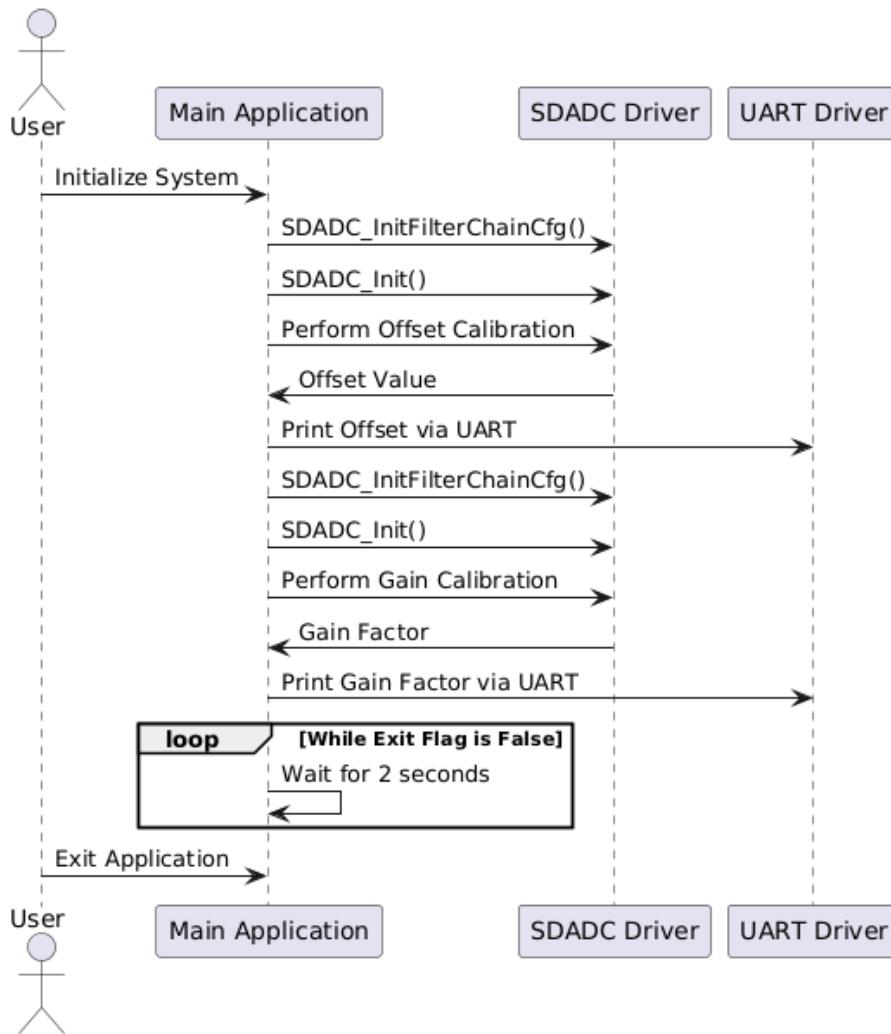
This demo demonstrates how to perform offset and gain calibration for the SDADC. The code calculates the offset and gain factor for a channel and outputs the results via UART.

- **Offset Calibration:** Measures the offset error of the SDADC.
- **Gain Calibration:** Measures the gain factor of the SDADC.
- **UART Output:** Prints the calibration results to the UART interface.

Note:

- 1) Ensure proper SDADC configuration for calibration.
- 2) Adjust the sample count for calibration based on accuracy requirements.

The sequence diagram of the demo is as follows:



2.47.3.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 USB-2-UART cable
- 1 Personal computer
- 1 J-Link debugger

2.47.3.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

2.47.3.4 Steps to Run

The steps to run the example are as follows:

- 1) Import the SDADC_Calibration project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

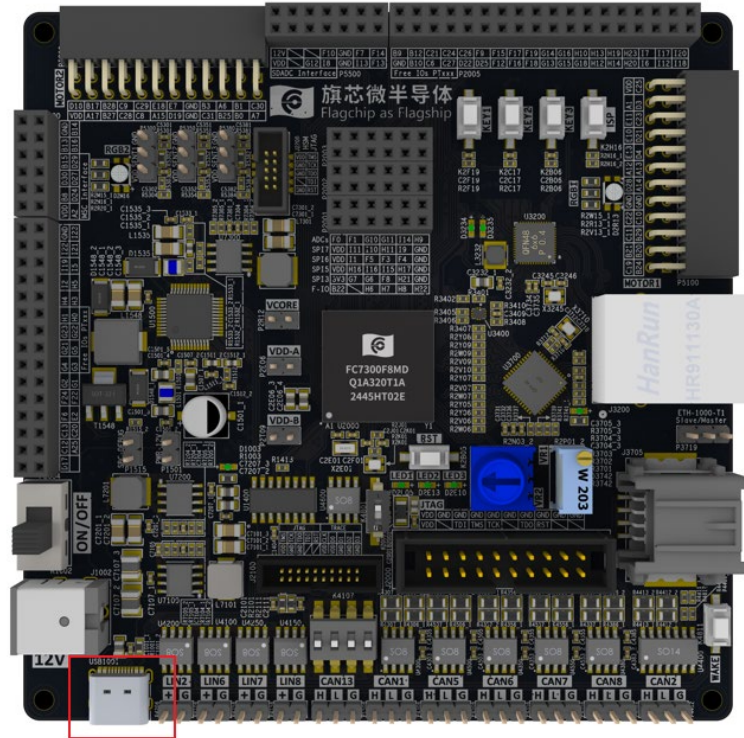
2) Print the calibration results via UART.

2.48 SEC

2.48.1 Description

This demo project shows how to read the SEC data.

Note: Most of the configuration of the SEC module comes from NVR, so this demo mainly serves as a display.



2.48.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power Adapter 12 V or Micro USB line
- 1 Personal computer
- 1 J-Link debugger

2.48.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

2.48.4 Steps to Run

2.48.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: Make sure "**Debug_Flash**" is selected as the active build configuration.

2.48.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\SEC\SEC_Demo_Project\IAR_Project, and double-click the **SEC_demo** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.48.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

After building and debugging, take the following steps:

- 1) Plug in the USB-Type C cable and connect inner chip CH340 to UART1 pin (UART1 115200/8/N/1).
- 2) MCU will upload the toggle message. If you transmit message to it and it will respond same message.

```

1 [2023-02-14 16:45:09.583 R]System Start!!
2 IspMode is (1 means enable;0 means disable) : 0
3 Secure boot status is (1 means enable;0 means disable): 0
4 boot address is : ffffffff
5
6 [2023-02-14 16:47:08.171 R]System Start!!
7 IspMode is (1 means enable;0 means disable) : 0
8 Secure boot status is (1 means enable;0 means disable): 0
9 boot address is : ffffffff
10
11

```

2.49 SEMA

2.49.1 Description

This example demonstrates how to use the Sema module to implement the spin lock function.

If core0 acquires the spin lock, LED1 will light up and print "Core 0 Spin Lock Acquire!", then core0 releases the spin lock, LED1 will turn off and print "Core 0 Spin Lock Release!".

If core1 acquires the spin lock, LED2 will light up and print "Core 1 Spin Lock Acquire!", then core1 releases the spin lock, LED2 will turn off and print "Core 1 Spin Lock Release!".

2.49.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.49.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1_RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1_TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A26: MUX = ALT2, GPIO Output	LED1
Port D31: MUX = ALT1, GPIO Output	LED2

2.49.4 Steps to Run

First, plug in the USB Type-C cable, and connect the PC and demo board. Then, open UART debug terminal (USB-SERIAL CH340/115200/8/N/1).

2.49.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

2.49.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\SEMA\SpinLock_Project\IAR_Project, and double-click the **Sema_SpinLock** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.49.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.50 SENT

There are four SENT demos: SENT_Polling, SENT_Interrupt, SENT_Dma, and SENT_SPC.

2.50.1 SENT_Polling

2.50.1.1 Description

The purpose of this demo is to show you the general configuration process to use the SENT with polling method.

2.50.1.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 MLX90367LDC Sent sensor
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.50.1.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port E19: MUX = ALT4, SENT0_RXD2	SENT0, Channel 2, RXD

2.50.1.4 Steps to Run

2.50.1.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: Make sure "**Debug_Flash**" is selected as the active build configuration.

After building and debugging, you can see the below output via the UART tool:

```

1 [2023-02-06 14:41:36.456 R]=====Sent polling demo START=====
2
3 [2023-02-06 14:41:37.237 R]Receive 50 Fast messages and 3 slow messages
4 Number 1 :Fast Message TimeStamp: 109913, Data: A96105, status: 1, CRC: 13
5 Number 2 :Fast Message TimeStamp: 115949, Data: A8F115, status: 1, CRC: 2
6 Number 3 :Fast Message TimeStamp: 121985, Data: A89125, status: 1, CRC: 14
7 Number 4 :Fast Message TimeStamp: 128021, Data: A71135, status: 1, CRC: 13
8 Number 5 :Fast Message TimeStamp: 134057, Data: A62145, status: 9, CRC: 1
9 Number 6 :Fast Message TimeStamp: 140093, Data: A80155, status: 1, CRC: 12
10 Number 7 :Fast Message TimeStamp: 146130, Data: A44165, status: 1, CRC: 1
11 Number 8 :Fast Message TimeStamp: 152166, Data: B06174, status: 9, CRC: 12
12 Number 9 :Fast Message TimeStamp: 158202, Data: A4D185, status: 1, CRC: 3
13 Number 10 :Fast Message TimeStamp: 164238, Data: AB2195, status: 9, CRC: 14
14 Number 11 :Fast Message TimeStamp: 170274, Data: AB31A5, status: 9, CRC: 12
15 Number 12 :Fast Message TimeStamp: 176310, Data: AC71B5, status: 1, CRC: 14
16 Number 13 :Fast Message TimeStamp: 182345, Data: A781C5, status: 13, CRC: 1
17 Number 14 :Fast Message TimeStamp: 188381, Data: A891D5, status: 13, CRC: 2
18 Number 15 :Fast Message TimeStamp: 194417, Data: A9F1E5, status: 13, CRC: 15
19 Number 16 :Fast Message TimeStamp: 200452, Data: B0B1F4, status: 13, CRC: 15
20 Number 17 :Fast Message TimeStamp: 206489, Data: AC1205, status: 13, CRC: 9
21 Number 18 :Fast Message TimeStamp: 212525, Data: A85215, status: 13, CRC: 4
22 Number 19 :Fast Message TimeStamp: 218561, Data: AAD225, status: 5, CRC: 4
23 Number 20 :Fast Message TimeStamp: 224597, Data: A7A235, status: 1, CRC: 15
24 Number 21 :Fast Message TimeStamp: 230633, Data: AD6245, status: 1, CRC: 14
25 Number 22 :Fast Message TimeStamp: 236669, Data: ADF255, status: 1, CRC: 15
26 Number 23 :Fast Message TimeStamp: 242706, Data: AAB265, status: 1, CRC: 9
27 Number 24 :Fast Message TimeStamp: 248741, Data: A91275, status: 1, CRC: 11
28 Number 25 :Fast Message TimeStamp: 254777, Data: A92285, status: 1, CRC: 4
29 Number 26 :Fast Message TimeStamp: 260813, Data: ACF295, status: 1, CRC: 4
30 Number 27 :Fast Message TimeStamp: 266849, Data: ACF2A5, status: 1, CRC: 2
31 Number 28 :Fast Message TimeStamp: 272885, Data: A382B5, status: 1, CRC: 4
32 Number 29 :Fast Message TimeStamp: 278922, Data: AD12C5, status: 9, CRC: 2
33 Number 30 :Fast Message TimeStamp: 284958, Data: A8A2D5, status: 5, CRC: 7
34 Number 31 :Fast Message TimeStamp: 290994, Data: AED2E5, status: 9, CRC: 3
35 Number 32 :Fast Message TimeStamp: 297030, Data: AD52F5, status: 13, CRC: 9
36 Number 33 :Fast Message TimeStamp: 303066, Data: AA4305, status: 9, CRC: 4
37 Number 34 :Fast Message TimeStamp: 309102, Data: A8E315, status: 13, CRC: 8
38 Number 35 :Fast Message TimeStamp: 315138, Data: A6F325, status: 9, CRC: 13
39 Number 36 :Fast Message TimeStamp: 321174, Data: A9E335, status: 9, CRC: 2
40 Number 37 :Fast Message TimeStamp: 327210, Data: A56345, status: 1, CRC: 13
41 Number 38 :Fast Message TimeStamp: 333246, Data: A8A355, status: 1, CRC: 13
42 Number 39 :Fast Message TimeStamp: 339282, Data: B45364, status: 1, CRC: 14
43 Number 40 :Fast Message TimeStamp: 345318, Data: A66375, status: 1, CRC: 4
44 Number 41 :Fast Message TimeStamp: 351353, Data: A6A385, status: 9, CRC: 13
45 Number 42 :Fast Message TimeStamp: 357389, Data: A79395, status: 1, CRC: 13
46 Number 43 :Fast Message TimeStamp: 363425, Data: ABC3A5, status: 1, CRC: 4
47 Number 44 :Fast Message TimeStamp: 369461, Data: A6C3B5, status: 9, CRC: 14
48 Number 45 :Fast Message TimeStamp: 375497, Data: A883C5, status: 9, CRC: 6
49 Number 46 :Fast Message TimeStamp: 381533, Data: B0B3D4, status: 1, CRC: 5
50 Number 47 :Fast Message TimeStamp: 387569, Data: A503E5, status: 1, CRC: 1
51 Number 48 :Fast Message TimeStamp: 393605, Data: A6A3F5, status: 1, CRC: 3
52 Number 49 :Fast Message TimeStamp: 399641, Data: B22404, status: 13, CRC: 8
53 Number 50 :Fast Message TimeStamp: 405677, Data: ACB415, status: 13, CRC: 11
54 =====slow message list=====
55 Number 1 :slow Message TimeStamp: 176310, Data: 0, ID: 43, CRC: 19
56 Number 2 :slow Message TimeStamp: 176310, Data: 1, ID: 1, CRC: 63
57 Number 3 :slow Message TimeStamp: 284958, Data: 0, ID: 44, CRC: 20
58 =====Sent polling demo End=====
59
60

```

2.50.1.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\SENT\SENT_Polling_Project\IAR_Project, and double-click the **SENT_polling** IAR project file with the suffix ".eww".

- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.50.1.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.50.2 SENT_Interrupt

2.50.2.1 Description

The purpose of this demo is to show you the general configuration process to use the SENT with Interrupt method.

2.50.2.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 MLX90367LDC Sent sensor
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.50.2.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port E19: MUX = ALT4, SENT0_RXD2	SENT0, Channel 2, RXD

2.50.2.4 Steps to Run

2.50.2.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: Make sure "**Debug_Flash**" is selected as the active build configuration.

After building and debugging, you can see the below output via the UART tool:

```

60 [2023-02-06 14:43:20.873 R]=====Sent Interrupt demo Start=====
61
62 [2023-02-06 14:43:21.654 R]Receive 50 Fast messages and 2 slow messages
63 =====Fast message list=====
64 Number 1 :Fast Message TimeStamp: 18619, Data: AC6885, status: 9, CRC: 0
65 Number 2 :Fast Message TimeStamp: 85022, Data: A8EB95, status: 5, CRC: 7
66 Number 3 :Fast Message TimeStamp: 151425, Data: AC7BA5, status: 9, CRC: 0
67 Number 4 :Fast Message TimeStamp: 217825, Data: A648B5, status: 13, CRC: 11
68 Number 5 :Fast Message TimeStamp: 284221, Data: B1CBC4, status: 9, CRC: 10
69 Number 6 :Fast Message TimeStamp: 350622, Data: A318D5, status: 9, CRC: 2
70 Number 7 :Fast Message TimeStamp: 417016, Data: A28BE5, status: 13, CRC: 9
71 Number 8 :Fast Message TimeStamp: 483411, Data: A87BF5, status: 13, CRC: 8
72 Number 9 :Fast Message TimeStamp: 549809, Data: A53C05, status: 1, CRC: 6
73 Number 10 :Fast Message TimeStamp: 616202, Data: A65C15, status: 1, CRC: 14
74 Number 11 :Fast Message TimeStamp: 682594, Data: A60C25, status: 1, CRC: 1
75 Number 12 :Fast Message TimeStamp: 748991, Data: A5EC35, status: 1, CRC: 14
76 Number 13 :Fast Message TimeStamp: 815386, Data: B52C44, status: 9, CRC: 15
77 Number 14 :Fast Message TimeStamp: 881784, Data: AE4C55, status: 1, CRC: 6
78 Number 15 :Fast Message TimeStamp: 948185, Data: A41C65, status: 1, CRC: 12
79 Number 16 :Fast Message TimeStamp: 1014580, Data: AA5C75, status: 9, CRC: 4
80 Number 17 :Fast Message TimeStamp: 1080977, Data: B0BC84, status: 1, CRC: 5
81 Number 18 :Fast Message TimeStamp: 1147380, Data: A8EC95, status: 9, CRC: 15
82 Number 19 :Fast Message TimeStamp: 1213780, Data: A65CA5, status: 9, CRC: 5
83 Number 20 :Fast Message TimeStamp: 1280177, Data: A59C85, status: 1, CRC: 2
84 Number 21 :Fast Message TimeStamp: 1346575, Data: A08CC5, status: 13, CRC: 12
85 Number 22 :Fast Message TimeStamp: 1412969, Data: AA1CD5, status: 13, CRC: 0
86 Number 23 :Fast Message TimeStamp: 1479365, Data: A98CE5, status: 13, CRC: 6
87 Number 24 :Fast Message TimeStamp: 1545765, Data: A81CF5, status: 13, CRC: 5
88 Number 25 :Fast Message TimeStamp: 1612161, Data: A46D05, status: 13, CRC: 6
89 Number 26 :Fast Message TimeStamp: 1678556, Data: A3CD15, status: 13, CRC: 6
90 Number 27 :Fast Message TimeStamp: 1744955, Data: A6DD25, status: 5, CRC: 8
91 Number 28 :Fast Message TimeStamp: 1811352, Data: A2DD35, status: 1, CRC: 8
92 Number 29 :Fast Message TimeStamp: 1877750, Data: B2DD44, status: 1, CRC: 5
93 Number 30 :Fast Message TimeStamp: 1944151, Data: A77D55, status: 1, CRC: 7
94 Number 31 :Fast Message TimeStamp: 2010551, Data: A68D65, status: 1, CRC: 5
95 Number 32 :Fast Message TimeStamp: 2076948, Data: ABCD75, status: 1, CRC: 14
96 Number 33 :Fast Message TimeStamp: 2143348, Data: A74D85, status: 1, CRC: 12
97 Number 34 :Fast Message TimeStamp: 2209746, Data: A94D95, status: 1, CRC: 9
98 Number 35 :Fast Message TimeStamp: 2276141, Data: AAFDA5, status: 1, CRC: 11
99 Number 36 :Fast Message TimeStamp: 2342540, Data: A1FD85, status: 1, CRC: 2
00 Number 37 :Fast Message TimeStamp: 2408938, Data: AA5DC5, status: 9, CRC: 8
01 Number 38 :Fast Message TimeStamp: 2475330, Data: A67D05, status: 5, CRC: 4
02 Number 39 :Fast Message TimeStamp: 2541722, Data: A8DDE5, status: 9, CRC: 10
03 Number 40 :Fast Message TimeStamp: 2608116, Data: A66DF5, status: 13, CRC: 4
04 Number 41 :Fast Message TimeStamp: 2674510, Data: B00E04, status: 9, CRC: 13
05 Number 42 :Fast Message TimeStamp: 2740905, Data: A8FE15, status: 13, CRC: 8
06 Number 43 :Fast Message TimeStamp: 2807303, Data: A64E25, status: 9, CRC: 2
07 Number 44 :Fast Message TimeStamp: 2873697, Data: AE0E35, status: 9, CRC: 9
08 Number 45 :Fast Message TimeStamp: 2940092, Data: AB1E45, status: 1, CRC: 15
09 Number 46 :Fast Message TimeStamp: 3006491, Data: A57E55, status: 1, CRC: 15
10 Number 47 :Fast Message TimeStamp: 3072888, Data: ABCE65, status: 1, CRC: 5
11 Number 48 :Fast Message TimeStamp: 3139288, Data: A62E75, status: 1, CRC: 13
12 Number 49 :Fast Message TimeStamp: 3205692, Data: A65E85, status: 9, CRC: 15
13 Number 50 :Fast Message TimeStamp: 3272088, Data: A2CE95, status: 1, CRC: 12
14 =====slow message list=====
15 Number 1 :slow Message TimeStamp: 1280177, Data: 0, ID: 43, CRC: 19
16 Number 2 :slow Message TimeStamp: 1280177, Data: 1, ID: 1, CRC: 63
17 =====Sent Interrupt demo End=====

```

2.50.2.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\SENT\SENT_Interrupt_Project\IAR_Project, and double-click the **SENT_Interrupt** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.50.2.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.50.3 SENT_Dma

2.50.3.1 Description

The purpose of this demo is to show you the general configuration process to use the SENT with DMA module.

2.50.3.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 MLX90367LDC Sent sensor
- 1 USB Type-C cable

- 1 Personal computer
- 1 J-Link debugger

2.50.3.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port E19: MUX = ALT4, SENT0_RXD2	SENT0, Channel 2, RXD

2.50.3.4 Steps to Run

2.50.3.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: Make sure "Debug_Flash" is selected as the active build configuration.

After building and debugging, you can see the below output via the UART tool:

```

119 [2023-02-06 14:50:56.449 R]=====Sent dma demo Start=====
120
121 [2023-02-06 14:50:57.216 R]Receive 50 Fast messages and 2 slow messages
122 =====Fast message list=====
123 Number 1 :Fast Message TimeStamp: 1975, Data: A9CD65, status: 9, CRC: 13
124 Number 2 :Fast Message TimeStamp: 8012, Data: A3BD75, status: 13, CRC: 11
125 Number 3 :Fast Message TimeStamp: 14048, Data: 9E5D86, status: 1, CRC: 13
126 Number 4 :Fast Message TimeStamp: 20084, Data: A71D95, status: 1, CRC: 7
127 Number 5 :Fast Message TimeStamp: 26121, Data: A88DA5, status: 1, CRC: 7
128 Number 6 :Fast Message TimeStamp: 32157, Data: 9ACD86, status: 1, CRC: 10
129 Number 7 :Fast Message TimeStamp: 38193, Data: A01DC5, status: 1, CRC: 0
130 Number 8 :Fast Message TimeStamp: 44230, Data: A61DD5, status: 1, CRC: 1
131 Number 9 :Fast Message TimeStamp: 50266, Data: AE4DE5, status: 1, CRC: 10
132 Number 10 :Fast Message TimeStamp: 56302, Data: A20DF5, status: 1, CRC: 3
133 Number 11 :Fast Message TimeStamp: 62339, Data: AA6E05, status: 9, CRC: 8
134 Number 12 :Fast Message TimeStamp: 68375, Data: AA9E15, status: 9, CRC: 12
135 Number 13 :Fast Message TimeStamp: 74412, Data: AEEE25, status: 5, CRC: 9
136 Number 14 :Fast Message TimeStamp: 80449, Data: A4CE35, status: 5, CRC: 6
137 Number 15 :Fast Message TimeStamp: 86485, Data: A7DE45, status: 13, CRC: 3
138 Number 16 :Fast Message TimeStamp: 92521, Data: A9FE55, status: 13, CRC: 14
139 Number 17 :Fast Message TimeStamp: 98557, Data: A63E65, status: 13, CRC: 11
140 Number 18 :Fast Message TimeStamp: 104594, Data: AA4E75, status: 13, CRC: 14
141 Number 19 :Fast Message TimeStamp: 110630, Data: A39E85, status: 13, CRC: 9
142 Number 20 :Fast Message TimeStamp: 116666, Data: AB6E95, status: 13, CRC: 9
143 Number 21 :Fast Message TimeStamp: 122702, Data: A4CEA5, status: 5, CRC: 9
144 Number 22 :Fast Message TimeStamp: 128739, Data: A55EB5, status: 1, CRC: 6
145 Number 23 :Fast Message TimeStamp: 134775, Data: A54EC5, status: 1, CRC: 12
146 Number 24 :Fast Message TimeStamp: 140811, Data: A7FED5, status: 1, CRC: 4
147 Number 25 :Fast Message TimeStamp: 146847, Data: A02EE5, status: 1, CRC: 1
148 Number 26 :Fast Message TimeStamp: 152884, Data: 9F2EF6, status: 1, CRC: 5
149 Number 27 :Fast Message TimeStamp: 158920, Data: A71F05, status: 1, CRC: 6
150 Number 28 :Fast Message TimeStamp: 164956, Data: AE7F15, status: 1, CRC: 11
151 Number 29 :Fast Message TimeStamp: 170992, Data: A90F25, status: 1, CRC: 1
152 Number 30 :Fast Message TimeStamp: 177028, Data: A12F35, status: 1, CRC: 15
153 Number 31 :Fast Message TimeStamp: 183064, Data: AEAFA5, status: 9, CRC: 15
154 Number 32 :Fast Message TimeStamp: 189101, Data: A5AF55, status: 5, CRC: 6
155 Number 33 :Fast Message TimeStamp: 195137, Data: B00F64, status: 13, CRC: 6
156 Number 34 :Fast Message TimeStamp: 201173, Data: A3AF75, status: 13, CRC: 1
157 Number 35 :Fast Message TimeStamp: 207209, Data: A14F85, status: 13, CRC: 1
158 Number 36 :Fast Message TimeStamp: 213245, Data: B1BF94, status: 13, CRC: 0
159 Number 37 :Fast Message TimeStamp: 219281, Data: A1AFA5, status: 13, CRC: 7
160 Number 38 :Fast Message TimeStamp: 225317, Data: A7BF85, status: 9, CRC: 2
161 Number 39 :Fast Message TimeStamp: 231353, Data: A88FC5, status: 1, CRC: 9
162 Number 40 :Fast Message TimeStamp: 237389, Data: A68FD5, status: 1, CRC: 12
163 Number 41 :Fast Message TimeStamp: 243425, Data: A61FE5, status: 1, CRC: 9
164 Number 42 :Fast Message TimeStamp: 249462, Data: ACBFF5, status: 1, CRC: 1
165 Number 43 :Fast Message TimeStamp: 255497, Data: A6E005, status: 1, CRC: 4
166 Number 44 :Fast Message TimeStamp: 261534, Data: A66015, status: 1, CRC: 1
167 Number 45 :Fast Message TimeStamp: 267570, Data: A1D025, status: 1, CRC: 1
168 Number 46 :Fast Message TimeStamp: 273606, Data: 9D5036, status: 1, CRC: 13
169 Number 47 :Fast Message TimeStamp: 279642, Data: A71045, status: 9, CRC: 4
170 Number 48 :Fast Message TimeStamp: 285678, Data: A26055, status: 1, CRC: 11
171 Number 49 :Fast Message TimeStamp: 291714, Data: AA3065, status: 9, CRC: 0
172 Number 50 :Fast Message TimeStamp: 297750, Data: A99075, status: 1, CRC: 2
173 =====slow message list=====
174 Number 1 :slow Message TimeStamp: 189101, Data: 801, ID: 1, CRC: 63
175 Number 2 :slow Message TimeStamp: 297750, Data: 0, ID: 5, CRC: 62
176 =====Sent dma demo End=====

```

2.50.3.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\SENT\SENT_Dma_Project\IAR_Project, and double-click the **SENT_DMA** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.50.3.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.50.4 SENT_SPC

2.50.4.1 Description

The purpose of this demo is to show you the general configuration process to use the SENT with SPC mode.

2.50.4.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 TLE4998 Sent sensor
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.50.4.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port E19: MUX = ALT4, SENT0_RXD2	SENT0, Channel 2, RXD

2.50.4.4 Steps to Run

2.50.4.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: Make sure "**Debug_Flash**" is selected as the active build configuration.

After building and debugging, you can see the below output via the UART tool:

```

183 [2023-02-06 14:57:32.127 R]=====Sent SPC demo Start=====
184
185 [2023-02-06 14:57:32.884 R]Receive Fast messages
186 =====Fast message list=====
187 Number 1 :Fast Message TimeStamp: 1318774, Data: 7FA452, status: 11, CRC: 6
188 Number 2 :Fast Message TimeStamp: 1382326, Data: 7F8852, status: 3, CRC: 5
189 Number 3 :Fast Message TimeStamp: 1444269, Data: 7F9152, status: 3, CRC: 4
190 Number 4 :Fast Message TimeStamp: 1504708, Data: 7F8852, status: 3, CRC: 5
191 Number 5 :Fast Message TimeStamp: 1566649, Data: 7F9152, status: 3, CRC: 4
192 Number 6 :Fast Message TimeStamp: 1627078, Data: 7F8852, status: 3, CRC: 12
193 Number 7 :Fast Message TimeStamp: 1691166, Data: 7F7F52, status: 3, CRC: 11
194 Number 8 :Fast Message TimeStamp: 1755657, Data: 7F9152, status: 3, CRC: 4
195 Number 9 :Fast Message TimeStamp: 1816086, Data: 7F8852, status: 3, CRC: 12
196 Number 10 :Fast Message TimeStamp: 1880192, Data: 7F9452, status: 3, CRC: 2
197 Number 11 :Fast Message TimeStamp: 1940872, Data: 7F8852, status: 3, CRC: 5
198 Number 12 :Fast Message TimeStamp: 2002831, Data: 7F8852, status: 3, CRC: 5
199 Number 13 :Fast Message TimeStamp: 2064772, Data: 7F8852, status: 3, CRC: 12
200 Number 14 :Fast Message TimeStamp: 2128859, Data: 7F8852, status: 3, CRC: 5
201 Number 15 :Fast Message TimeStamp: 2190799, Data: 7F8852, status: 3, CRC: 12
202 Number 16 :Fast Message TimeStamp: 2254906, Data: 7F8852, status: 3, CRC: 5
203 Number 17 :Fast Message TimeStamp: 2316945, Data: 7F9452, status: 3, CRC: 2
204 Number 18 :Fast Message TimeStamp: 2377660, Data: 7F8852, status: 3, CRC: 5
205 Number 19 :Fast Message TimeStamp: 2439620, Data: 7F8852, status: 3, CRC: 5
206 Number 20 :Fast Message TimeStamp: 2501579, Data: 7F9852, status: 3, CRC: 8
207 Number 21 :Fast Message TimeStamp: 2565041, Data: 7F8852, status: 3, CRC: 5
208 Number 22 :Fast Message TimeStamp: 2626982, Data: 7F8252, status: 3, CRC: 9
209 Number 23 :Fast Message TimeStamp: 2688531, Data: 7F7F52, status: 3, CRC: 11
210 Number 24 :Fast Message TimeStamp: 2753022, Data: 7F8852, status: 3, CRC: 12
211 Number 25 :Fast Message TimeStamp: 2817111, Data: 7F8252, status: 3, CRC: 9
212 Number 26 :Fast Message TimeStamp: 2878659, Data: 7F8852, status: 3, CRC: 12
213 Number 27 :Fast Message TimeStamp: 2942746, Data: 7F9152, status: 3, CRC: 4
214 Number 28 :Fast Message TimeStamp: 3003174, Data: 7F9852, status: 3, CRC: 8
215 Number 29 :Fast Message TimeStamp: 3066653, Data: 7F8852, status: 3, CRC: 12
216 Number 30 :Fast Message TimeStamp: 3130742, Data: 7F8852, status: 3, CRC: 5
217 Number 31 :Fast Message TimeStamp: 3192683, Data: 7F8852, status: 3, CRC: 5
218 Number 32 :Fast Message TimeStamp: 3254624, Data: 7F9E52, status: 3, CRC: 14
219 Number 33 :Fast Message TimeStamp: 3319991, Data: 7F8852, status: 3, CRC: 5
220 Number 34 :Fast Message TimeStamp: 3381930, Data: 7F9152, status: 3, CRC: 4
221 Number 35 :Fast Message TimeStamp: 3442368, Data: 7F8852, status: 3, CRC: 12
222 Number 36 :Fast Message TimeStamp: 3506472, Data: 7F8852, status: 3, CRC: 12
223 Number 37 :Fast Message TimeStamp: 3570560, Data: 7F9852, status: 3, CRC: 8
224 Number 38 :Fast Message TimeStamp: 3634031, Data: 7F9452, status: 3, CRC: 2
225 Number 39 :Fast Message TimeStamp: 3694720, Data: 7F7F52, status: 3, CRC: 11
226 Number 40 :Fast Message TimeStamp: 3759209, Data: 7F8852, status: 3, CRC: 12
227 Number 41 :Fast Message TimeStamp: 3823315, Data: 7F8852, status: 3, CRC: 12
228 Number 42 :Fast Message TimeStamp: 3887403, Data: 7F8852, status: 3, CRC: 12
229 Number 43 :Fast Message TimeStamp: 3951509, Data: 7F8852, status: 3, CRC: 12
230 Number 44 :Fast Message TimeStamp: 4015596, Data: 7F7F52, status: 3, CRC: 11
231 Number 45 :Fast Message TimeStamp: 4080095, Data: 7F9152, status: 3, CRC: 4
232 Number 46 :Fast Message TimeStamp: 4140525, Data: 7F8852, status: 3, CRC: 5
233 Number 47 :Fast Message TimeStamp: 4202465, Data: 7F8852, status: 3, CRC: 12
234 Number 48 :Fast Message TimeStamp: 4266571, Data: 7F9E52, status: 3, CRC: 14
235 Number 49 :Fast Message TimeStamp: 4331938, Data: 7F8852, status: 3, CRC: 12
236 Number 50 :Fast Message TimeStamp: 4396026, Data: 7F8852, status: 3, CRC: 12
237 =====Sent SPC demo End=====

```

2.50.4.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\SENT\SENT_SPC_Project\IAR_Project, and double-click the **SENT_SPC** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.50.4.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.51 SMC

2.51.1 Description

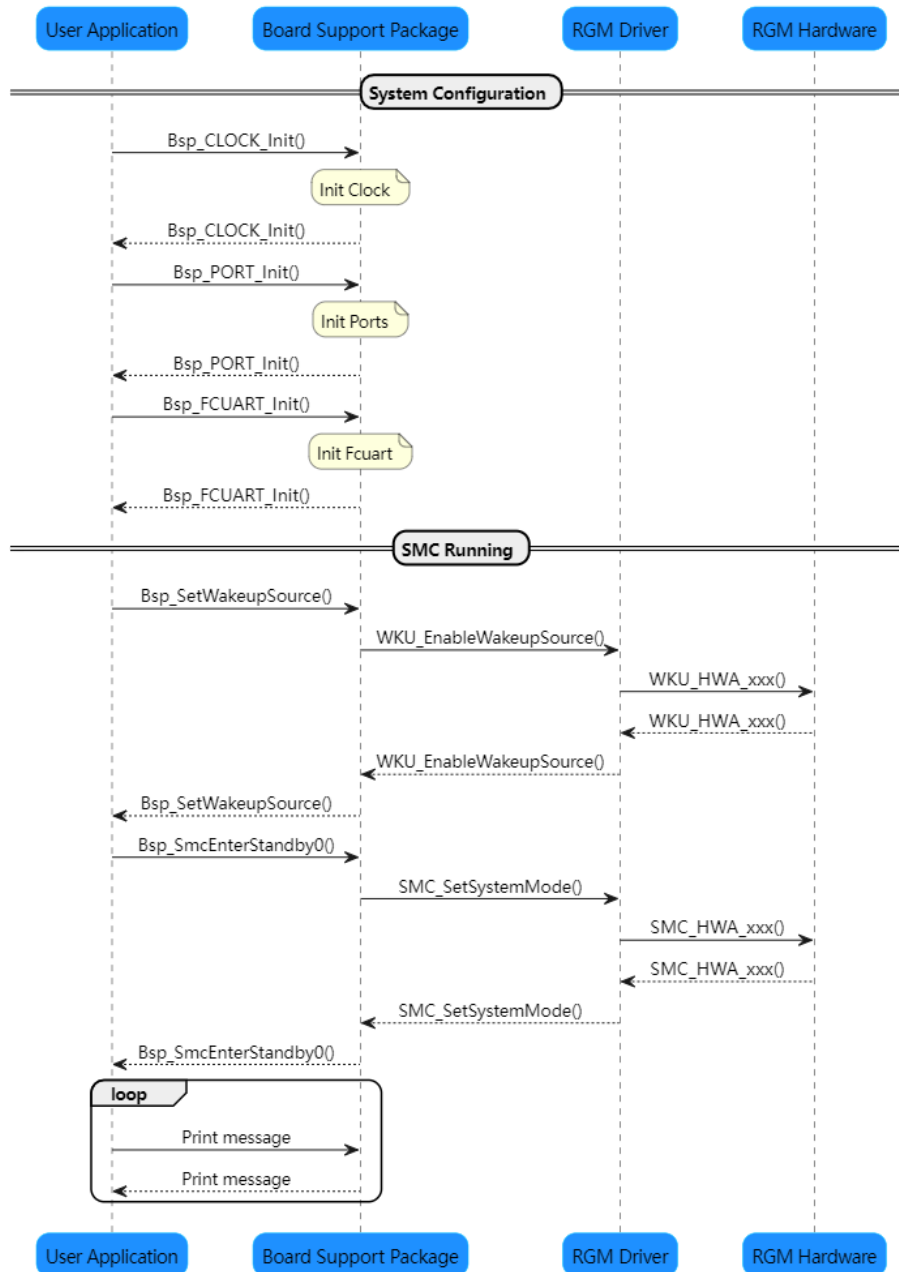
This demo shows how to enter low power mode and how to wake up from low power mode.

This demo contains two projects: SDK_FC7xxx_SMC_Demo, SDK_FC7xxx_SMC_Demo_Multicore. The two projects correspond to three cores respectively.

This demo use Key3 (PTA11) as the wakeup source, the configured KEY3 need to be pressed for the chip to wake up.

In this demo, the low power mode refers to standby mode. And the code will reset after the chip is woken up.

The sequence diagram of the demo is as follows:



2.51.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger or other debug tool

2.51.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port F23: MUX = ALT3, UART0_TX	FCUART0 TXD
Port G21: MUX = ALT3, UART0_RX	FCUART0 RXD
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1_RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A26: MUX = ALT1, GPIO Output	LED1
Port D31: MUX = ALT1, GPIO Output	LED2
Port A14: MUX = ALT1, GPIO Output	LED3
Port A11: MUX = ALT7, NMI	-
Port D20: MUX = ALT1, GPIO IN	-
Port D26: MUX = ALT1, GPIO IN	-
Port F21: MUX = ALT1, GPIO IN wakeup	-

Note: Use UART by USB-Type C port and connect 12V power.

2.51.4 Steps to Run

The steps to run the example are as follows:

- 1) Plug in the USB Type-C cable, and connect the PC and demo board. Turn on the power, and then the demo board will power on.
- 2) Download the program to the MCU by JTAG.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 3) UART information will print regularly and LED will toggle.

2.52 STCU

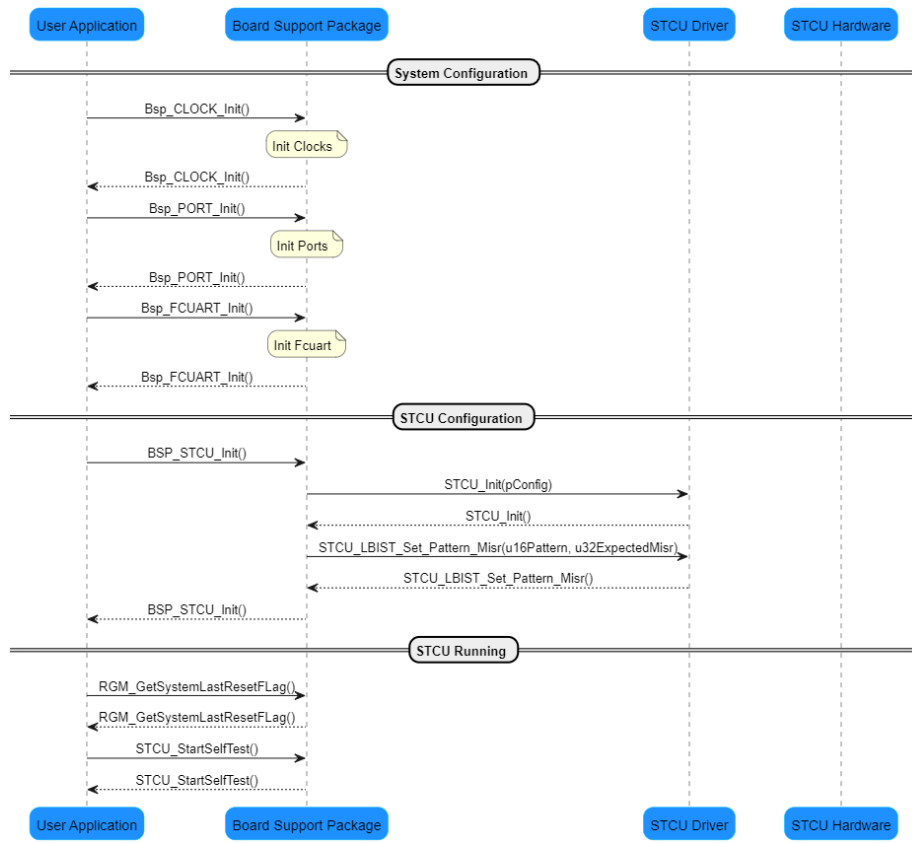
2.52.1 Description

The purpose of this demo is to show you the general configuration process to use the STCU to software trigger the LBIST and MBIST, and use STCU to initialize the SRAM.

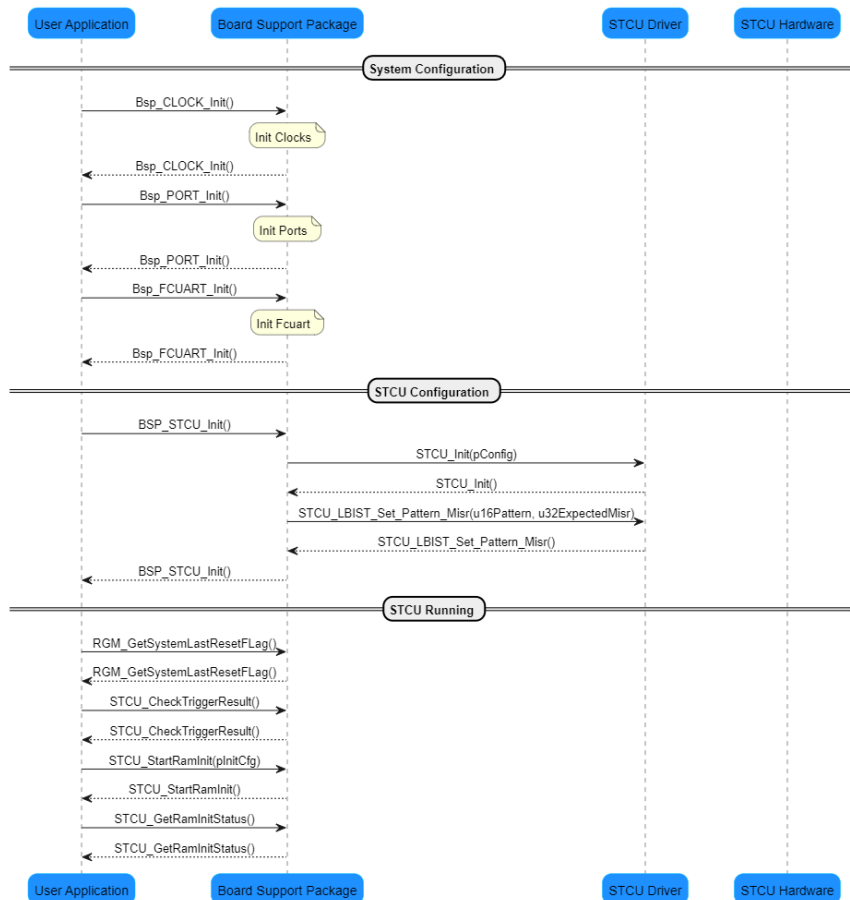
Note: The demo does not support downloading to SRAM because STCU would initialize SRAM in demo.

The sequence diagram of the demo before self test done is as follows:

FC7300F8MDQ SDK Quick Start



the sequence diagram of the demo after self test done is as follows:



2.52.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger

2.52.3 MCU Ports

N/A

2.52.4 Steps to Run

2.52.4.1.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: Make sure "**Debug_Flash**" is selected as the active build configuration.

After building and debugging, you can see the below output via the UART tool:

```
=====Mbist&Lbist test start=====
Lbist test result is PASS!
Mbist test result is PASS!
=====Mbist&Lbist test Done=====
=====Ram Init test Start=====
SRAM0 has been initialized to whole zero
SRAM1 has been initialized to whole zero
SRAM2 has been initialized to whole zero
=====Ram Init test Done=====
█
```

2.52.4.1.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\STCU\STCU_RAM_Project\IAR_Project, and double-click the **STCU_Demo** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.52.4.1.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.53 SysTick

2.53.1 Description

This demo project uses a timer to enter the interrupt for 1s for IO operation.

2.53.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 USB Type-C cable

- 1 Personal computer
- 1 J-Link debugger

2.53.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A26: MUX = PORT_GPIO_MODE	LED1
Port D31: MUX = PORT_GPIO_MODE	LED2
Port A13: MUX = PORT_GPIO_MODE	LED3

2.53.4 Steps to Run

2.53.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

2.53.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\Systick\Systick_demo_Project\IAR_Project, and double-click the **Systick_Demo** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

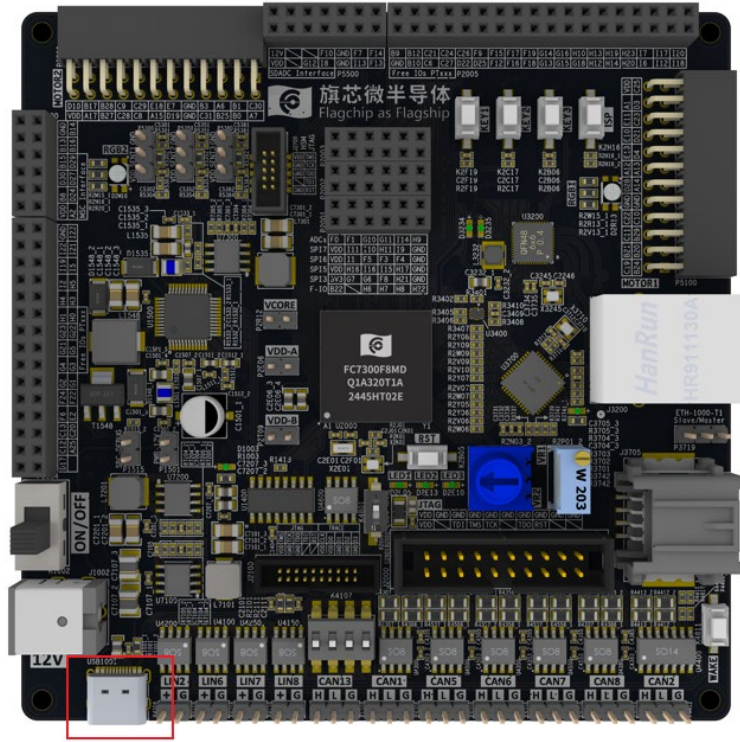
2.53.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

2.54 TMU

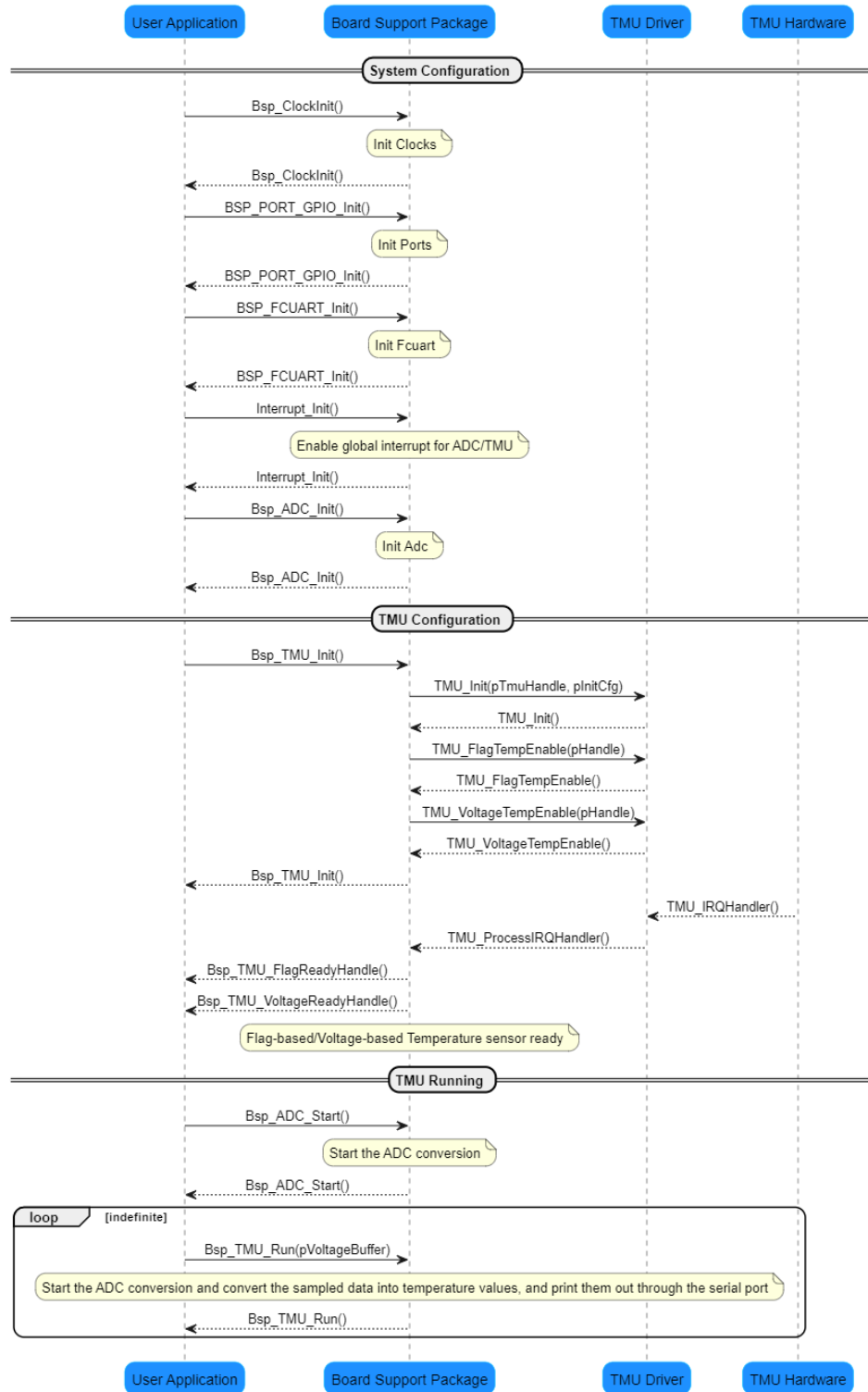
2.54.1 Description

The TMU module provides a circuit for monitoring temperature, including voltage-based temperature sensor and flag-based temperature sensor. The purpose of this demo is to print the current temperature through the serial port.



The sequence diagram of the demo is as follows:

FC7300F8MDQ SDK Quick Start



2.54.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power adapter 12 V
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link debugger
- 1 PC Uart tool

2.54.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection

Users use the serial to display the temperature.

2.54.4 Steps to Run

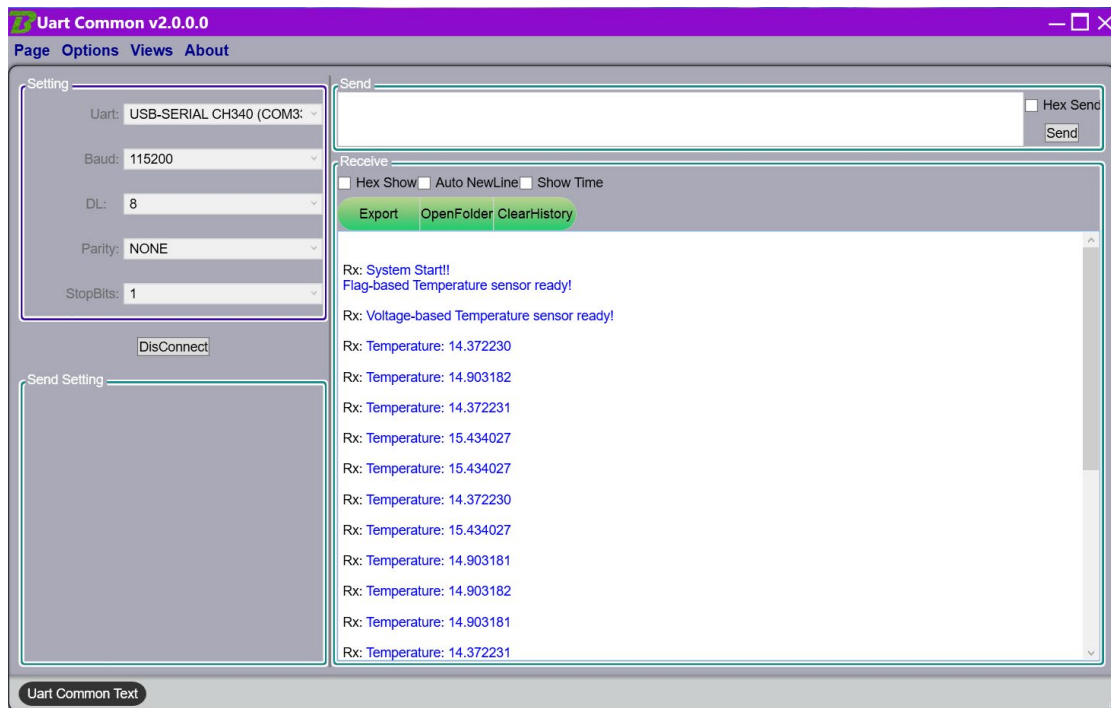
The steps to run the example are as follows:

- 1) Plug in the USB Type-C cable and open the up-machine (UART1 115200/8/N/1).
- 2) Import the TMU_Demo project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

- 3) Normally, MCU will start printing temperature message, and respond to the corresponding interrupt event.



2.55 TPU

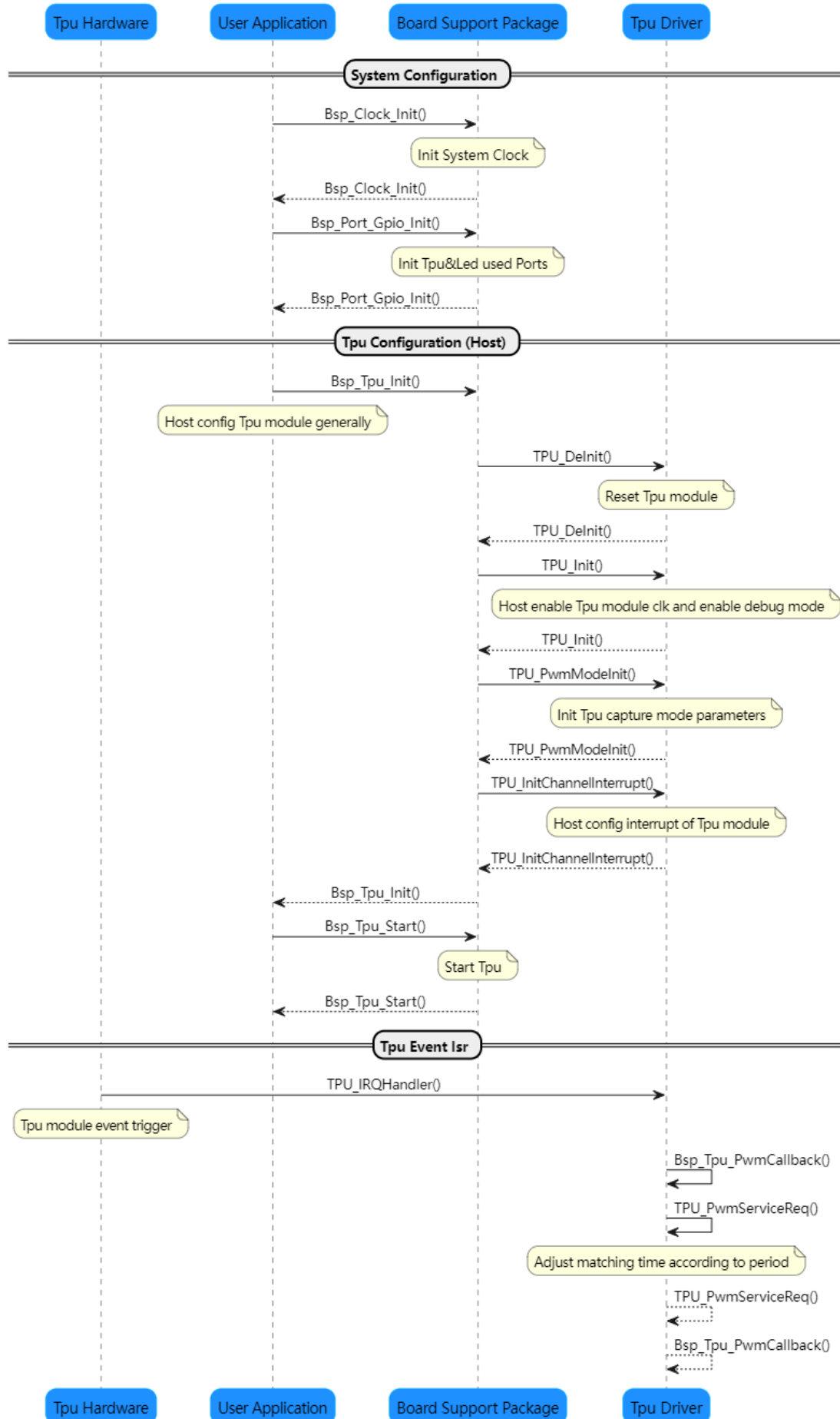
There are five TPU demos: Tpu_Pwm_NonFlexCore, Tpu_Capture_Host, Tpu_Host, Tpu_Pwm_FlexCore, and Tpu_Capture_FlexCore.

2.55.1 Tpu_Pwm_NonFlexCore

2.55.1.1 Description

This demo project shows how to use TPU module to output PWM through the SDK. This demo project uses PTC6 (TPUCH0) to output PWM. This demo project also toggles LED3 every second.

Note: Using this method will greatly consume CPU resources, especially when the PWM frequency is high, the interrupt frequency will be too high, causing the software to be unable to perform other tasks.



2.55.1.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power Adapter 12 V
- 1 Personal computer
- 1 J-Link debugger
- Dupont Lines
- Logic analyzer or oscilloscope

2.55.1.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port C6: MUX = ALT3, TPU0_CH0	For outputting PWM pulse
Port A26: MUX = ALT1, GPIO Output	For toggling LED3

2.55.1.4 Steps to Run

The steps to run the example are as follows:

- 1) Connect the PWM pin and GND pin to a logic analyzer or oscilloscope (if necessary).
- 2) Import the Tpu_Pwm_NonFlexCore project, then build and download it. Note that the unit of `u32ActiveTime` and `u32PeriodTime` are ticks, which are determined by TCR1 clock (50 MHz in demo project).

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

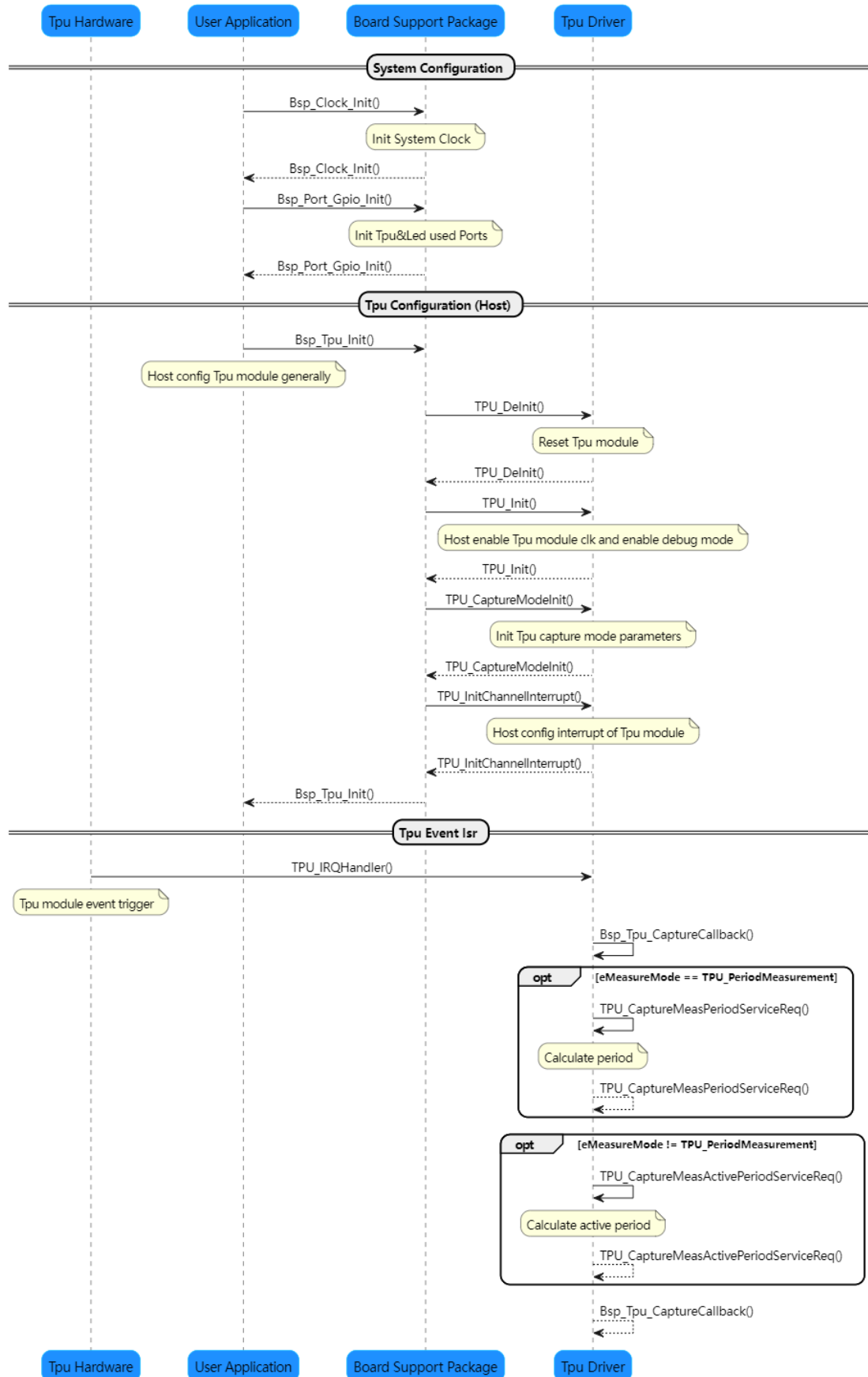
2.55.2 Tpu_Capture_Host

2.55.2.1 Description

This demo project shows how to use TPU module to capture frequency and duty through the SDK. This demo project uses PTG14 (TPU_CH8) to capture frequency and duty. This demo project also toggles LED3 every second.

Note: Using this method will not consume CPU resources so much, especially when the input PWM frequency is high, the interrupt frequency will be too high, causing the software to be unable to perform other tasks.

FC7300F8MDQ SDK Quick Start



2.55.2.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board

- 1 Power Adapter 12 V
- 1 Personal computer
- 1 J-Link debugger
- Dupont Lines
- Signal generator

2.55.2.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port G14: MUX = ALT2, TPU0_CH8	For capturing PWM pulse
Port A26: MUX = ALT1, GPIO Output	For toggling LED3

2.55.2.4 Steps to Run

The steps to run the example are as follows:

- 1) Connect the Capture pin and GND pin to a signal generator.
- 2) Import the Tpu_Capture_Host project, then build and download it. Note that the unit of u32SampleTime is ticks, which is determined by TCR1 clock (50 MHz in demo project).

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

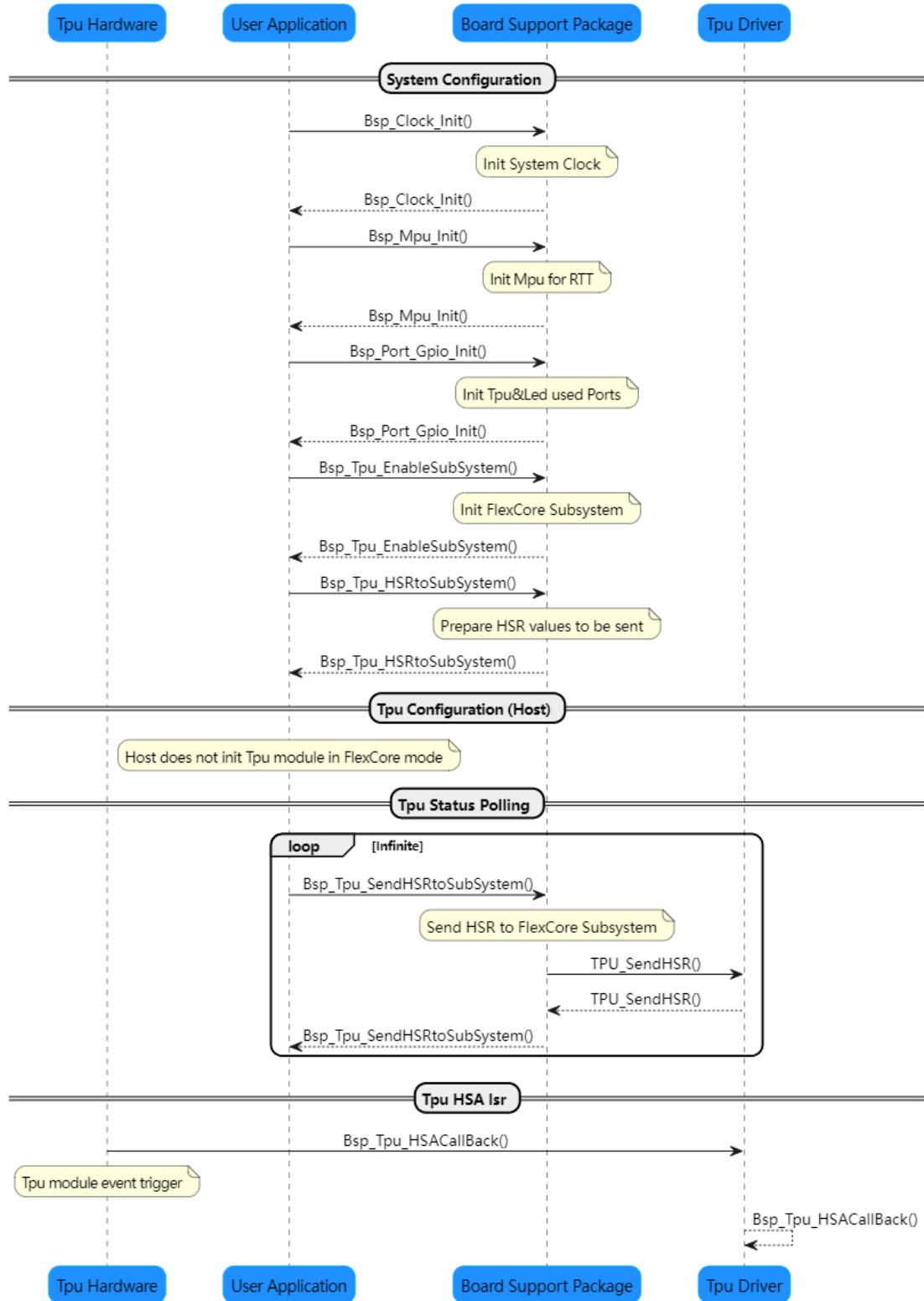
2.55.3 Tpu_Host

2.55.3.1 Description

This demo project demonstrates the combination of HST and FlexCore, and that project is used as host (CM7). This project shall be used in conjunction with the TPU project in FlexCore. (TPU\Tpu_Capture_FlexCore or TPU\Tpu_Pwm_FlexCore).

This demo project just enables FlexCore subsystem and then inform FlexCore to Start output PWM or capture PWM.

Note: Using this method will not consume CPU resources so much, due to most interrupt tasks are executed in FlexCore.



2.55.3.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power Adapter 12 V
- 1 Personal computer
- 1 J-Link debugger

2.55.3.3 MCU Ports

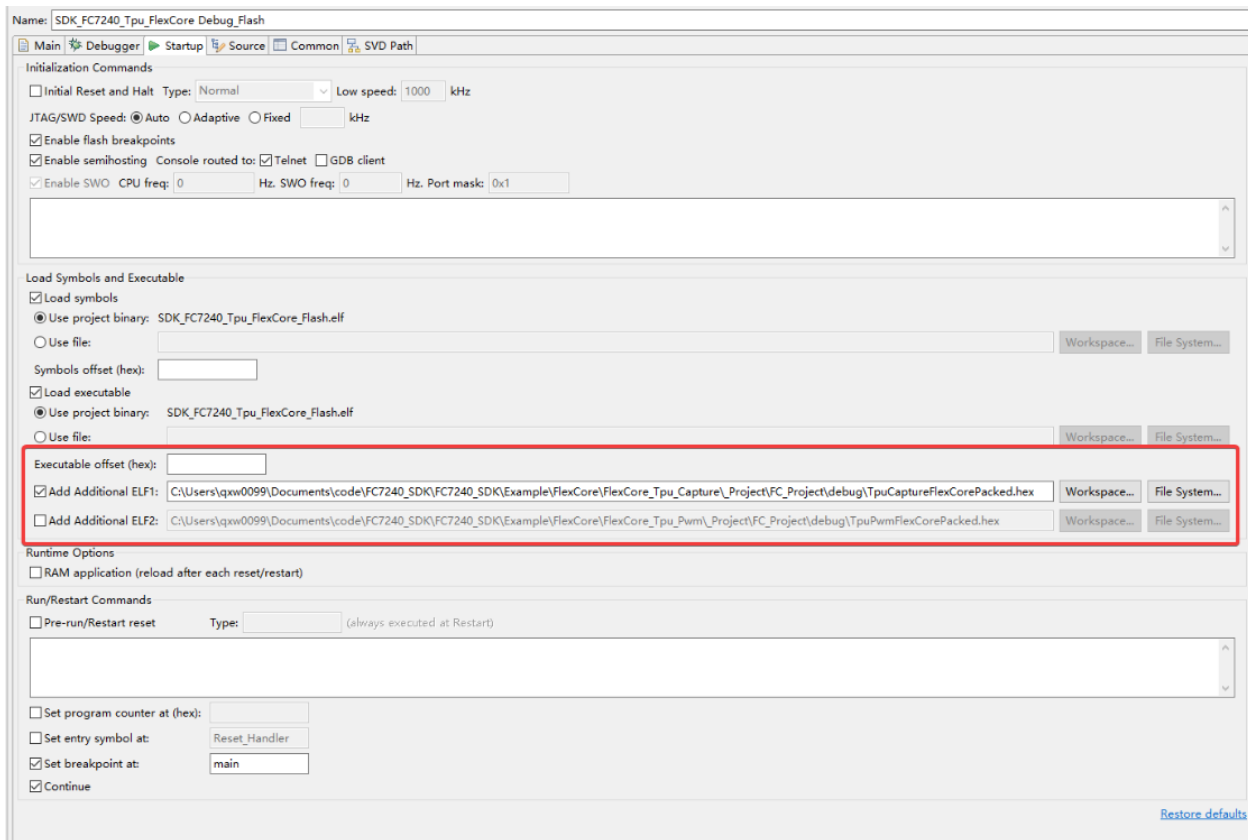
Pin Function	FC7300F8MDQ Demo Board
Port G14: MUX = ALT2, TPU0_CH8	For capturing PWM pulse
Port C6: MUX = ALT3, TPU0_CH0	For outputting PWM pulse
Port A26: MUX = ALT2, GPIO Output	For toggling LED3

2.55.3.4 Steps to Run

2.55.3.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: After opening the Tpu_Host project, choose one of the projects to be downloaded in FlexCore subsystem (Carefully to choose packed.hex of Tpu_xxx_FlexCore project).



2.55.3.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\TPU\Tpu_Host_Project\IAR_Project, and double-click the IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.55.3.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

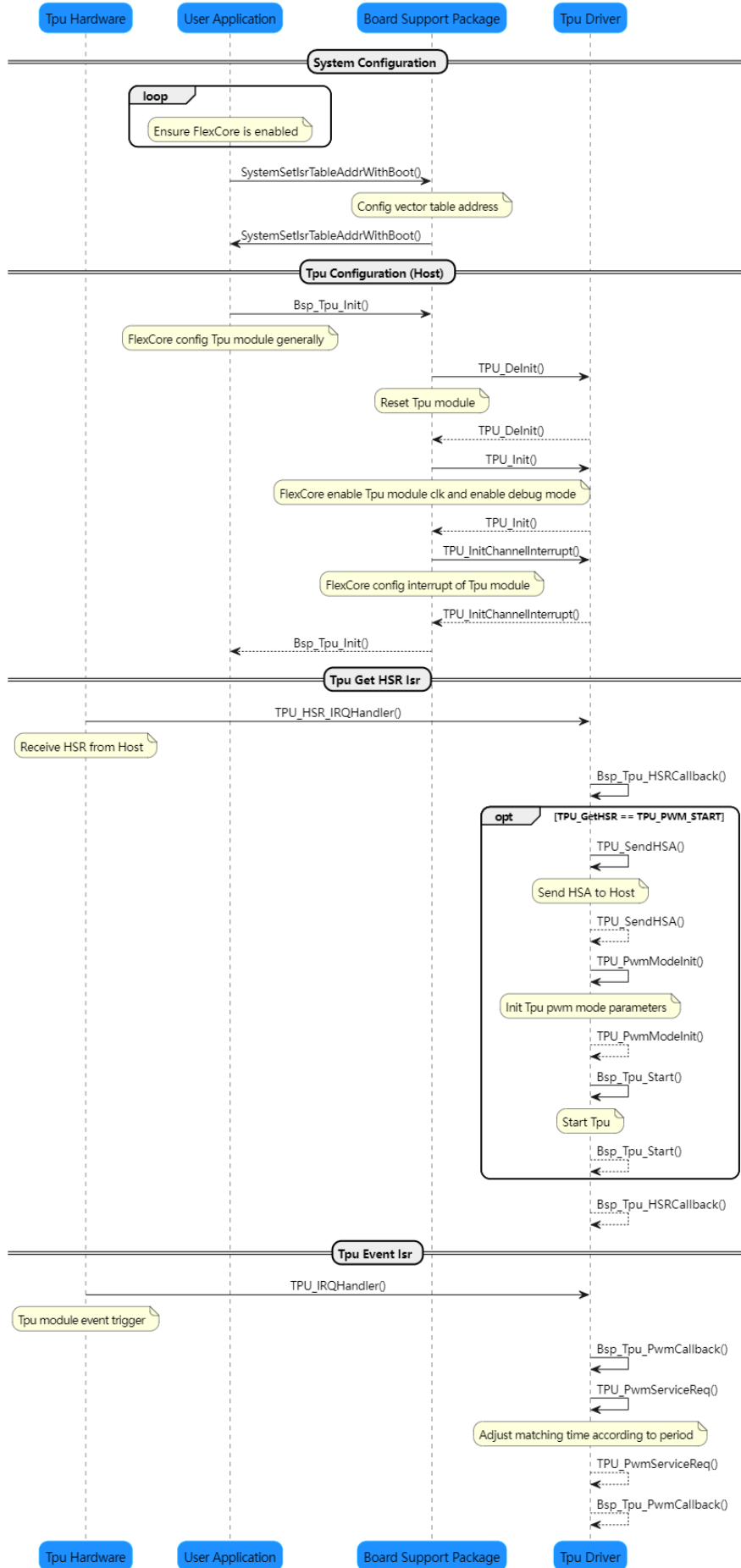
2.55.4 Tpu_Pwm_FlexCore

2.55.4.1 Description

This demo project shows how to be used in conjunction with the project Tpu_Host. Post-build hex of this project will be downloaded by the project Tpu_Host and can be debugged by downloading the hex to RAM directly.

Note: After the compilation of this project, there will be an additional post-processing hex that needs to be downloaded by the project Tpu_Host. If you want to debug, you need to download the original hex directly from the project to RAM.

FC7300F8MDQ SDK Quick Start



2.55.4.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power Adapter 12 V
- 1 Personal computer
- 1 J-Link debugger
- Dupont Lines
- Logic analyzer or oscilloscope

2.55.4.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port C6: MUX = ALT3, TPU0_CH0	For outputting PWM pulse
Port A26: MUX = ALT1, GPIO Output	For toggling LED3

2.55.4.4 Steps to Run

The steps to run the example are as follows:

- 1) Connect the PWM pin and GND pin to a logic analyzer or oscilloscope (if necessary).
- 2) Import the Tpu_Pwm_FlexCore project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

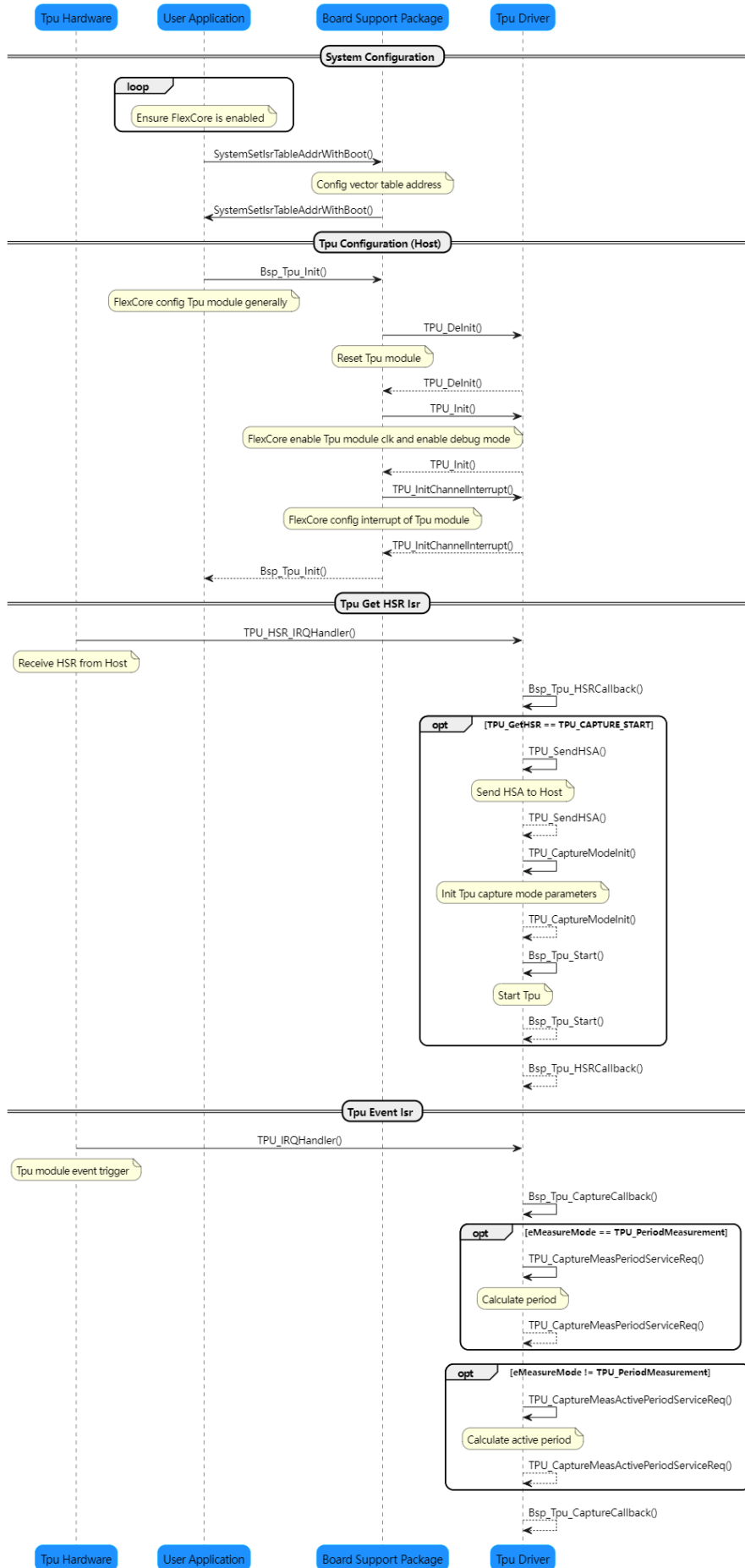
2.55.5 Tpu_Capture_FlexCore

2.55.5.1 Description

This demo project shows how to be used in conjunction with the project Tpu_Host. Post-build hex of this project will be downloaded by the project Tpu_Host and can be debugged by downloading the hex to RAM directly.

Note: After the compilation of this project, there will be an additional post-processing hex that needs to be downloaded by the project Tpu_Host. If you want to debug, you need to download the original hex directly from the project to RAM.

FC7300F8MDQ SDK Quick Start



2.55.5.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 Power Adapter 12 V
- 1 Personal computer
- 1 J-Link debugger
- Dupont Lines
- Signal generator

2.55.5.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port G14: MUX = ALT2, TPU0_CH8	For capturing PWM pulse
Port A26: MUX = ALT1, GPIO Output	For toggling LED3

2.55.5.4 Steps to Run

The steps to run the example are as follows:

- 1) Connect the PWM pin and GND pin to a signal generator.
- 2) Import the Tpu_Capture_FlexCore project, then build and download it.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
- To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
- To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).

2.56 TSTMP

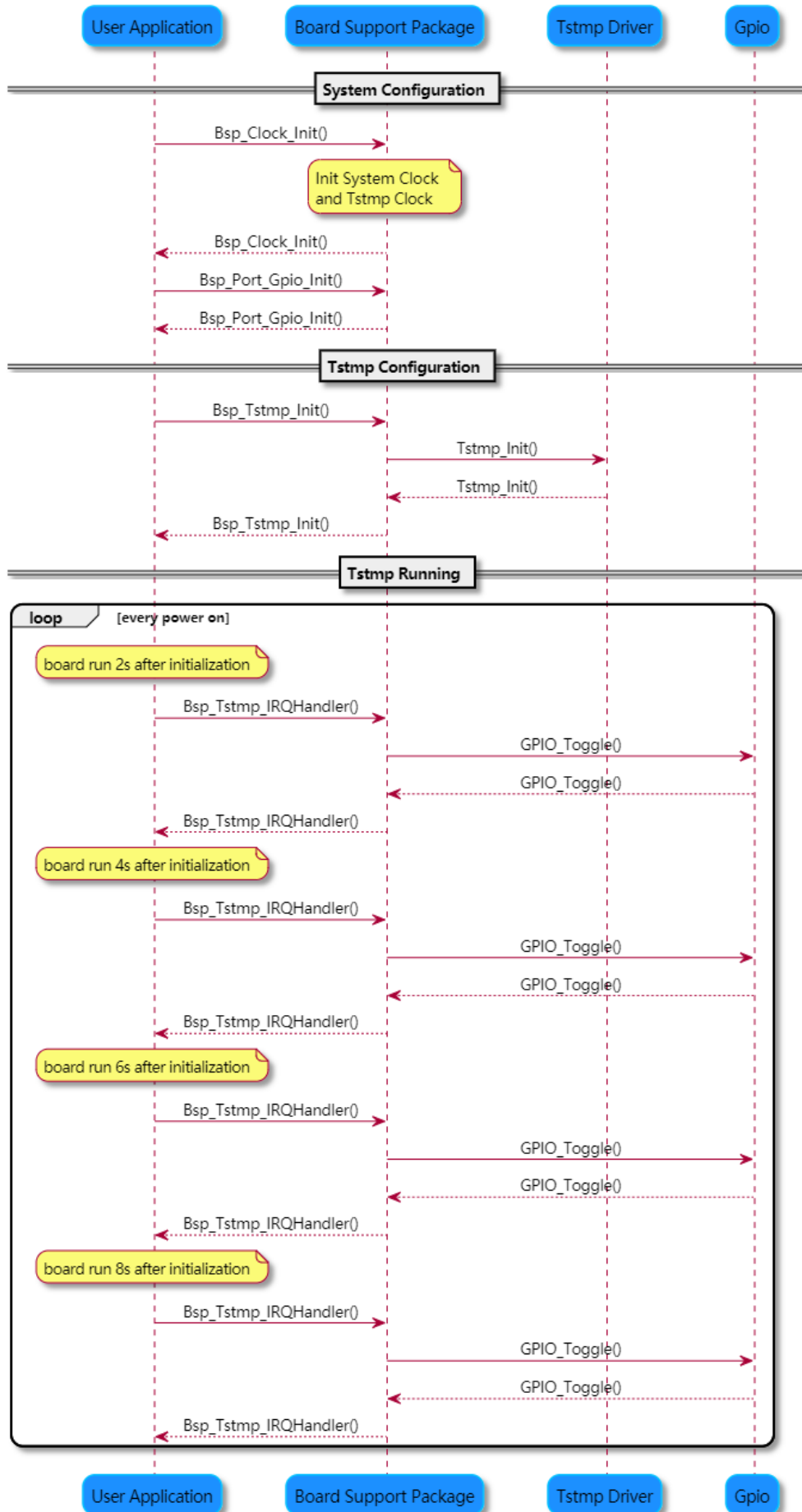
2.56.1 Description

This demo shows how to use the TSTMP1 to generate an interrupt immediately after POR through the SDK.

This demo project will toggle LED every 2s/4s/6s/8s after POR.

In addition, the module counter of TSTMP0 can be reset after POR, while the module counter of TSTMP1-3 can be reset after POR-reset, PCC-reset and PIN-reset.

The sequence diagram of the demo is as follows:



2.56.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board
- 1 USB Type-C cable
- 1 Personal computer
- 1 J-Link Debugger

2.56.3 MCU Ports

N/A.

2.56.4 Steps to Run

2.56.4.1 For FC_IDE

To run the example by using the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).

Note: Make sure the device name "FC7300F8MD_CORE0" is selected when debugging.

2.56.4.2 For IAR Embedded Workbench

To run the example by using the IAR, take the following steps:

Note: Before using the IAR, make sure you have performed the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).

- 1) Open the directory: SDK\Example\TSTMP\TSTMP_Interrupt_Project\IAR_Project, and double-click the **TSTMP_Interrupt** IAR project file with the suffix ".eww".
- 2) After the project being opened in the IAR, click the **Download and Debug** button in the toolbar.

2.56.4.3 For Keil

Note: Before using the Keil, make sure you have performed the steps described in [Chapter 1.3.3 Keil](#).

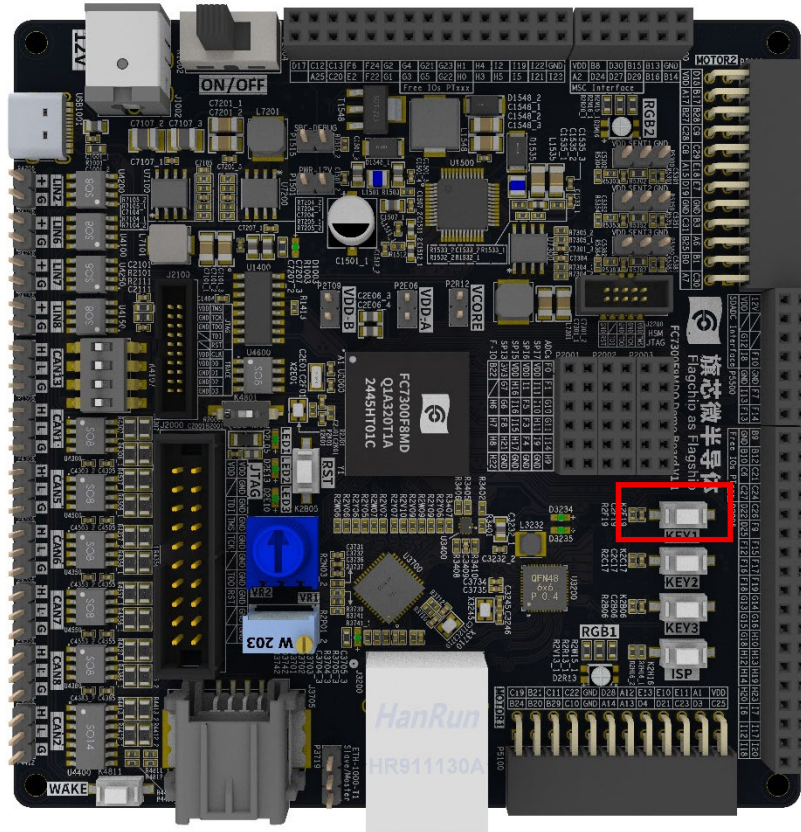
2.57 WDOG

2.57.1 Description

This demo shows how to set a windowed watch dog, and feed it in the windowed time.

Note: The demo just supports flash mode, but not RAM mode, because the demo will use POR reset.

FC7300F8MDQ SDK Quick Start



WDOG initialization settings are as following:

```
WDOG_CfgType Wdog_InitStruct = {0};
```

```
Wdog_InitStruct.bWinEnable= true;
```

```
/* set WDOG source as SIRC, form the EVB board, current value is 12 000 000 Hz */
```

```
Wdog_InitStruct.eClkSource= WDOG_SIRC_CLK;
```

```
/* 256 pre-scaler enable */
```

```
Wdog_InitStruct.bPrescalerEnable =true;
```

```
Wdog_InitStruct.eTimeoutReaction = WDOG_REACTION_INT_1024_CYCLE;
```

```
Wdog_InitStruct.eTesttype = WDOG_TST_DISABLED;
```

```
Wdog_InitStruct.bEnableInDebug= false;
```

```
Wdog_InitStruct.bEnableInWait = false;
```

```
Wdog_InitStruct.bEnableInStop= false;
```

```
/* set the time out value as 37500 * 256 /12000000 = 0.8s */
```

```
Wdog_InitStruct.u16WindowValue = 37500U;
```

```
/* set the time out value as 56250 * 256 /12000000 = 1.2s */
```

```
Wdog_InitStruct.u16TimeoutValue= 56250U;
```

```
g_tWdogHandle.eInstance = WDOG_INSTANCE_0;
```

```
/* set the IRQ callback function */
```

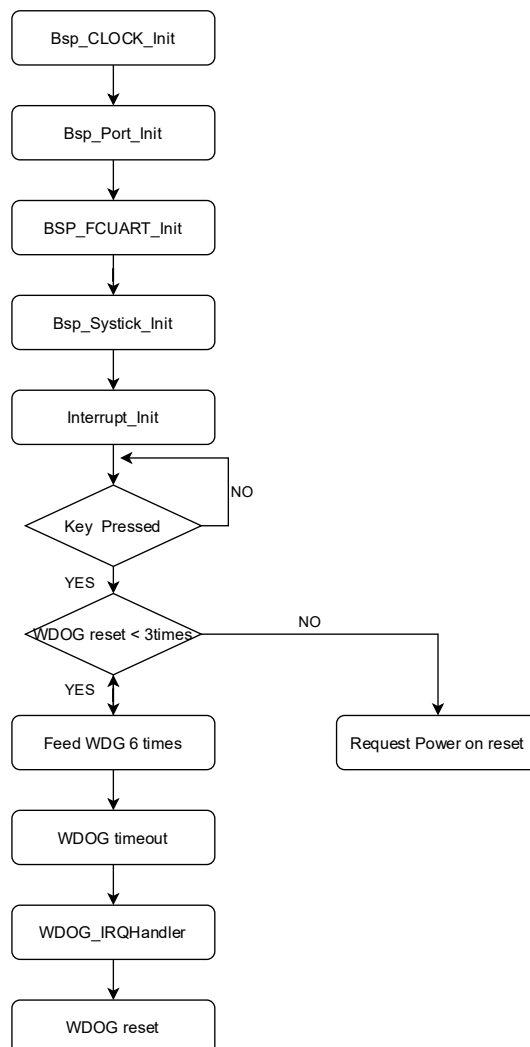
```
g_tWdogHandle.tSettings.plSRCallback = WDOG0_IRQHandler_Callback;
```

```
WDOG_Init(&g_tWdogHandle, &Wdog_InitStruct);
```

Enable the windowed watchdog, and set the windowed time between 0.8 s to 1.2 s. In this demo, feed the watchdog every 1 s. Use the SYSTICK to control the timing window.

When a watchdog timeout occurs, an interrupt handler would increase (RGM->[SRIE](#)). This register value would change only on POR, so use it to record WDOG reset times.

The detailed demo sequence flow is as follows:



2.57.2 Prerequisites

To run the example, make sure you have the following materials:

- 1 FC7300F8MDQ Demo Board

- 1 Power adapter 12 V
- 1 USB Type-C cable
- 1 Personal computer
- 1 PC UART tool

2.57.3 MCU Ports

Pin Function	FC7300F8MDQ Demo Board
Port A18: MUX = ALT3, UART1_TX	FCUART1 TXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port A19: MUX = ALT3, UART1_RX	FCUART1 RXD, UART converted to USB. Connector (USB TYPE-C) for external UART connection
Port D20: MUX = GPIO, Key 1	Pressing Key 1 would trigger the watchdog feed process

2.57.4 Steps to Run

The steps to run the example are as follows:

- 1) Plug in the micro-USB cable, connect the PC and demo board. Turn the power key to USB, and then the demo board will power on.
- 2) Download the program to the MCU by JTAG.

Note:

- To use the FC_IDE, refer to the steps described in [Chapter 1.3.1 FC_IDE](#).
 - To use the IAR, refer to the steps described in [Chapter 1.3.2 IAR Embedded Workbench](#).
 - To use the Keil, refer to the steps described in [Chapter 1.3.3 Keil](#).
- 3) Open the PC UART tool and connect it (parameter setting 115200/8/N/1).
 - 4) Press KEY 1. System initializes the windowed watchdog, and starts feeding.
 - 5) The MCU will stop feeding after feeding 6 times, then the watchdog will time out, the watchdog interrupt will be triggered, and the watchdog reset will occur.
 - 6) If the watchdog resets 3 times, perform a power-on reset.

Rx: System Start

Rx: Watch dog initialization start!
Watch dog reset increased value as 0 !

Rx: Watch dog feed the 1 time

Rx: Watch dog feed the 2 time

Rx: Watch dog feed the 3 time

Rx: Watch dog feed the 4 time

Rx: Watch dog feed the 5 time

Rx: Watch dog feed the 6 time

Rx: System Start
Please press KEY 1 bottom to continue the demo

Rx: Watch dog reset occurred
Watch dog initialization start!
Watch dog reset increased value as 1 !

Rx: Watch dog feed the 1 time

Rx: Watch dog feed the 2 time

Rx: Watch dog feed the 3 time

Rx: Watch dog feed the 4 time

Rx: Watch dog feed the 5 time

Rx: Watch dog feed the 6 time

Rx: System Start
Please press KEY 1 bottom to continue the demo

Rx: Watch dog reset occurred
Watch dog initialization start!
Watch dog reset increased value as 2 !

Rx: Watch dog feed the 1 time

Rx: Watch dog feed the 2 time

Rx: Watch dog feed the 3 time

Rx: Watch dog feed the 4 time

Rx: Watch dog feed the 5 time

Rx: Watch dog feed the 6 time

Rx: System Start
Please press KEY 1 bottom to continue the demo

Rx: Watch dog reset occurred
Please perform Power ON Reset

Rx: Watch dog reset occurred
Please perform Power ON Reset

Revision History

Revision	Date	Changes
0.1	2024/12/24	Preliminary version
0.2	2026/01/15	- Updated the example descriptions in <i>Section 2.16 FCSPI</i>, <i>Section 2.18 Flash</i>, <i>Section 2.30 LIN</i>, <i>Section 2.31 LU</i>, and <i>Section 2.38 OTA</i>; - Added the examples <i>Section 2.27 HSADC</i>, and <i>Section 2.44 SC6258XQ</i>; - Updated the demo board related information; - Fixed typos.

Disclaimer

The drivers and example applications provided in this document are intended for skilled software developers who want to get familiar with the MCU peripherals, as well as design, validate and test their applications. These resources are not designed or authorized for designing or fabricating any integrated circuits. This document is for illustrative purposes only and shall not be used for any commercial purposes.

The code and other information included in this SDK are the intellectual property of the Suzhou Flagchip Semiconductor Co., Ltd. (hereinafter referred to as "Flagchip"). The information described herein is subject to change without further notice. Flagchip makes no warranties of the accuracy or completeness of the included information, and reserves the right of final interpretation for the content hereinabove. Flagchip hereby disclaims any responsibilities for any losses, expenses, claims, damages, and liabilities related to the use of these resources.

